

Índice →

[Diagrama de la Red](#)

[Instalación](#)

[Interfaces de red de Proxmox](#)

[Wake on LAN](#)

[Migración de .vdi \(de virtualbox\) a Proxmox](#)

[POC \(Proof of concept\)](#)

[Configuración con claves ssh y usuario root](#)

[Playbooks](#)

[Reglas de firewall.](#)

[VPN](#)

[Instalación de Wireguard Server desde CLI](#)

[Generación de las claves](#)

[Clientes \(peers\)](#)

[DNS dinámica](#)

[Instalación](#)

[Docker-compose](#)

[Persistencia \(servicio linux en systemd\)](#)

[Comprobaciones](#)

[Configuración de los Contenedores](#)

[/ NEXTCLOUD /](#)

[/PIHOLE/](#)

[/Uptime Kuma/](#)

[Red 192.168.20.0/24 \(Windows net\)FCT2](#)

[Creación de Usuarios Unidades Organizativas y Grupos](#)

[WebDAV](#)

[Instalación de los otros puestos por red WDS](#)

[Configuración del Servidor DHCP](#)

[Equipo plantilla.](#)

[GPO Para conexiones remotas a través de la VPN](#)

[Instalamos squid](#)

[configuramos squid](#)

[Wazuh](#)

[Componentes principales de Wazuh:](#)

[Instalar W.Indexer](#)

[Instalar W.Server](#)

[Instalar W.Dashboard](#)

[Propuesta de Implementación de Snort](#)

[1. Preparar una Máquina para Snort](#)

[2. Instalar Snort](#)

[3. Configurar Snort](#)

[Acción de la Regla](#)

[Notificaciones al Teléfono e interfaz Web \(NTFY\)](#)

[Tunel de Cloudfare](#)

[¿Cómo funciona el tunel de cloudfare?](#)

[¿Qué es NTFY?](#)

[Automatización con Proceso del sistema](#)

[Automatización con Crontab](#)

[Configuración de los clientes Syslog con ansible](#)

1. INTRODUCCIÓN

1.1 ESTRUCTURA DEL PROYECTO

Este proyecto se encuentra redactado con la siguiente organización:

En primer lugar se plantea el diagrama de red para entender de manera más clara el funcionamiento de la infraestructura , en un proyecto de esta escala u infraestructuras más grandes es importante organizar bien dónde debe de estar cada endpoint

Remarcar la importancia de probar la infraestructura antes de implantarla, pues a veces pueden surgir imprevistos que si se implantan sin las debidas pruebas pueden suponer un gran quebradero de cabeza para cualquier técnico o ingeniero que lo tenga que solucionar.

También se hará una contabilización del hardware necesario y un análisis económico sencillo, así como una comparación respecto a un sistema no virtualizado tradicional.

Se mostrará en este documento la parte de instalación y configuración necesaria para el funcionamiento de la infraestructura.

Diagrama de la Red

2. **Gestión Centralizada:** Proxmox proporciona una interfaz de usuario web centralizada que permite la gestión y administración de todas las máquinas virtuales y contenedores desde un solo lugar. Esta interfaz intuitiva facilita la creación, configuración y supervisión de las VMs y contenedores.
3. **Virtualización Completa y Basada en Contenedores:** Proxmox admite tanto la virtualización completa (VMs) como la virtualización basada en contenedores (LXC)
4. **Alta Disponibilidad:** Proxmox ofrece funciones avanzadas de alta disponibilidad que garantizan la continuidad del servicio mediante la migración automática de máquinas virtuales en caso de falla de hardware o mantenimiento planificado.
5. **Clustering:** Proxmox permite crear clústeres de servidores para una mayor escalabilidad y redundancia. Los clústeres pueden incluir varios nodos Proxmox que comparten recursos de almacenamiento y cómputo para mejorar el rendimiento y la tolerancia a fallos.

Instalación

Instalar Proxmox en el servidor central donde se van a gestionar todos los servidores, si la instalación se realiza con éxito podríamos conectarnos mediante ssh a la máquina con las credenciales que le proporcionamos durante la instalación y se abre un servidor web en el puerto **8006** donde podremos gestionar Proxmox desde una interfaz gráfica

Conexión **ssh**:

```

ssh root@192.168.5.149
~ ssh root@192.168.5.149
root@192.168.5.149's password:
Linux proxmox 6.8.4-2-pve #1 SMP PREEMPT_DYNAMIC PMX 6.8.4-2 (2024-04-10T17:36Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

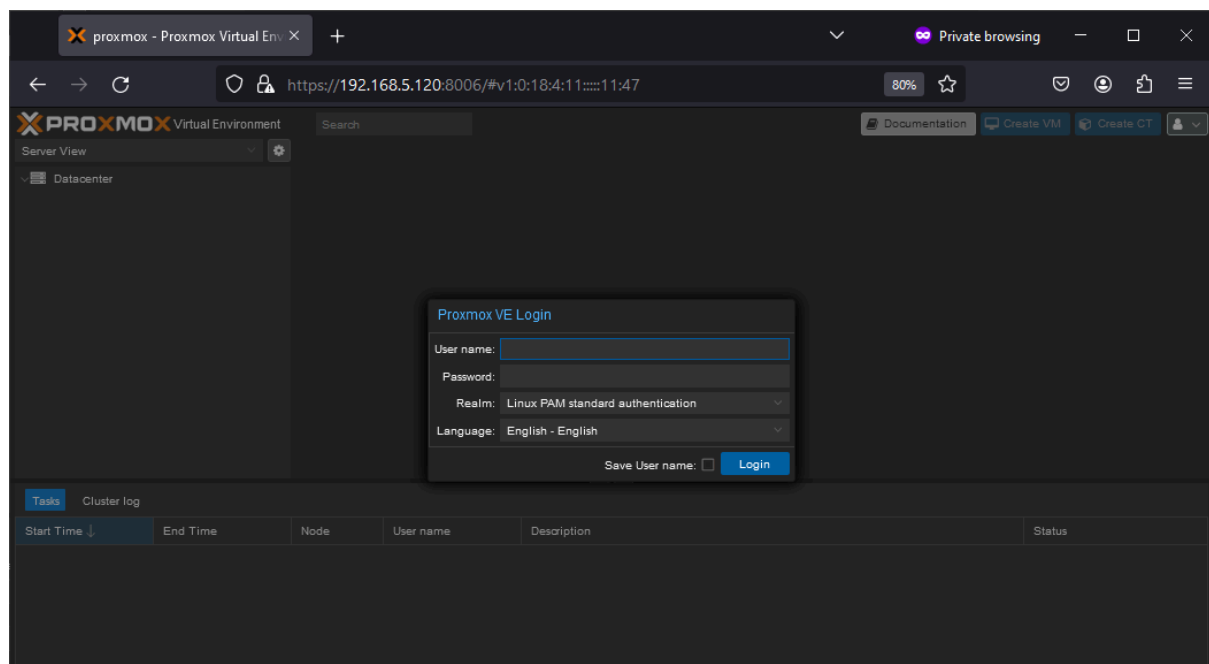
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed May  8 09:00:00 2024 from 192.168.5.10
root@proxmox:~# neofetch

      .:/:.
    `hMMMMMMd/      /dMMMMMMh`
      sMMMMMMd:      :mMMMMMMMs`
-/+oo+/:` .yMMMMMMh- -hMMMMMMMy`:/+oo+/-`
`:/ooooooo/`-hMMMMMMMyMMMMMMh-`/ooooooo:
`/ooooooo:~mMMMMMMMMm~`:/ooooooo/
./ooooooo+~+NMMMMMMMN+~+ooooooo/
./+ooooooo+~oNMMMMMN~+ooooooo+
.+ooooooo/. sMMs`./ooooooo+
:ooooooo/..`/ooooooo:
:ooooooo/..`/ooooooo:
.+ooooooo/. sMMs`./ooooooo+
./+ooooooo+~oNMMMMMN~+ooooooo+
./ooooooo+~+NMMMMMMMN+~+ooooooo/
`/ooooooo:~mMMMMMMMMm~`:/ooooooo/
:ooooooo/`-hMMMMMMMyMMMMMMh-`/ooooooo:
-/+oo+/:` .yMMMMMMh- -hMMMMMMMy`:/+oo+/-
      sMMMMMMm:      :dMMMMMMs`
      hMMMMMMd/      /dMMMMMMh`
      `:/:~          `:/:~

root@proxmox
OS: Proxmox VE 8.2.2 x86_64
Host: MS-7C91 2.0
Kernel: 6.8.4-2-pve
Uptime: 36 mins
Packages: 785 (dpkg)
Shell: bash 5.2.15
Terminal: /dev/pts/0
CPU: AMD Ryzen 9 5900X (24) @ 2.800GHz
GPU: NVIDIA GeForce RTX 3060
Memory: 1527MiB / 23952MiB

```

Servicio web:



Interfaces de red de Proxmox

Vamos a crear varias interfaces de red puente (nat) para que podamos tener salida a internet en nuestras máquinas

Vamos a crear un puente en la red , este es mi /etc/network/interfaces de proxmox (actualizado a día 12/06/24)

```
auto lo
iface lo inet loopback

iface enp5s0 inet manual

auto enp42s0
iface enp42s0 inet manual

auto vmbr0
iface vmbr0 inet static
    address 192.168.5.254/24
    gateway 192.168.5.1
    bridge-ports enp42s0
    bridge-stp off
    bridge-fd 0
    post-up iptables -t nat -A PREROUTING -i vmbr0 -p udp --dport 51821 -j DNAT
--to-destination 10.0.2.1:51821
    post-up iptables -A FORWARD -p udp -d 10.0.2.1 --dport 51821 -m state --state
NEW,ESTABLISHED,RELATED -j ACCEPT
    pre-down iptables -t nat -D PREROUTING -i vmbr0 -p udp --dport 51821 -j DNAT
--to-destination 10.0.2.1:51821
    pre-down iptables -D FORWARD -p udp -d 10.0.2.1 --dport 51821 -m state --state
NEW,ESTABLISHED,RELATED -j ACCEPT
# Reglas IPtables para reditigir tráfico de Wireguard al router
#puente Principal

auto vmbr2
iface vmbr2 inet static
    address 10.0.2.2/24
    bridge-ports none
    bridge-stp off
    bridge-fd 0
    up /usr/sbin/brctl setageing vmbr2 0
    up /usr/sbin/brctl setfd vmbr2 0
#NAT (bridge)

auto vmbr1
iface vmbr1 inet manual
    bridge-ports none
    bridge-stp off
    bridge-fd 0
#Red interna FCT1

auto vmbr3
```

```

iface vmbr3 inet manual
    bridge-ports none
    bridge-stp off
    bridge-fd 0
#RED INTERNA FCT2

auto vmbr4
iface vmbr4 inet static
    address 20.20.20.1/24
    bridge-ports none
    bridge-stp off
    bridge-fd 0
    bridge.stp off
    up /usr/sbin/brctl setageing vmbr4 0
    up /usr/sbin/brctl setfd vmbr4 0

#RED PRUEBAS
post-up echo 1 > /proc/sys/net/ipv4/ip_forward
post-up iptables -t nat -A POSTROUTING -s '10.0.2.0/24' -o vmbr0 -j MASQUERADE
post-down iptables -t nat -D POSTROUTING -s '10.0.2.0/24' -o vmbr0 -j MASQUERADE
## redirigir tráfico a red de pruebas
post-up iptables -t nat -A POSTROUTING -s '20.20.20.0/24' -o vmbr0 -j MASQUERADE
post-down iptables -t nat -D POSTROUTING -s '20.20.20.0/24' -o vmbr0 -j
MASQUERADE

source /etc/network/interfaces.d/*

```

Ahora la interfaz vmbr2 enrutará el tráfico de red puente la con la red 10.0.2.0/24 dándole salida a internet

Realizo una prueba con contenedores de proxmox

- TestRouter
 - eth0 - 192.168.11.0/24
 - enp5s0 - 10.0.2.0/24
- Test
 - eht0 - 192.168.11.0/24

En el router ejecuto

```

post-up echo 1 > /proc/sys/net/ipv4/ip_forward
post-up iptables -t nat -A POSTROUTING -s '192.168.11.0/24' -o enp5s0 -j
MASQUERADE
post-down iptables -t nat -D POSTROUTING -s '192.168.11.0/24' -o enp5s0 -j
MASQUERADE

```

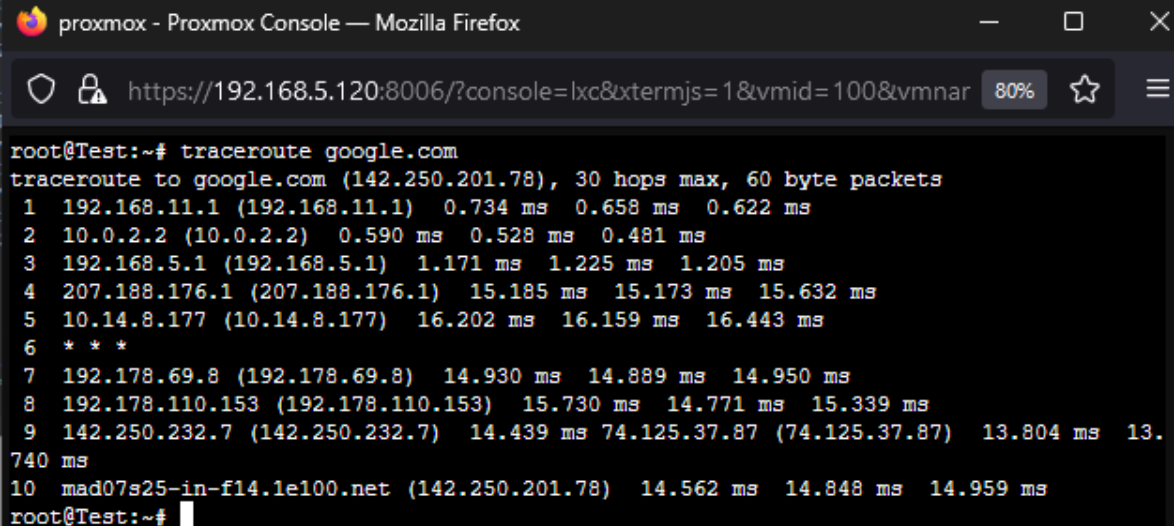
Probamos el enrutamiento:

```

root@test-router:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if55: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:31:f8:73 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.11.1/24 brd 192.168.11.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe31:f873/64 scope link
        valid_lft forever preferred_lft forever
3: enp5s22@if59: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:2d:61:d9 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.2.3/24 brd 10.0.2.255 scope global enp5s22
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe2d:61d9/64 scope link
        valid_lft forever preferred_lft forever
root@test-router:~# ping 192.168.11.3
PING 192.168.11.3 (192.168.11.3) 56(84) bytes of data.
64 bytes from 192.168.11.3: icmp_seq=1 ttl=64 time=0.160 ms
64 bytes from 192.168.11.3: icmp_seq=2 ttl=64 time=0.076 ms
^C
--- 192.168.11.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1048ms
rtt min/avg/max/mdev = 0.076/0.118/0.160/0.042 ms
root@test-router:~#

```

Salida a internet:



The screenshot shows a web browser window titled "proxmox - Proxmox Console — Mozilla Firefox". The address bar displays the URL "https://192.168.5.120:8006/?console=lxcl&xtermjs=1&vmid=100&vmnar=80%". The console output shows a successful traceroute to google.com (142.250.201.78) with 30 hops max and 60 byte packets. The route starts at 192.168.11.1 and ends at 142.250.232.7, with intermediate hops including 10.0.2.2, 192.168.5.1, 207.188.176.1, 10.14.8.177, 192.178.69.8, 192.178.110.153, and 74.125.37.87.

```

root@Test:~# traceroute google.com
traceroute to google.com (142.250.201.78), 30 hops max, 60 byte packets
 1 192.168.11.1 (192.168.11.1) 0.734 ms 0.658 ms 0.622 ms
 2 10.0.2.2 (10.0.2.2) 0.590 ms 0.528 ms 0.481 ms
 3 192.168.5.1 (192.168.5.1) 1.171 ms 1.225 ms 1.205 ms
 4 207.188.176.1 (207.188.176.1) 15.185 ms 15.173 ms 15.632 ms
 5 10.14.8.177 (10.14.8.177) 16.202 ms 16.159 ms 16.443 ms
 6 * * *
 7 192.178.69.8 (192.178.69.8) 14.930 ms 14.889 ms 14.950 ms
 8 192.178.110.153 (192.178.110.153) 15.730 ms 14.771 ms 15.339 ms
 9 142.250.232.7 (142.250.232.7) 14.439 ms 74.125.37.87 (74.125.37.87) 13.804 ms 13.740 ms
10 mad07s25-in-f14.1e100.net (142.250.201.78) 14.562 ms 14.848 ms 14.959 ms
root@Test:~#

```

Wake on LAN

Wake on Lan (WoL) es una tecnología que permite encender un equipo de forma remota a través de la red local , no viene integrada en todas las placas base así que necesitaremos una placa base compatible con esto

Vamos a implementar el Wake on LAN en nuestro servidor para que en caso de apagarse podamos volver a encender el equipo desde un equipo de nuestra red local . gracias a la conexión de VPN podremos a acceder de forma segura a la raspberry pi dentro de nuestra Red de area local (Lan) [[ver diagrama de red](#)] y mandarle un [paquete mágico](#)

Antes de activar el WoL en el sistema operativo debemos entrar en la BIOS de nuestra pc y encontrar la opción que nos permita activar wake-on-lan , normalmente podemos encontrar la opción cómo WoL o Wake on PCI-E/PCI

Para configurar nuestro Wake on Lan debemos hacer varias configuraciones dentro de la máquina de Proxmox:

- Instalar el Ehttools

Ehttools es una herramienta de Unix que nos permitirá ver más información de las interfaces de red, entre ellas si está habilitado wake-on-lan , desde el punto de vista de hardware si mi tarjeta de red tiene varios puertos de red y por ende varias interfaces es importante determinar en que interfaz quiero activar wake-on-lan, además mi servidor proxmox tiene varias interfaces de red puente que le dan salida a internet a varias máquinas de mi red , Para mandar el [paquete mágico](#) será necesario también encontrar la mac de esa interfaz de red. ya que vamos a mandar un paquete por la red que apunte a esa dirección mac.

Para instalar el paquete podemos descargarlo en linux con el instalador de paquetes de preferencia en nuestro sistema

Vamos a ejecutar la herramienta indicándo la interfaz que queremos comprobar, en mi caso enp42s0 que es la que uso para hacer los puentes.

```
root@proxmox:~# ethtool enp42s0
Settings for enp42s0:
    Supported ports: [ TP MII ]
    Supported link modes:  10baseT/Half 10baseT/Full
                          100baseT/Half 100baseT/Full
                          1000baseT/Full
                          2500baseT/Full
    Supported pause frame use: Symmetric Receive-only
    Supports auto-negotiation: Yes
    Supported FEC modes: Not reported
    Advertised link modes:  10baseT/Half 10baseT/Full
                          100baseT/Half 100baseT/Full
```

```
1000baseT/Full
2500baseT/Full
Advertised pause frame use: Symmetric Receive-only
Advertised auto-negotiation: Yes
Advertised FEC modes: Not reported
Link partner advertised link modes: 10baseT/Half 10baseT/Full
                                   100baseT/Half 100baseT/Full
                                   1000baseT/Half 1000baseT/Full
Link partner advertised pause frame use: Symmetric
Link partner advertised auto-negotiation: Yes
Link partner advertised FEC modes: Not reported
Speed: 1000Mb/s
Duplex: Full
Auto-negotiation: on
master-slave cfg: preferred slave
master-slave status: slave
Port: Twisted Pair
PHYAD: 0
Transceiver: external
MDI-X: Unknown
Supports Wake-on: pumbg
Wake-on: d
Link detected: yes
```

La opción que nos interesa es el Wake-on que por defecto aparecerá cómo d(disabled) debemos activarla de modo que aparezca como g (granted) con el parámetro -s (set)

```
ethtool -s enp42s0 wol g
```

Persistencia

Para que sea persistente nuestra configuración debemos editar el archivo nano /etc/network/if-up.d/ethtool

Lo abrimos con un editor de texto y le añadimos la línea `ethtool -s enp42s0 wol g` Esto hace que cada vez que se inicie ifup (herramienta de unix para levantar las interfaces de red) se ejecute ethool y le de granted a la opción de wol, debemos darle permisos de ejecución con el comando:

```
chmod +x /etc/network/if-up.d/ethtool
```

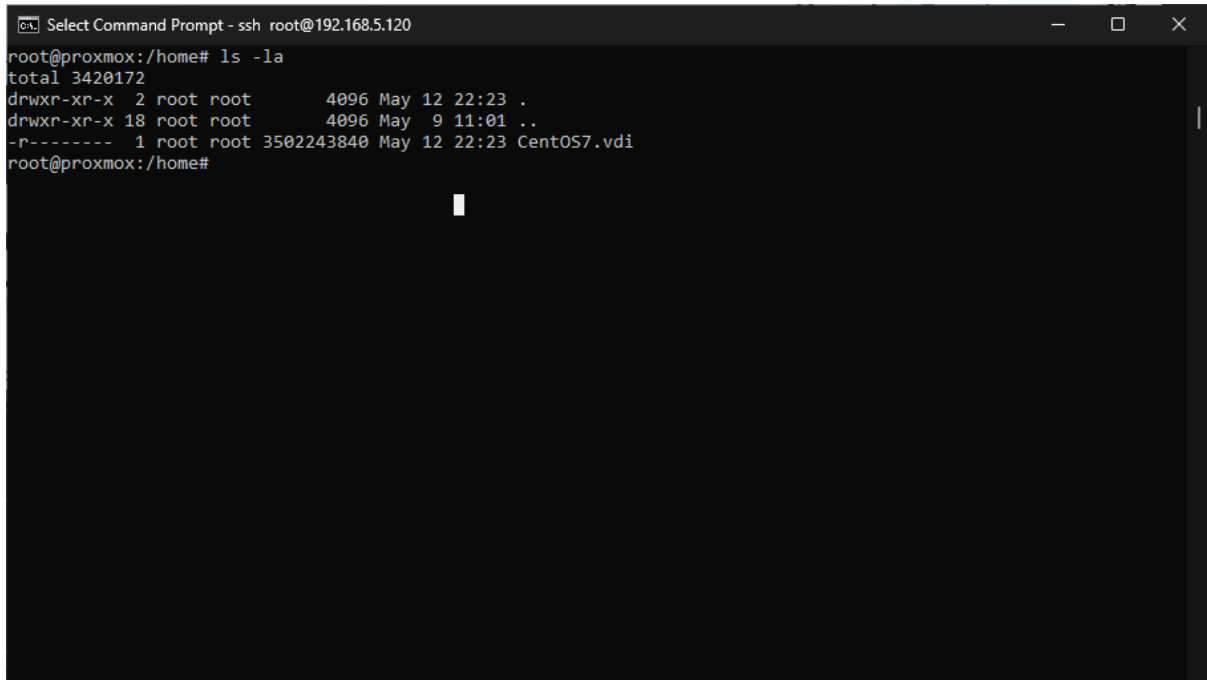
para testearlo podemos desactivar y activar la interfaz con el siguiente comando

```
ifdown enp42s0 && ifup enp42s0
```

Si volvemos a ejecutar la herramienta ethtool podemos ver que la opción de WoL está activada. (`ethtool enp42s0`)

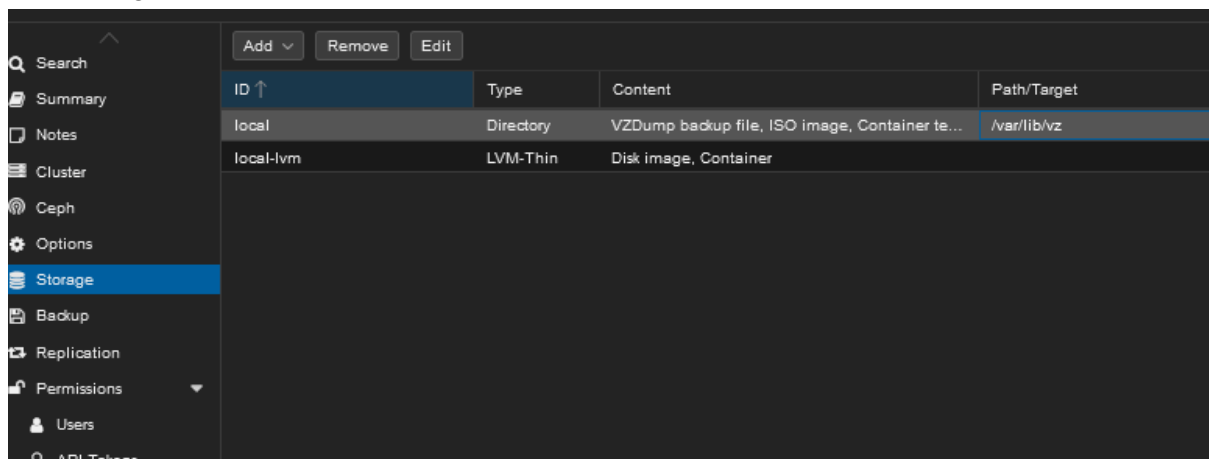
Migración de .vdi (de virtualbox) a Proxmox

- 1-Copiamos nuestro archivo .vdi (virtual disk image)



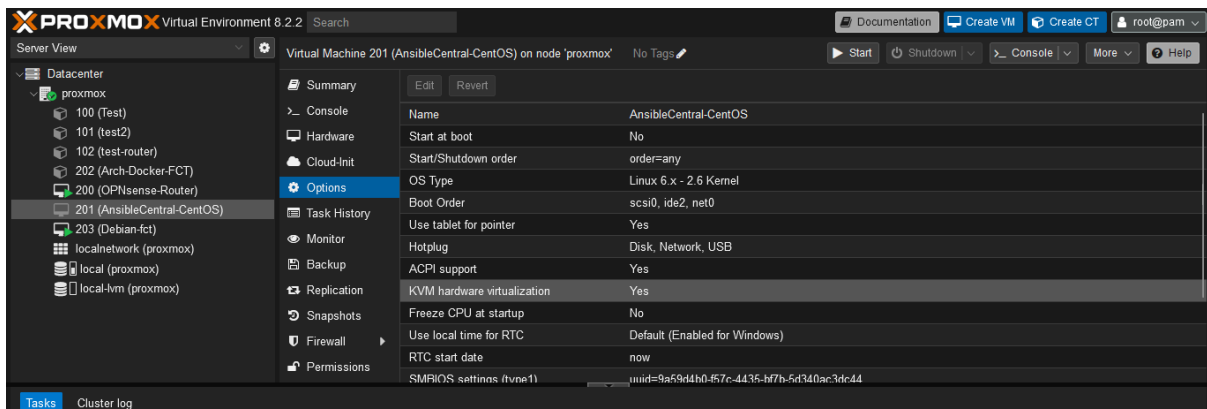
```
Select Command Prompt - ssh root@192.168.5.120
root@proxmox:/home# ls -la
total 3420172
drwxr-xr-x  2 root root    4096 May 12 22:23 .
drwxr-xr-x 18 root root    4096 May  9 11:01 ..
-r-----  1 root root 3502243840 May 12 22:23 CentOS7.vdi
root@proxmox:/home#
```

- 2 - vamos a mirar la ruta donde se encuentra nuestro storage de proxmox y vamos a migrar los archivos allí



ID ↑	Type	Content	Path/Target
local	Directory	VZDump backup file, ISO image, Container te...	/var/lib/vz
local-lvm	LVM-Thin	Disk image, Container	

- 3 - Vamos a crear una Instancia para el .vdi que vamos a importar .



En mi caso creo la máquina con el ID 201 .

El motivo por el que debemos crear una nueva VM es porque proxmox gestiona sus discos virtuales en **volúmenes virtuales**

Un volumen virtual viene a ser parecido a una partición disco sobre el papel ,de todas formas las LV tienen unas ventajas frente a las particiones de disco:

- Los LVs pueden crecer o reducir su tamaño de manera dinámica sin afectar el disco físico o el resto de los LVs y también pueden tener su propio sistema de archivos y se pueden montar en directorios del sistema, similar a las particiones de disco.

Las particiones de disco si que no pueden reducir su tamaño de forma dinámica , por eso proxmox se decanta por crear LVs en el disco para el almacén de sus discos virtuales.

- 4- Ejecutamos el siguiente comando en el host de proxmox:

qm importdisk 201 centOS7.vdi local-lvm

```

Command Prompt - ssh root@192.168.5.120
root@proxmox:/home# qm importdisk 201 CentOS7.vdi local-lvm
importing disk 'CentOS7.vdi' to VM 201 ...
Rounding up size to full physical extent 20.35 GiB
Logical volume "vm-201-disk-2" created.
transferred 0.0 B of 20.4 GiB (0.00%)
transferred 212.6 MiB of 20.4 GiB (1.02%)
transferred 425.1 MiB of 20.4 GiB (2.04%)
transferred 637.7 MiB of 20.4 GiB (3.06%)
transferred 848.2 MiB of 20.4 GiB (4.07%)
transferred 1.0 GiB of 20.4 GiB (5.09%)
transferred 1.2 GiB of 20.4 GiB (6.11%)
transferred 1.5 GiB of 20.4 GiB (7.13%)
transferred 1.7 GiB of 20.4 GiB (8.15%)
transferred 1.9 GiB of 20.4 GiB (9.17%)
transferred 2.1 GiB of 20.4 GiB (10.19%)
transferred 2.3 GiB of 20.4 GiB (11.20%)
transferred 2.5 GiB of 20.4 GiB (12.22%)
transferred 2.7 GiB of 20.4 GiB (13.24%)
transferred 2.9 GiB of 20.4 GiB (14.26%)
transferred 3.1 GiB of 20.4 GiB (15.28%)
transferred 3.3 GiB of 20.4 GiB (16.30%)
transferred 3.5 GiB of 20.4 GiB (17.32%)
transferred 3.7 GiB of 20.4 GiB (18.33%)
transferred 3.9 GiB of 20.4 GiB (19.35%)
transferred 4.1 GiB of 20.4 GiB (20.37%)
transferred 4.4 GiB of 20.4 GiB (21.39%)
transferred 4.6 GiB of 20.4 GiB (22.41%)
transferred 4.8 GiB of 20.4 GiB (23.43%)
transferred 5.0 GiB of 20.4 GiB (24.45%)
transferred 5.2 GiB of 20.4 GiB (25.46%)
transferred 5.4 GiB of 20.4 GiB (26.48%)
transferred 5.6 GiB of 20.4 GiB (27.50%)

```

- **qm importdisk**: Este es el comando principal que indica a Proxmox que queremos importar un disco virtual.
- **201**: Este es el ID del disco virtual al que queremos importar el nuevo disco.
- **centOS.qcow2**: Este es el nombre del archivo del disco virtual que queremos importar. En este caso, estamos importando un disco virtual en formato **vdi** llamado **centOS.vdi**.
- **local-lvm**: Este es el almacenamiento de Proxmox al que queremos importar el disco. En este caso, estamos importando el disco al almacenamiento **local-lvm**.

```
Select Command Prompt - ssh root@192.168.5.120
transferred 19.7 GiB of 20.4 GiB (96.76%)
transferred 19.9 GiB of 20.4 GiB (97.78%)
transferred 20.1 GiB of 20.4 GiB (98.80%)
transferred 20.3 GiB of 20.4 GiB (99.82%)
transferred 20.4 GiB of 20.4 GiB (100.00%)
transferred 20.4 GiB of 20.4 GiB (100.00%)
Successfully imported disk as 'unused0:local-lvm:vm-201-disk-2'
root@proxmox:/home#
```

- 5 - Detach al disco que estaba en el VM

Proxmox Virtual Environment 8.2-2

Virtual Machine 201 (AnsibleCentral-CentOS) on node 'proxmox'

Summary

- Memory: 1.00 GiB
- Processors: 1 (1 sockets, 1 cores) [x86-64-v2-AES]
- BIOS: Default (SeaBIOS)
- Display: Default
- Machine: Default (i440fx)
- SCSI Controller: VirtIO SCSI single

Confirm

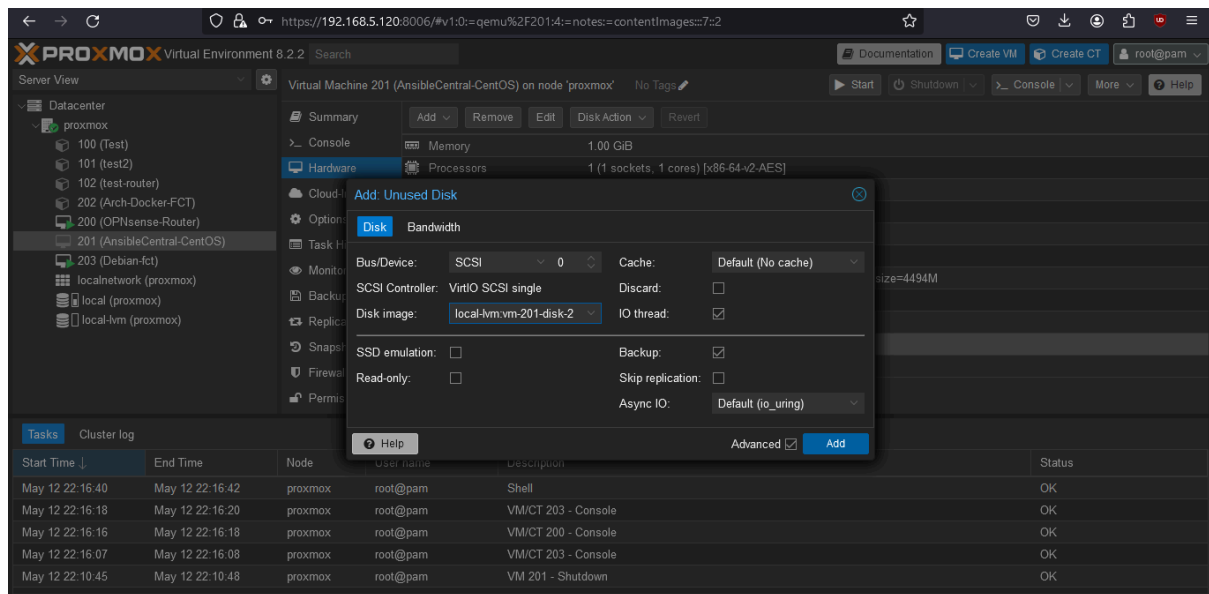
Are you sure you want to detach entry 'Hard Disk (scsi0)'?

Yes No

Tasks Cluster log

Start Time	End Time	Node	User name	Description	Status
May 12 22:16:40	May 12 22:16:42	proxmox	root@pam	Shell	OK
May 12 22:16:18	May 12 22:16:20	proxmox	root@pam	VM/CT 203 - Console	OK
May 12 22:16:16	May 12 22:16:18	proxmox	root@pam	VM/CT 200 - Console	OK
May 12 22:16:07	May 12 22:16:08	proxmox	root@pam	VM/CT 203 - Console	OK
May 12 22:10:45	May 12 22:10:48	proxmox	root@pam	VM 201 - Shutdown	OK

Ahora debemos añadirle un disco y asignarle el disco que importamos con el nombre de vm-201-disk-2



Ansible

Ansible es una herramienta de automatización de TI de código abierto que se utiliza para la gestión de la configuración, la implementación de aplicaciones y la automatización de tareas muy usada en entornos de TI.

Utiliza un enfoque basado en lenguaje **YAML** (Yet Another Markup Language) para describir los procesos de automatización de manera simple y legible.

Principales características de Ansible:

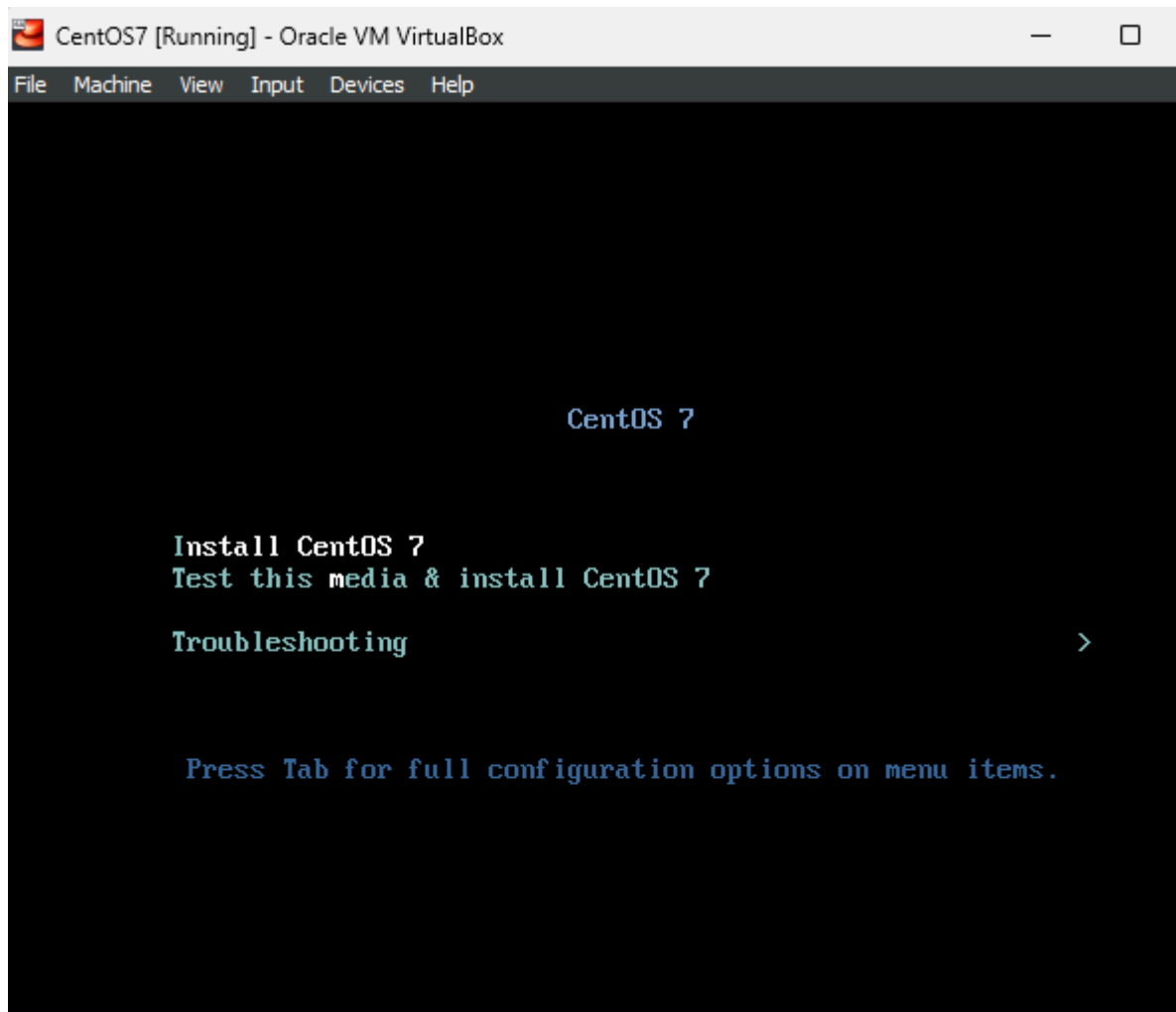
1. **Gestión de la Configuración:** Ansible permite definir y gestionar la configuración de sistemas y servicios de forma declarativa. Esto significa que puedes describir el estado deseado de tus sistemas en archivos de configuración YAML, y Ansible se encargará de asegurarse de que los sistemas están configurados de acuerdo a esa especificación.
2. **Tareas:** Ansible puede orquestar tareas que involucren múltiples servidores y dispositivos de red. Podemos definir trabajos que incluyan la instalación de software, la configuración de servicios, la gestión de usuarios y grupos....
3. **Agentless:** Ansible opera de forma "agentless", lo que significa que no requiere la instalación de agentes en los sistemas que gestiona. **Utiliza SSH** para ejecutar comandos y configuraciones en los sistemas objetivo.

Este es el punto fuerte de ansible, imaginemos que tenemos X servidores en nuestra empresa repartidos en diferentes localizaciones y quiero cambiar el servidor dns de los X servidores, bastaría con editar el /etc/resolv.conf si se tratase de un sistema linux pero

hacerlo uno por uno va a llevar tiempo que nos podemos ahorrar ya que con ansible podríamos automatizar esa tarea.

POC (Proof of concept)

Vamos a instalar nuestro equipo de control que tendrá como sistema operativo un CentOS-7 basado en redhat .



Ansible guardará todo lo que necesita para funcionar en la ruta : /etc/ansible

```
[root@CentralitaOS ansible]# ls
ansible.cfg  hosts  roles
[root@CentralitaOS ansible]# pwd
/etc/ansible
[root@CentralitaOS ansible]# _
```

Una vez instalado ansible en la configuración debemos descomentar el check de ssh host checking

```
File Machine View Input Devices Help
GNU nano 2.3.1 File: ansible.cfg Modified

# timeout to something more suitable for the
# environment.
# gather_timeout = 10

# Ansible facts are available inside the ansible_facts.* dictionary
# namespace. This setting maintains the behaviour which was the default prior
# to 2.5, duplicating these variables into the main namespace, each with a
# prefix of 'ansible_'.
# This variable is set to True by default for backwards compatibility. It
# will be changed to a default of 'False' in a future release.
# ansible_facts.
# inject_facts_as_vars = True

# additional paths to search for roles in, colon separated
#roles_path = /etc/ansible/roles

# uncomment this to disable SSH key host checking
host_key_checking = False

# change the default callback, you can only have one 'stdout' type enabled at a time.
#stdout_callback = skippy

## Ansible ships with some plugins that require whitelisting,
## this is done to avoid running all of a type by default.
## These setting lists those that you want enabled for your system.
## Custom plugins should not need this unless plugin author specifies it.

# enable callback plugins, they can output to stdout but cannot be 'stdout' type.
#callback_whitelist = timer, mail

# Determine whether includes in tasks and handlers are "static" by

^G Get Help    ^O WriteOut    ^R Read File   ^Y Prev Page   ^K Cut Text     ^C Cur Pos
^X Exit        ^J Justify     ^W Where Is    ^N Next Page   ^U UnCut Text   ^T To Spell

CTRL DERECHA
```

En nuestro archivo hosts indicaremos las rutas hacia las máquinas así cómo las credenciales para loguearse por ssh en ambas máquinas

El archivo de host tiene la siguiente sintaxis

```
GNU nano 2.3.1      File: hosts      Modified
[prueba] — 1
10.0.5.5 #maquina debian 1 en red NAT
192.168.10.3 #maquina debian 2 en red Interna

[prueba:vars] — 2
ansible_user=ansible
ansible_password=ansible1234_

^G Get Help      ^O WriteOut      ^R Read File     ^Y Prev Page     ^K Cut Text      ^C Cur Pos
^X Exit          ^J Justify       ^W Where Is      ^U Next Page     ^U UnCut Text    ^T To Spell
```

1. Entre corchetes el grupo de hosts en este caso “prueba” es el nombre con el que vamos a referenciar a ese grupo de Hosts
2. (Grupo:vars) : Para especificar las credenciales con las que nos vamos a conectar. por ssh a las máquinas

En los equipos debemos de crear el usuario con la contraseña que indicamos en el archivo “hosts” de ansible para ello:

```
sudo useradd -m -p $(openssl passwd -1 ansible1234) ansible
```

`$(openssl passwd -1 ansible1234)` genera un hash de la contraseña "ansible1234" utilizando el algoritmo de cifrado MD5, que es el mismo algoritmo que utiliza Linux para almacenar las contraseñas en el archivo `/etc/shadow`. Esto nos permite crear un usuario con una contraseña ya cifrada sin necesidad de introducir el hash manualmente.

Además debemos habilitar y activar ssh

```
sudo systemctl enable ssh
sudo systemctl start ssh
```

Creo un script par automatizar el proceso en máquinas linux, ademas de instalar y habilitar el ssh

```
#!/bin/bash

# Verificar si SSH está instalado
if ! command -v ssh &> /dev/null; then
    echo "SSH no está instalado. Instalando SSH..."
    sudo apt update
    sudo apt install -y openssh-server
fi

# Crear el usuario "ansible" con contraseña "ansible1234"
sudo useradd -m ansible
echo "ansible:ansible1234" | sudo chpasswd

# Agregar el usuario "ansible" al grupo sudoers con permisos NOPASSWD
echo "ansible ALL=(ALL) NOPASSWD: ALL" | sudo tee -a /etc/sudoers.d/ansible

# Cambiar el propietario del archivo sudoers.d
sudo chown root:root /etc/sudoers.d/ansible

# Cambiar los permisos del archivo sudoers.d
sudo chmod 440 /etc/sudoers.d/ansible

# Habilitar SSH
sudo systemctl enable ssh
```

Una vez realizada la configuración vamos a probar un módulo simple de Ansible, vamos a hacer un ping al grupo de equipos para ver que la conexión se establece

```
admin@CentralitaOS:~  
[admin@CentralitaOS ~]$ ansible prueba -m ping  
[WARNING]: Platform linux on host 192.168.10.3 is using the discovered Python  
interpreter at /usr/bin/python3, but future installation of another Python  
interpreter could change this. See  
https://docs.ansible.com/ansible/2.9/reference\_appendices/interpreter\_discovery.html  
for more information.  
192.168.10.3 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "changed": false,  
  "ping": "pong"  
}  
[WARNING]: Platform linux on host 10.0.5.5 is using the discovered Python interpreter  
at /usr/bin/python, but future installation of another Python interpreter could  
change this. See  
https://docs.ansible.com/ansible/2.9/reference\_appendices/interpreter\_discovery.html  
for more information.  
10.0.5.5 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python"  
  },  
  "changed": false,  
  "ping": "pong"  
}  
[admin@CentralitaOS ~]$
```

1 fe80::7e50:e13e:2fd2:6a13 | 14.6 GiB | W: down | E: 10.0.5.254 (1000 Mbit/s) | No battery | 0.13 | 2024-05-07 11:00:25

la sintaxis es:

Ansible (grupo de hosts) -m (indicamos módulo)

Con -a podemos lanzar un comando a todos los equipos del grupo, por ejemplo un "whoami"

```
admin@Centralita05:~$ ansible prueba -a "whoami"
[WARNING]: Platform linux on host 10.0.5.5 is using the discovered Python interpreter
at /usr/bin/python, but future installation of another Python interpreter could
change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html
for more information.
10.0.5.5 | CHANGED | rc=0 >>
ansible
[WARNING]: Platform linux on host 192.168.10.3 is using the discovered Python
interpreter at /usr/bin/python3, but future installation of another Python
interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html
for more information.
192.168.10.3 | CHANGED | rc=0 >>
ansible
[admin@Centralita05 ~]$
```

1 fe80::7e50:e13e:2fd2:6a13 | 14.6 GiB | W: down | E: 10.0.5.254 (1000 Mbit/s) | No battery | 0.02 | 2024-05-07 11:02:25

Configuración con claves ssh y usuario root

Vamos a crear un script para que cree el usuario ansible con el que nos vamos a logear y que copie la clave ssh pública del usuario root del servidor central de Ansible desde donde nos vamos a conectar al resto de equipos, este script se debe de ejecutar en todos los equipos de la empresa

Prefiero este método de conexión mediante claves frente a una conexión por contraseña por motivos de seguridad, de esta forma nos ahorramos posibles ataques de fuerza bruta por diccionario y guardar la contraseña en texto claro dentro del script.

```
#!/bin/bash

# Verifica si el script se ejecuta como root
if [ "$EUID" -ne 0 ]; then
    echo "Por favor, ejecuta este script como root."
    exit 1
fi
```

```
# Configura el usuario ansible con contraseña 'ansible'
echo "Creando el usuario ansible con contraseña 'ansible' y
directorio /home/ansible..."
useradd -m -d /home/ansible ansible
echo "ansible:ansible" | chpasswd

# Otorga permisos de administrador al usuario ansible sin solicitar
contraseña de sudo
echo "Configurando sudo para el usuario ansible sin solicitar
contraseña..."
echo "ansible ALL=(ALL) NOPASSWD: ALL" > /etc/sudoers.d/ansible

# Verifica si SSH esta instalado y lo activa si es necesario
echo "Verificando si SSH esta instalado..."
if ! dpkg -s openssh-server >/dev/null 2>&1; then
    echo "SSH no esta instalado. Instalando SSH..."
    apt update
    apt install -y openssh-server
    # Habilita el servicio SSH
    systemctl enable ssh
fi

# Configura SSH para permitir la autentificacion con clave
echo "Configurando SSH para permitir la autentificacion con clave..."
sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/'
/etc/ssh/sshd_config
sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/'
/etc/ssh/sshd_config
systemctl restart ssh

# Verifica si el directorio .ssh existe y lo crea si es necesario
echo "Verificando si el directorio /home/ansible/.ssh existe..."
if [ ! -d /home/ansible/.ssh ]; then
    echo "El directorio /home/ansible/.ssh no existe. Creándolo..."
    mkdir -p /home/ansible/.ssh
    chmod 700 /home/ansible/.ssh
fi

# Agrega la clave publica del servidor Ansible al usuario ansible
echo "Agregando la clave publica del servidor Ansible al usuario
ansible..."
echo "ssh-rsa
```

```

AAAAB3NzaClyc2EAAAADAQABAAQDAtdVDNlQYMR8IXHiL7LqURbk0S6tDLDje9P89ra
ART5u+ClyvMfnX/w4EKkh1iY20GDc7U2/wuklQB9vUcT694Wbyg7OEHZYTydCsQ5X5jP0
Oj4jcmydOQvepIfA193w/zDaklP8H0k5caNmZLwk5U0yGbYofzd4d60GNP0KtmQk7M/nO
kwxQCtYPnZtP4gj3mMxyPdoBSDTkO2/+J5GqxR+oDowlEepBBFrYZnV3gUhQsAi7II8d7
oQmC0tvXchic0phc3T36SggbcvM6x/BGoOqG+uNBasOAackOWiZzrm4IftxPn0JlN135r
0PlfjQi9t2JNxmUTxHC1aL52t6g4DODmh5yiAA2KF0z+WFRmsjAhCT9nzJ+orxudC7V78
S/zfqHsLTw+9un2dJnkPT2+4tNxkM0t7Nu0ld1fG0+StAAmN9oSswJAdfCLfs40TlgpL8
ErumjRsFyjifv/9Cuu3OyZaQFFTKsW0G4n68DobiTO57L7YNIwljRnONKBps=
root@Deb-Ansible" >> /home/ansible/.ssh/authorized_keys
chmod 600 /home/ansible/.ssh/authorized_keys
chown -R ansible:ansible /home/ansible/.ssh

echo "Agregando el usuario ansible al grupo systemd-journal..."
usermod -aG systemd-journal ansible

echo "Configuracion completada."

```

Con eso, nuestro archivo ansible.conf puede verse tal que así sin ninguna contraseña visible, solo el uso de nuestra clave ssh

```

root@Deb-Ansible:~# cat /etc/ansible/ansible.cfg

[defaults]
inventory = /root/inventory.yml
remote_user = ansible
private_key_file = /root/.ssh/id_rsa

```

Nuestro archivo inventory.yml se ve tal que así

```

root@Deb-Ansible:~# cat inventory.yml
all:
  hosts:
    debian-fct:
      ansible_host: 192.168.10.14
    equipo2:
      ansible_host: 192.168.10.11
    equipo3:
      ansible_host: 192.168.10.12

```

Este archivo se va a ir actualizando a medida que se añadan nuevos equipos en la empresa

En el cliente, vamos a probar a ejecutar el script .

```
QEMU (Debian-fct) - noVNC - Opera
No es seguro localhost:8006
Applications Live View Log Files Fire... Terminal - debian@debian...
Terminal - debian@debianfct:~
File Edit View Terminal Tabs Help
debian@debianfct:~$ whoami
debian
debian@debianfct:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:32:1b:2e brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    inet 192.168.10.14/24 brd 192.168.10.255 scope global dynamic noprefixroute ens18
        valid_lft 5119sec preferred_lft 5119sec
    inet6 fe80::b9fd:c9d3:flc0:ae55/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
debian@debianfct:~$ chmod +x AnsibleClientInstal.sh
```

```
QEMU (Debian-fct) - noVNC - Opera
No es seguro localhost:8006
Applications Live View Log Files Fire... Terminal - debian@debian...
Terminal - debian@debianfct:~
File Edit View Terminal Tabs Help
Preparing to unpack .../runit-helper_2.15.2_all.deb ...
Unpacking runit-helper (2.15.2) ...
Selecting previously unselected package openssh-server.
Preparing to unpack .../openssh-server_1%3a9.2p1-2+deb12u2_amd64.deb ...
Unpacking openssh-server (1:9.2p1-2+deb12u2) ...
Setting up runit-helper (2.15.2) ...
Setting up openssh-client (1:9.2p1-2+deb12u2) ...
Setting up openssh-sftp-server (1:9.2p1-2+deb12u2) ...
Setting up openssh-server (1:9.2p1-2+deb12u2) ...

Creating config file /etc/ssh/sshd_config with new version
*creating SSH2 RSA key; this may take some time ...
2072 SHA256:TSTEEeEQ++HYDvcmB1ST9l1qKfLXdpfGA4ufDtNoRU root@debianfct (RSA)
Creating SSH2 ECDSA key; this may take some time ...
256 SHA256:9L1lFN4/wftrROANimAQpUdreQ9mp+T4PpJOxUs+BPQ root@debianfct (ECDSA)
Creating SSH2 ED25519 key; this may take some time ...
256 SHA256:AyCkUWVqlaJ5IpJ+P8Z9+JC/S/2m/rVw01sQy9Tm4og root@debianfct (ED25519)
Created symlink /etc/systemd/system/ssh.service → /lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /lib/systemd/system/ssh.service.
rescue-ssh.target is a disabled or a static unit, not starting it.
ssh.socket is a disabled or a static unit, not starting it.
Processing triggers for man-db (2.11.2-2) ...
Synchronizing state of ssh.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
Configurando SSH para permitir la autenticacion con clave...
Agregando la clave publica del servidor Ansible al usuario ansible...
./AnsibleClientInstal.sh: line 36: /home/ansible/.ssh/authorized_keys: No such file or directory
chown: cannot access '/home/ansible/.ssh/authorized_keys': No such file or directory
chmod: cannot access '/home/ansible/.ssh/authorized_keys': No such file or directory
Configuracion completada.
debian@debianfct:~$
```

Una vez se ejecuta el script compruebo que el usuario ansible esta creado, que la clave pública del servidor de ansible está configurada y que el ssh esté activo.

```

Configuracion completada.
debian@debianfct:~$ sudo su
root@debianfct:/home/debian# cat /home/ansible/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDAtVDNlQYMR8IXHl7LqURbk0S6tDLdJe9P89raART5u+ClyvMfnX/w4EKkh1iY20GDc7U2/wuklQB9vUcT694Wb
yg70EHZYTydCsQ5X5jP00j4jcmYd0QvepIfA193w/zDaklP8H0k5caNmZLwKSU0yGbYofzd4d60GNP0KtmQk7M/n0kwxQctYPnZtP4g3mMxyPdoB5DTk02/+J5Gqx
R+oD0w1EepBBFrYZnV3gUhQsAi7II8d7oQmC0tVchic0phc3T36SggbcvM6x/BGo0qG+uNBas0Aack0WiZrm4IftxPn0JJN135x0P1fjQi9t2JNxMuTxHC1aL52t
6g4D0Dmh5yiAA2KF0z+WFrmSjAhCT9nzJ+orxudC7V78S/zfqHsLTw+9un2dJnkPT2+4tNxkM0t7Nu0ld1fG0+StAamN9oSswJAdfCLfs40T1gpL8ErUmjRsFyjifv
/9Cuu30yZaQFFTKsW0G4n68DobiT057L7YNIw1jRnONKBps= root@Deb-Ansible
root@debianfct:/home/debian# systemctl status ssh
bash: systemctl: command not found
root@debianfct:/home/debian# systemctl status ssh
* ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Tue 2024-05-14 13:45:30 CEST; 4min 21s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
  Process: 4032 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
 Main PID: 4033 (sshd)
    Tasks: 1 (limit: 2255)
   Memory: 1.4M
      CPU: 21ms
   CGroup: /system.slice/ssh.service
           └─4033 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

may 14 13:45:30 debianfct systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
may 14 13:45:30 debianfct sshd[4033]: Server listening on 0.0.0.0 port 22.
may 14 13:45:30 debianfct sshd[4033]: Server listening on :: port 22.
may 14 13:45:30 debianfct systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@debianfct:/home/debian# cat /etc/passwd | grep "ansible"
ansible:x:1001:1001::/home/ansible:/bin/sh
root@debianfct:/home/debian#

```

Ahora probamos a mandar ping desde el servidor ansible y ejecutar un comando

```

proxmox - Proxmox Console - Opera
No es seguro localhost:8006

root@Deb-Ansible:~# ansible -m ping debian-fct -v
Using /etc/ansible/ansible.cfg as config file
debian-fct | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
root@Deb-Ansible:~# ansible debian-fct -a "whoami"
debian-fct | CHANGED | rc=0 >>
ansible
root@Deb-Ansible:~#

```

Playbooks

Un playbook de Ansible es un archivo **YAML** (yet another mark language) que contiene una serie de tareas y configuraciones que se aplican a uno o más servidores remotos. Estas

tareas pueden incluir desde la instalación de paquetes y la configuración de servicios, hasta la gestión de archivos y usuarios.

Los playbooks son el componente principal de Ansible y permiten automatizar de forma declarativa las tareas de administración de sistemas.

Ejemplo de un playbook de ansible:

Checknano.yaml

He creado este archivo yalm par ansible que se va a encargar de ver en las máquinas de [prueba] que el nano (herramienta de UNIX para escribir en archivos) está instalado, si no está instalado lo instala

- become: yes → “become” indica que vamos a usar permisos de superuser(sudo)
- task:

```
---
- name: Verificar e instalar nano
  hosts: prueba
  become: yes

  tasks:
    - name: Verificar si nano está instalado
      command: "which nano"
      register: nano_check
      ignore_errors: yes

    - name: Instalar nano si no está instalado
      package:
        name: nano
        state: present
      when: nano_check.rc != 0
```

Para ejecutar un playbook podemos usar el siguiente comando:
ansible-playbook (nombre del yalm)

La ejecución del yalm anterior se ve tal que así:

```

admin@Centralita05: ~/yaml-ansible

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host 10.0.5.5 is using the discovered Python interpreter
at /usr/bin/python, but future installation of another Python interpreter could
change this. See
https://docs.ansible.com/ansible/2.9/reference\_appendices/interpreter\_discovery.html
for more information.
ok: [10.0.5.5]
[WARNING]: Platform linux on host 192.168.10.3 is using the discovered Python
interpreter at /usr/bin/python3, but future installation of another Python
interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference\_appendices/interpreter\_discovery.html
for more information.
ok: [192.168.10.3]

TASK [Verificar si nano está instalado] *****
changed: [10.0.5.5]
changed: [192.168.10.3]

TASK [Instalar nano si no está instalado] *****
skipping: [10.0.5.5]
skipping: [192.168.10.3]

PLAY RECAP *****
10.0.5.5 : ok=2    changed=1    unreachable=0    failed=0    skipped
=1    rescued=0    ignored=0
192.168.10.3 : ok=2    changed=1    unreachable=0    failed=0    skipped
=1    rescued=0    ignored=0

[admin@Centralita05 yaml-ansible]$

```

En este caso nos devuelve un estado ok ya que nano ya está instalado, vamos a probar a desinstalar nano en uno de los equipos

```
sudo apt remove nano
```

Ahora ejecutamos de nuevo el playbook

```
File Machine View Input Devices Help
admin@Centralita05:~/yaml-ansible

change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html
for more information.
ok: [10.0.5.5]
[WARNING]: Platform linux on host 192.168.10.3 is using the discovered Python
interpreter at /usr/bin/python3, but future installation of another Python
interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html
for more information.
ok: [192.168.10.3]

TASK [Verificar si nano está instalado] *****
fatal: [10.0.5.5]: FAILED! => {"changed": true, "cmd": ["which", "nano"], "delta": "0
00:00.007327", "end": "2024-05-07 07:16:01.623233", "msg": "non-zero return code", "r
": 1, "start": "2024-05-07 07:16:01.615906", "stderr": "", "stderr_lines": [], "stdou
": "", "stdout_lines": []}
...ignoring
changed: [192.168.10.3]

TASK [Instalar nano si no está instalado] *****
skipping: [192.168.10.3]
changed: [10.0.5.5]

PLAY RECAP *****
10.0.5.5 : ok=3 changed=2 unreachable=0 failed=0 skippe
=0 rescued=0 ignored=1
192.168.10.3 : ok=2 changed=1 unreachable=0 failed=0 skippe
=1 rescued=0 ignored=0

[admin@Centralita05 yaml-ansible]$
```

Podemos observar que la primera task dió error en la primera task “which nano” ya que lo desinstalé en la MV 10.0.5.5
Cómo dió error la primera task el script de yaml pasa a la segunda task que es instalar nano

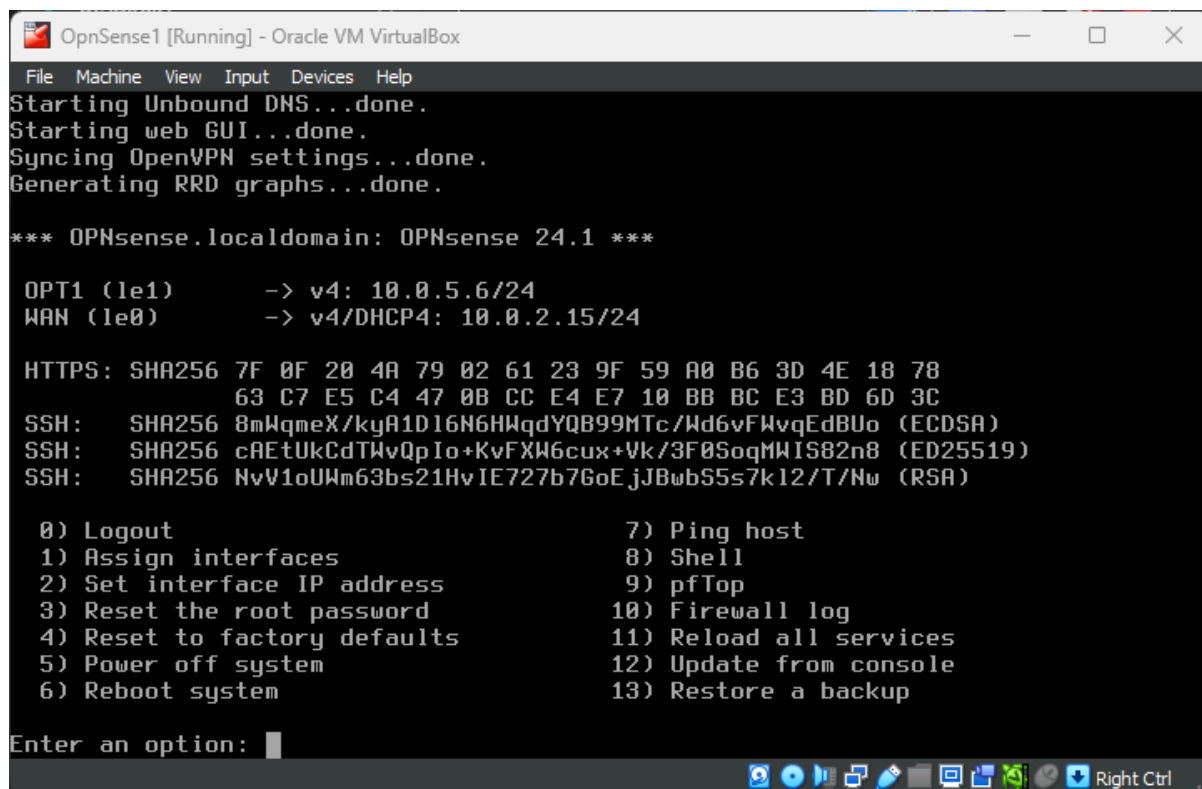
OpnSense

Opsense va a ser nuestro sistema operativo para el router principal.
decido utilizar OPNsense como solución para enrutar ya que me baso en una serie de consideraciones clave que destacan las ventajas y beneficios que OPNsense puede aportar a la infraestructura de red y seguridad.

OPNsense es un sistema operativo de firewall y enrutador basado en FreeBSD, que ofrece una solución completa de seguridad de red y administración de políticas (firewall de próxima generación). Una de las principales razones por las que he optado por OPNsense es su capacidad para proporcionar una seguridad avanzada mediante una amplia gama de funciones de seguridad, como filtrado de paquetes, prevención de intrusiones (IPS/IDS) y VPN. Esto nos permitirá proteger la red empresarial contra amenazas externas e internas de manera efectiva y confiable.

Además, la interfaz gráfica intuitiva de OPNsense facilita la configuración y administración de los routers de la infraestructura sin la necesidad de conocimientos avanzados en línea de comandos. Esto nos permitirá configurar reglas de firewall, políticas de seguridad y VPN desde la GUI

Una vez realizada la instalación encontramos una terminal con diferentes opciones, podemos realizar pings para hacer pruebas de conectividad o iniciar una shell de comandos para realizar configuración en el equipo. De todas formas el OS inicia un servidor web https en el momento de la instalación que puede ser accedida desde cualquier red que tenga conexión con el router.



```
OpnSense1 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Starting Unbound DNS...done.
Starting web GUI...done.
Syncing OpenVPN settings...done.
Generating RRD graphs...done.

*** OPNsense.localdomain: OPNsense 24.1 ***

OPT1 (le1)      -> v4: 10.0.5.6/24
WAN (le0)      -> v4/DHCP4: 10.0.2.15/24

HTTPS: SHA256 7F 0F 20 4A 79 02 61 23 9F 59 A0 B6 3D 4E 18 78
        63 C7 E5 C4 47 0B CC E4 E7 10 BB BC E3 BD 6D 3C
SSH:   SHA256 8mWqmeX/kyA1D16N6HWqdYQB99MTc/Wd6vFWvqEdBUo (ECDSA)
SSH:   SHA256 cAEtUkCdTWvQpIo+KvFXW6cux+Vvk/3F0SoqMWIS82n8 (ED25519)
SSH:   SHA256 NvV1oUWm63bs21HvIE727b7GoE.jJBwbS5s7k12/T/Nw (RSA)

0) Logout
1) Assign interfaces
2) Set interface IP address
3) Reset the root password
4) Reset to factory defaults
5) Power off system
6) Reboot system
7) Ping host
8) Shell
9) pfTop
10) Firewall log
11) Reload all services
12) Update from console
13) Restore a backup

Enter an option: 
```

El protocolo a usar para entrar en la interfaz web es https (puerto 443) . Accedemos a ella desde cualquier equipo para empezar con la configuración

Desde la opción 1) de asignar interfaces le asigno las siguientes 3

WAN -> 10.0.2.0/24
OPT1 (fct1 red linux) -> 192.168.10.0/24
OPT2 (fct2 red Windows -> 192.168.20.0./24

Con la opción 2 le asigno una IP al rúter en las 3 redes respectivamente.

Reglas de firewall.

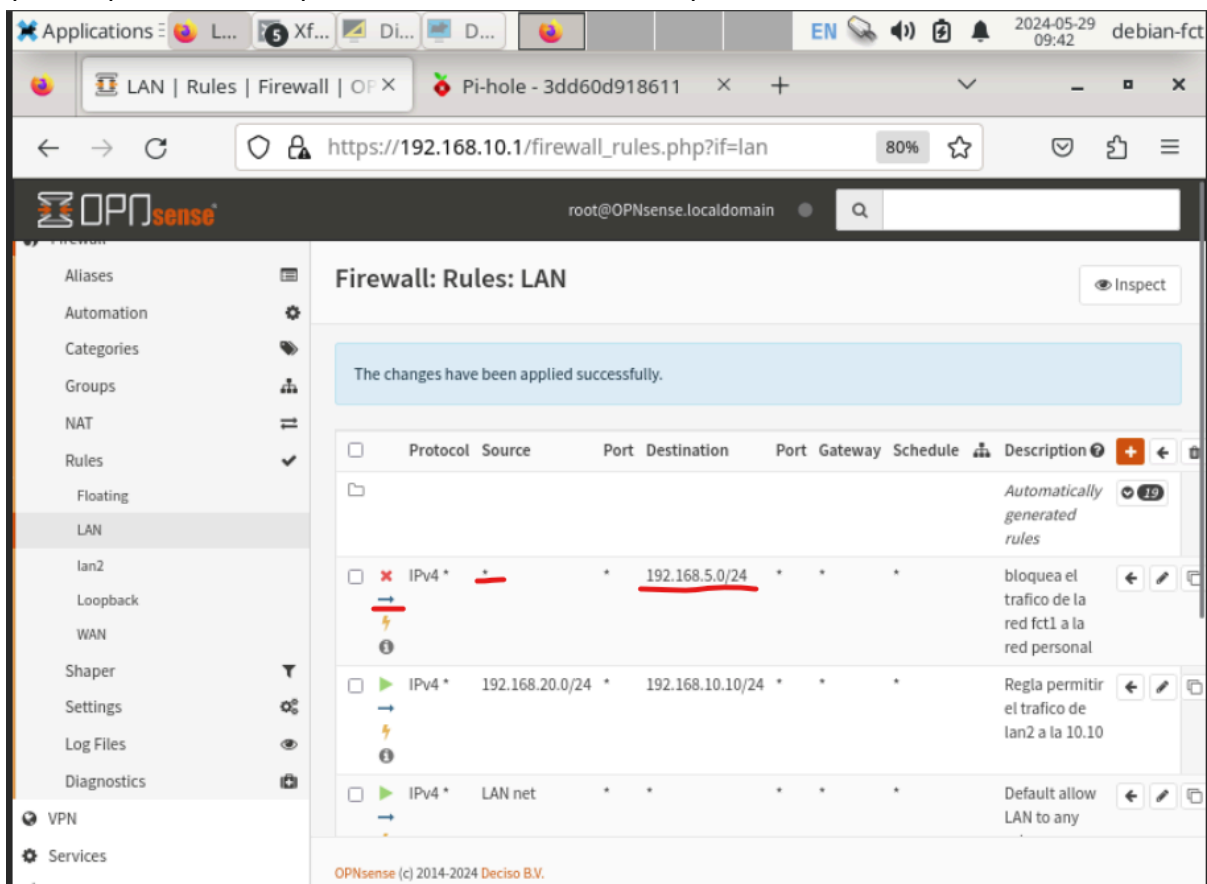
Primero me interesa que OPNSense no permita el tráfico entre las redes internas a mi red personal de casa. Para ello, vamos a crear una regla de firewall que impida que el tráfico que se genera desde la red **fct1(192.168.10.0/24)** pueda llegar a mi red personal . la **(192.168.5.0/24)**

```
Terminal - debian@debianfct: ~
File Edit View Terminal Tabs Help

debian@debianfct:~$ ip a | grep inet
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host noprefixroute
    inet 192.168.10.14/24 brd 192.168.10.255 scope global dynamic noprefixroute
ens18
    inet6 fe80::b9fd:c9d3:f1c0:ae55/64 scope link noprefixroute
debian@debianfct:~$ ping 192.168.5.1
PING 192.168.5.1 (192.168.5.1) 56(84) bytes of data.
64 bytes from 192.168.5.1: icmp_seq=1 ttl=62 time=1.06 ms
64 bytes from 192.168.5.1: icmp_seq=2 ttl=62 time=1.42 ms
^C
--- 192.168.5.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.056/1.237/1.418/0.181 ms
debian@debianfct:~$
```

Para ello nos dirigimos a la configuración de **OPNSense > Firewall > Rules > LAN**

Una vez estemos en la configuración dentro del entorno gráfico creamos una nueva regla que bloquee el tráfico que sale desde la fct1 a la red personal



En la regla ponemos que el bloquee el tráfico saliente (->), desde cualquier dispositivo de la red **fct1 (*)** a la **192.168.5.0/24**

Una vez aplicada la regla podemos probar a lanzar una traza icmp a cualquier dispositivo de la red 192.168.5.0/24

```
PING 192.168.5.1 (192.168.5.1) 56(84) bytes of data.  
^C  
--- 192.168.5.1 ping statistics ---  
2 packets transmitted, 0 received, 100% packet loss, time 1030ms  
  
debian@debianfct:~$ traceroute google.es  
traceroute to google.es (142.250.178.163), 30 hops max, 60 byte packets  
 1 OPNsense.localdomain (192.168.10.1)  0.265 ms  0.291 ms  0.342 ms  
 2 10.0.2.2 (10.0.2.2)  1.203 ms  1.267 ms  1.286 ms  
 3 192.168.5.1 (192.168.5.1)  1.579 ms  1.741 ms  1.823 ms  
 4 213.177.208.1 (213.177.208.1)  16.263 ms  15.956 ms  16.014 ms  
 5 10.14.8.177 (10.14.8.177)  16.138 ms  16.008 ms  15.834 ms  
 6 * * *  
 7 192.178.69.8 (192.178.69.8)  15.145 ms  15.202 ms  15.132 ms  
 8 192.178.110.73 (192.178.110.73)  14.686 ms  14.843 ms  192.178.110.87 (192.  
178.110.87)  16.796 ms  
 9 142.251.54.153 (142.251.54.153)  16.233 ms  15.940 ms  142.251.54.155 (142.  
251.54.155)  15.921 ms  
10 mad41s08-in-f3.1e100.net (142.250.178.163)  14.888 ms  15.250 ms  15.163 m  
s  
debian@debianfct:~$
```

Por curiosidad lanzo un traceroute a una ip de internet para ver cómo viaja el paquete por la red y si que observo que pasa por mi linea personal pero una vez pasado primero por el servidor de proxmox. No hay conexión directa con mi red personal desde la red interna.

Vamos a aplicar esta misma regla para las máquinas de la red fct2 (192.168.20.0/24) y la red WAN (10.0.2.0/24)

VPN

Vamos a configurar varias vpn en el router, entre ellas una para la red de windows y otra para la red de linux , para ello debemos irnos a la interfaz gráfica de OPNSENSE

QEMU (Debian-fct) - noVNC — Mozilla Firefox

https://192.168.5.254:8006/?console=kvm&novnc=1&vmid=203&vmname=Debian-fct&node=proxmox&

Applications | WireGuard | VP... | Xfce Terminal | EN | 2024-06-05 18:58 | debian-fct

← → ↻ | https://192.168.10.1/ui/wireguard/general#instances | 80% | ☆ | 🔒 | 📄 | ☰

OPNsense® root@OPNsense.localdomain 🔍

Lobby

Reporting

System

Interfaces

Firewall

VPN

IPsec

OpenVPN

WireGuard

Instances

Peers

Peer generator

Status

Log File

Services

Power

Help

VPN: WireGuard

Instances

Peers

Peer generator

🔍 Search

🔄 7 📄

☐ Enabled	▼ Name	Device	Tunnel A...	Port	Peers	Commands
No results found!						
⏪ ⏩ 1 ⏪ ⏩ Showing 0 to 0 of 0 entries						

🔔 Enable WireGuard

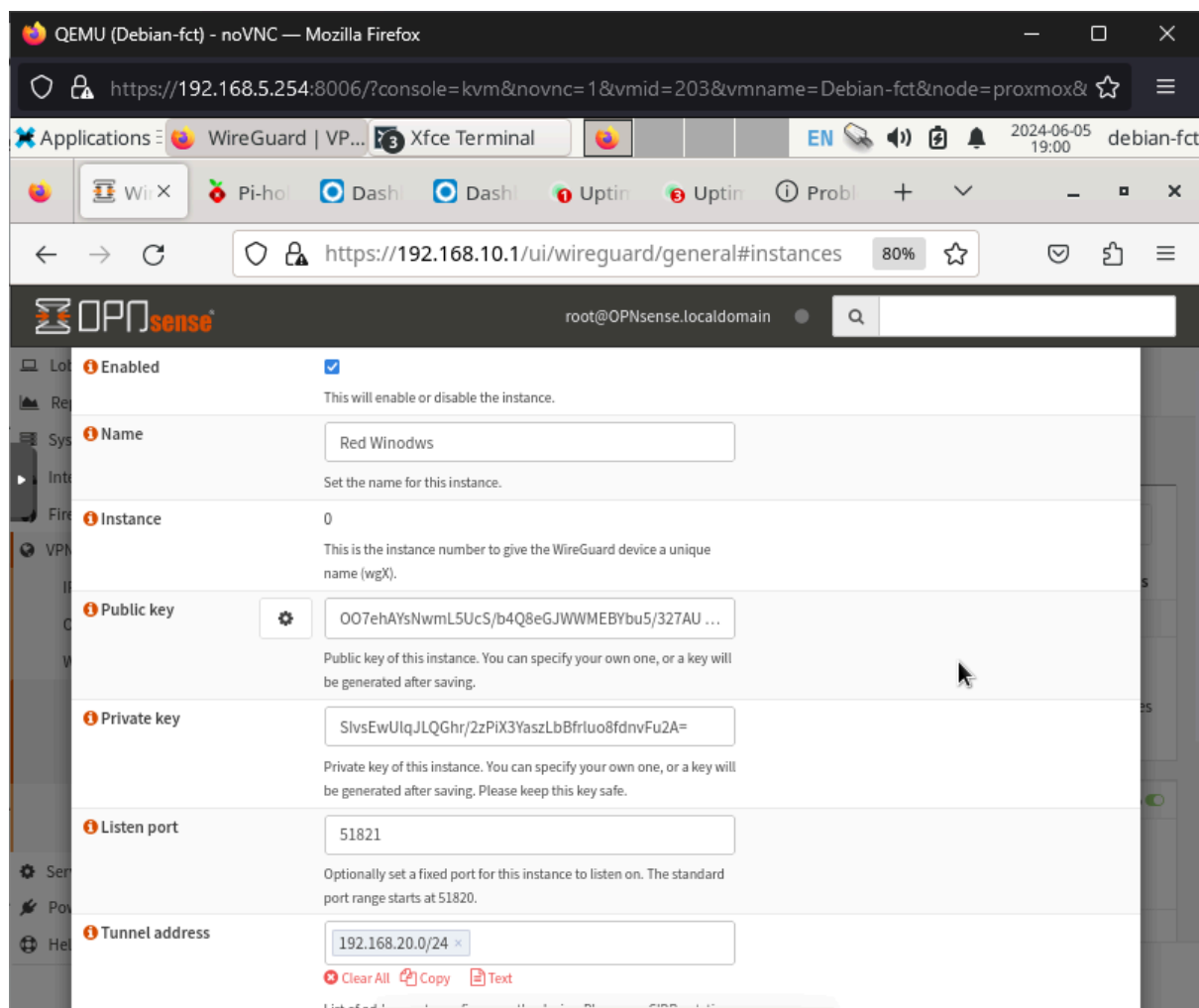
☒

This will activate WireGuard and start all enabled instances.

full help 🟢

OPNsense (c) 2014-2024 Deciso B.V.

Nos dirigimois a **VPN - Wireguard - Add**



Dentro de la configuración de la interfaz debemos indicarle el puerto de escucha donde wireguard estará escuchando así como el segmento de red donde queremos que se pueda acceder

Además debemos añadir el puerto de escucha que es el puerto que usará la VPN para comunicarse

Tenemos que agregar una regla de port forward a la máquina proxmox en opnsense para que se redirija el tráfico de la 10.0.2.2 que le llegue al puerto de wireguard y se redirija al router

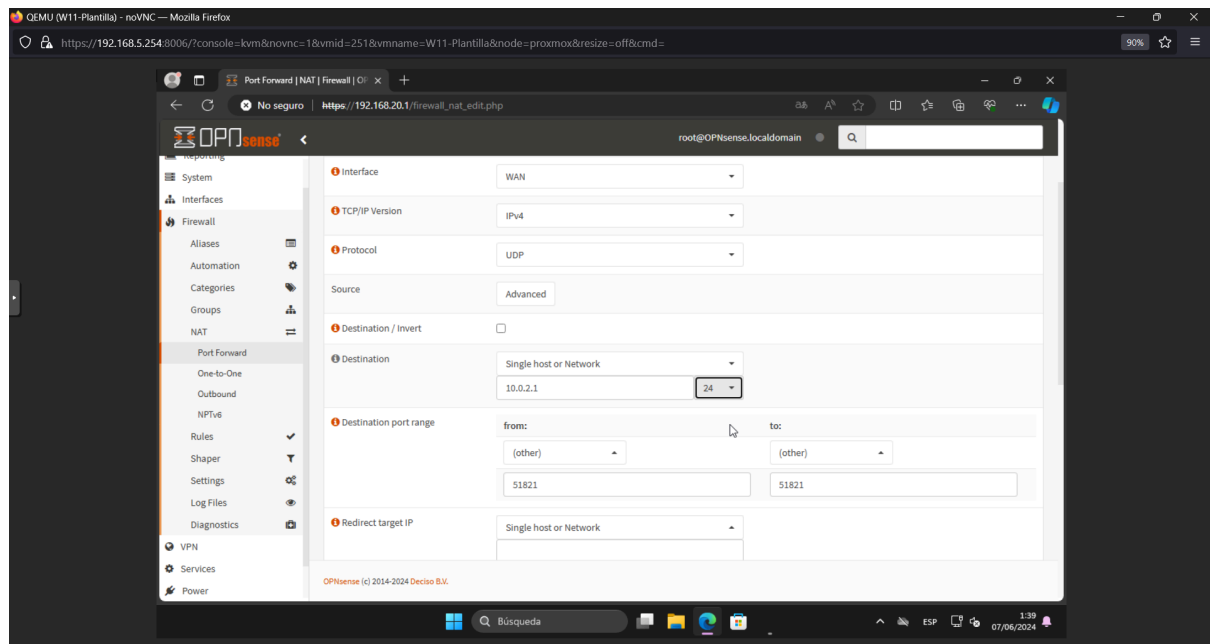
Cuando configuremos el peer(pareja) del cliente para que se conecte al servidor de wireguard vamos a necesitar un **endpoint** . El endpoint va a ser a donde envíe el cliente los paquetes para conectarse al servidor VPN (la ip de salida a internet de mi router) de esta forma, haremos un script que se ejecute con cron(tarea programada) para que esté constantemente actualizando con el servidor dns la dirección IP de salida a internet de mi router.

Configuramos la [Dns Dinámica](#)

Firewall > NAT > Port Forward.

Configuramos la regla

- **Interface:** WAN (la interfaz conectada a tu red externa/internet).
- **Protocol:** UDP (WireGuard utiliza UDP).
- **Destination:** 10.0.2.1.
- **Destination port range:** 51821
- **Redirect target IP:** 10.0.2.1 (Opnsense).
- **Redirect target port:** 51821



Para conectarnos a la VPN vamos también a necesitar redireccionar el tráfico del puerto de wireguard al router, la conexión entre la máquina proxmox y el router es dentro de la WAN (10.0.2.0/24) donde el router es la 10.0.2.1

Para ello debemos crear reglas de enrutamiento desde el servidor proxmox al router:
Modificamos /etc/network/interfaces

nuestra interfaz vmbr0 queda de esta manera

```
auto vmbr0
iface vmbr0 inet static
    bridge-ports enp42s0
    bridge-stp off
    bridge-fd 0
    address 192.168.5.254/24
    gateway 192.168.5.1
    # Reglas IPtables para reditigir tráfico de Wireguard al router
```

```

post-up iptables -t nat -A PREROUTING -i vmbr0 -p udp --dport 51821 -j DNAT
--to-destination 10.0.2.1:51821
post-up iptables -A FORWARD -p udp -d 10.0.2.1 --dport 51821 -m state --state
NEW,ESTABLISHED,RELATED -j ACCEPT
pre-down iptables -t nat -D PREROUTING -i vmbr0 -p udp --dport 51821 -j DNAT
--to-destination 10.0.2.1:51821
pre-down iptables -D FORWARD -p udp -d 10.0.2.1 --dport 51821 -m state --state
NEW,ESTABLISHED,RELATED -j ACCEPT

```

Con el comando iptables podemos listar las reglas de iptables , despues del reinicio comprobamos que se aplicaron y que son persistentes

```

root@proxmox:~# iptables -L --line-numbers
Chain INPUT (policy ACCEPT)
num  target      prot opt source                destination
1    ts-input    all  --  anywhere              anywhere

Chain FORWARD (policy ACCEPT)
num  target      prot opt source                destination
1    ts-forward  all  --  anywhere              anywhere
2    ACCEPT      udp  --  anywhere              10.0.2.1              udp dpt:51821 state NEW,RELATED,ESTABLISHED

```

Vamos a ver que el tráfico este llegando a la máquina proxmox , para ello podemos usar la herramienta tcpdump de captura de paquetes

Tengamos en cuenta que el tráfico va a seguir la siguiente ruta

Red externa (4g del movil por ejemplo) > DNS dinamica a la ip de salida de internet de mi ruter (192.168.5.1) > servidor proxmox 192.168.5.254:51821 > Router OPNSESNE (10.0.2.1:51821)

```

root@proxmox:~# tcpdump -i vmbr0 udp port 51821
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on vmbr0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:31:25.592467 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:31:30.746494 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:31:35.792986 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:31:40.786694 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:31:45.786737 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:31:50.800268 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:31:55.986217 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:32:01.240310 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:32:06.387091 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:32:11.602286 IP 16.red-88-29-107.staticip.rima-tde.net.9837 > proxmox.fct.51821: UDP, length 148
10:32:26.146924 IP 16.red-88-29-107.staticip.rima-tde.net.9752 > proxmox.fct.51821: UDP, length 148
10:32:31.163258 IP 16.red-88-29-107.staticip.rima-tde.net.9752 > proxmox.fct.51821: UDP, length 148
10:32:36.282795 IP 16.red-88-29-107.staticip.rima-tde.net.9752 > proxmox.fct.51821: UDP, length 148
10:32:41.307050 IP 16.red-88-29-107.staticip.rima-tde.net.9752 > proxmox.fct.51821: UDP, length 148
10:32:46.283115 IP 16.red-88-29-107.staticip.rima-tde.net.9752 > proxmox.fct.51821: UDP, length 148
10:32:51.394060 IP 16.red-88-29-107.staticip.rima-tde.net.9752 > proxmox.fct.51821: UDP, length 148
10:32:56.553962 IP 16.red-88-29-107.staticip.rima-tde.net.9752 > proxmox.fct.51821: UDP, length 148

```

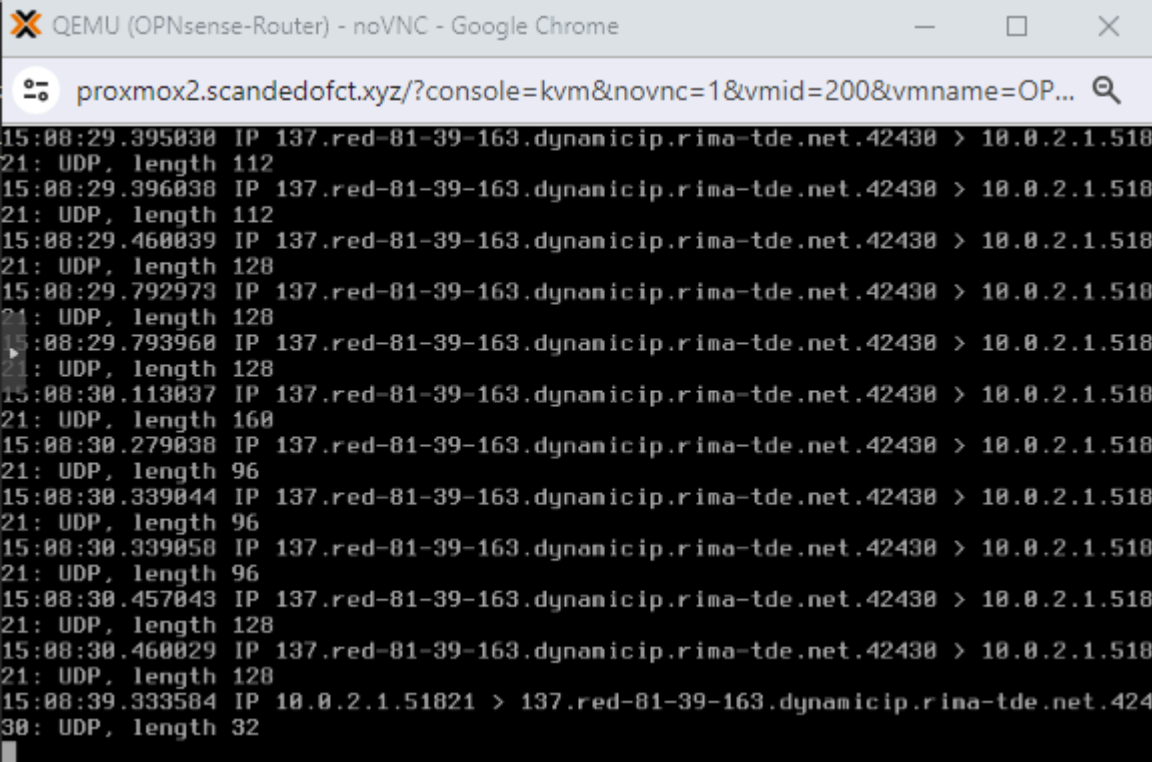
Vemos que están llegando peticiones una vez me conecto desde mi movil a la vpn, con -i especificamos la interfaz y luego el protocolo y el puerto

Vamos a crear una regla en el endpoint de snort que capture paquetes udp dirigidos a ese puerto en la red interna

para ello modificamos el /etc/snort/rules/local.rules

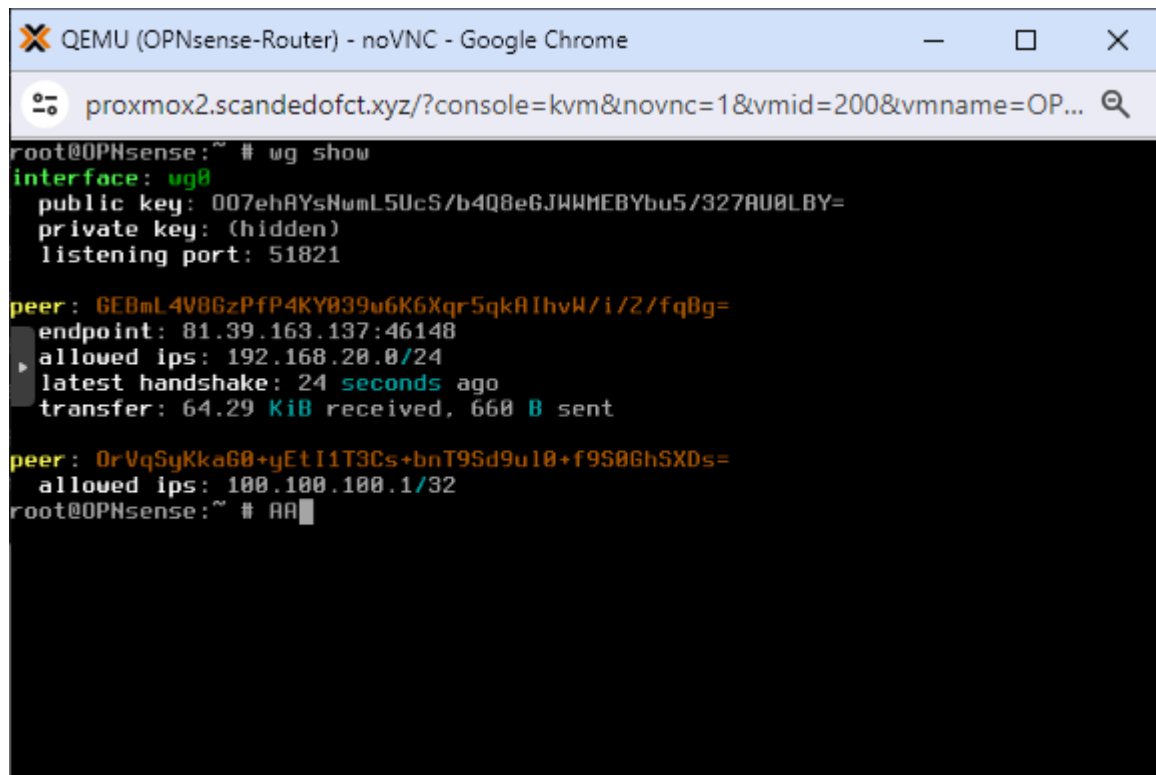
```
alert udp any any -> any 51821 (msg:"UDP al puerto 51821 detectado"; sid:1000001; rev:1;)
```

también podemos usar tcpdump en el router y ver si llegan los paquetes UDP
tcpdump -i em0 udp port 51821



```
15:08:29.395030 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 112
15:08:29.396038 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 112
15:08:29.460039 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 128
15:08:29.792973 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 128
15:08:29.793960 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 128
15:08:30.113037 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 160
15:08:30.279038 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 96
15:08:30.339044 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 96
15:08:30.339058 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 96
15:08:30.457043 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 128
15:08:30.460029 IP 137.red-81-39-163.dynanicip.rima-tde.net.42430 > 10.0.2.1.518
21: UDP, length 128
15:08:39.333584 IP 10.0.2.1.51821 > 137.red-81-39-163.dynanicip.rima-tde.net.424
30: UDP, length 32
```

con **wg show** podemos ver si nuestro peer está conectado



The screenshot shows a terminal window titled "QEMU (OPNsense-Router) - noVNC - Google Chrome". The address bar shows the URL "proxmox2.scandedofct.xyz/?console=kvm&novnc=1&vmid=200&vmname=OP...". The terminal output shows the command "wg show" being executed, displaying the configuration for the "wg0" interface and two peers. The first peer is connected, showing its endpoint, allowed IPs, and handshake time.

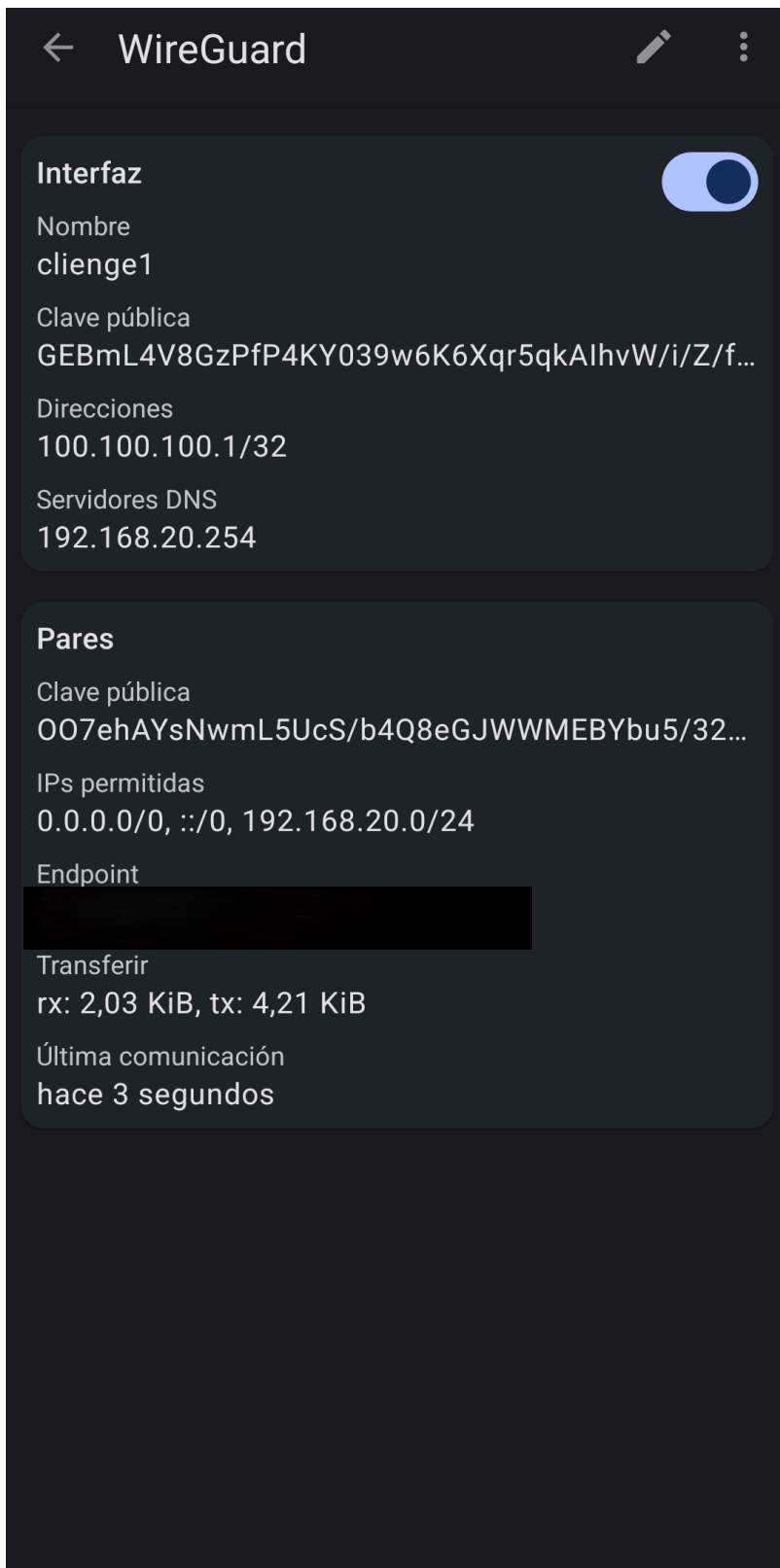
```
root@OPNsense:~ # wg show
interface: wg0
  public key: 007ehAYsNwmL5UcS/b4Q8e6JWWMEBYbu5/327AU0LBY=
  private key: (hidden)
  listening port: 51821

peer: GEBmL4V8GzPfP4KY039u6K6Xqr5qkAIhvW/i/2/fqBg=
  endpoint: 81.39.163.137:46148
  allowed ips: 192.168.20.0/24
  latest handshake: 24 seconds ago
  transfer: 64.29 KiB received, 660 B sent

peer: 0rVqSyKka60+yEtI1T3Cs+bnT9Sd9u10+f9S0GhSXDs=
  allowed ips: 100.100.100.1/32
root@OPNsense:~ #
```

Podemos ver que nuestro primer peer está conectado

El servidor DNS del cliente de wireguard será nuestro windows server para que nos resuelva por nombre las ips de las máquinas windows.

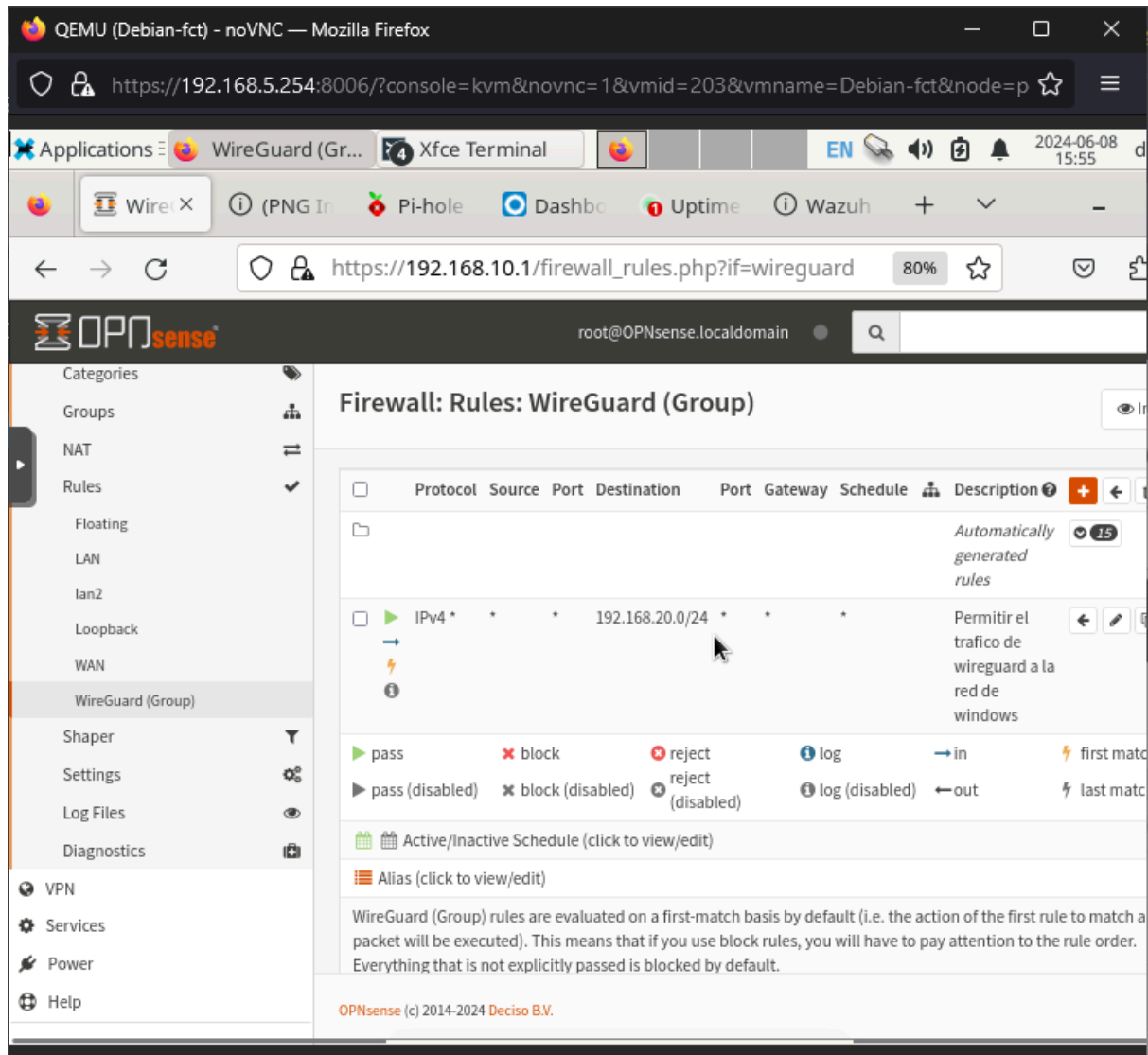


Una vez la conexión está establecida es momento de crear las reglas de firewall que permitan que la red de wireguard tenga conexión con la red de Windows(192.168.20.0/24)

Firewall > Rules > WireGuard

Action: Pass

Interface: WireGuard
Address Family: IPv4
Protocol: Any
Source: WireGuard subnet
Destination: 192.168.20.0/24



Para comprobar la conectividad lanzo paquetes desde el móvil donde tengo configurado el cliente wireguard con el Peer de wireguard que creamos con anterioridad

Realizo las siguientes pruebas:

- Ping a la dns de google 8.8.8.8 (comprobar salida a internet)
 - **No debe tener salida a internet ya que la conexión vpn sólo nos interesa para conectarnos a las máquinas windows por escritorio remoto y nada más**
- Ping al router (192.168.20.1)
- Ping al servidor Windows (192.168.20.254)

```

Welcome to Termux

Docs:      https://termux.dev/docs
Community: https://termux.dev/community

Working with packages:

- Search:  pkg search <query>
- Install: pkg install <package>
- Upgrade: pkg upgrade

Report issues at https://termux.dev/issues
- $ ping 192.168.20.244
- $ ping 192.168.20.254
PING 192.168.20.254 (192.168.20.254) 56(84) bytes of data.
64 bytes from 192.168.20.254: icmp_seq=1 ttl=127 time=72.4 ms
S
64 bytes from 192.168.20.254: icmp_seq=2 ttl=127 time=70.5 ms
S
64 bytes from 192.168.20.254: icmp_seq=3 ttl=127 time=69.4 ms
S
64 bytes from 192.168.20.254: icmp_seq=4 ttl=127 time=172 ms
64 bytes from 192.168.20.254: icmp_seq=5 ttl=127 time=57.4 ms
S
64 bytes from 192.168.20.254: icmp_seq=6 ttl=127 time=90.3 ms
S
^C
--- 192.168.20.254 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010
ms
rtt min/avg/max/mdev = 57.434/88.747/172.276/38.578 ms
- $ ping 192.168.20.1
PING 192.168.20.1 (192.168.20.1) 56(84) bytes of data.
64 bytes from 192.168.20.1: icmp_seq=1 ttl=64 time=150 ms
64 bytes from 192.168.20.1: icmp_seq=2 ttl=64 time=149 ms
^C
--- 192.168.20.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002
ms
rtt min/avg/max/mdev = 149.539/150.237/150.936/0.798 ms
- $ █

```

Instalación de Wireguard Server desde CLI

Por el mero hecho de enseñarlo voy a instalar un wireguard server en una máquina debian
 Para la instalación de wireguard desde la cli voy a seguir la siguiente [documentación](#) para la
 instalación por linea de comandos

La guía está orientada a instalarlo en una raspberry pi pero no debería de darnos problema
 seguirla orientado a una máquina debian

```
sudo apt-get install wireguard wireguard-tools
```

Generación de las claves

Para que wireguard funcione vamos a necesitar una clave pública y una clave privada, estas claves son las que wireguard va a intercambiar con el cliente para establecer la comunicación

```
wg genkey | tee server.key | wg pubkey > server.pub
```

Creamos el archivo de configuración que va a ser también el nombre de la interfaz de red que se levante, para este caso usamos wg0.conf

```
sudo nano /etc/wireguard/wg0.conf
```

El archivo para levantar la interfaz debe de verse así

```
GNU nano 5.4
[Interface]
Address = 10.100.0.1/24, fd08:4711::1/64
ListenPort = 47111
PrivateKey = OL4gzNLDCHAsn7mEEo/2g5tD52skZ5C/qcuo
```

En Address podemos poner el rango de ips que queramos, son las ips que van a asignarse a la interfaz de wireguard.

En ListenPort debemos especificar un puerto de escucha por donde wireguard se va a comunicar por UDP

Ponemos la Privatekey que generamos previamente en el archivo.

Ya podemos levantar la interfaz

```
sudo systemctl enable wg-quick@wg0.service
sudo systemctl daemon-reload
sudo systemctl start wg-quick@wg0
```

Con el comando wg podemos verificar si está funcionando la interfaz, con ip a debería aparecernos la interfaz wg0 con el rango de ips que le especificamos en el .conf

```
proxmox - Proxmox Console - Opera
proxmox.scandedofct.xyz
root@Debian-Plantilla-VPN:/etc/wireguard# wg
interface: wg0
  public key: EGYuxc33QlSPsov7ei0kkgAho1//LbH9rf5+t+DxWW8=
  private key: (hidden)
  listening port: 47111
root@Debian-Plantilla-VPN:/etc/wireguard# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if31: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:87:8b:dc brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 20.20.20.2/24 brd 20.20.20.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe87:8bdc/64 scope link
        valid_lft forever preferred_lft forever
3: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN group default qlen 1000
    link/none
    inet 10.100.0.1/24 scope global wg0
        valid_lft forever preferred_lft forever
    inet6 fd08:4711::1/64 scope global
        valid_lft forever preferred_lft forever
root@Debian-Plantilla-VPN:/etc/wireguard#
```

Clientes (peers)

Para configurar los clientes vamos a tener que especificarlos en el archivo .conf de nuestro /etc/wireguard y además configurar un archivo .conf para enviar al cliente

Genero las llaves publica y privada para mi nuevo cliente con nombre movil:

```
root@Debian-Plantilla-VPN:/etc/wireguard# name="movil"
root@Debian-Plantilla-VPN:/etc/wireguard# wg genkey | tee "${name}.key" | wg pubkey >
"${name}.pub"
```

Añadimos al archivo de configuración el apartado [peer] con la clave publica del cliente y el rango de ips que se van a permitir:

Ahora nuestro wg.conf se ve así:

```
proxmox.scandedofct.xyz
root@Debian-Plantilla-VPN:/etc/wireguard# wg
interface: wg0
  public key: EGYuxc33QlSPsov7ei0kkgAho1//LbH9rf5+t+DxWW8=
  private key: (hidden)
  listening port: 47111

peer: CQB30iHBd8wA0KmHi7UcJZCz2VD38plXRulYev0w2yU=
  allowed ips: 10.100.0.2/32, fd08:4711::2/128
root@Debian-Plantilla-VPN:/etc/wireguard#
```

[Interface]

```
Address = 10.100.0.1/24, fd08:4711::1/64
ListenPort = 47111
PrivateKey = 0L4gzNLDCHAsn7mEEo/2g5tD52skZ5C/qcucFXBAM0A=
[Peer]
PublicKey = CQB30iHBd8wA0KmHi7UcJZCz2VD38pIXRuIYev0w2yU=
AllowedIPs = 10.100.0.2/32, fd08:4711::2/128
```

Vamos a crear el archivo .conf que debe tener el cliente e importarlo a wireguard:

Para crear nuestro archivo de configuración de cliente debe de tener

- Endpoint: Una DNS dinámica que apunta a mi rúter
- Publickey La llave publica del servidor, que verifica que se esté comunicando con el servidor correcto.
- Allowed Ips rango de ips permitidas

movil.conf:

```
[Interface]
Address = 10.100.0.2/32, fd08:4711::2/128
DNS = 8.8.8.8
PrivateKey = oCaoBVjDxVA/gcmKgh0QLj6ELoGul6DO9Kh0H9oCanY=
[Peer]
Endpoint = scandedofct.duckdns.org:47111
PublicKey = EGYuxc33QISPsov7ei0kggAho1//LbH9rf5+t+DxWW8=
AllowedIPs = 10.100.0.2.32/32
```

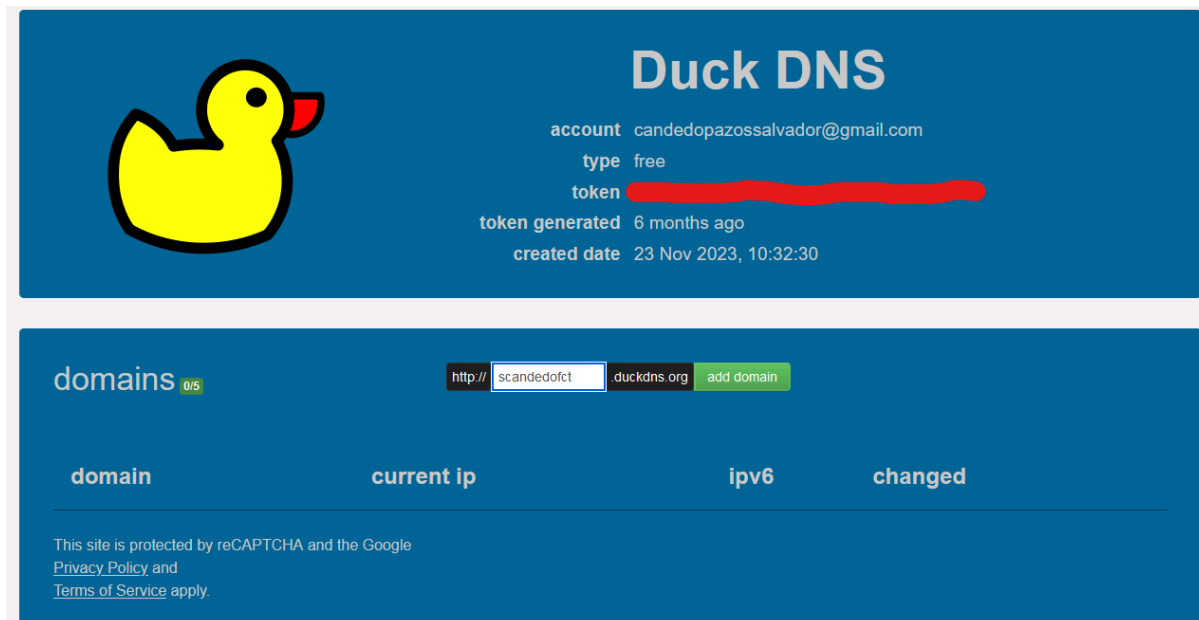
Podemos generar un código qr para escanearlo con una cámara e importarlo en el teléfono directamente:

```
qrencode -t ansiutf8 < "/etc/wireguard/${name}.conf"
```

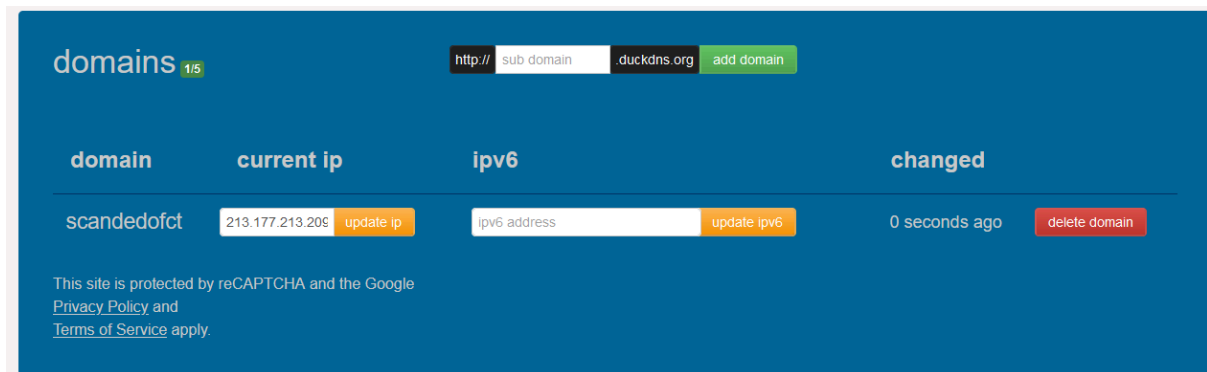


DNS dinámica

Para configurar la Dns dinámica, vamos a crear un subdominio en [duckdns](https://duckdns.org/) , que es un servicio de nombres de dominio totalmente gratuito que nos permite crear dns dinámicas de forma sencilla.



Una vez añadido el subdominio Duckdns nos coge directamente la ip de salida a internet de nuestro router



La siguiente configuración que vamos a hacer se debe a que no tenemos una **Dirección IP fija** en nuestra casa por lo que cuando nuestro ISP(proveedor de internet) cambie la dirección ip de salida de nuestro el subdominio que acabamos de crear en duckdns se quedará apuntando a la dirección ip vieja y nos quedaremos sin manera de comunicarnos con nuestro rúter.

Para ello debemos crear un script que mande una petición curl a duckdns haciendo uso de nuestro token (para autenticarnos de forma segura en el sitio) y actualizando la ip de salida del rúter para que siempre esté apuntando a la dirección correcta

Primero creamos la carpeta y el script

```
mkdir ~/duckdns
```

```
cd duckdns && nano duck.sh
```

El script es el siguiente

```
echo url="https://www.duckdns.org/update?domains=exampledomain&token='mi token'&ip=" | curl -k -o ~/duckdns/duck.log -K -
```

Vamos a analizar el script:

- **echo url=""**

Esta parte del comando utiliza echo para generar una cadena de texto. La cadena es una URL para actualizar el dominio exampledomain en DuckDNS.

El token es una clave única que autentica la solicitud de actualización.

El parámetro ip está vacío en esta URL, lo que indica que DuckDNS debe actualizar el dominio con la dirección IP desde la que se realiza la solicitud.

- | (pipe)

Pasa el output del comando al siguiente que sería el curl

- **curl -k -o ~/duckdns/duck.log -K -**

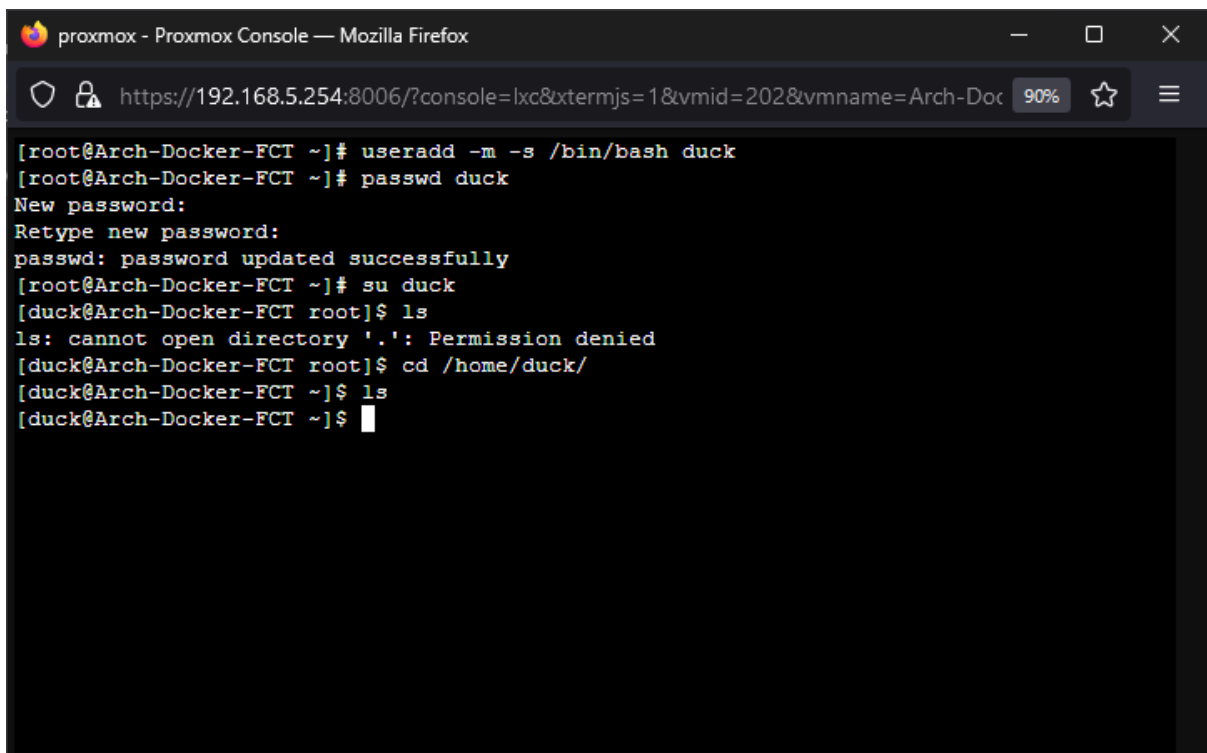
curl es una herramienta de línea de comandos para transferir datos a un servidor, lo usamos previamente para sacar por línea de comandos el contenido de un servidor http

-k indica a curl que no verifique los certificados SSL. Esto es útil si el certificado del servidor no es de confianza o es autofirmado.

-o ~/duckdns/duck.log especifica que la salida de curl se debe guardar en el archivo duck.log en el directorio ~/duckdns.

-K - indica a curl que lea las opciones de un archivo de configuración. El guion (-) después de -K le dice a curl que lea las opciones de la entrada estándar, que en este caso es la salida del comando echo.

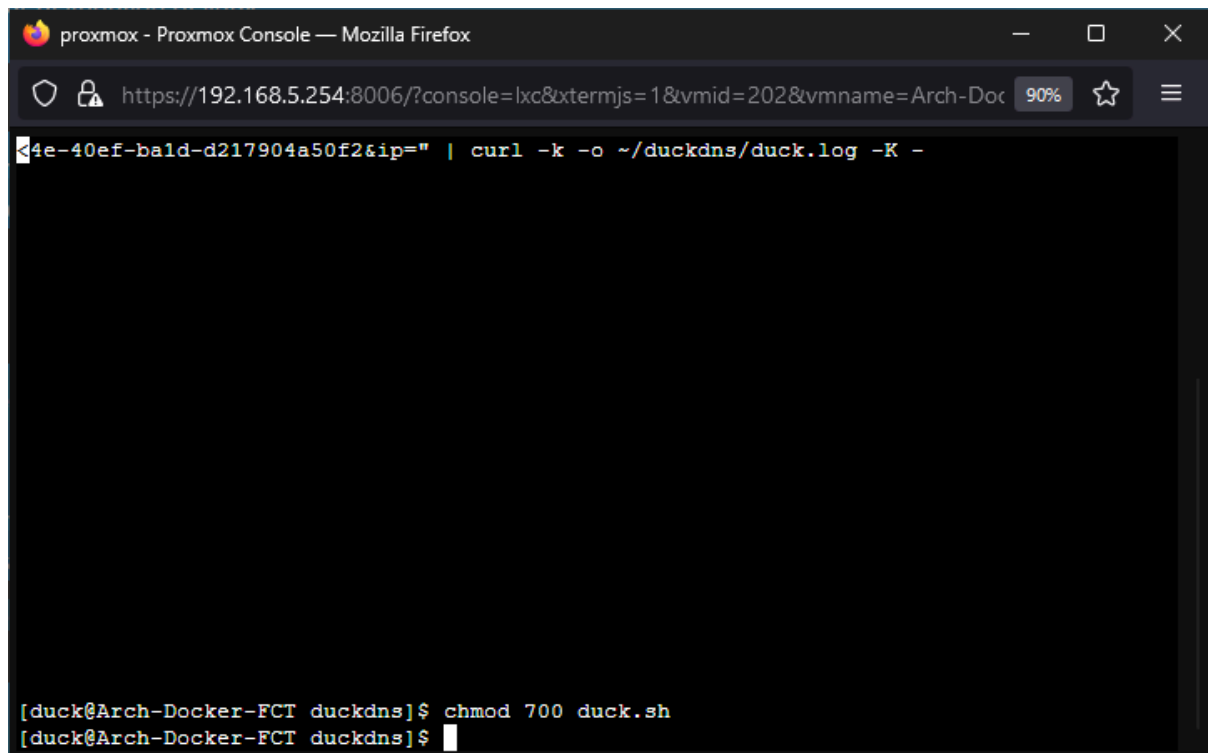
Primero creo un usuario duck para añadir mayor seguridad



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lxcs&xtermjs=1&vmid=202&vmname=Arch-Doc 90% ☆ ≡

[root@Arch-Docker-FCT ~]# useradd -m -s /bin/bash duck
[root@Arch-Docker-FCT ~]# passwd duck
New password:
Retype new password:
passwd: password updated successfully
[root@Arch-Docker-FCT ~]# su duck
[duck@Arch-Docker-FCT root]$ ls
ls: cannot open directory '.': Permission denied
[duck@Arch-Docker-FCT root]$ cd /home/duck/
[duck@Arch-Docker-FCT ~]$ ls
[duck@Arch-Docker-FCT ~]$
```

Creamos el script y le damos permisos

A screenshot of a Proxmox console window titled "proxmox - Proxmox Console — Mozilla Firefox". The address bar shows a URL: "https://192.168.5.254:8006/?console=lxcl&termjs=1&vmid=202&vmname=Arch-Doc". The terminal window has a dark background and shows the following commands and output:

```
4e-40ef-ba1d-d217904a50f2&ip=" | curl -k -o ~/duckdns/duck.log -K -  
  
[duck@Arch-Docker-FCT duckdns]$ chmod 700 duck.sh  
[duck@Arch-Docker-FCT duckdns]$
```

con crontab -e abrimos la lista de trabajos de cron para el usuario con el que estemos, en este caso duck

Dentro del editor de cron pondremos la siguiente tarea.

```
*/* * * * * ~/duckdns/duck.sh >/dev/null 2>&1
```

Esto ejecutará el script cada 5 min por lo que si nuestro isp cambia la ip de nuestro router duckdns actualizará de forma automática la ip pública de nuestro router

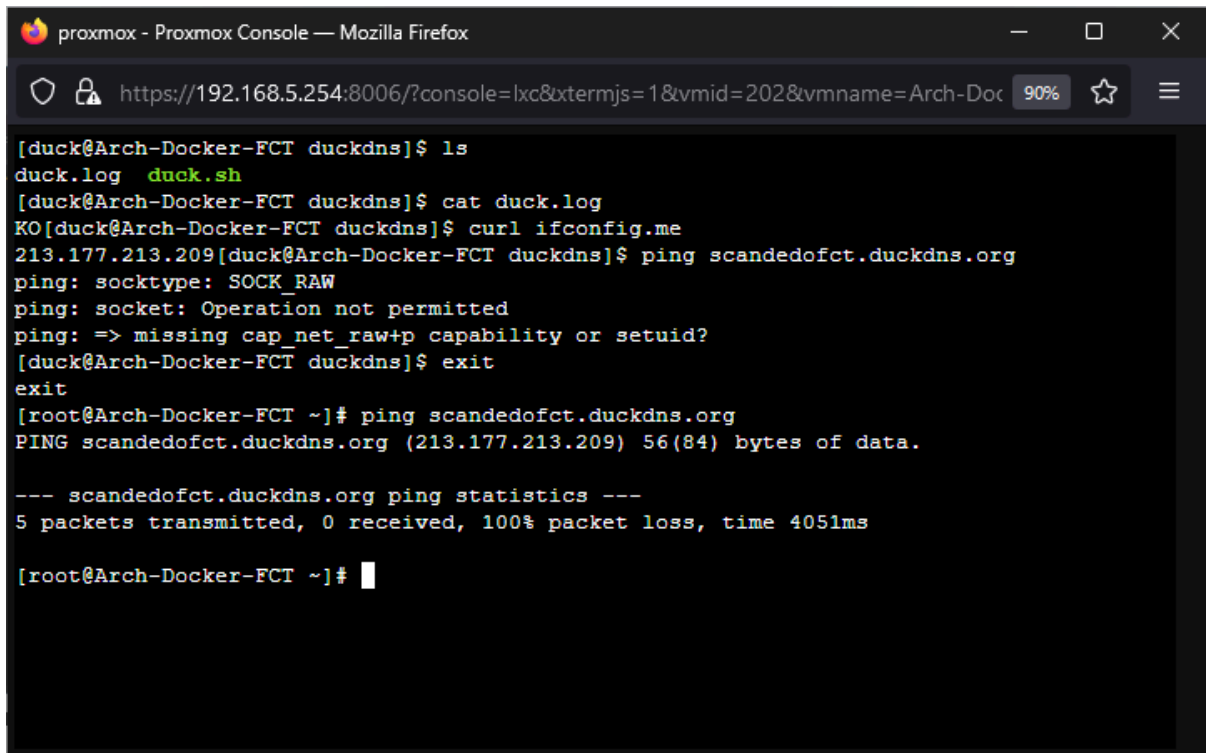
para ver que todo funciona realizo las siguientes pruebas.

Espero 5 minutos y reviso el log para ver si se ejecutó bien el curl del script.

Hago un ping con el usuario duck pero no tiene permisos más que para ejecutar el script y la tarea de cron

hago un ping con root al servidor dns y me resuelve la ip de mi router previamente sacada con un curl a ifconfig.me

Los pings fallan pero es dado que estamos en la red local de nuestro router y se provoca un loopback, además que mi router tiene deshabilitado resolver peticiones **icmp (ping)**



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lxcl&xtermjs=1&vmid=202&vmname=Arch-Doc 90% ☆ ≡

[duck@Arch-Docker-FCT duckdns]$ ls
duck.log  duck.sh
[duck@Arch-Docker-FCT duckdns]$ cat duck.log
KO[duck@Arch-Docker-FCT duckdns]$ curl ifconfig.me
213.177.213.209[duck@Arch-Docker-FCT duckdns]$ ping scandefoct.duckdns.org
ping: socktype: SOCK_RAW
ping: socket: Operation not permitted
ping: => missing cap_net_raw+p capability or setuid?
[duck@Arch-Docker-FCT duckdns]$ exit
exit
[root@Arch-Docker-FCT ~]# ping scandefoct.duckdns.org
PING scandefoct.duckdns.org (213.177.213.209) 56(84) bytes of data.

--- scandefoct.duckdns.org ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4051ms

[root@Arch-Docker-FCT ~]#
```

Docker

Docker es una plataforma de código abierto diseñada para simplificar el desarrollo, implementación y ejecución de aplicaciones en contenedores. Los contenedores Docker permiten empaquetar una aplicación junto con sus dependencias y configuraciones en una unidad estándar, lo que garantiza que la aplicación se ejecute de manera consistente en cualquier entorno, ya sea un entorno de desarrollo, prueba o producción.

Para el proyecto, Docker es crucial debido a varias razones:

- **Portabilidad:** Los contenedores Docker proporcionan un entorno aislado y portátil para ejecutar aplicaciones, nos garantiza que las aplicaciones se ejecuten de la misma manera en cualquier lugar donde se desplieguen, independientemente de las diferencias en la infraestructura, por ejemplo, podríamos ejecutar un servidor de base de datos mssql y otro servidor MongoDB de forma completamente aislada, cada servicio funcionando en su contenedor
- **Seguridad :** Los contenedores son sistemas operativos minimalistas, apenas tienen paquetes instalados más de los necesarios para hacer funcionar el servicio que contenga el container, en caso de que un cibercriminal lograra ejecución remota de código en el equipo se va a encontrar con que lo que puede hacer dentro del container es muy limitado, es por ello que la mayoría de nuestros servicios van a funcionar en docker (Bases de datos, Paneles de control, páginas web...)

Instalación

La instalación la vamos a realizar en un sistema **arch linux** ya que es un sistema mucho más ligero que la gran mayoría de distribuciones de linux , y queremos que consuma pocos recursos porque la optimización es realmente importante en este proyecto .

Arch linux usa un gestor de paquetes diferente al de debian por ejemplo. Así que vamos a instalar los paquetes con **Pacman** , instalarlo en una máquina con gestor de paquetes **apt** no varía demasiado.

Vamos a instalar Docker en una instancia **arch linux**

Primero actualizamos los paquetes del sistema

```
sudo pacman -Syu
```

Instalar Docker y Docker Compose:

```
sudo pacman -S docker docker-compose
```

Inicia el servicio Docker y habilita su inicio automático en el arranque:

```
sudo systemctl start docker  
sudo systemctl enable docker
```

```
[root@Arch-Docker-FCT ~]# systemctl start docker  
[root@Arch-Docker-FCT ~]# systemctl enable docker  
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.  
[root@Arch-Docker-FCT ~]# docker --version  
Docker version 26.1.2, build 211e74b240  
[root@Arch-Docker-FCT ~]#
```

Docker instalado!

Docker-compose

Docker Compose es una herramienta que permite definir y ejecutar Contenedores Docker desde un mismo archivo de configuración

Utiliza un archivo YAML para configurar los servicios de la aplicación, lo que facilita la definición de la estructura de la aplicación y sus dependencias.

En el o proyecto vamos a utilizar Docker Compose para crear y administrar contenedores que ejecuten servicios específicos, vamos a empezar con:

- **Pi-hole** : Pi-Hole nos va a servir de servidor DNS para toda la infraestructura , también actúa de servidor DHCP pero como ya configuramos el router para eso vamos a configurarlo como servidor DNS
- Nextcloud: Va a ser nuestro gestor de contenido para compartir archivos entre sistemas, además de tener un servidor web desde donde poder entrar con el navegador guardaremos los archivos en local dentro de nuestra máquina ARCH para hacer un backup y para poder compartir la carpeta por samba
- PHPMYAdmin: Next Cloud necesita una base de datos para funcionar, decido instalar un contenedor PHPMYAdmin como el que usamos en el módulo de aplicaciones web para administrar la base de datos

services:

watchtower:

```
image: containrrr/watchtower
volumes:
- /var/run/docker.sock:/var/run/docker.sock
```

pihole:

```
container_name: pihole
image: pihole/pihole:latest
ports:
- "53:53/tcp"
- "53:53/udp"
- "8080:80/tcp"
environment:
TZ: 'Europe/Madrid'
volumes:
- './pihole/etc-pihole:/etc/pihole'
- './pihole/etc-dnsmasq.d:/etc/dnsmasq.d'
restart: unless-stopped
```

nextcloud:

```
image: nextcloud
ports:
- "1100:80/tcp"
volumes:
- nextcloud_data:/var/www/html/data
environment:
- NEXTCLOUD_TRUSTED_DOMAINS=localhost
- MYSQL_HOST=nextcloud_db
- MYSQL_DATABASE=nextcloud
- MYSQL_USER=nextcloud
- MYSQL_PASSWORD=Qwerty22
```

```

    depends_on:
      - nextcloud_db

nextcloud_db:
  image: mariadb
  environment:
    - MYSQL_ROOT_PASSWORD=Qwerty22
    - MYSQL_DATABASE=nextcloud
    - MYSQL_USER=nextcloud
    - MYSQL_PASSWORD=Qwerty22
  volumes:
    - nextcloud_db_data:/var/lib/mysql

phpmyadmin:
  image: phpmyadmin/phpmyadmin
  ports:
    - "8000:8080"
  environment:
    - PMA_HOST=test_db
    - MYSQL_ROOT_PASSWORD=Qwerty22

volumes:
  phpmyadmin_data:
  nextcloud_db_data:

```

Sintaxis	
<code>services</code>	Inicia el archivo .yalm con los servicios que van a lanzarse en docker compose .
<code>image</code>	Especifica la imagen que va a usar de los servidores de docker seguido de / la versión (latest)
<code>ports</code>	Especifica los puertos que se van a reenviar a la máquina donde se aloja el docker , a la izquierda se especifica el puerto del host y a la derecha el del contenedor separados por un ":" En el caso de pihole por ejemplo, se comparten los puertos para el servidor DNS y para la interfaz web (80 en el contenedor) se redirige al 8080 de ARCH-FCT

restart	Podemos especificar varias opciones, (no,on-failure,always...) Esto va a determinar cuándo se debe iniciar el contenedor .
volumes	Especifica donde se van a guardar en el host archivos del sistema importantes de monitorizar de dentro del contenedor.

Ejecutamos el **docker compose** con el comando:

```
docker-compose up
```

Debemos asegurarnos de que el archivo yaml está en la ruta donde nos encontramos, en mi caso **/root/compose/docker-compose.yml**

```
[root@Arch-Docker-FCT Compose]# docker-compose up -d
WARN[0000] /root/Compose/docker-compose.yml: 'version' is obsolete
[+] Running 13/22
  # pihole [#####] Pulling                                10.2s
  # nextcloud [#####] Pulling                             10.2s
  # phpmyadmin [#####] Pulling                             10.2s
  # watchtower [#####] Pulling                             10.2s
```

Una vez ejecutado el comando se empiezan a descargar los contenedores

Con **docker ps** podemos ver los contenedores en ejecución así como información de los mismos:

```
[root@Arch-Docker-FCT Compose]# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        NAMES                PORTS
f9db4d8999ae   nextcloud:fpm        "/entrypoint.sh php-..." 7 minutes ago  Up 6 minutes  compose-nextcloud-1   9000/tcp
5d94d73c321c   mariadb              "docker-entrypoint.s..." 7 minutes ago  Up 7 minutes  compose-nextcloud_db-1 3306/tcp
2ff58096ee50   phpmyadmin/phpmyadmin "/docker-entrypoint...." 7 minutes ago  Up 7 minutes  compose-phpmyadmin-1   0.0.0.0:8000->80/tcp, :::8000->80/tcp
39f35f44dd02   pihole/pihole:latest "/s6-init"              7 minutes ago  Up About a minute (healthy) 0.0.0.0:53->53/tcp, 0.0.0.0:53->53/udp, :::53->53/tcp, 67/udp, 0.0.0.0:8080->80/tcp, :::8080->80/tcp  pihole
efde1563b19   containrrr/watchtower "/watchtower"            7 minutes ago  Up 7 minutes (healthy) 8080/tcp              compose-watchtower-1
```

En caso de que necesitemos entrar por terminal a alguno de nuestros contenedores podemos hacerlo de la siguiente forma:

docker exec -ti (nombre del contenedor) (terminal)

```
proxmox - Proxmox Console - Opera
No es seguro localhost:8006

[root@Arch-Docker-FCT Compose]# docker exec -ti nextcloud "/bin/bash"
Error response from daemon: No such container: nextcloud
[root@Arch-Docker-FCT Compose]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
60fcaeb278c4   nextcloud     "/entrypoint.sh apac..." 2 minutes ago  Up About a minute  0.0.0.0:8080->80/tcp
, :::8080->80/tcp   compose-nextcloud-1
442a373e6e53   mariadb      "docker-entrypoint.s..." 2 minutes ago  Up About a minute  3306/tcp
compose-db-1
[root@Arch-Docker-FCT Compose]# docker exec -ti 60fcaeb278c4 "/bin/bash"
root@60fcaeb278c4:/var/www/html# ls
3rdparty      composer.lock  custom_apps  lib          package-lock.json  robots.txt
AUTHORS       config         data         nextcloud-init-sync.lock  package.json        status.php
COPYING       console.php    dist         occ           public.php         themes
apps          core          index.html   ocs          remote.php         version.php
composer.json  cron.php      index.php    ocs-provider  resources
root@60fcaeb278c4:/var/www/html#
```

Persistencia (servicio linux en systemd)

Para que Docker Compose se inicie automáticamente al iniciar la (MV) en Linux, podemos crear un servicio systemd.

Para ello debemos de crear un archivo de servicio systemd para ello debemos escribir un archivo en la siguiente ruta:

`/etc/systemd/system/(nombre del servicio).service`

Vamos a crear el archivo con la siguiente sintaxis:

```
[Unit]
Description=Inicia los contenedores de Docker-compose al iniciar la VM
After=docker.service
Requires=docker.service

[Service]
WorkingDirectory=/root/compose/
ExecStart=/usr/bin/docker-compose up -d
ExecStop=/usr/bin/docker-compose down
Restart=always
TimeoutStartSec=0

[Install]
WantedBy=multi-user.target
```

Ahora debemos reiniciar el daemon-reload para que se apliquen las nuevas configuraciones en los servicios del sistema y hacerle un **enable** para crear un link simbólico que incie el servicio al iniciar el OS

```
sudo systemctl daemon-reload
sudo systemctl restart docker-compose-startup.service
```

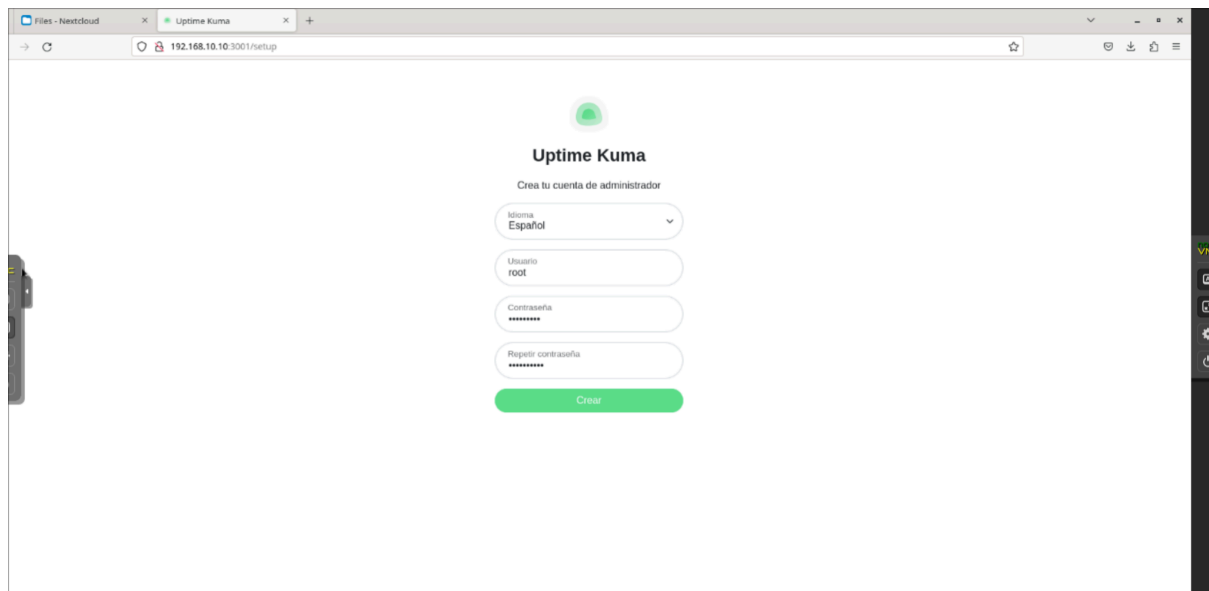
Si sale todo bien probamos a comprobar el status del servicio y a reiniciar el sistema para ver si está habilitado correctamente

```
[root@Arch-Docker-FCT Compose]# systemctl status docker-compose-startup
● docker-compose-startup.service - Inicia los contenedores de Docker-compose al iniciar la VM
   Loaded: loaded (/etc/systemd/system/docker-compose-startup.service; enabled; preset: disabled)
   Active: active (running) since Sat 2024-05-18 11:48:15 UTC; 397ms ago
     Main PID: 2371 (docker-compose)
       Tasks: 6 (limit: 28663)
    Memory: 16.5M (peak: 16.7M)
       CPU: 55ms
    CGroup: /system.slice/docker-compose-startup.service
            └─2371 /usr/bin/docker-compose up -d

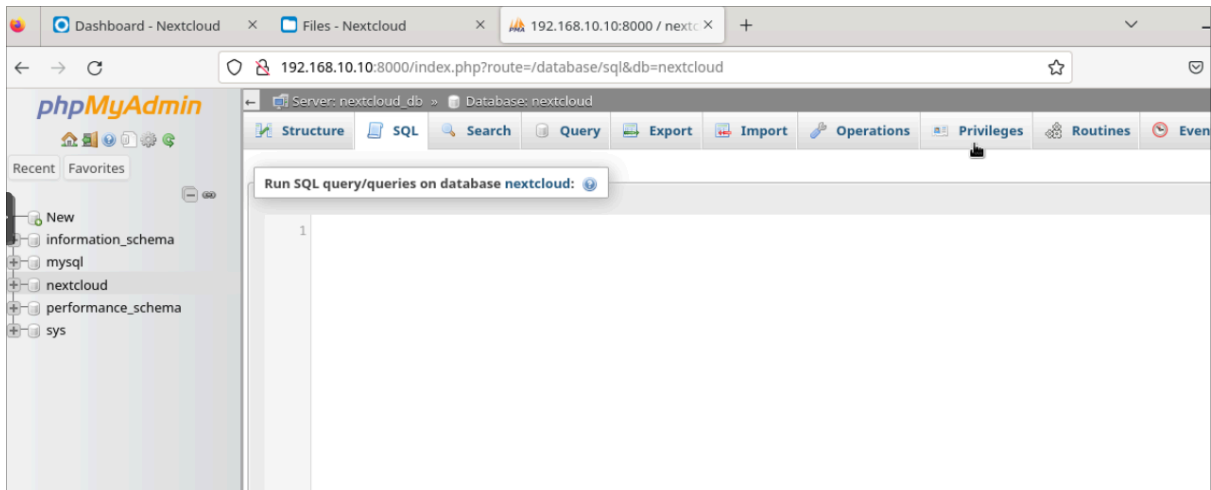
May 18 11:48:14 Arch-Docker-FCT systemd[1]: docker-compose-startup.service: Scheduled restart job, restart counter is at 4.
May 18 11:48:15 Arch-Docker-FCT systemd[1]: Started Inicia los contenedores de Docker-compose al iniciar la VM.
May 18 11:48:15 Arch-Docker-FCT docker-compose[2371]: time="2024-05-18T11:48:15Z" level=warning msg="/root/Compose/docker-compos
May 18 11:48:15 Arch-Docker-FCT docker-compose[2371]: Container compose-phpmyadmin-1 Running
May 18 11:48:15 Arch-Docker-FCT docker-compose[2371]: Container compose-watchtower-1 Running
May 18 11:48:15 Arch-Docker-FCT docker-compose[2371]: Container compose-nextcloud_db-1 Running
May 18 11:48:15 Arch-Docker-FCT docker-compose[2371]: Container compose-nextcloud-1 Running
May 18 11:48:15 Arch-Docker-FCT docker-compose[2371]: Container pi-hole Starting
lines 1-18/18 (END)
```

Comprobaciones

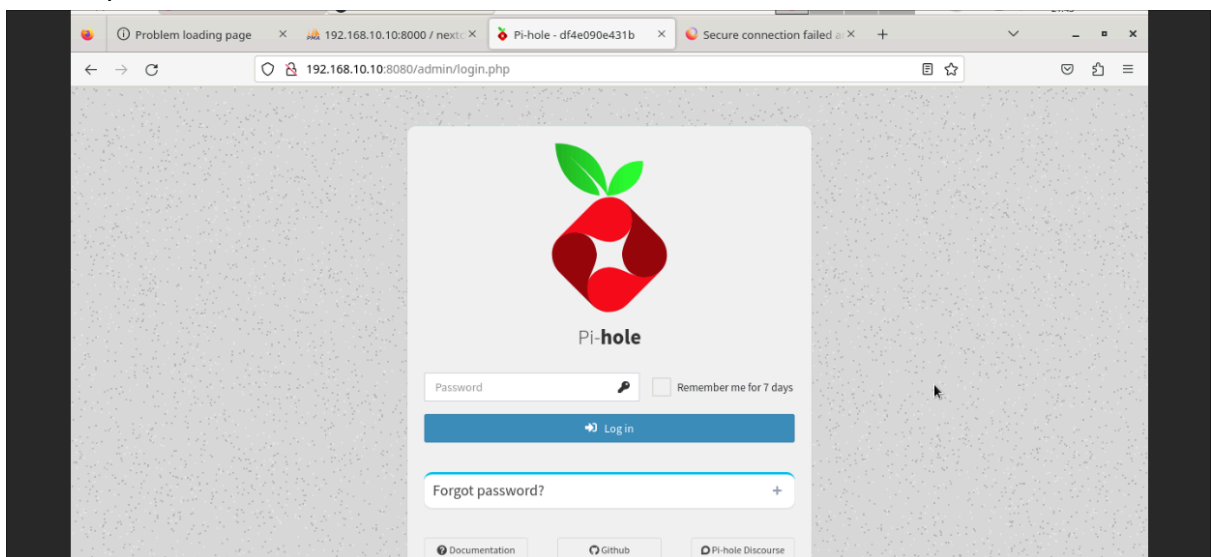
Uptime Kuma puerto 3001 (http)



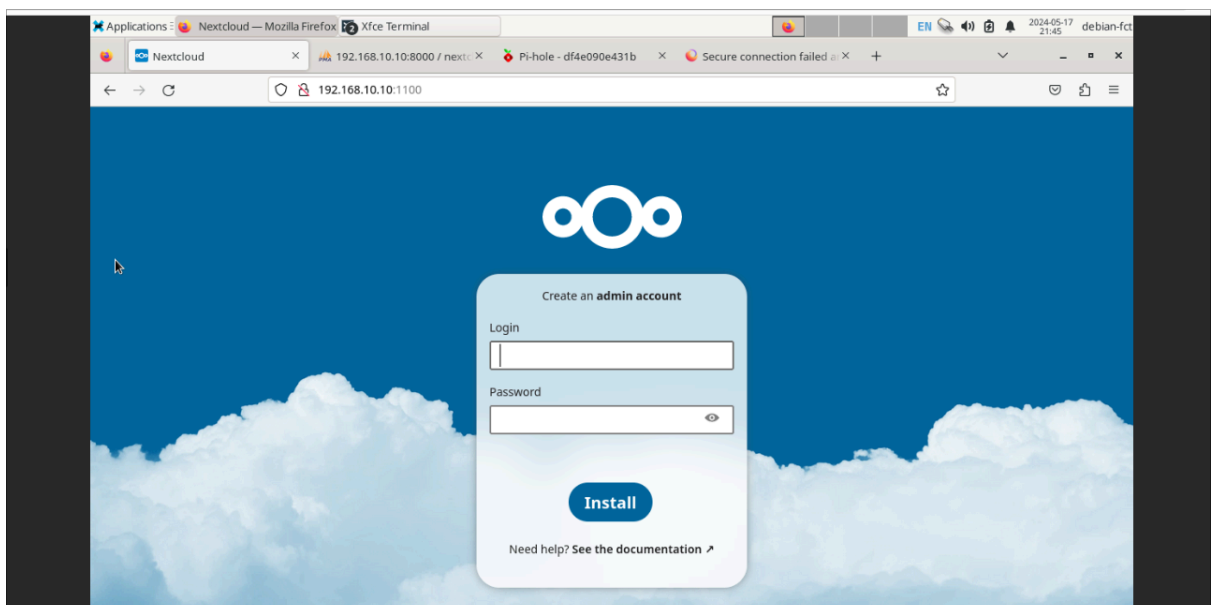
PhpMyadmin puerto 8000



Pi-Hole puerto 8080



Nextcloud puerto 1100



Logs del watchtower

```
[root@Arch-Docker-FCT Compose]# docker-compose logs watchtower
WARN[0000] /root/Compose/docker-compose.yml: 'version' is obsolete
watchtower-1 | time="2024-05-18T11:48:13Z" level=info msg="Watchtower 1.7.1"
watchtower-1 | time="2024-05-18T11:48:13Z" level=info msg="Using no notifications"
watchtower-1 | time="2024-05-18T11:48:13Z" level=info msg="Checking all containers (except explicitly disabled with label)"
watchtower-1 | time="2024-05-18T11:48:13Z" level=info msg="Scheduling first run: 2024-05-19 11:48:13 +0000 UTC"
watchtower-1 | time="2024-05-18T11:48:13Z" level=info msg="Note that the first check will be performed in 23 hours, 59 minutes, 59 seconds"
[root@Arch-Docker-FCT Compose]#
```

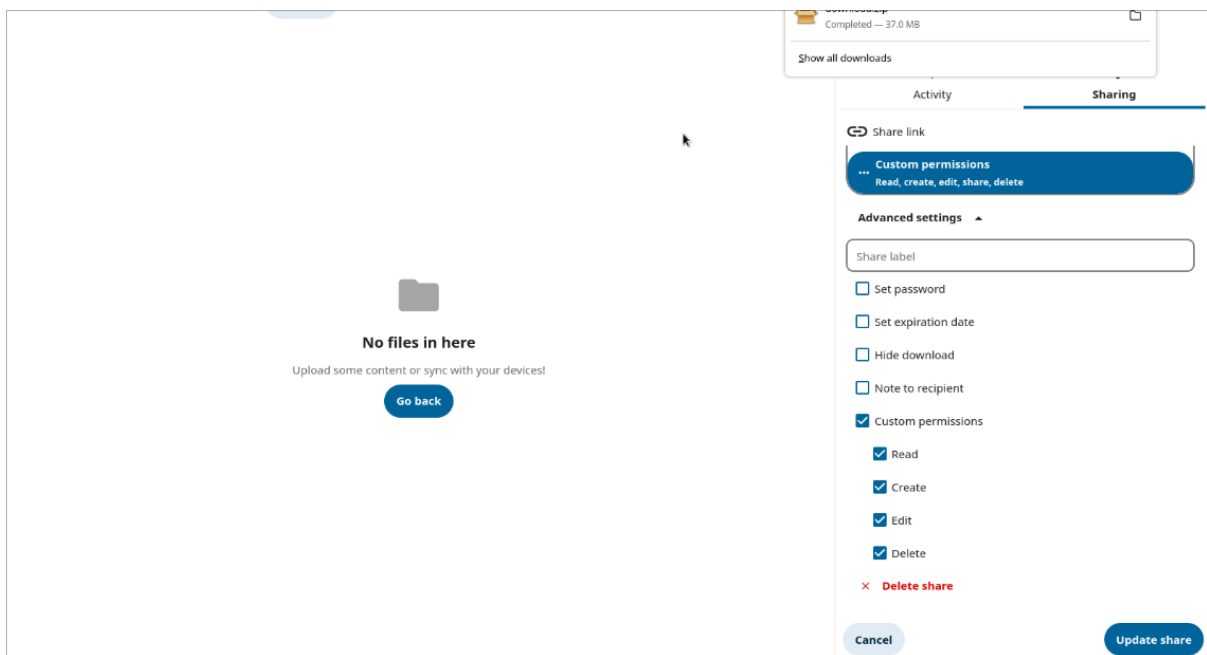
Configuración de los Contenedores

/ NEXTCLOUD /

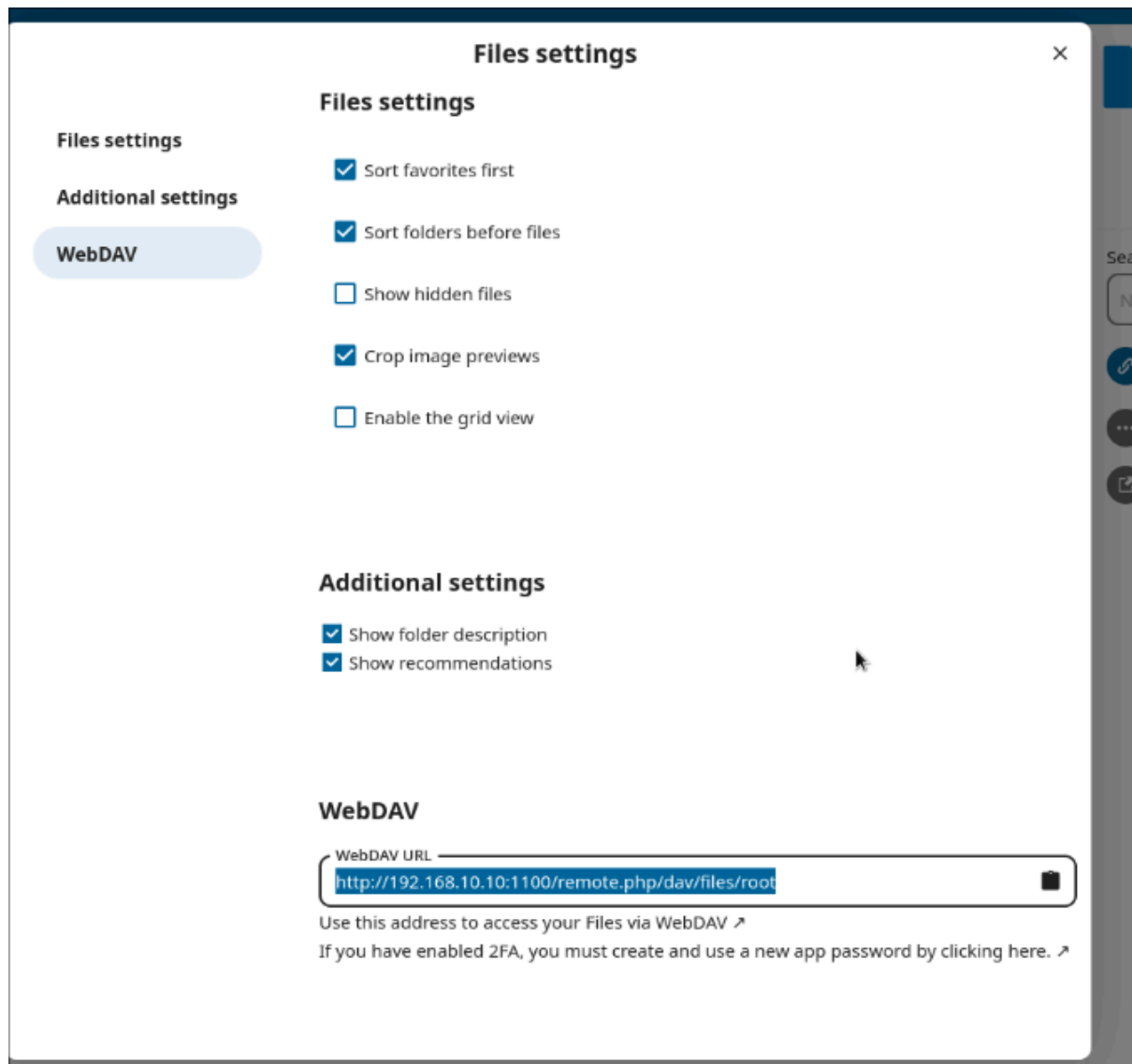
Vamos a configurar una carpeta “Cajon” que podamos montar en los sistemas linux como una carpeta de red para poder pasar archivos de manera eficiente y segura entre los dispositivos de la red

// Crear la carpeta y permisos de la misma

Vamos a crear la carpeta Cajon en Nextcloud desde la interfaz web y configurarla para darle permisos **read write create edit delete**



//WebDAV//



WebDAV es una tecnología que nos permite acceder a los archivos en remoto por el protocolo http

Para montar la url en un sistema Linux como si fuese un disco necesitamos instalar el paquete **devfs2** que es una herramienta que te permite montar sistemas de archivos WebDAV como si fueran sistemas de archivos locales en un sistema operativo Linux.

1. Instalamos el paquete devfs2

```
apt update && apt install devfs2
```

2. Creamos una carpeta cajon en /mnt/cajon y montamos con devfs2

```
mkdir -p /mnt/cajon/
```

```
chmod 777 /mnt/cajon/
```

```
mount -t davfs2 http://192.168.10.10:1100/remote.php/dav/files/root/Cajon /mnt/cajon
```

Nos pedirá credenciales de usuario para montar el repositorio, esta es una medida de seguridad de Nextcloud en conjunto con el protocolo WebDAV

```
debian@debianfct:/mnt$ sudo mount -t davfs http://192.168.10.10:1100/remote.php/dav/files/root/ /mnt/cajon
Please enter the username to authenticate with server
http://192.168.10.10:1100/remote.php/dav/files/root/ or hit enter for none.
Username: root
Please enter the password to authenticate user root with server
http://192.168.10.10:1100/remote.php/dav/files/root/ or hit enter for none.
Password:
mount: (hint) your fstab has been modified, but systemd still uses
the old version; use 'systemctl daemon-reload' to reload.
debian@debianfct:/mnt$ ls -la cajon/
total 1
drwxr-xr-x 4 root root 104 may 18 15:56 .
drwxr-xr-x 2 root root  0 may 18 15:56 Cajon
drwx----- 2 root root  0 may 18 15:05 lost+found
debian@debianfct:/mnt$
```

Con el fin de automatizarlo davfs nos permite crear un archivo **secrets** con permisos “600” y debe de tener la siguiente estructura

```
http://192.168.10.10:1100/remote.php/dav/files/root/cajon {usuario} {contraseña}
```

El archivo debe encontrarse en /etc/davfs2/secrets y debe contener el usuario y la password de Nextcloud

El archivo estará protegido ya que solo se puede leer con privilegios de superusuario

```
debian@debianfct:/mnt$ ls
cajon
debian@debianfct:/mnt$ sudo cat /etc/davfs2/secrets
http://192.168.10.10:1100/remote.php/dav/files/root/Cajon root root
debian@debianfct:/mnt$
```

3. Crear entrada en fstab para que lo monte al iniciar el sistema

```
echo "http://192.168.10.10:1100/remote.php/dav/files/root/cajon /mnt/cajon davfs
user,rw,noauto 0 0" | sudo tee -a /etc/fstab
```

El siguiente script de bash detalla todo el proceso del montaje en cualquier sistema linux

```
#!/bin/bash
```

```

# Verificar permisos de sudo
if [ "$(id -u)" -ne 0 ]; then
    echo "Este script necesita permisos de superusuario para instalar paquetes y montar
sistemas de archivos."
    sudo "$0" "$@" # Ejecutar este script con sudo
    exit $?
fi

# Verificar si davfs2 está instalado y, si no lo está, instalarlo
if ! command -v mount.davfs &> /dev/null; then
    echo "davfs2 no está instalado. Instalando..."
    if command -v apt &> /dev/null; then
        apt update
        apt install -y davfs2
    elif command -v pacman &> /dev/null; then
        pacman -Sy --noconfirm davfs2
    elif command -v yum &> /dev/null; then
        yum update -y
        yum install -y davfs2
    else
        echo "No se pudo encontrar un gestor de paquetes compatible."
        exit 1
    fi
fi

# Crear el directorio de montaje si no existe
MOUNT_DIR="/mnt/cajon"
if [ ! -d "$MOUNT_DIR" ]; then
    mkdir -p "$MOUNT_DIR"
    echo "Directorio $MOUNT_DIR creado."
fi

# Crear el archivo "secrets"
sudo echo "http://192.168.10.10:1100/remote.php/dav/files/root/Cajon root ***** >>
/etc/davfs2/secrets
sudo chmod 600 /etc/davfs2/secrets

# Montar el repositorio WebDAV
echo "Montando el repositorio WebDAV en $MOUNT_DIR..."
echo "Ingrese su nombre de usuario y contraseña de Nextcloud cuando se le solicite."
echo

```

```

mount -t davfs "http://192.168.10.10:1100/remote.php/dav/files/root/cajon" "$MOUNT_DIR"

# Verificar si el montaje fue exitoso
if [ $? -eq 0 ]; then
    echo "El repositorio WebDAV se montó correctamente en $MOUNT_DIR."
else
    echo "Error al montar el repositorio WebDAV."
fi

# Agregar entrada en /etc/fstab para montar automáticamente en el inicio
echo "http://192.168.10.10:1100/remote.php/dav/files/root/cajon $MOUNT_DIR davfs
user,rw,noauto 0 0" | sudo tee -a /etc/fstab
echo "Entrada agregada en /etc/fstab para montar automáticamente en el inicio."

```

Como lo automatizamos todo desde el servidor ansible , creo un playbook que automatiza todo el proceso:

```

---
- name: Configuración de montaje WebDAV en Linux
  hosts: APT
  become: true

  tasks:
    - name: Instalar paquete davfs2 en Debian
      apt:
        name: davfs2
        state: present

    - name: Crear directorio de montaje
      file:
        path: /mnt/cajon
        state: directory
        mode: "0777"

    - name: Crear archivo de secretos de davfs2 con las credenciales
      template:
        src: /root/playbooks/templates/secrets.j2
        dest: /etc/davfs2/secrets

```

```

owner: root
group: root
mode: '0600'

- name: Montar repositorio WebDAV
  command: "mount -t davfs http://192.168.10.10:1100/remote.php/dav/files/root/Cajon
/mnt/cajon"
  ignore_errors: yes # Ignorar errores si ya está montado

- name: Agregar entrada en /etc/fstab
  lineinfile:
    path: /etc/fstab
    line: "http://192.168.10.10:1100/remote.php/dav/files/root/Cajon /mnt/cajon davfs
user,rw,noauto 0 0"
  state: present
  notify: Remount fstab

handlers:
- name: Remount fstab
  command: "mount -a"

```

Lo bueno de este ansible es que podemos guardar el archivo “secrets” en la máquina local de Ansible, concretamente en la ruta [/root/playbooks/templates/secrets.j2](#)

```

root@Deb-Ansible:~# ls -la playbooks/templates/secrets.j2
-rw-r--r-- 1 root root 74 May 18 13:32 playbooks/templates/secrets.j2
root@Deb-Ansible:~# █

```

De esta forma nuestras credenciales estan protegidas en el servidor de ansible y solo son accesibles con permisos 0600 en la máquina de debian que es donde vamos a realizar la prueba.

Vamos a probar a ejecutar el playbook con -v para verbose

```
ansible-playbook -i inventory.yml --limit APT playbooks/WebDAV.yaml -v
```

Ejecutamos el script , yo lo ejecute varias veces para hacer pruebas y posteriormente desmonte el repositorio y borre el archivo de “secrets” y podemos ver como ansible lo automatiza de tal forma que va revisando los procesos y si falta algo lo hace pero si no lo deja igual

```
root@Deb-Ansible:~# ansible-playbook -i inventory.yml playbooks/WebDAV.yaml -v
Using /etc/ansible/ansible.cfg as config file

PLAY [Configuración de montaje WebDAV en Linux] *****

TASK [Gathering Facts] *****
ok: [debian-fct]

TASK [Instalar paquete davfs2 en Debian] *****
ok: [debian-fct] -> {"cache_update_time": 1716035475, "cache_updated": false, "changed": false}

TASK [Crear directorio de montaje] *****
ok: [debian-fct] -> {"changed": false, "gid": 0, "group": "root", "mode": "0777", "owner": "root", "path":
"/mnt/cajon", "size": 4096, "state": "directory", "uid": 0}

TASK [Crear archivo de secretos de davfs2 con las credenciales] *****
changed: [debian-fct] -> {"changed": true, "checksum": "2709d5f1d96f4b5e873ade29bb78b8c2cd78c4db", "dest":
"/etc/davfs2/secret", "gid": 0, "group": "root", "md5sum": "5208ed5ce0db73133114d3e3e13b2c19", "mode": "
0600", "owner": "root", "size": 74, "src": "/home/ansible/.ansible/tmp/ansible-tmp-1716042361.8061342-2042
-75175156173706/source", "state": "file", "uid": 0}

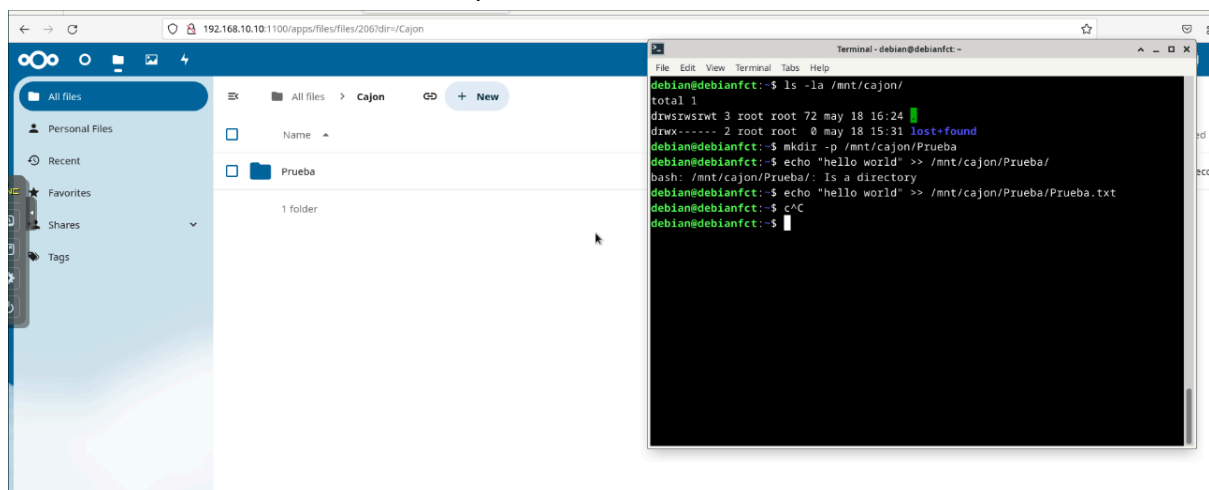
TASK [Montar repositorio WebDAV] *****
changed: [debian-fct] -> {"changed": true, "cmd": {"mount", "-t", "davfs", "http://192.168.10.10:1100/remo
te.php/dav/files/root/Cajon", "/mnt/cajon"}, "delta": "0:00:01.761573", "end": "2024-05-18 16:26:04.336980
", "msg": "", "rc": 0, "start": "2024-05-18 16:26:02.575407", "stderr": "", "stderr_lines": [], "stdout":
"", "stdout_lines": []}

TASK [Agregar entrada en /etc/fstab] *****
ok: [debian-fct] -> {"backup": "", "changed": false, "msg": ""}

PLAY RECAP *****
debian-fct : ok=6 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 i
gnored=0

root@Deb-Ansible:~#
```

Si volvemos a nuestra mv observamos cómo los cambios creados desde la carpeta se actualizan automáticamente en el repositorio de Next Cloud.



Con el playbook deberíamos poder realizar esta instalación de forma remota en cualquier dispositivo linux de forma completamente automática .

Lanzo desde el remoto de ansible un ls y un cat al directorio y a la derecha comparo los logs de docker-compose para ver que efectivamente se produce un GET cuando leemos el archivo desde terminal.

```

root@Deb-Ansible:~# ansible debian-fct -a 'ls /mnt/cajon'
debian-fct | CHANGED | rc=0 >>
hola.txt
lost+found
root@Deb-Ansible:~# ansible debian-fct -a "cat /mnt/cajon/hola.txt"
debian-fct | CHANGED | rc=0 >>
hola.txt
root@Deb-Ansible:~#

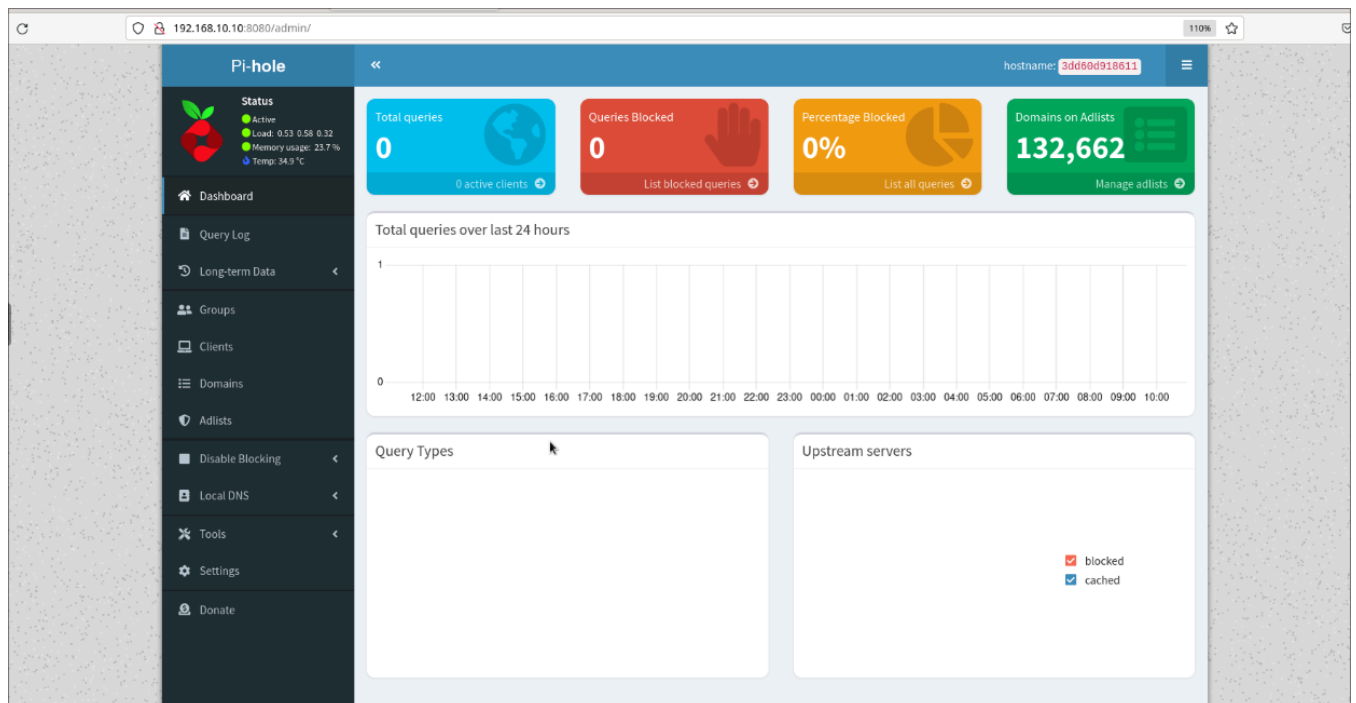
```

```

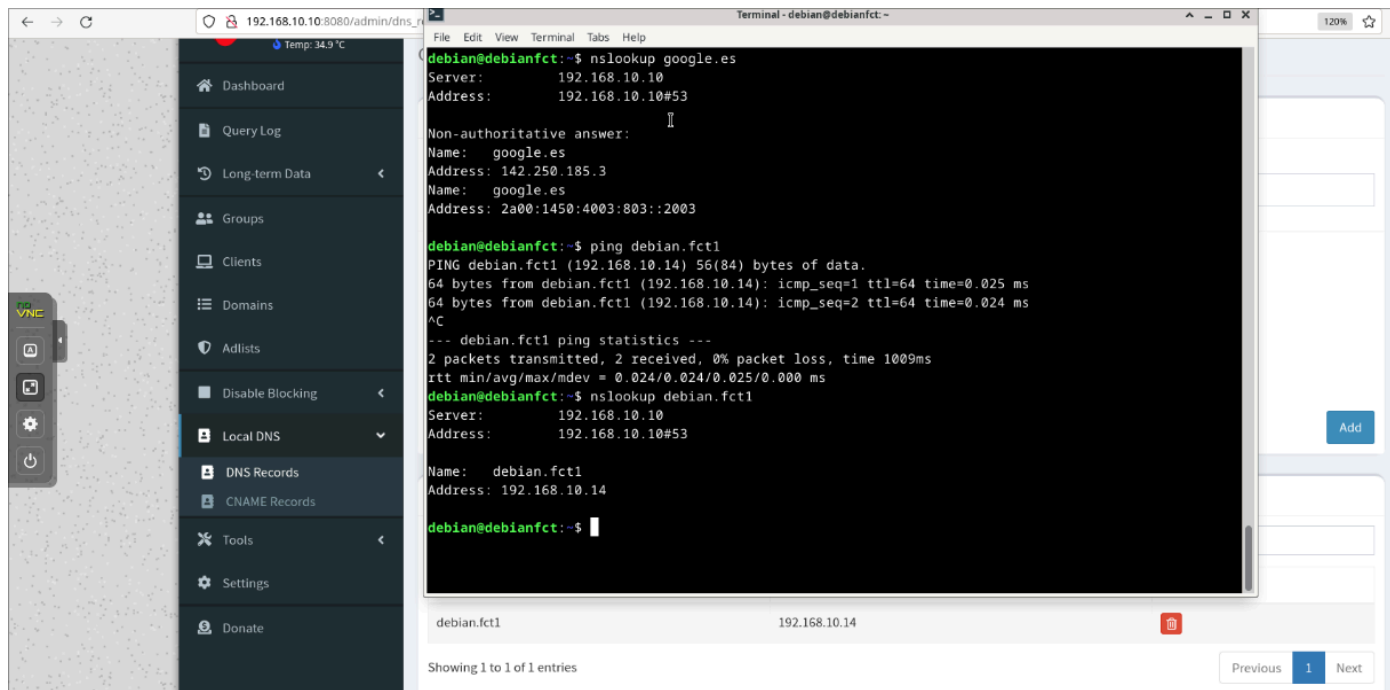
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:10:22 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2188 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - - [18/May/2024:16:10:26 +0000] "GET /ocs/v2.php/apps/not
ifications/api/v2/notifications HTTP/1.1" 304 821 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:
109.0) Gecko/20100101 Firefox/115.0"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:10:32 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2912 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:10:35 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2908 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:10:35 +0000] "GET /remote.php/dav/f
iles/root/Cajon/hola.txt HTTP/1.1" 200 1807 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:10:40 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2906 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:10:43 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2916 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:10:50 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2912 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - - [18/May/2024:16:10:55 +0000] "GET /apps/files/api/v1/s
tats HTTP/1.1" 200 982 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firef
ox/115.0"
nextcloud-1 | 192.168.10.14 - - [18/May/2024:16:10:56 +0000] "GET /ocs/v2.php/apps/not
ifications/api/v2/notifications HTTP/1.1" 304 820 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:
109.0) Gecko/20100101 Firefox/115.0"
nextcloud-1 | 192.168.10.14 - - [18/May/2024:16:21:33 +0000] "OPTIONS /remote.php/dav/
files/root/Cajon/ HTTP/1.1" 401 1923 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:21:33 +0000] "OPTIONS /remote.php/d
av/files/root/Cajon/ HTTP/1.1" 200 1903 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:21:33 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2918 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:21:34 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2188 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:21:51 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2916 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - - [18/May/2024:16:23:51 +0000] "OPTIONS /remote.php/dav/
files/root/Cajon/ HTTP/1.1" 401 1925 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:23:51 +0000] "OPTIONS /remote.php/d
av/files/root/Cajon/ HTTP/1.1" 200 1907 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:23:51 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2910 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:23:52 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2188 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:24:14 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2914 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:24:23 +0000] "PROPFIND /remote.php/
dav/files/root/Cajon/ HTTP/1.1" 207 2908 "-" "davfs2/1.6.1 neon/0.32.5"
nextcloud-1 | 192.168.10.14 - root [18/May/2024:16:24:43 +0000] "GET /remote.php/dav/f
iles/root/Cajon/hola.txt HTTP/1.1" 200 1809 "-" "davfs2/1.6.1 neon/0.32.5"

```

/PIHOLE/



Pihole nos va a servir de DNS para la red local, aunque también es un bloqueador de anuncios y analizador de tráfico de red, así que podríamos usarlo para bloquear algunos sitios web que no nos interesen por motivos de seguridad



Para configurar nuestro registro DNS es tan sencillo como entrar a LOCAL DNS> DNS RECORDS y añadir el CNAME y la IP , también podemos hacerlo en los archivos de configuración de pi-hole.

Ahora debemos añadir la IP de nuestro ARCH-DOCKER que tiene los puertos DNS forwarded del contenedor para poder comunicarnos con el DNS en nuestro archivo `/etc/hosts` .

Para resoluciones fuera del Local DNS como el de la imagen (google.es) Pi-hole redirecciona automáticamente las peticiones que no están en su lista con los DNS del router.

/Uptime Kuma/

Uptime Kuma es un software con interfaz web que nos permite monitorizar tódo tipo de servidores o servicios. Podemos configurarlo para que haga ping a una IP para ver si está activa o a un DNS para que le haga un nslookup y ver si está resolviendo.

También podemos crear queries de sql para que se ejecuten en un servidor SQL por ejemplo, nos ofrece desde la interfaz web una vistas con gráficas muy llamativas y cómodas. Es de código abierto por lo que podemos modificarlo a nuestro antojo.

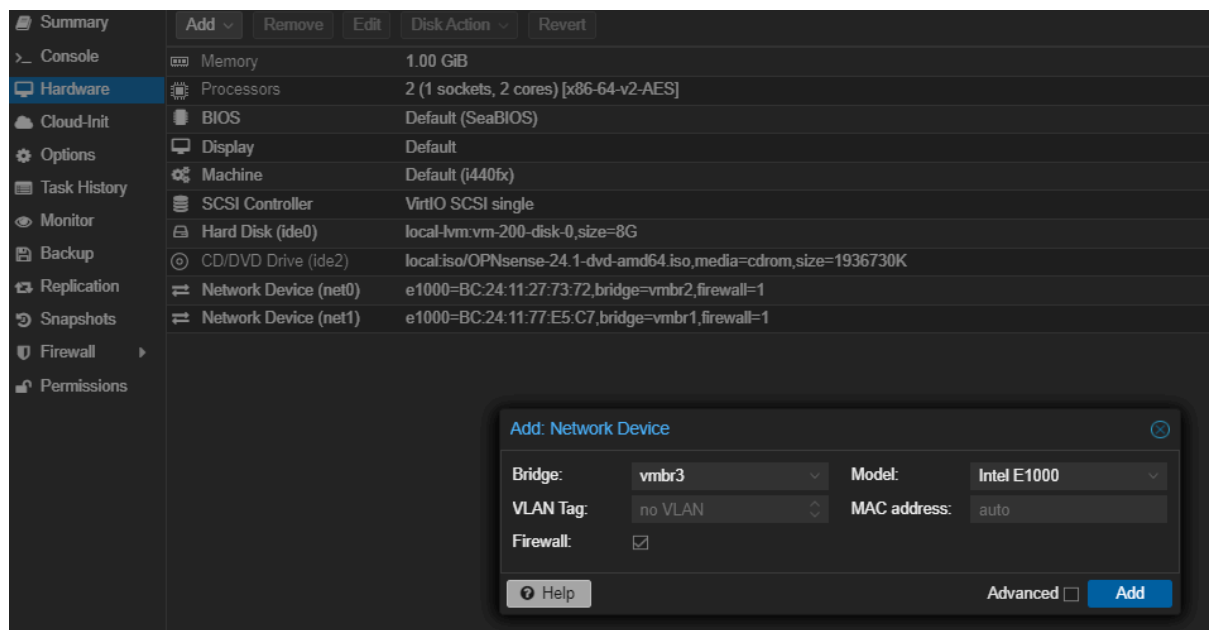
Red 192.168.20.0/24 (Windows net)FCT2

Para configurar la red 192.168.20.0/24 primero creamos la red bridge en proxmox

Para ello añadimos una nueva interfaz de red en modo puente en **/etc/network/interfaces**

```
auto vmbr3
iface vmbr3 inet static
    address 192.168.20.1/24
    bridge-ports none
    bridge-stp off
    bridge-fd 0
```

Añadimos la nueva interfaz de red en el OPNSense



Una vez añadida la nueva interfaz, podemos acceder a la configuración desde el entorno gráfico en una máquina que tenga conexión con el router, en mi caso uso **debian.fct**

Vamos a Interfaces > Assignments.

Añadimos una nueva interfaz

OPNsense root@OPNsense.localdomain

Lobby

Reporting

System

Interfaces

- [LAN]
- [WAN]
- Assignments
- Overview
- Settings
- Neighbors
- Virtual IPs
- Wireless
- Point-to-Point
- Other Types

Interface	Identifier	Device	
[LAN]	lan	em1 (bc:24:11:77:e5:c7)	
[WAN]	wan	em0 (bc:24:11:27:73:72)	

Save

+ Assign a new interface

Device: em2 (bc:24:11:00:ea:98)

Description: lan2

Add

OPNsense (c) 2014-2024 Deciso B.V.

Configuramos la nueva Ip para la 20.0/24

OPNsense root@OPNsense.localdomain

Lobby

Reporting

System

Interfaces

- [LAN]
- [lan2]
- [WAN]
- Assignments
- Overview
- Settings
- Neighbors
- Virtual IPs
- Wireless
- Point-to-Point

Dynamic gateway policy ☐ This interface does not require an intermediate system to act as a gateway

Hardware settings

Overwrite global settings ☐

Static IPv4 configuration

IPv4 address: 192.168.20.1 24

IPv4 Upstream Gateway: Auto-detect

Save Cancel

OPNsense (c) 2014-2024 Deciso B.V.

Vemos que el ping desde la 192.168.10.14 funciona

```
Terminal - debian@debianfct: ~
File Edit View Terminal Tabs Help

debian@debianfct:~$ ping 192.168.20.1
PING 192.168.20.1 (192.168.20.1) 56(84) bytes of data.
64 bytes from 192.168.20.1: icmp_seq=1 ttl=64 time=0.357 ms
64 bytes from 192.168.20.1: icmp_seq=2 ttl=64 time=0.344 ms
^C
--- 192.168.20.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1022ms
rtt min/avg/max/mdev = 0.344/0.350/0.357/0.006 ms
debian@debianfct:~$
```

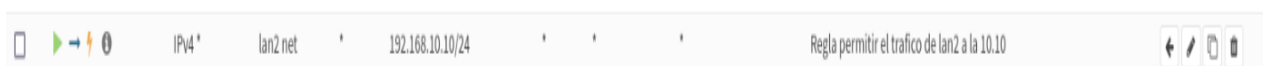
Mi objetivo es separar las dos redes de forma que los equipos de windows no tengan comunicación con la red de linux menos con la máquina de Docker.

Toda esta configuración la podemos hacer desde el firewall de OPNSENSE

Reglas para la red de Linux (192.168.10.0/24):

Firewall > Rules > LAN.

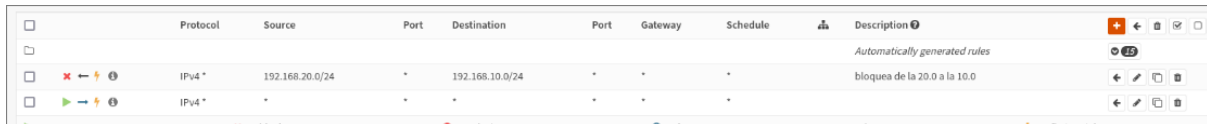
- Añade una regla específica que permita el acceso desde la red de Windows (192.168.20.0/24) solo a la IP 192.168.10.5.
 - **Action:** Pass
 - **Interface:** LAN
 - **Source:** 192.168.20.0/24
 - **Destination:** 192.168.10.5



Reglas para la red de Windows (192.168.20.0/24):

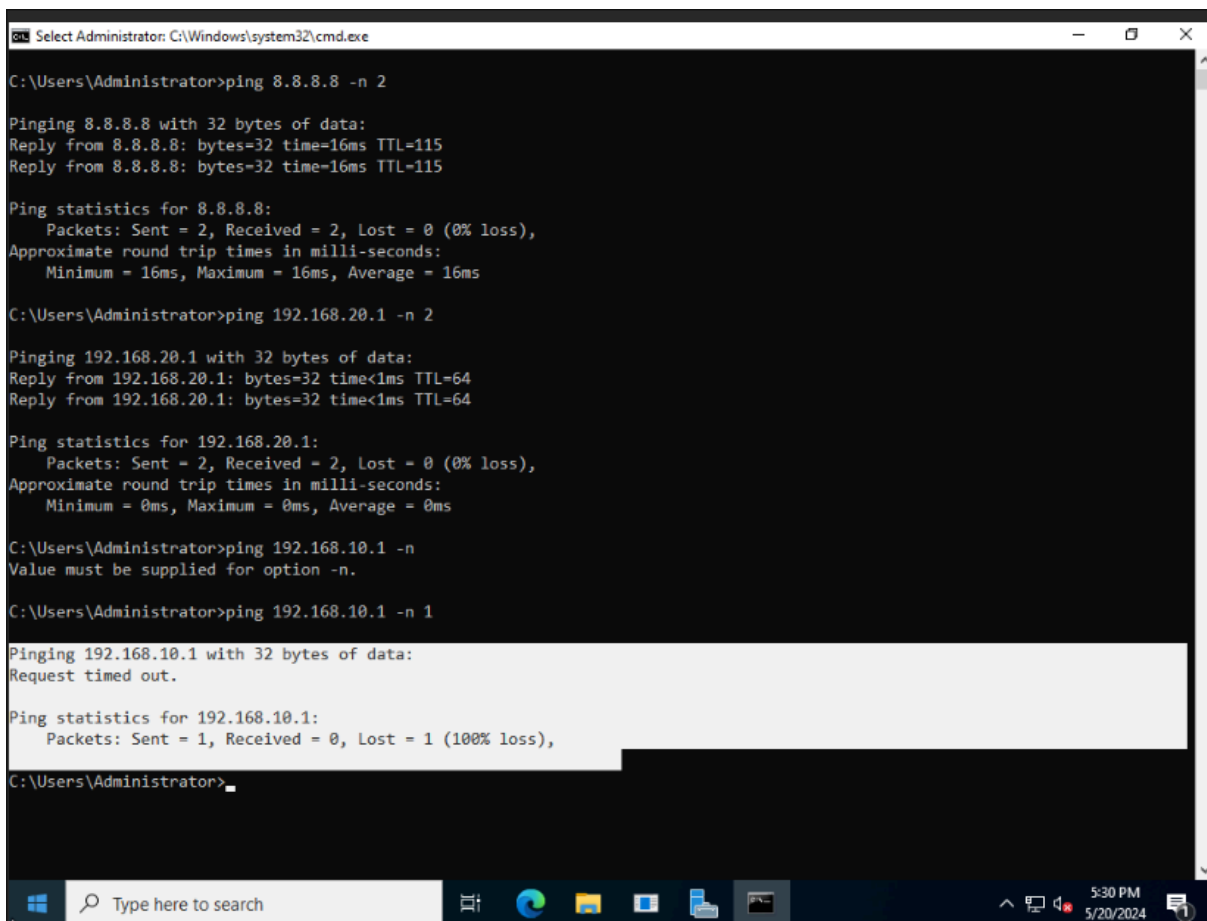
Firewall > Rules > LAN2

- Añade una regla para bloquear todo el tráfico hacia la red de Linux (192.168.10.0/24), excepto hacia 192.168.10.5.
 - **Action:** Block
 - **Interface:** OPT1
 - **Source:** 192.168.20.0/24
 - **Destination:** 192.168.10.0/24
 - **Destination Not:** 192.168.10.5



	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description
	IPv4 *	192.168.20.0/24	*	192.168.10.0/24	*	*	*	bloquea de la 20.0 a la 10.0

Comprobamos en **WS22** en la segunda red interna para ver si se aplican las reglas del firewall



```

C:\Users\Administrator>ping 8.8.8.8 -n 2

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=16ms TTL=115
Reply from 8.8.8.8: bytes=32 time=16ms TTL=115

Ping statistics for 8.8.8.8:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 16ms, Maximum = 16ms, Average = 16ms

C:\Users\Administrator>ping 192.168.20.1 -n 2

Pinging 192.168.20.1 with 32 bytes of data:
Reply from 192.168.20.1: bytes=32 time<1ms TTL=64
Reply from 192.168.20.1: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.20.1:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\Administrator>ping 192.168.10.1 -n
Value must be supplied for option -n.

C:\Users\Administrator>ping 192.168.10.1 -n 1

Pinging 192.168.10.1 with 32 bytes of data:
Request timed out.

Ping statistics for 192.168.10.1:
    Packets: Sent = 1, Received = 0, Lost = 1 (100% loss),
  
```

El tráfico desde la red 20.0/24 no llega a la 10.0

Podemos revisar los logs del OPNsense para ver si se está aplicando la regla

Interface	Time	Source	Destination	Proto	Label	
lan2	→ 2024-05-20T17:20:59	192.168.20.254:56259	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:59	192.168.20.254:55387	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:55	192.168.20.254:55784	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:53	192.168.20.254:54980	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:49	192.168.20.254:55725	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:49	192.168.20.254:54329	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:46	192.168.20.254:55405	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:42	192.168.20.254:54234	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:38	192.168.20.254:54473	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:34	192.168.20.254:55016	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:30	192.168.20.254:55866	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:28	192.168.20.254:56127	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:25	192.168.20.254:56212	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:24	192.168.20.254:55917	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:21	192.168.20.254:54762	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:15	192.168.20.254:56283	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:11	192.168.20.254:55444	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:06	192.168.20.254:54582	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:04	192.168.20.254:55591	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:01	192.168.20.254:54982	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:20:00	192.168.20.254:54232	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:19:59	192.168.20.254:56159	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:19:58	192.168.20.254:56241	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:19:56	192.168.20.254:54033	192.168.10.1:53	udp	Default deny / state violation rule	🔍
lan2	→ 2024-05-20T17:19:56	192.168.20.254:56215	192.168.10.1:53	udp	Default deny / state violation rule	🔍

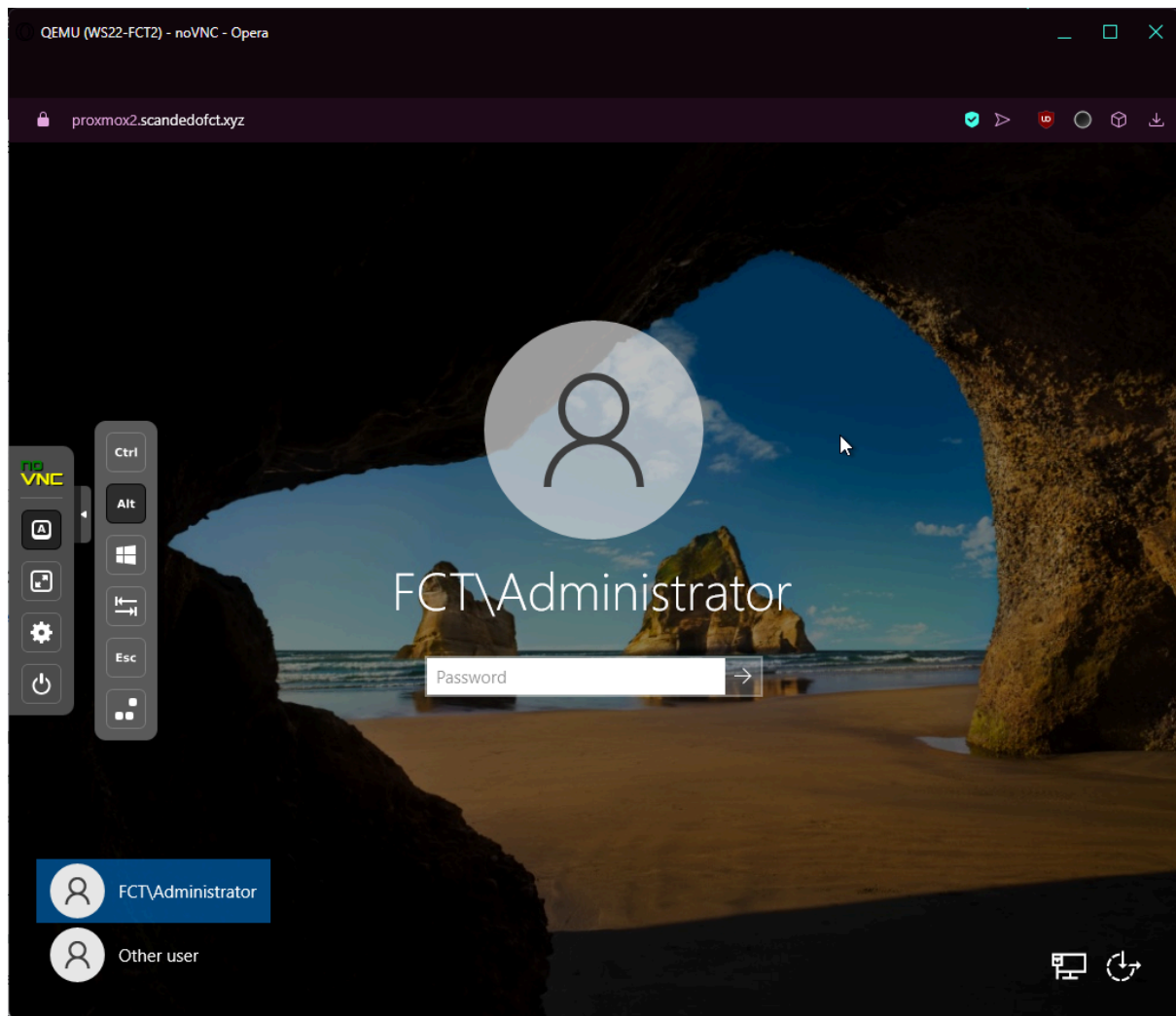
Podemos observar que se bloquea el tráfico dirigido a la red 192.168.10.1

—

Windows Server WSFCT22

Creo una MV en proxmox con ip 192.168.20.254 que va a ser nuestro AC y a donde se van a conectar los trabajadores de nuestra empresa por RDP

Creo la Máquina y configuro el dominio **FCT** que vamos a usar en la zona windows. No pongo capturas de esta parte ya que lo realizamos mil veces en clase y no es un proceso muy complicado



Creación de Usuarios Unidades Organizativas y Grupos

Creo un script de powershell que configure 3 usuarios para los 3 grupos que van a existir en nuestra infraestructura windows.

```
Administrator: Windows PowerShell
PS C:\Users\Administrator\Desktop> .\ConfDominio.ps1
PS C:\Users\Administrator\Desktop> net user /domain

User accounts for \\WS22-FCT

-----
Administrator      AdminUsuario1      AdminUsuario2
AdminUsuario3      Guest              krbtgt
SoporteUsuario1    SoporteUsuario2    SoporteUsuario3
TrabajadorUsuario1 TrabajadorUsuario2 TrabajadorUsuario3
The command completed successfully.

PS C:\Users\Administrator\Desktop>
```

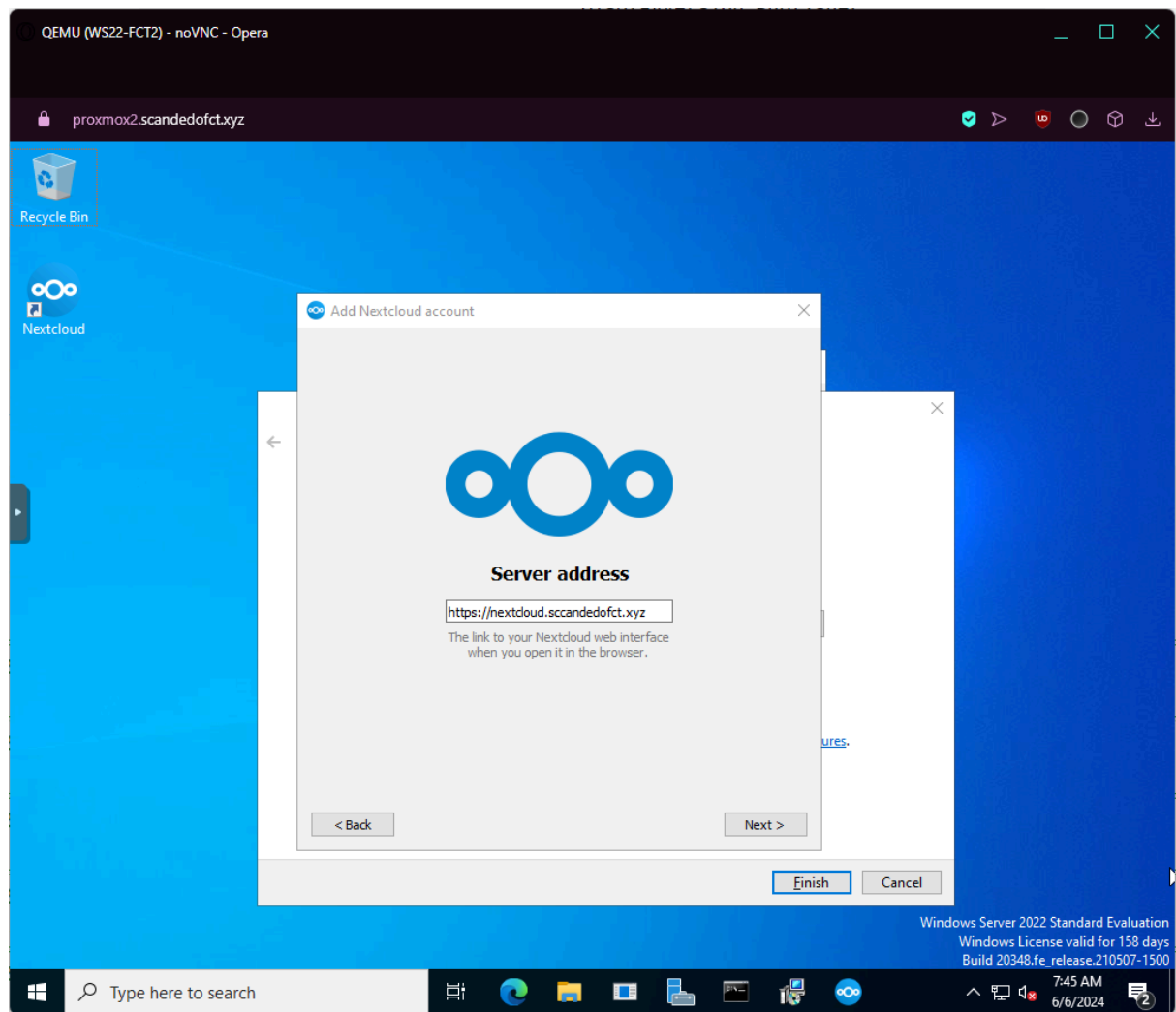
WebDAV

Vamos a configurar el **WebDAV** de nextcloud en el Windows server y compartirlo por SMB para toda la red

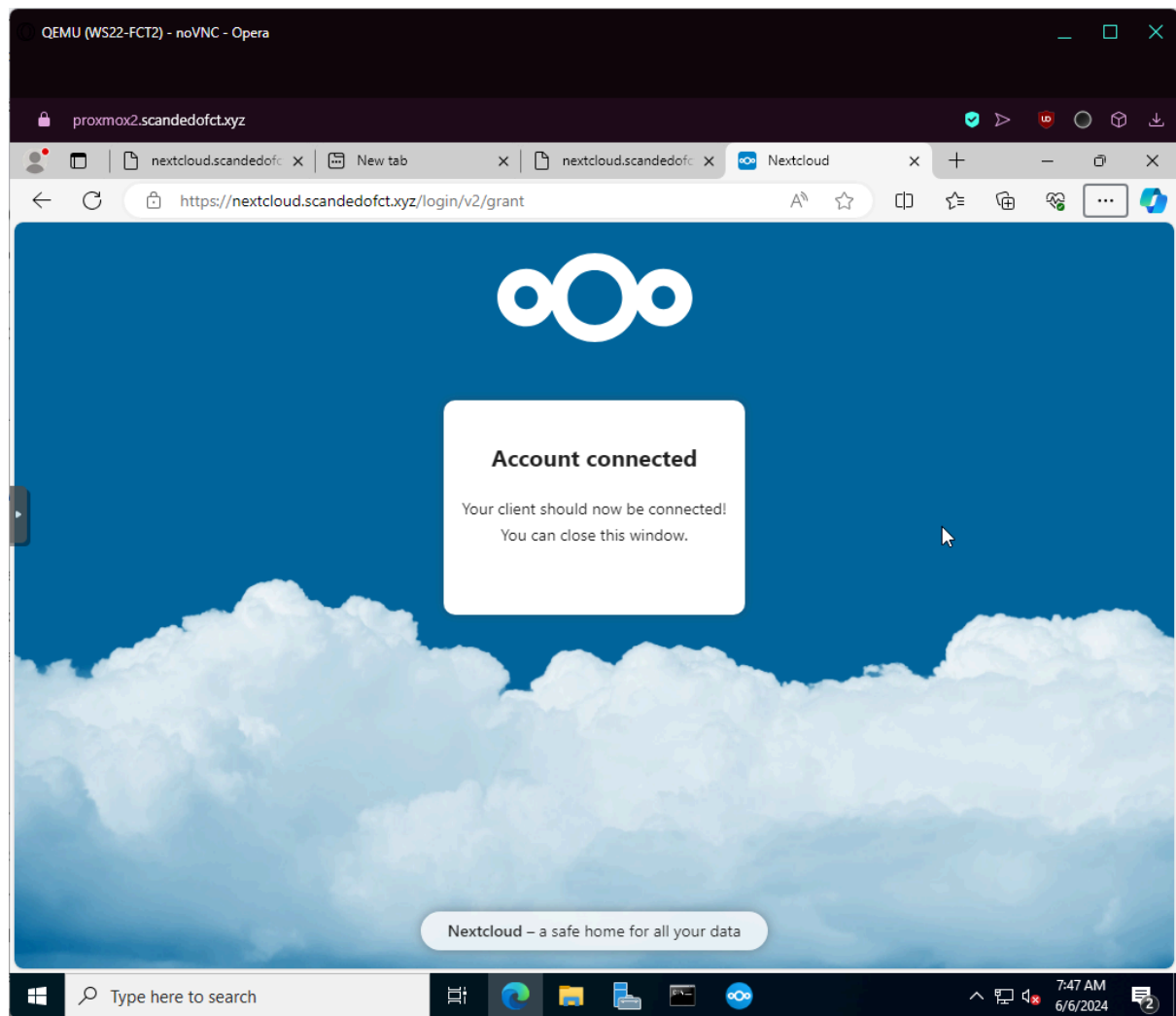
Para ello debemos descargarnos el nextcloud sync client para windows

[documentación](#)

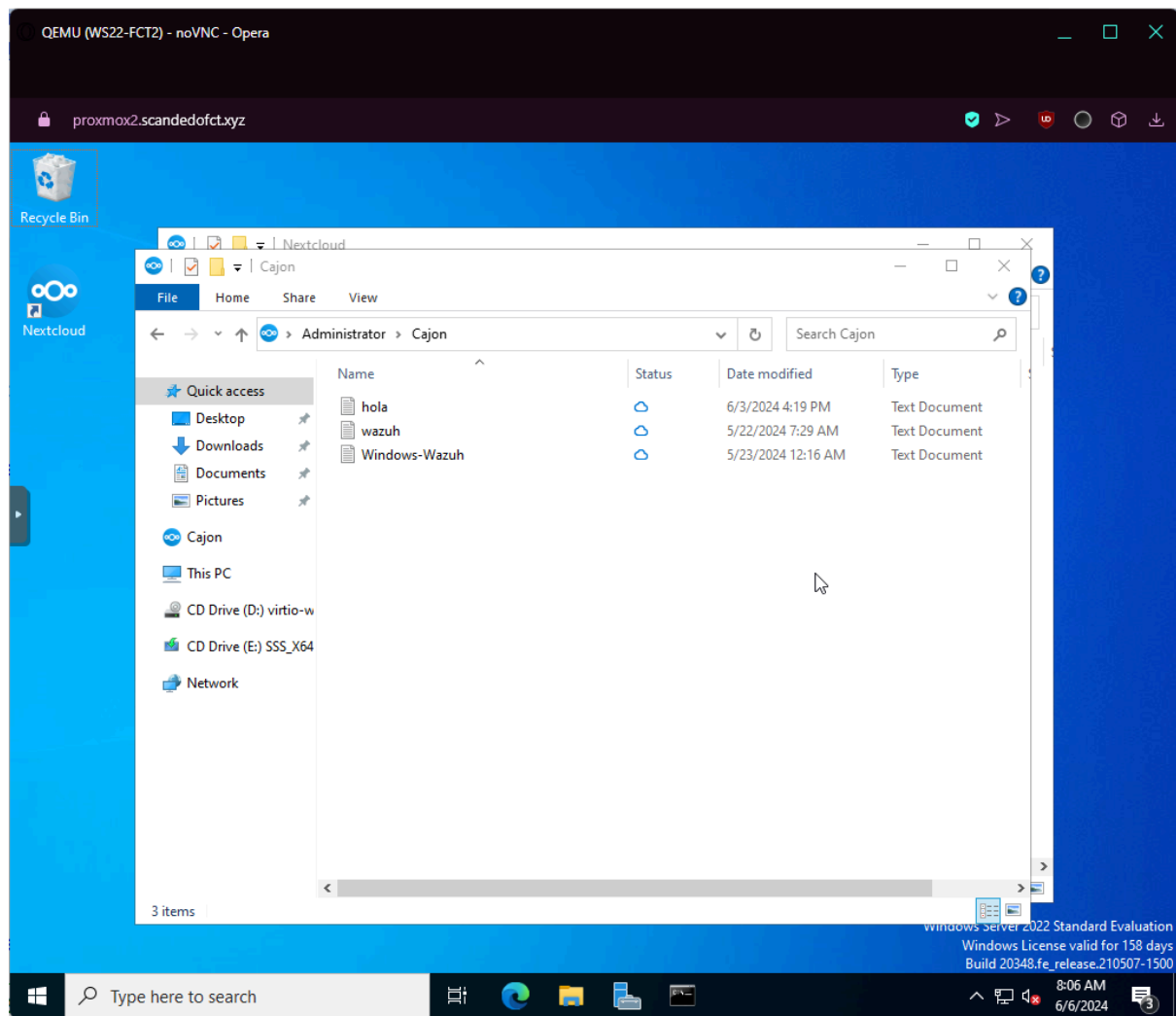
Le especifico la ruta de mi servidor de nextcloud apuntando al nombre de dominio de cloudfare que apunta a mi máquina en mi red linux (192.168.10.10) , en principio no tenemos conexión directa con la máquina por las reglas de firewall del router pero si que podemos acceder a nextcloud gracias al túnel de cloudfare



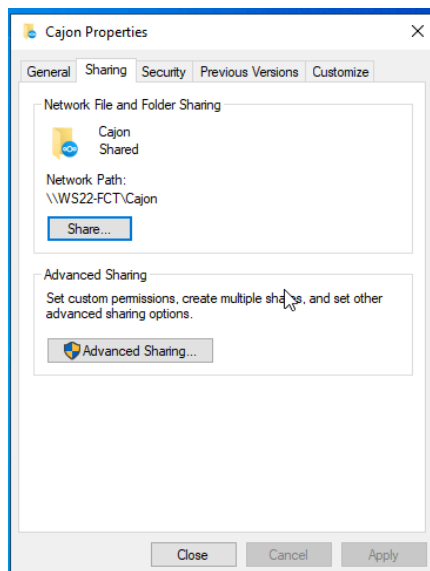
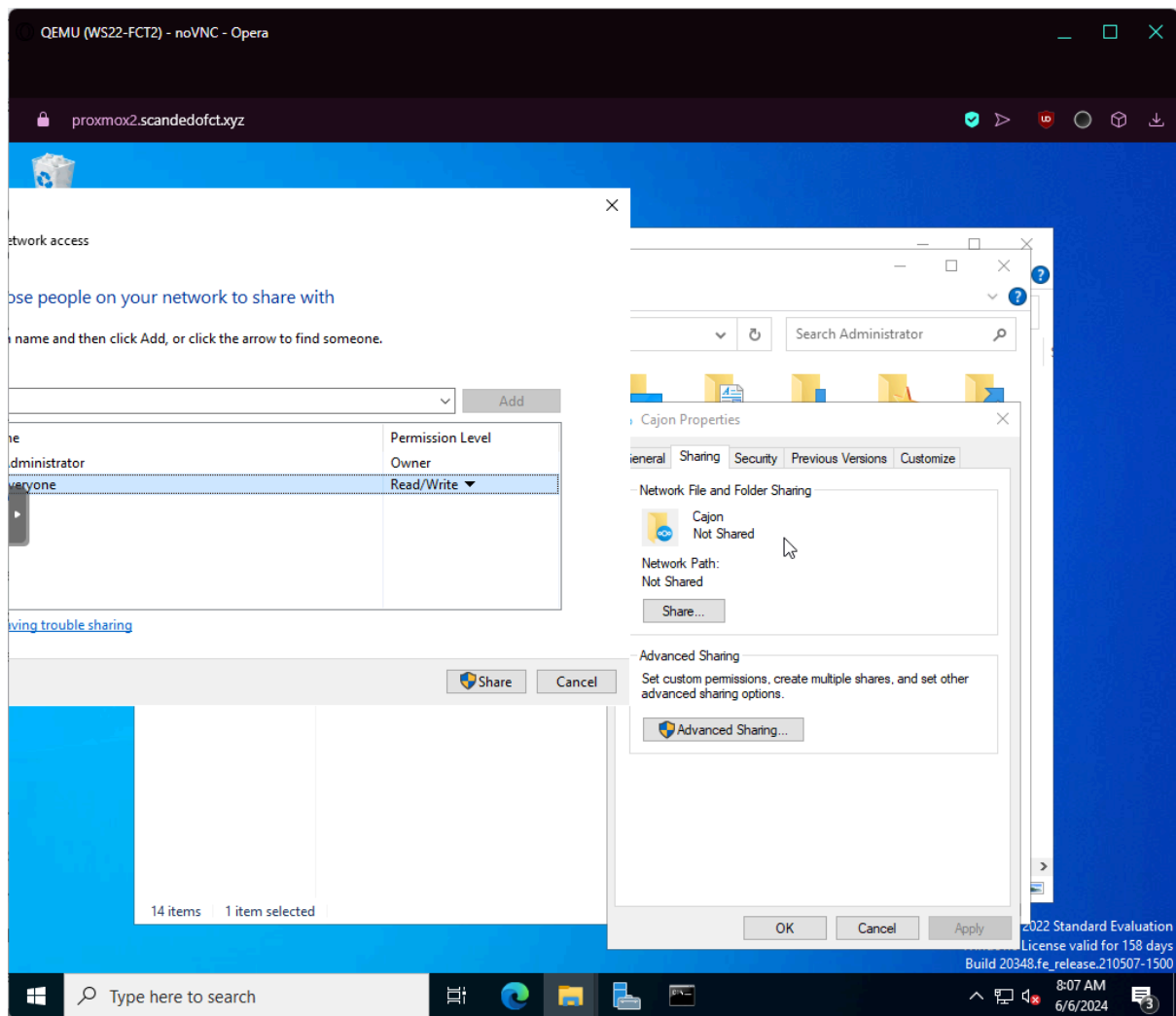
Debemos concederle acceso desde el navegador



Una vez terminada sincronización podemos decidir que carpetas queremos sincronizar en el equipo, en mi caso cajon y la deajo en C:\Users\Administrador\Cajon



Podemos usar smb para compartir esta misma carpeta en toda la red de Windows forma que los clientes puedan sincronizar la carpeta por smb y anclarla a una interfaz de disco, por ejemplo N:

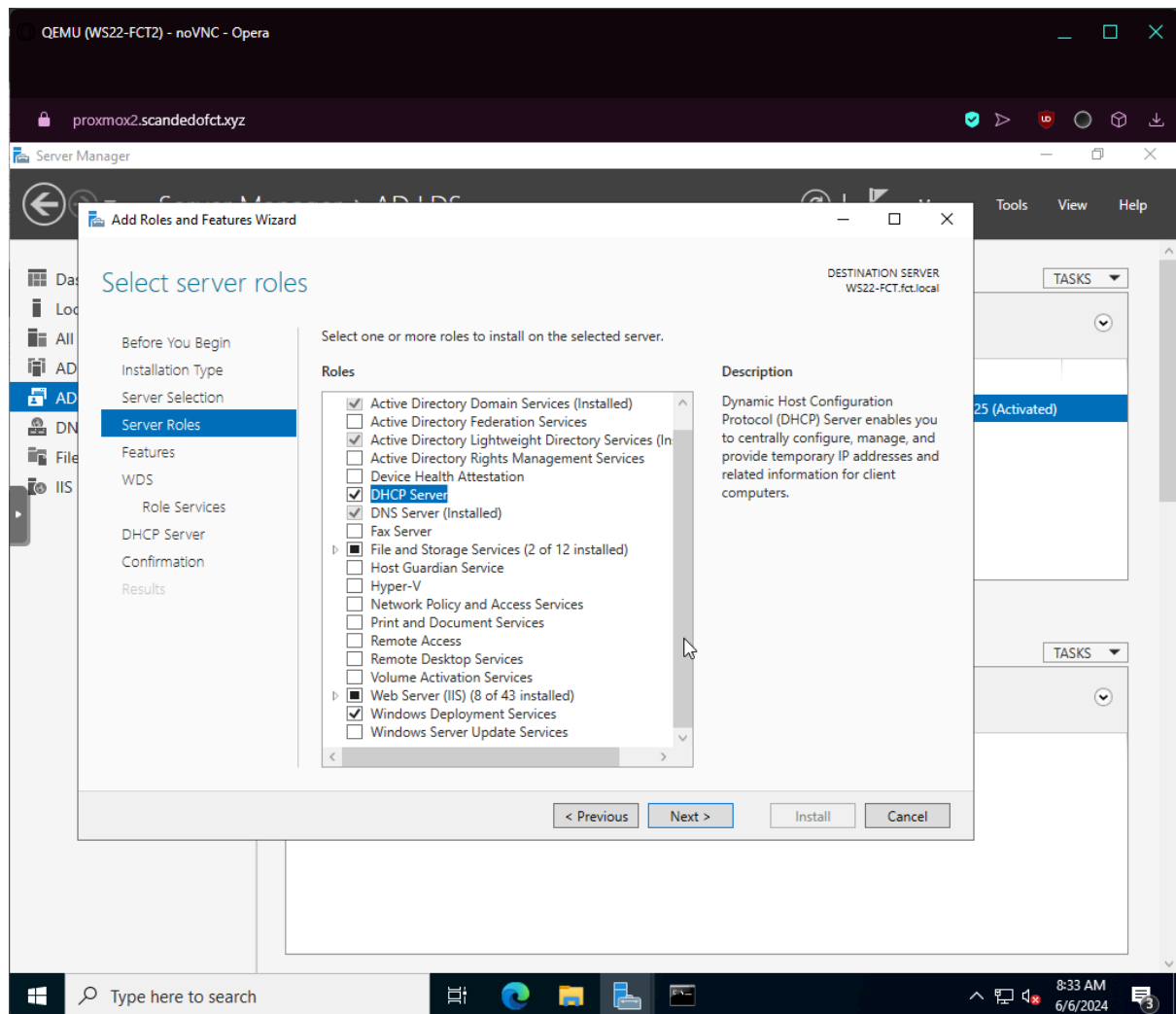


Queda compartida la carpeta en la red con la ruta \\WS22FCT\Cajon

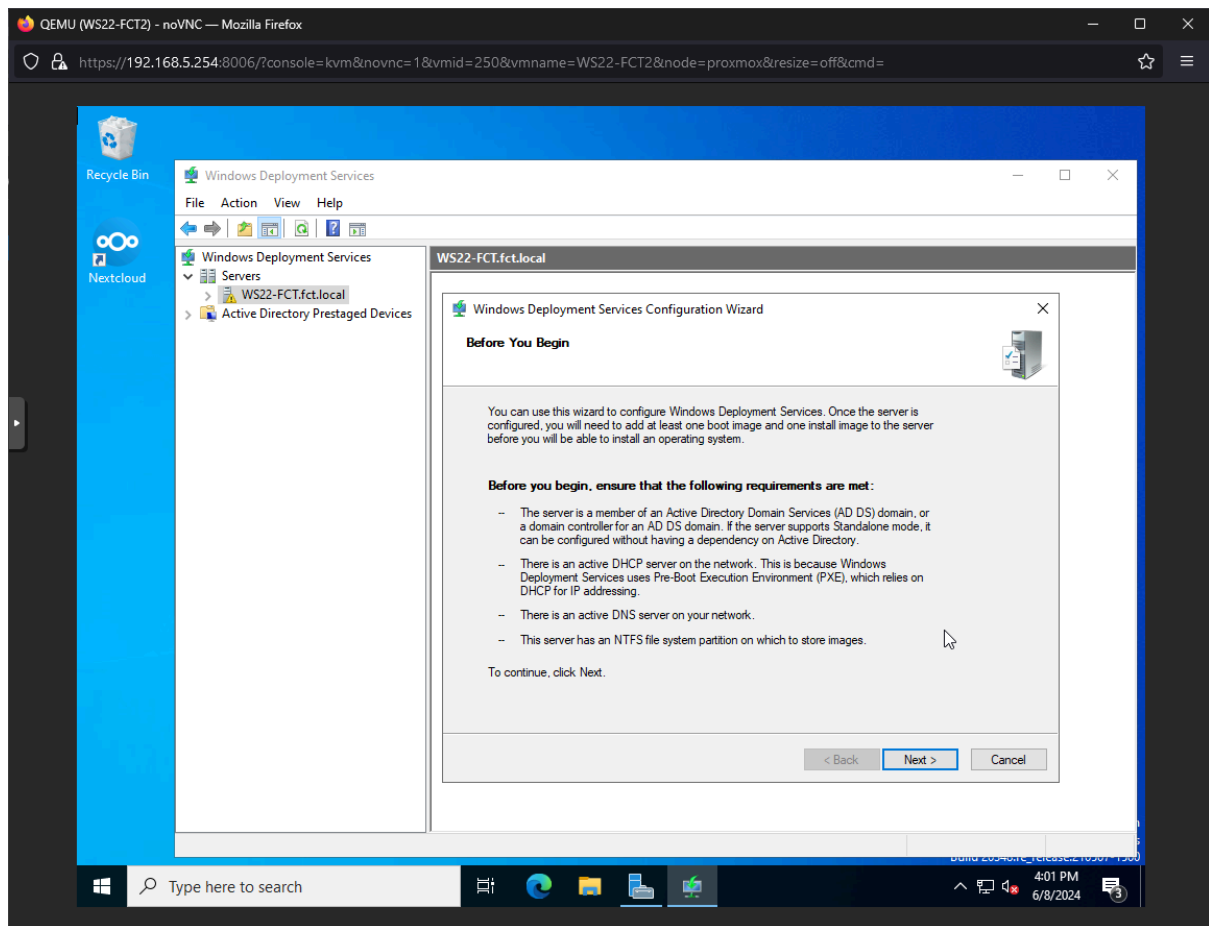
Instalación de los otros puestos por red WDS

Para el despliegue de dispositivos con tecnología PBX voy a seguir la siguiente [documentación](#)

Debemos configurar el Servidor DHCP para que windows server de ips a los equipos clientes y debemos también instalar el Windows Deployment Services

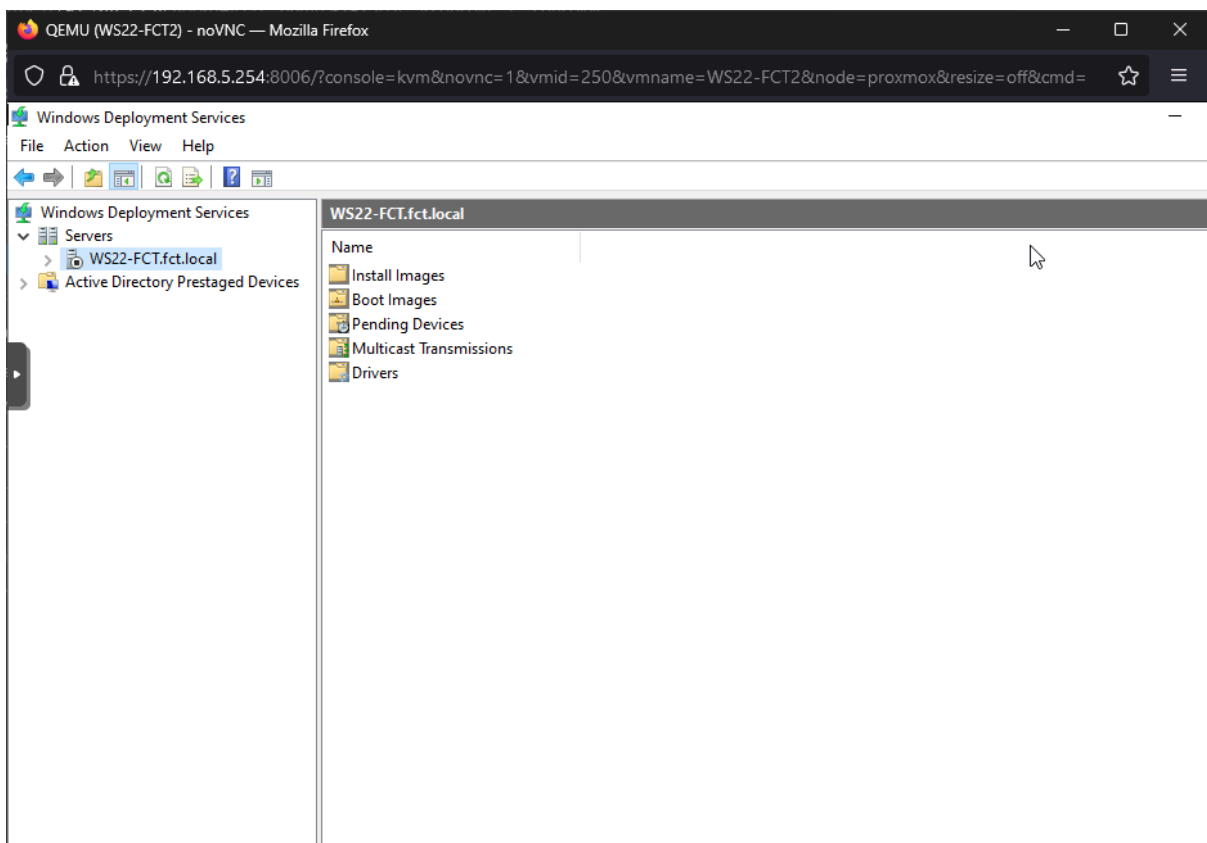
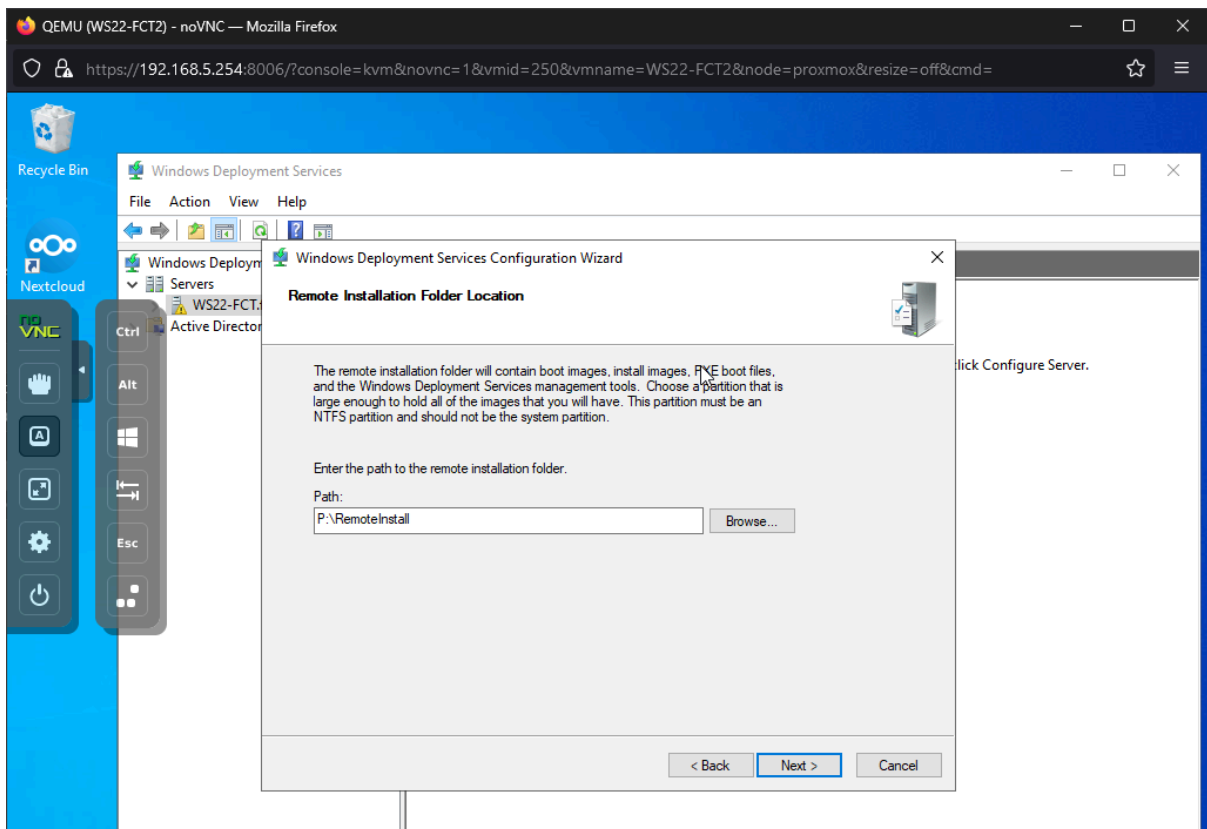


En la consola de WDS, nos dirigimos a "Configurar servidor".

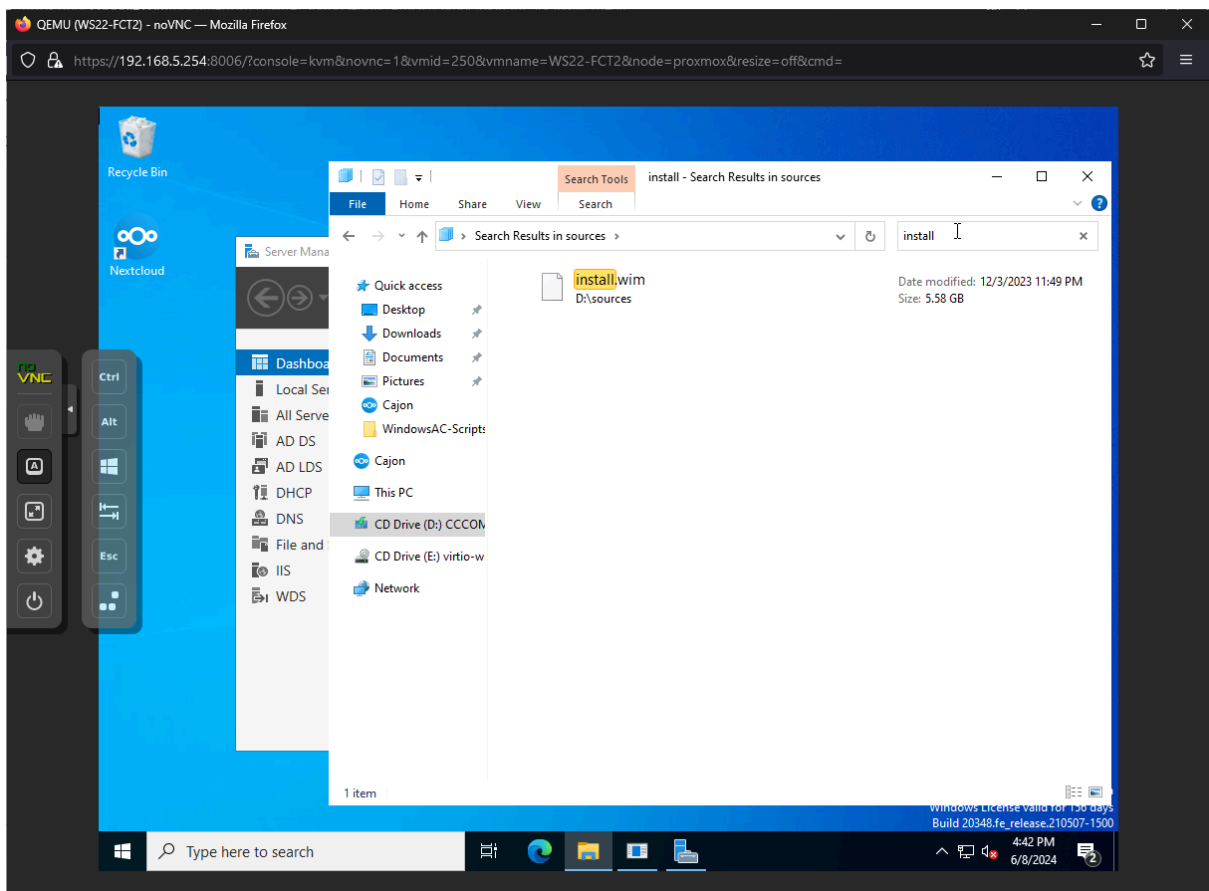
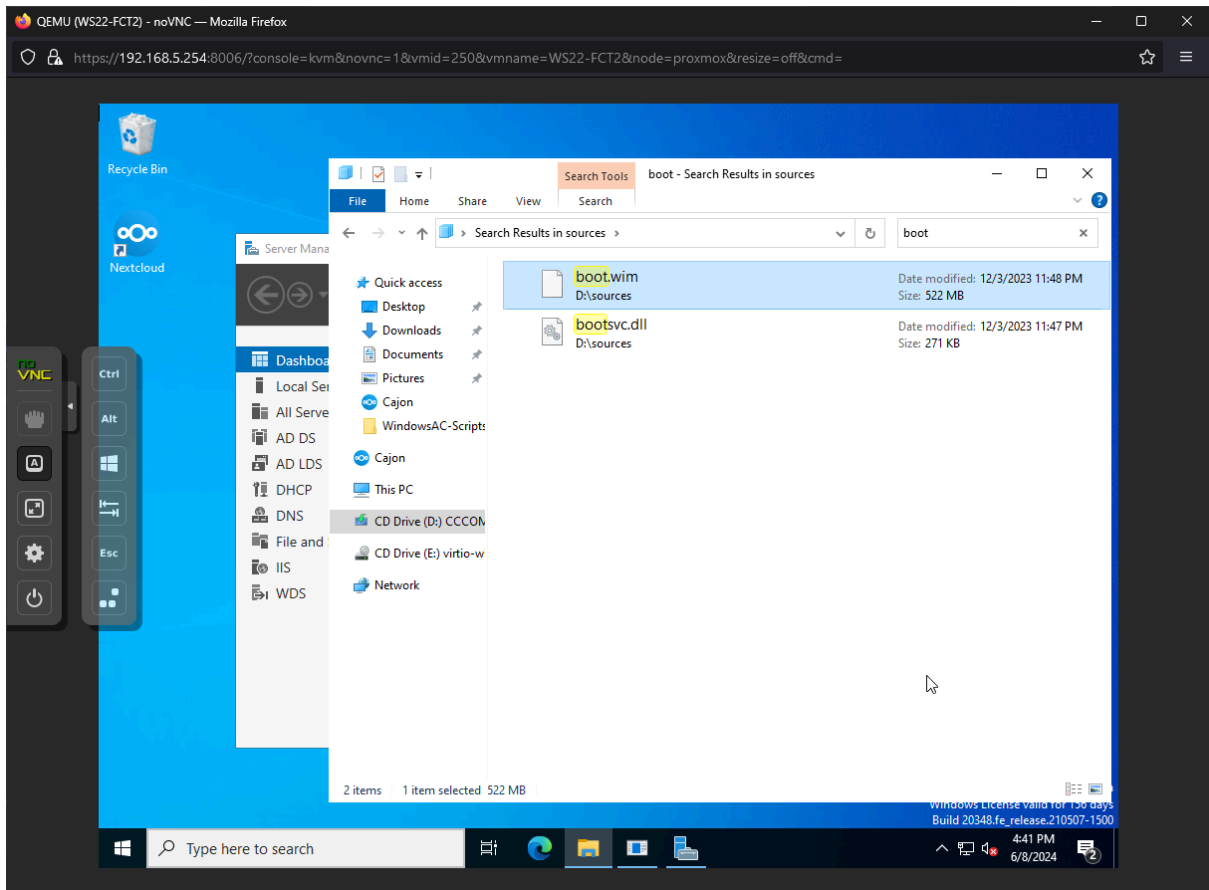


Ubicación de instalación: Especifica la ubicación en la que deseas almacenar las imágenes (por ejemplo, R:\RemoteInstall que será el disco).

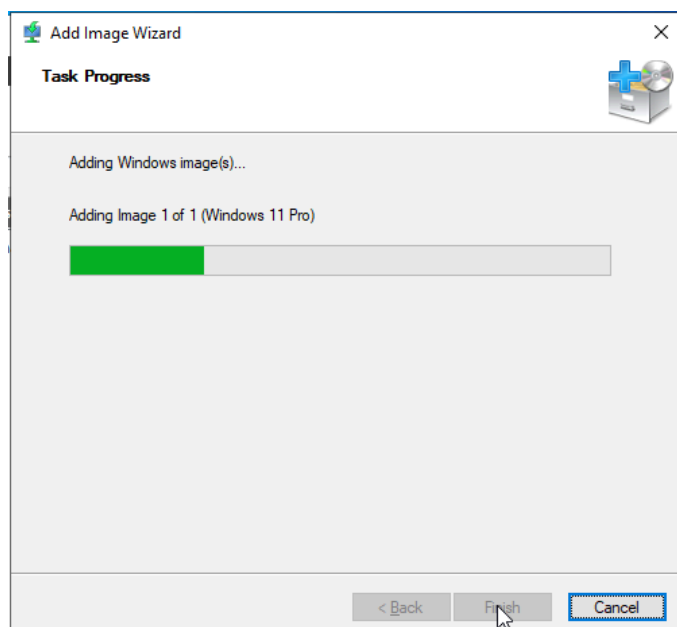
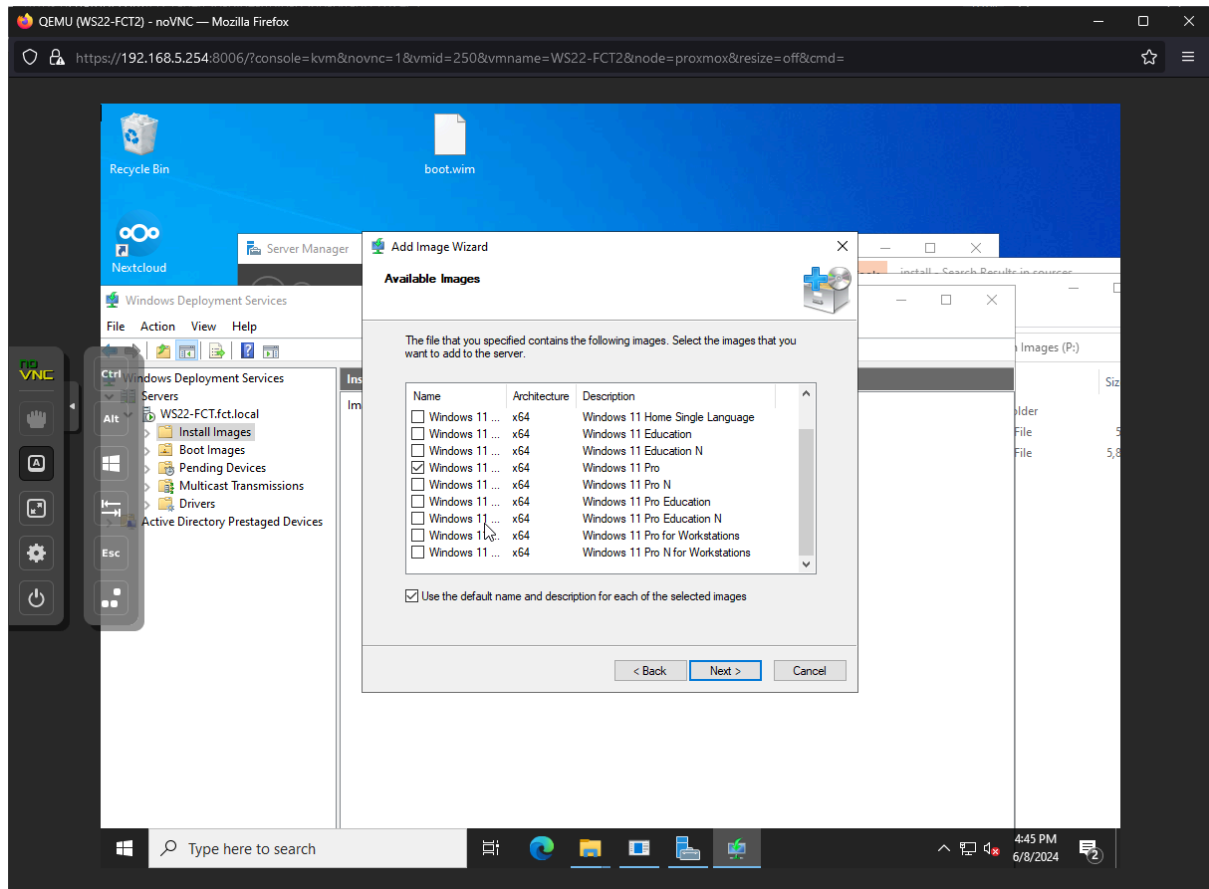
Opciones de PXE: seleccionamos "Responder a todos los clientes (conocidos y desconocidos)".



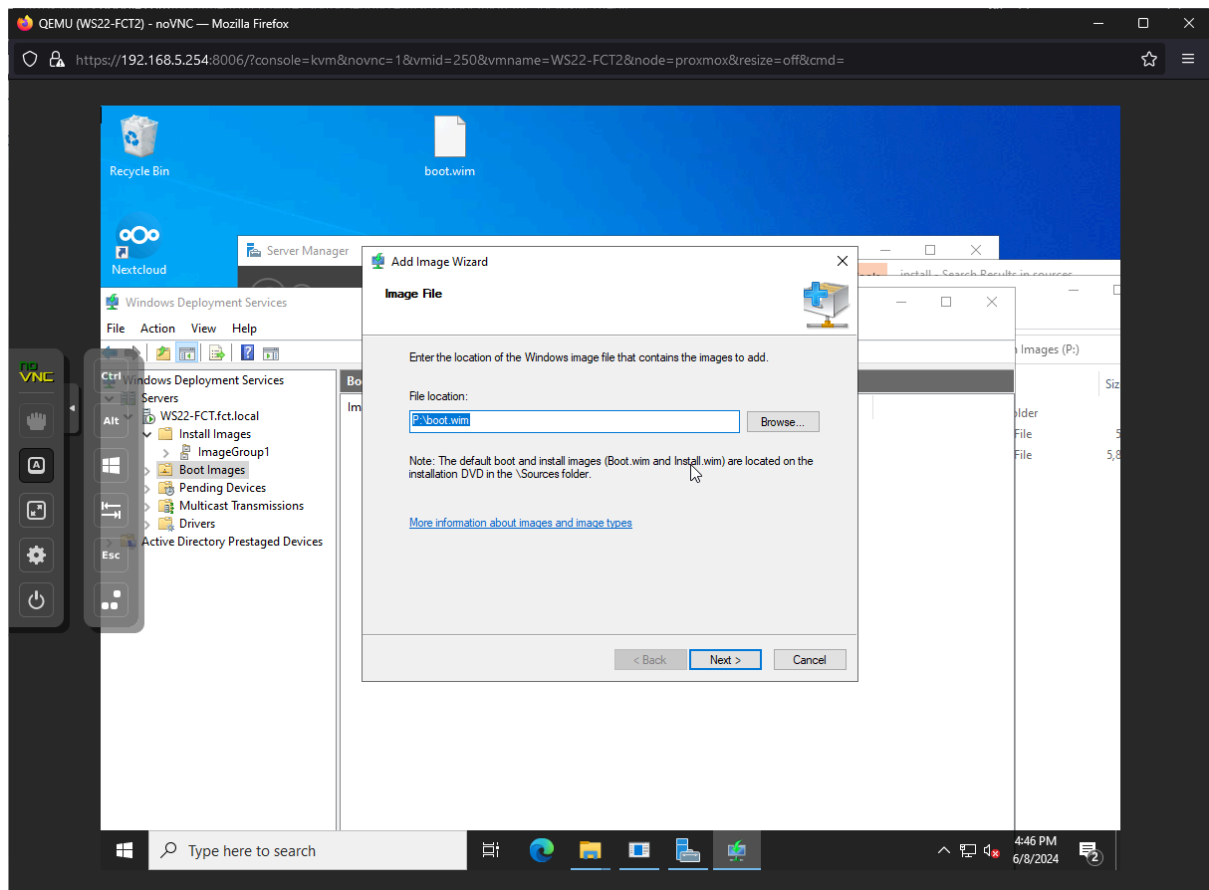
Una vez realizada la configuración debemos obtener los archivos install.wim y boot.wim dentro de una imagen de windows



Añadiendo la imagen de instalación dentro del administrador de WDS - Install Images nos pregunta que versiones de windows queremos, seleccionamos windows 11 pro ya que vamos a usar escritorio remoto

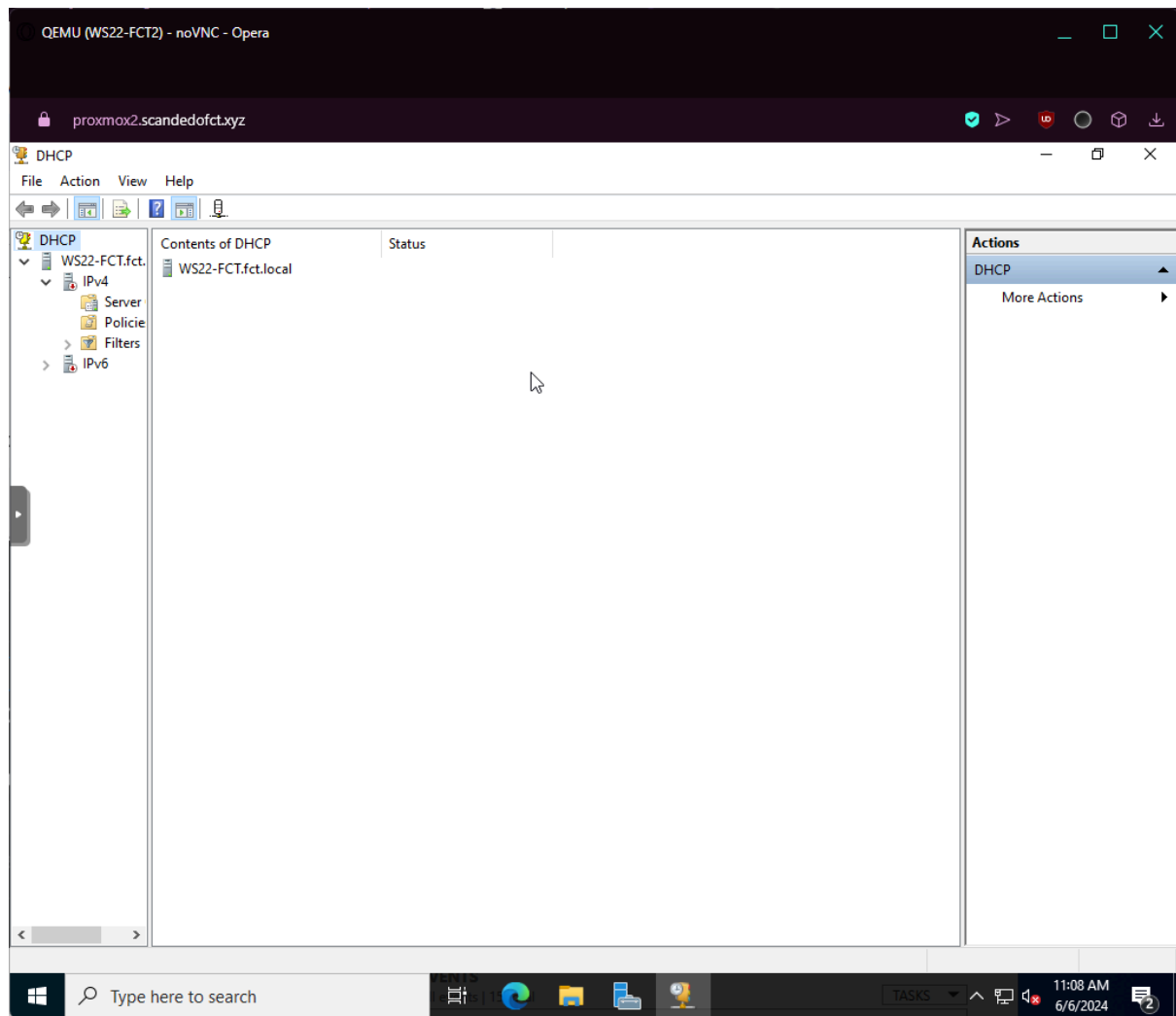


Añadimos también boot.wim



Configuración del Servidor DHCP

Vamos a configurar el servidor DHCP en windows server al igual que hicimos en clase



Vamos a establecer nuestro rango de IPS para que reciban los clientes, en mi caso de la 192.168.20.50 a la 192.168.20.100

New Scope Wizard

IP Address Range
You define the scope address range by identifying a set of consecutive IP addresses.

Configuration settings for DHCP Server

Enter the range of addresses that the scope distributes.

Start IP address: 192 . 168 . 20 . 50

End IP address: 192 . 168 . 20 . 100

Configuration settings that propagate to DHCP Client

Length: 24

Subnet mask: 255 . 255 . 255 . 0

< Back Next > Cancel

Vamos a necesitar un disco extra donde guardemos las imágenes para instalaciones por el WDS

Para ello añadido un nuevo disco desde la interfaz de Proxmox

Add: Hard Disk

Disk Bandwidth

Bus/Device: SCSI 1 Cache: Default (No cache)

SCSI Controller: VirtIO SCSI single Discard: ☐

Storage: local-lvm IO thread: ☒

Disk size (GiB): 50

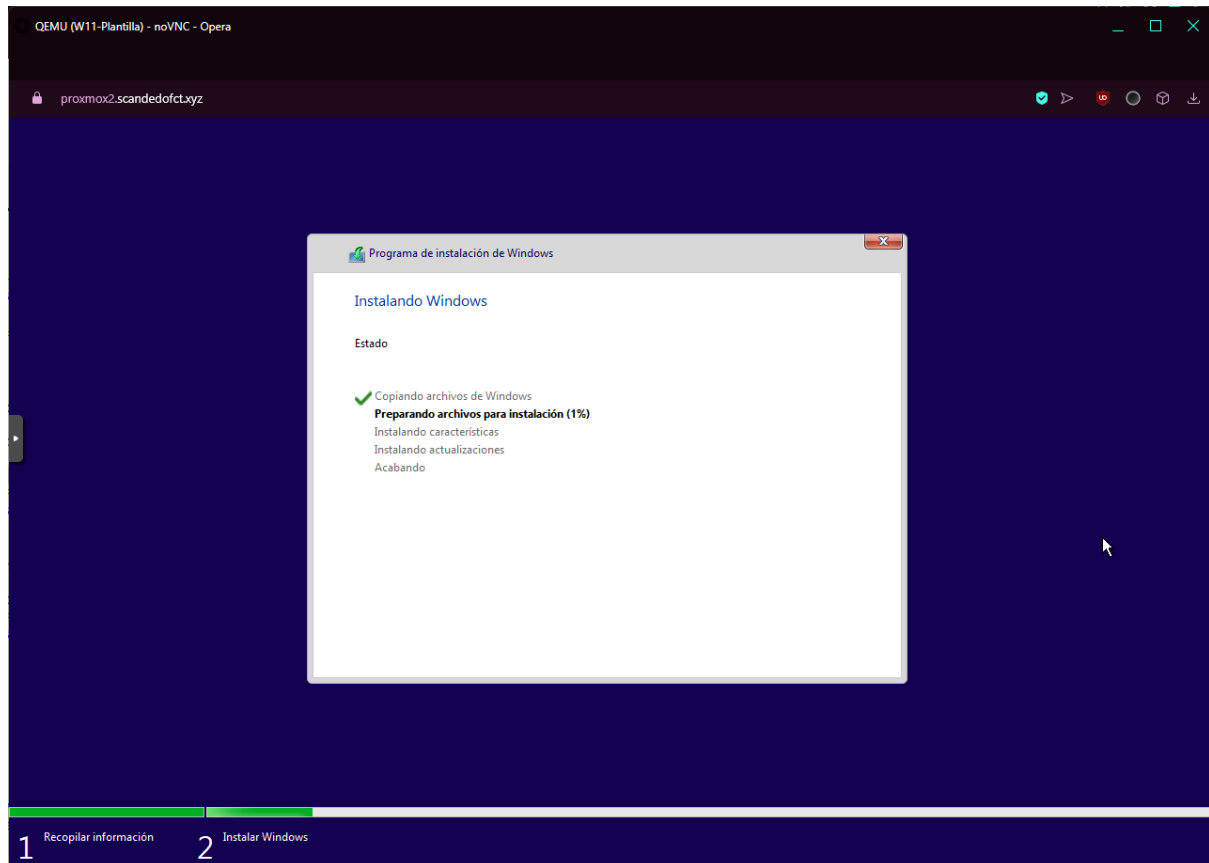
Format: Raw disk image (raw)

? Help Advanced ☐ Add

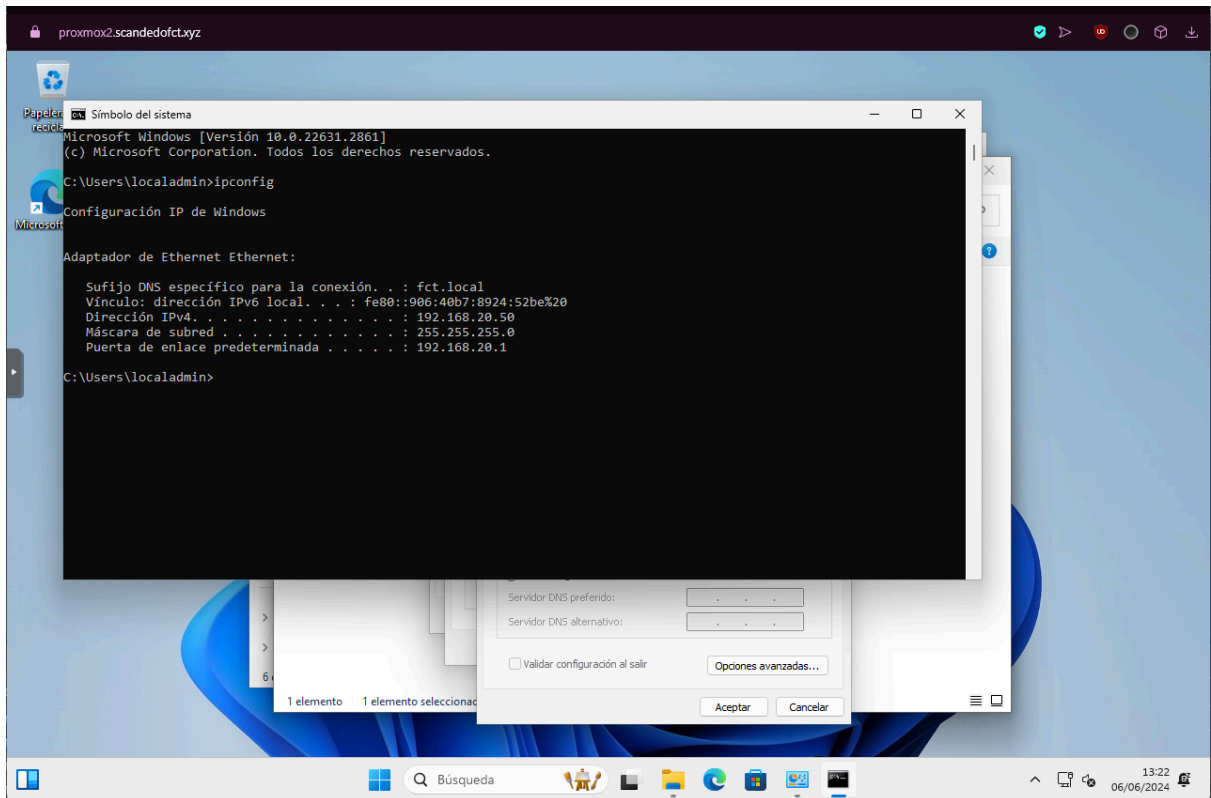
Equipo plantilla.

Vamos a instalar un Windows 11 cómo plantilla para instalar nuevos equipos por red posteriormente. Esta plantilla va a contener todos los programas necesarios para el uso empresarial y no se permitirá por medio de GPOS la instalación de software extra cómo medida de seguridad

Descargo la Imagen Iso de Windows y empiezo la instalación de un Windows11 Pro con normalidad

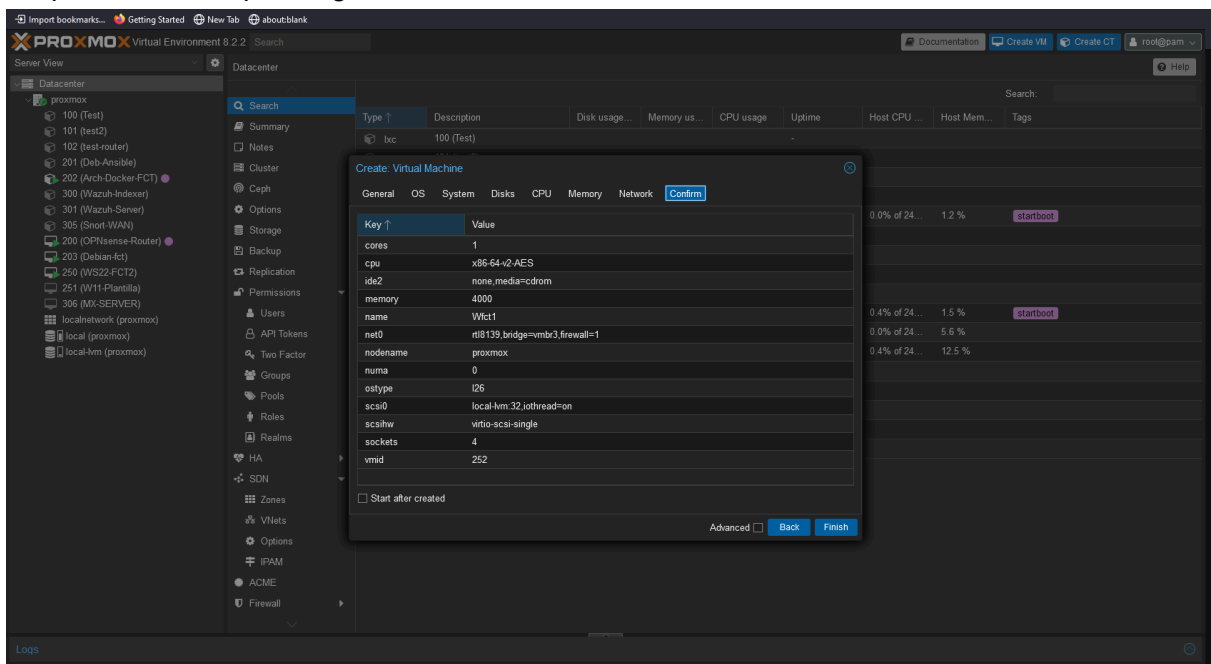


Una vez terminada la instalación reviso que el servidor DHCP esté funcionando correctamente. Sin tocar nada, le asigna la ip 192.168.20.50

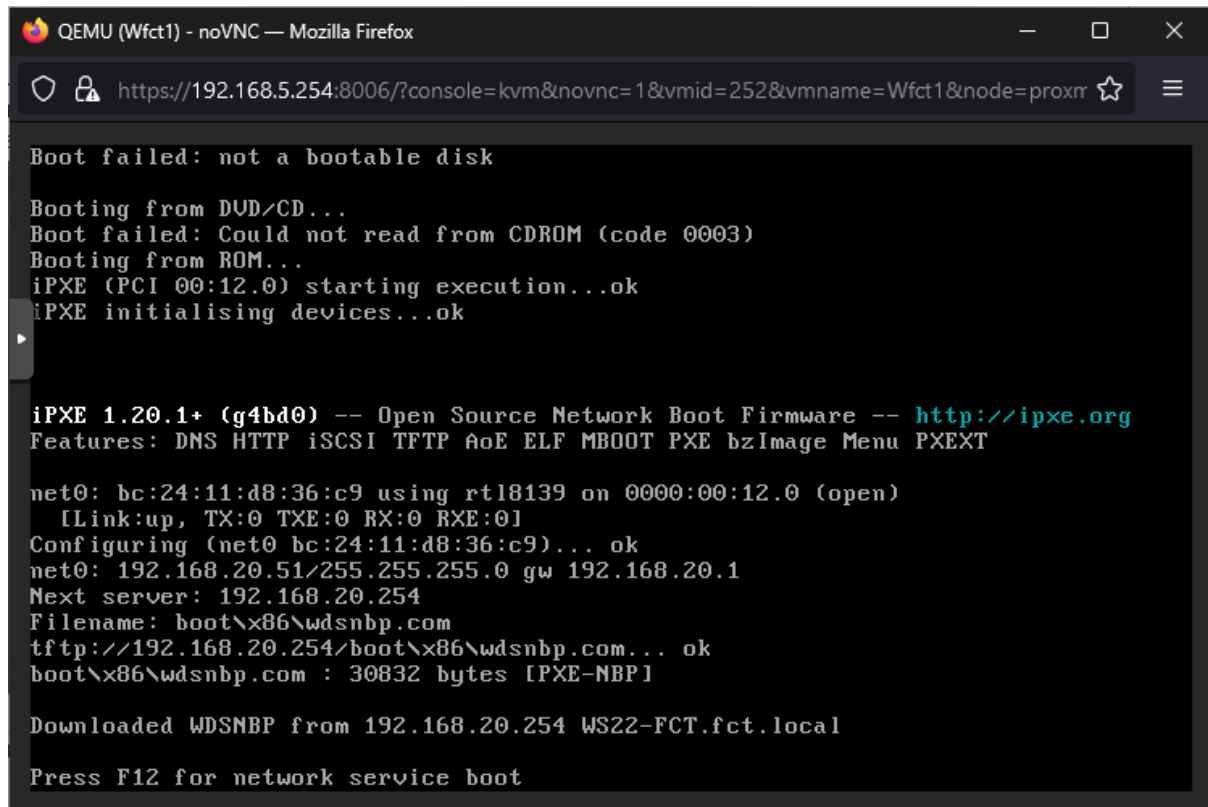


Para la instalación de los programas necesarios para el sistema operativo utilizo ninite como herramienta para el despliegue de software de forma segura y rapida

Una vez instalado el servidor DHCP y WDS y extraido el boot.wim y el install.wim e insertados en el servidor WDS podemos añadir una máquina en proxmox sólo con un adaptador de red que tenga conexión con la red interna FCT2 la de windows.



Iniciamos la máquina en pbx para ver si carga la imagen de windows



```
QEMU (Wfct1) - noVNC — Mozilla Firefox
https://192.168.5.254:8006/?console=kvm&novnc=1&vmid=252&vmname=Wfct1&node=proxmox

Boot failed: not a bootable disk

Booting from DVD/CD...
Boot failed: Could not read from CDROM (code 0003)
Booting from ROM...
iPXE (PCI 00:12.0) starting execution...ok
iPXE initialising devices...ok

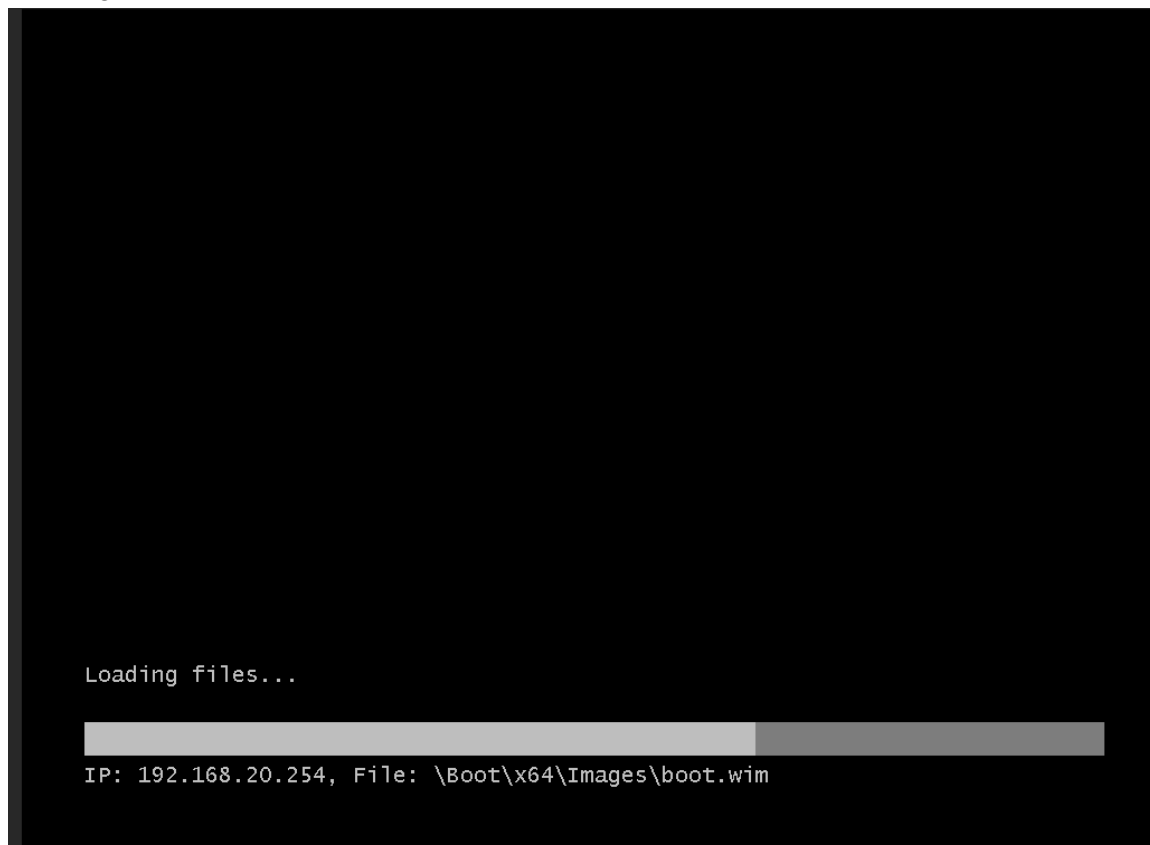
iPXE 1.20.1+ (g4bd0) -- Open Source Network Boot Firmware -- http://ipxe.org
Features: DNS HTTP iSCSI TFTP AoE ELF MBOOT PXE bzImage Menu PXEXT

net0: bc:24:11:d8:36:c9 using rtl8139 on 0000:00:12.0 (open)
[Link:up, TX:0 TXE:0 RX:0 RXE:0]
Configuring (net0 bc:24:11:d8:36:c9)... ok
net0: 192.168.20.51/255.255.255.0 gw 192.168.20.1
Next server: 192.168.20.254
Filename: boot\x86\wdsnbp.com
tftp://192.168.20.254/boot\x86\wdsnbp.com... ok
boot\x86\wdsnbp.com : 30832 bytes [IPXE-NBP]

Downloaded WDSNBP from 192.168.20.254 WS22-FCT.fct.local

Press F12 for network service boot
```

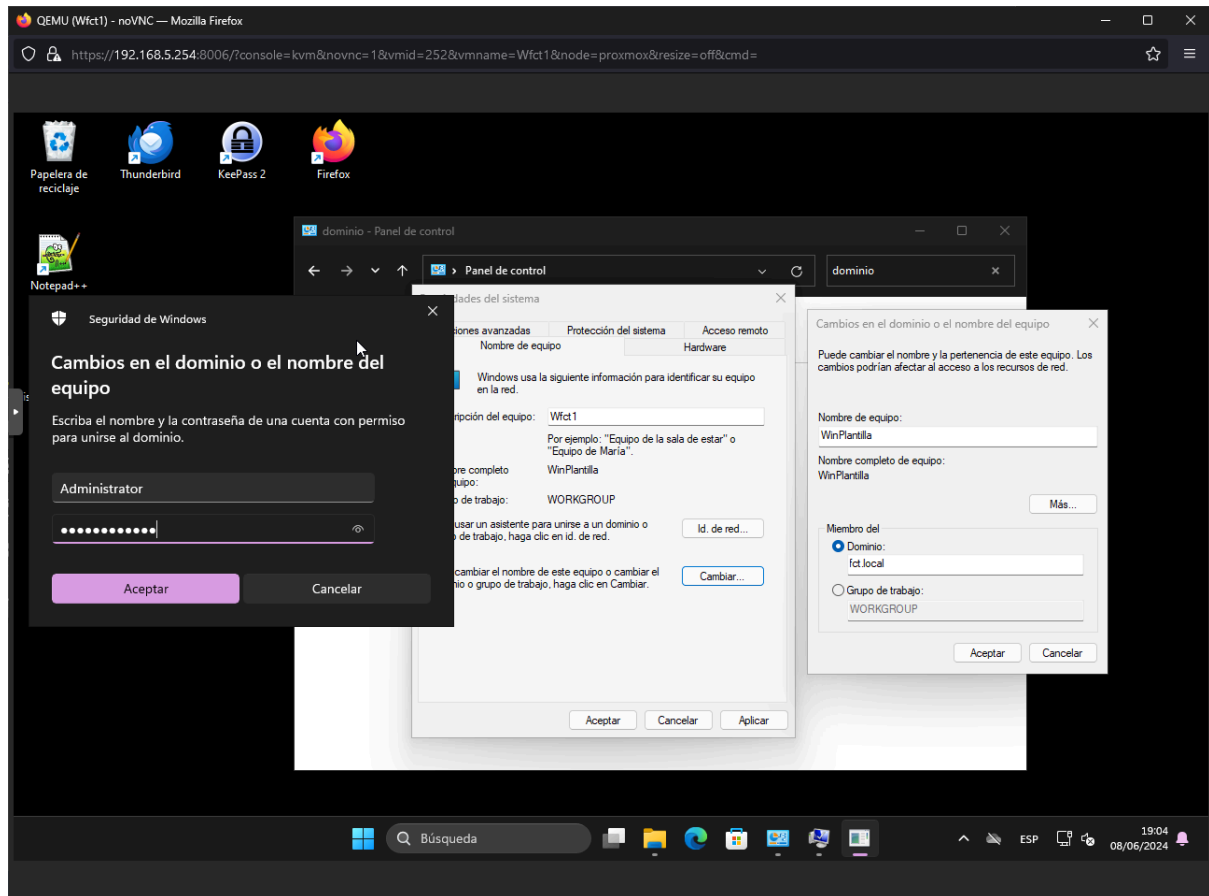
Recibimos una conexión por parte del servidor windows , pulsamos f12 para empezar con el despliegue



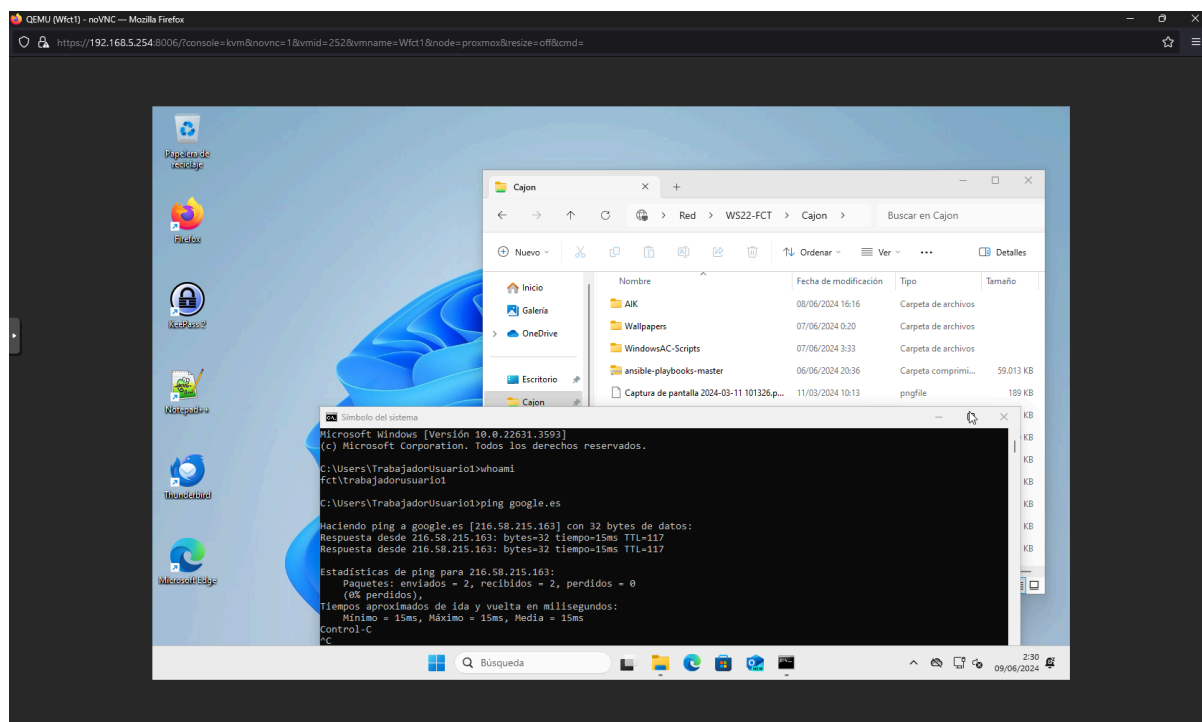
```
Loading files...

IP: 192.168.20.254, File: \Boot\x64\Images\boot.wim
```

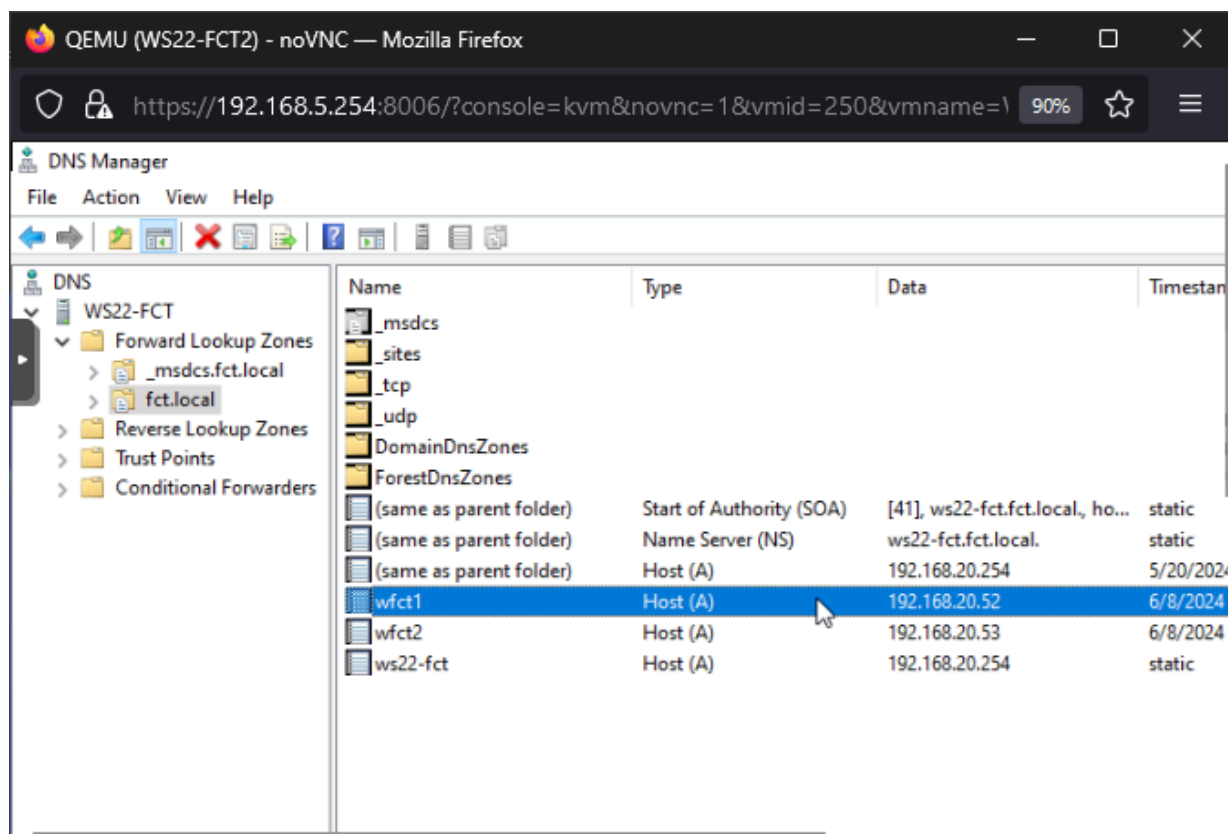
Unir al dominio



Una vez unido al dominio probamos iniciar sesión cómo uno de los usuarios y verificamos que todo funciona correctamente



También podemos comprobar que se añadió el registro dns del equipo en el servidor dns.



GPO Para conexiones remotas a través de la VPN

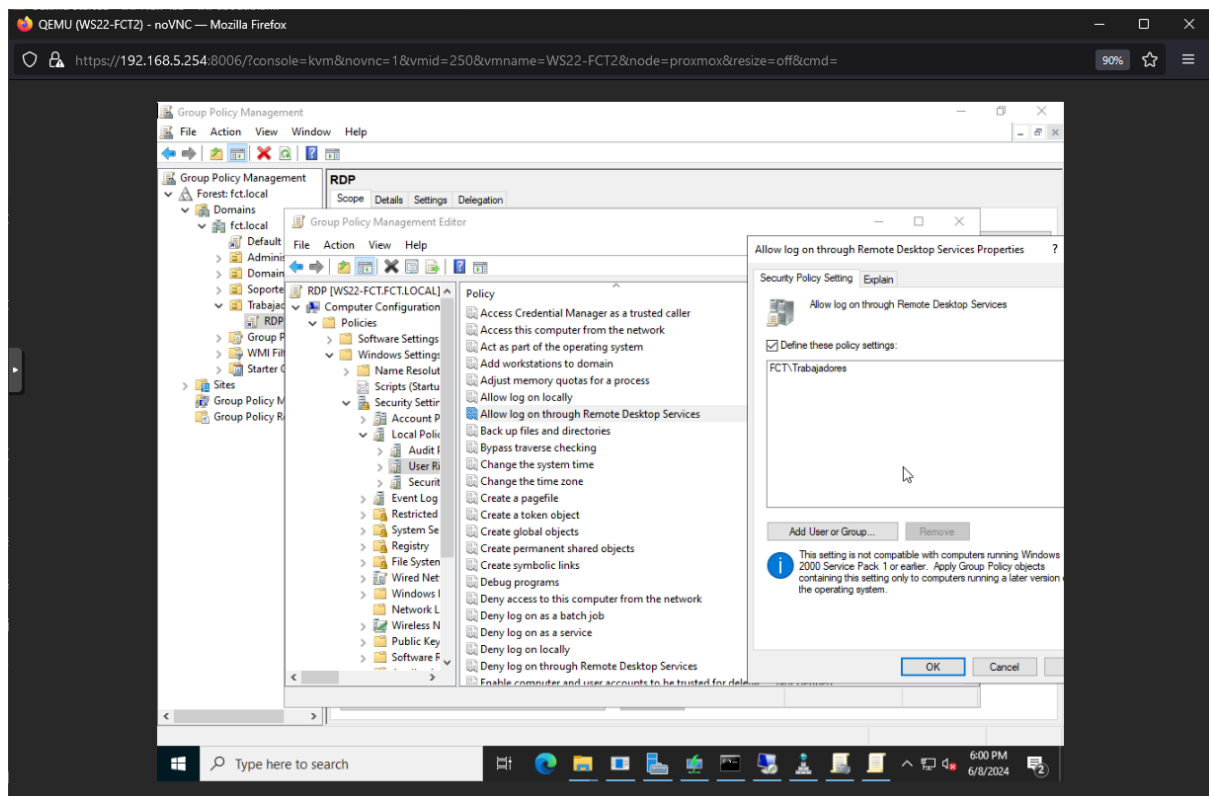
Vamos a crear una GPO que nos permita acceder por remoto a nuestros equipos windows con las cuentas de trabajadores

Abrimos el Editor de administración de directivas de grupo:

Configuración del equipo > Directivas > Configuración de Windows > Configuración de seguridad > Directivas locales > Asignación de derechos de usuario.

Doble clic en Permitir el inicio de sesión a través de Servicios de Escritorio Remoto.

Luego añadimos el grupo al cual queremos que se le aplique la política que se asignara localmente en los ordenadores del dominio:



Servidor Proxy

Para la prueba de conexión remota con los puestos de windows vamos a necesitar un servidor proxy para que pase todo nuestro tráfico a fuera de la red a ese servidor

Para ello me conecto a mi servidor de AWS de amazon por ssh, donde previamente configuré una Dns dinámica para que apunte siempre a la ip del servidor de la nube (que no es estática)

configuramos squid

```
sudo nano /etc/squid/squid.conf
```

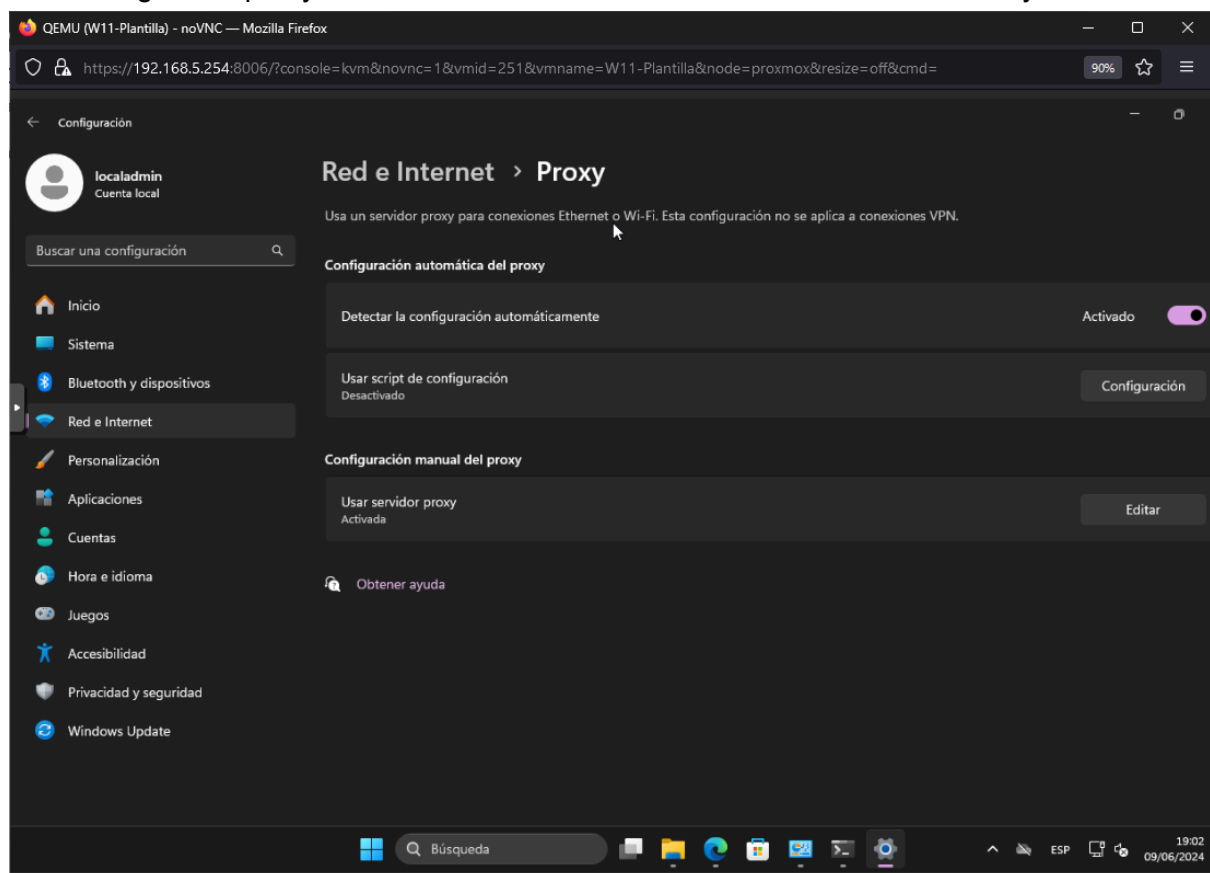
```
Select ec2-user@ip-172-31-7-113:~
GNU nano 5.8 /etc/squid/squid.conf Modified
coredump_dir /var/spool/squid

#
# Add any of your own refresh_pattern entries above these.
#
refresh_pattern ^ftp:      1440      20%      10080
refresh_pattern -i (/cgi-bin/|\?) 0     0%      0
refresh_pattern .          0         20%     4320

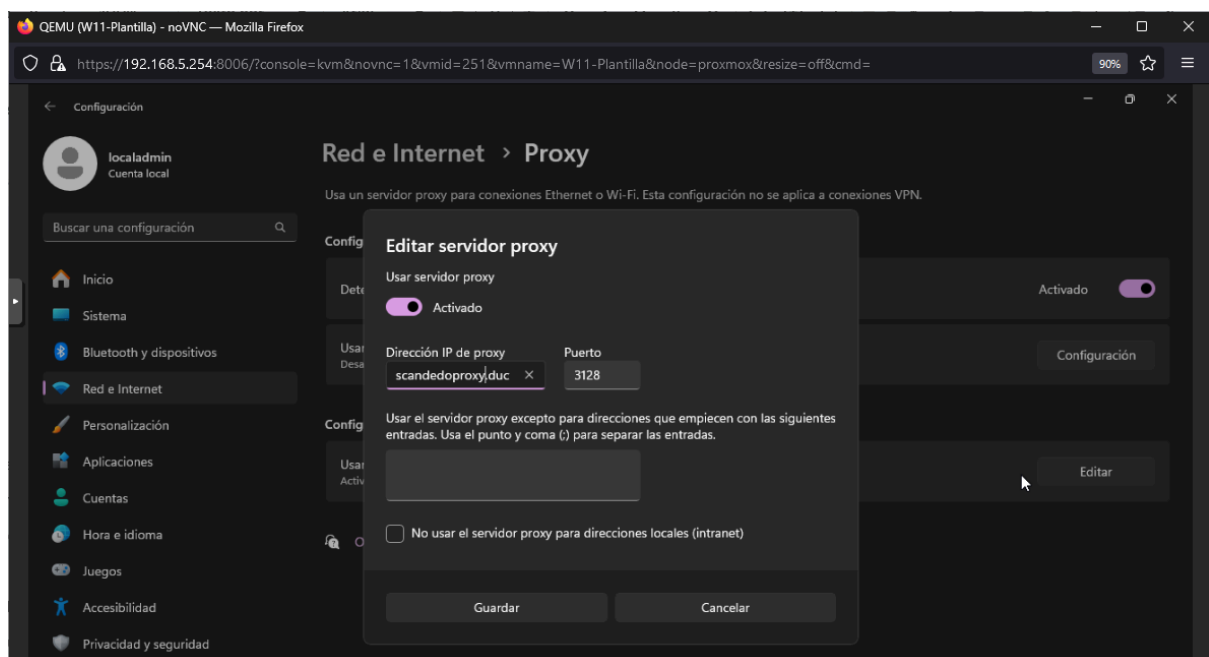
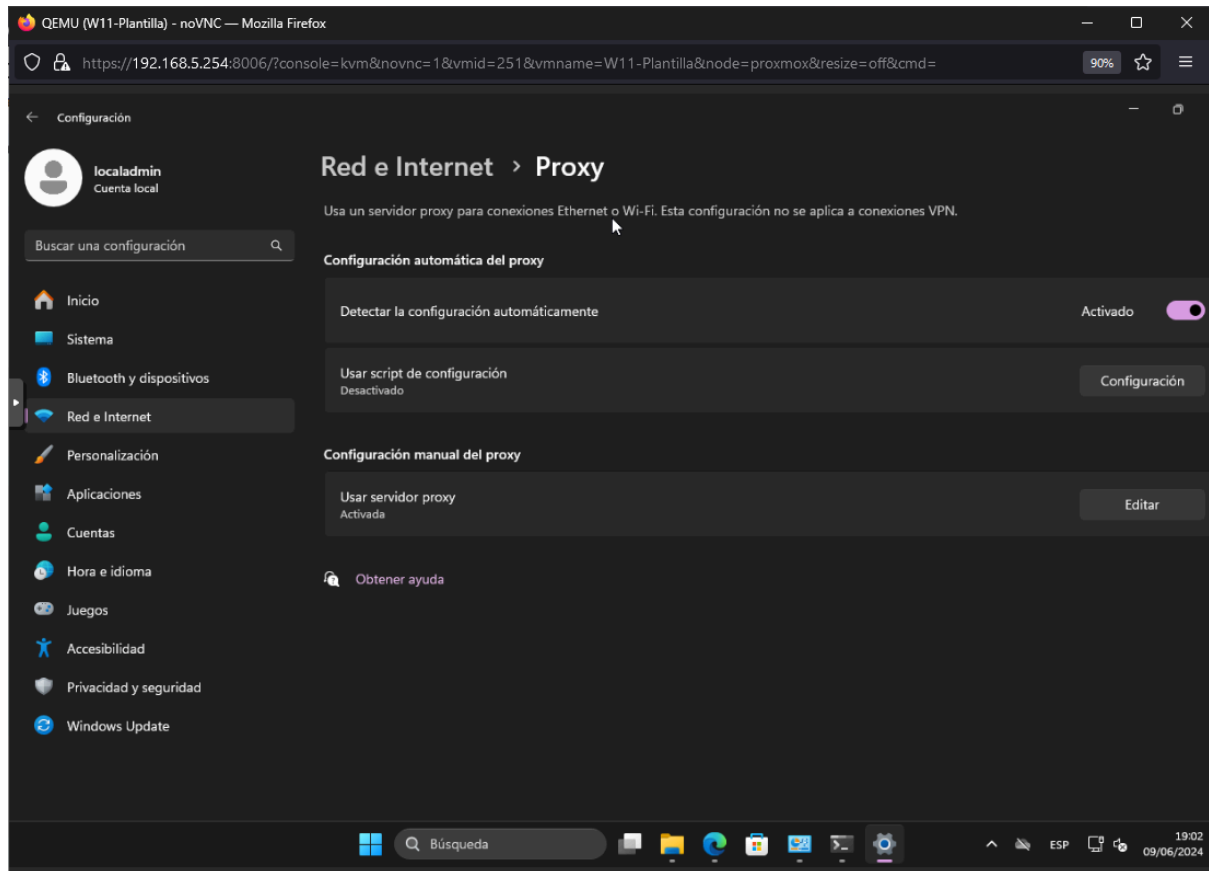
### Red Prueba windows
acl localnet src 20.20.20.0/24
http_access allow localnet
```

configuramos el proxy en windows

Para configurar el proxy en windows debemos acceder a Red e internet - Proxy



Abajo en servidorproxy debemos indicarle la dirección y el puerto del servidor proxy así como otras direcciones en las que **no** queremos que use proxy, cómo nos interesa que enrute todo el tráfico a través del proxy pues no ponemos nada



capturo paquetes con tcpdump para ver si le llegan las peticiones

```
sudo tcpdump -i any port 3128
```

```

ec2-user@ip-172-31-7-113:~$ sudo tcpdump -i eth0 -s 0 -w -
17:01:10.188496 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55902: Flags [S.], seq 1289040977, ack 2212653685, win 62727,
  options [mss 8961,nop,nop,sackOK,nop,wscale 7], length 0
17:01:10.191819 enX0 In IP 213.177.213.209.55901 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 1, win 513, options [nop,nop,sack 1 {18
51:3311}], length 0
17:01:10.191820 enX0 In IP 213.177.213.209.55901 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 391, win 511, options [nop,nop,sack 1 {
1851:3311}], length 0
17:01:10.191820 enX0 In IP 213.177.213.209.55901 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 3311, win 513, length 0
17:01:10.192750 enX0 In IP 213.177.213.209.55901 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [F.], seq 231, ack 3311, win 513, length 0
17:01:10.192750 enX0 In IP 213.177.213.209.55901 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [R.], seq 232, ack 4084, win 0, length 0
17:01:10.219380 enX0 In IP 213.177.213.209.55902 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [P.], seq 1:231, ack 1, win 513, length 230
17:01:10.219380 enX0 In IP 213.177.213.209.55902 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 1, win 513, length 0
17:01:10.219403 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55902: Flags [.], ack 231, win 489, length 0
17:01:10.219419 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55902: Flags [.], ack 231, win 489, length 0
17:01:10.219760 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55902: Flags [P.], seq 1:391, ack 231, win 489, length 390
17:01:10.219802 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55902: Flags [P.], seq 391:3311, ack 231, win 489, length 292
0
17:01:10.219914 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55902: Flags [P.], seq 3311:4084, ack 231, win 489, length 77
3
17:01:10.249825 enX0 In IP 213.177.213.209.55902 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 1851, win 513, length 0
17:01:10.250323 enX0 In IP 213.177.213.209.55902 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [F.], seq 231, ack 1851, win 513, length 0
17:01:10.250349 enX0 In IP 213.177.213.209.55902 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [R.], seq 232, ack 3311, win 0, length 0
17:01:10.304934 enX0 In IP 213.177.213.209.55903 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [S], seq 3020026349, win 64240, options [mss 146
0,nop,wscale 8,nop,nop,sackOK], length 0
17:01:10.304965 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55903: Flags [S.], seq 1607562292, ack 3020026350, win 62727,
  options [mss 8961,nop,nop,sackOK,nop,wscale 7], length 0
17:01:10.340417 enX0 In IP 213.177.213.209.55903 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 1, win 513, length 0
17:01:10.340849 enX0 In IP 213.177.213.209.55903 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [P.], seq 1:243, ack 1, win 513, length 242
17:01:10.340861 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55903: Flags [.], ack 243, win 489, length 0
17:01:10.343245 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55903: Flags [P.], seq 1:391, ack 243, win 489, length 390
17:01:10.343297 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55903: Flags [P.], seq 391:3311, ack 243, win 489, length 292
0
17:01:10.343443 enX0 Out IP ip-172-31-7-113.eu-west-3.compute.internal.squid > 213.177.213.209.55903: Flags [P.], seq 3311:4102, ack 243, win 489, length 79
1
17:01:10.377925 enX0 In IP 213.177.213.209.55903 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 1851, win 513, length 0
17:01:10.377925 enX0 In IP 213.177.213.209.55903 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [.], ack 4102, win 513, length 0
17:01:10.378855 enX0 In IP 213.177.213.209.55903 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [F.], seq 243, ack 4102, win 513, length 0
17:01:10.378855 enX0 In IP 213.177.213.209.55903 > ip-172-31-7-113.eu-west-3.compute.internal.squid: Flags [R.], seq 244, ack 4102, win 0, length 0

```

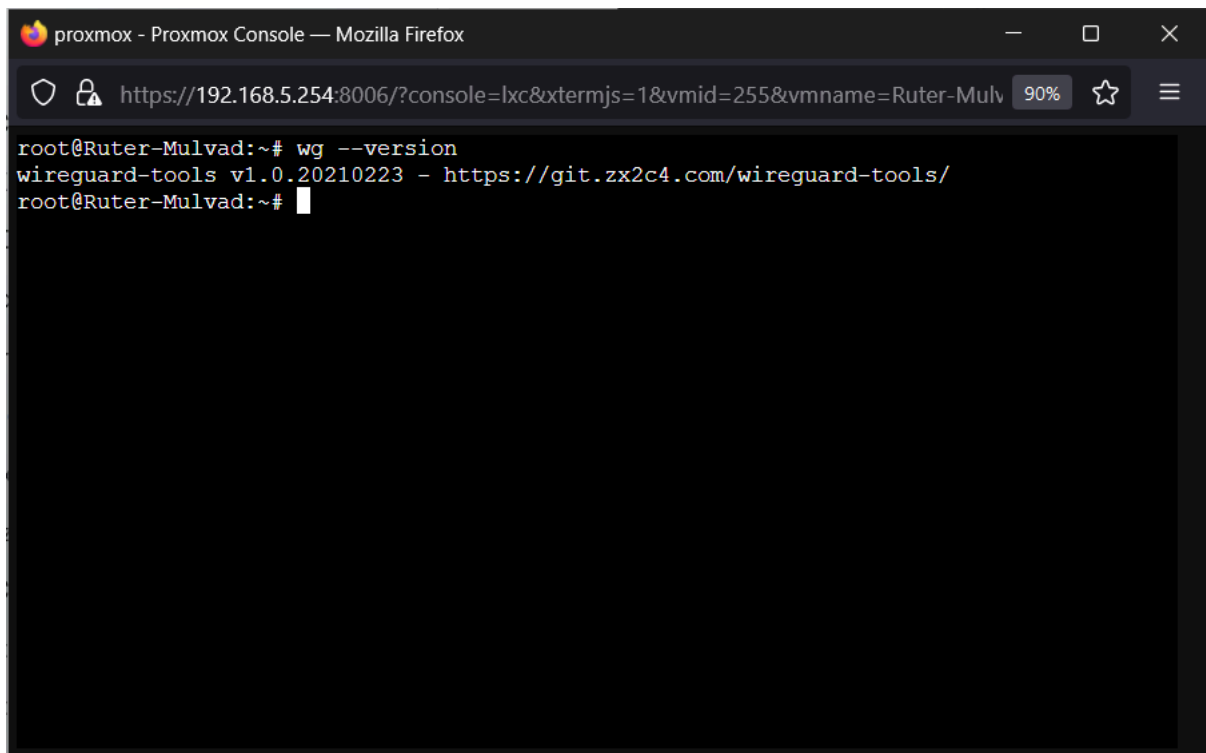
Rúter con Mullvad

Además del servidor proxy vamos a montar un rúter con la **vpn de mulvad** para que nuestro tráfico se redirija de forma segura y privada al exterior

para ello creo una máquina debian con proxmox

Descargo la wireguard-tools . Vamos a configurar un servidor de wireguard, igual que hicimos en el apartado [de la vpn](#) pero esta vez sin interfaz gráfica

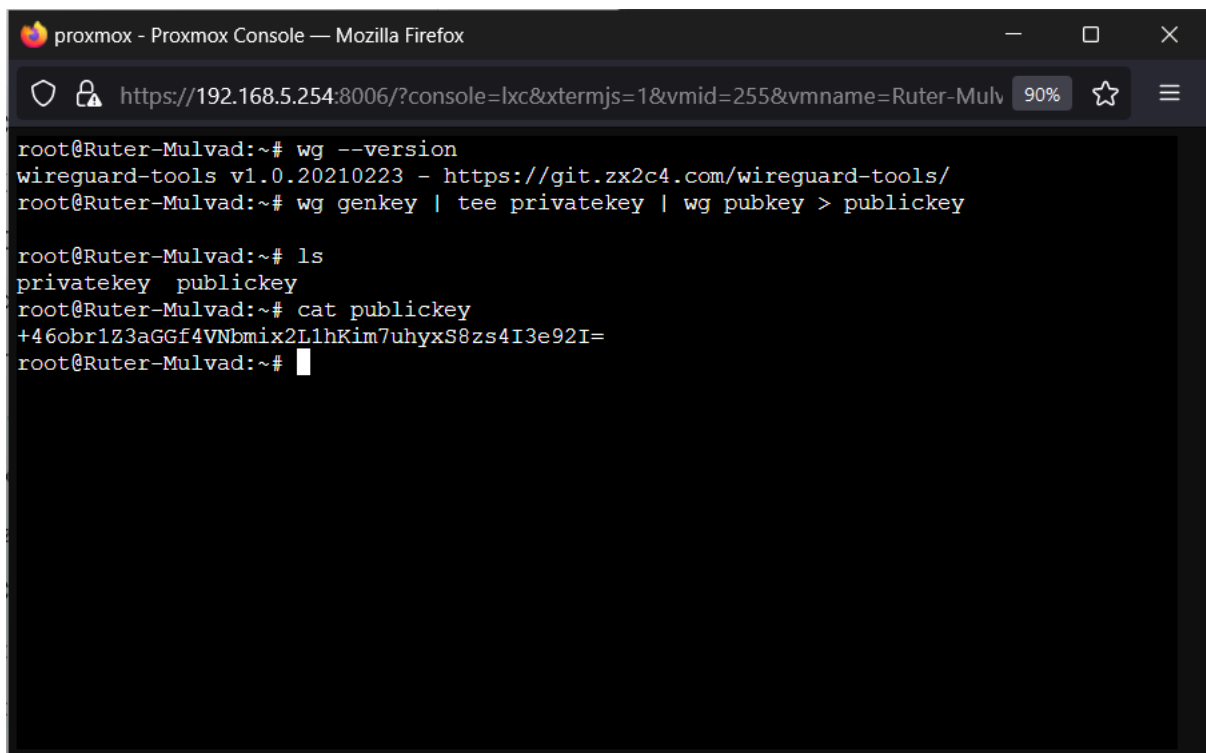
`sudo apt install wireguard-tools`



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lxc&xtermjs=1&vmid=255&vmname=Ruter-Mulv 90% ☆ ≡
root@Ruter-Mulvad:~# wg --version
wireguard-tools v1.0.20210223 - https://git.zx2c4.com/wireguard-tools/
root@Ruter-Mulvad:~#
```

Vamos a generar una clave pública y otra privada con el siguiente comando

```
wg genkey | tee privatekey | wg pubkey > publickey
```



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lxc&xtermjs=1&vmid=255&vmname=Ruter-Mulv 90% ☆ ≡
root@Ruter-Mulvad:~# wg --version
wireguard-tools v1.0.20210223 - https://git.zx2c4.com/wireguard-tools/
root@Ruter-Mulvad:~# wg genkey | tee privatekey | wg pubkey > publickey

root@Ruter-Mulvad:~# ls
privatekey  publickey
root@Ruter-Mulvad:~# cat publickey
+46obr1Z3aGGf4VNbmix2L1hKim7uhyxS8zs4I3e92I=
root@Ruter-Mulvad:~#
```

Ahora vamos a crear el archivo de configuración que es cómo la cartilla donde se guardan todos los datos necesarios para iniciar la conexión con la vpn

```
sudo nano /etc/wireguard/mullvad.conf
```

ahora realizaremos la siguiente configuración:

###

Para conseguir las addresses debemos introducir en la página web de mullvad o a través de una petición **curl** nuestra clave pública de wireguard esto es porque se utiliza para establecer una conexión segura entre nuestro dispositivo y sus servidores.

###

[Interface]

PrivateKey = **(la clave privada que acabamos de generar)**

Address = 10.69.71.75/32

DNS = 193.138.218.74 **(DNS que recomienda mullvad para privacidad)**

En la parte del peer debemos escoger de la siguiente página web que servidor queremos usar [servers](#)

[Peer]

PublicKey = bPfJDdgBXlY4w3ACs68zOMMhLUbbzktCKnLOFHqbxl4=

Endpoint = al-tia-wg-001.relays.mullvad.net:51820

AllowedIPs = 0.0.0.0/0

PersistentKeepalive = 25

Una vez configurado nuestro archivo de configuración podemos activarlo con **wg-quick up mullvad.conf**

The screenshot shows the Mullvad website's server selection interface. On the left, a table lists available servers with columns for Hostname, Country, and Status. The first two servers are 'al-tia-wg-001' and 'al-tia-wg-002', both located in Albania. Below them are servers in Australia. A modal window is open over the first server, displaying its configuration details in a terminal-like format. The configuration includes the interface name 'mullvad', a public key, a hidden private key, a listening port of 54902, and a remark 'Oxcac6c'. The peer configuration for 'al-tia-wg-001' is also shown, including its public key, endpoint (al-tia-wg-001.relays.mullvad.net:51820), allowed IPs (0.0.0.0/0), latest handshake time (24 seconds ago), transfer statistics (1.65 KiB received, 1.64 KiB sent), and persistent keepalive interval (every 25 seconds).

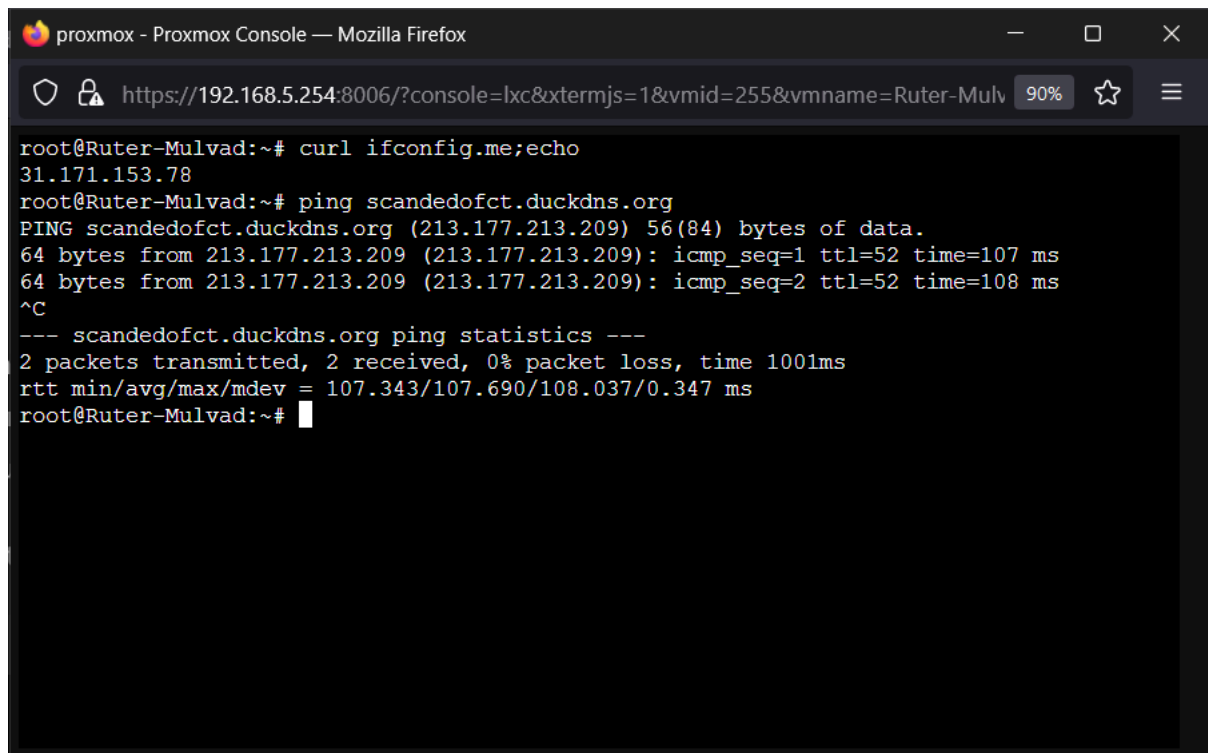
HOSTNAME	COUNTRY
al-tia-wg-001	Albania
al-tia-wg-002	Albania
au-adl-ovpn-301	Australia
au-adl-ovpn-302	Australia
au-adl-wg-301	Australia

```
root@Ruter-Mulvad:~# wg show
interface: mullvad
public key: ff+45XiuNRnRlaNi6+yWodUO+km/f9kInxg7qSctiC4=
private key: (hidden)
listening port: 54902
remark: Oxcac6c

peer: bPfJDdgBXlY4w3ACs68zOMMhLUbbzktCKnLOFHqbxl4=
endpoint: 31.171.153.66:51820
allowed ips: 0.0.0.0/0
latest handshake: 24 seconds ago
transfer: 1.65 KiB received, 1.64 KiB sent
persistent keepalive: every 25 seconds

root@Ruter-Mulvad:~#
```

Ahora podemos hacer un ping a ifconfig.me para verificar que estamos saliendo a internet con otra ip que no es la de nuestro rúter



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lxc&xtermjs=1&vmid=255&vmname=Ruter-Mulv 90% ☆ ≡

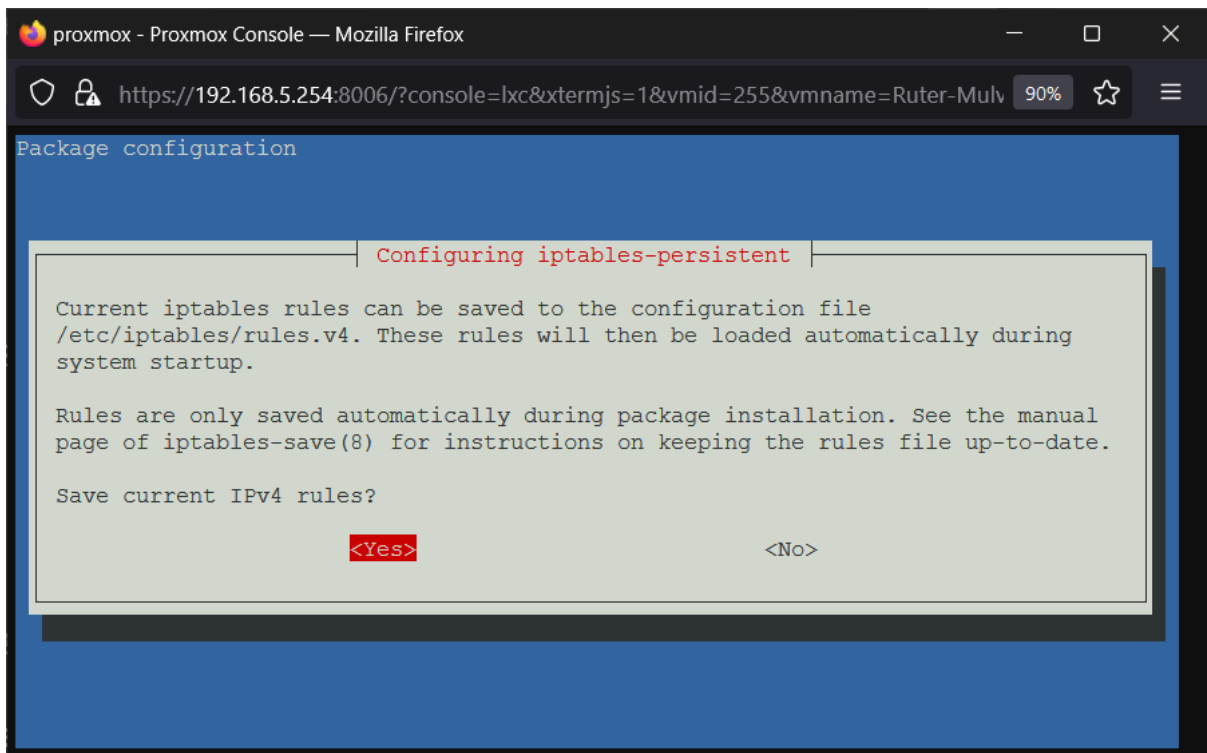
root@Ruter-Mulvad:~# curl ifconfig.me;echo
31.171.153.78
root@Ruter-Mulvad:~# ping scandedofct.duckdns.org
PING scandedofct.duckdns.org (213.177.213.209) 56(84) bytes of data.
64 bytes from 213.177.213.209 (213.177.213.209): icmp_seq=1 ttl=52 time=107 ms
64 bytes from 213.177.213.209 (213.177.213.209): icmp_seq=2 ttl=52 time=108 ms
^C
--- scandedofct.duckdns.org ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 107.343/107.690/108.037/0.347 ms
root@Ruter-Mulvad:~#
```

Le hago un ping a mi router con el uso de la [DNS dinámica](#) y vemos que recibimos respuesta porque no estamos dentro de nuestra red

Ahora vamos a añadir las reglas de iptables para que funcione el enrutamiento.
las reglas serían las siguientes:

```
sudo iptables -t nat -A POSTROUTING -o tun0 -j MASQUERADE
sudo iptables -A FORWARD -i mullvad -o eth0 -m state --state RELATED,ESTABLISHED
-j ACCEPT
sudo iptables -A FORWARD -i eth0 -o mullvad -j ACCEPT
```

Una vez aplicadas podemos descargar iptables-persistent para hacer persistentes los cambios



Ahora sólo falta configurar la persistencia con wireguard para que si reiniciamos el equipo se levante automáticamente la interfaz de mullvad(wireguard)

Wireguard ya genera un archivo en systemd específico para cada configuración bastaría con hacerle un systemctl enable, el comando es el siguiente:

```
sudo systemctl enable wg-quick@mullvad
```

```
proxmox - Proxmox Console - Opera
proxmox2scandedoft.xyz

root@Ruter-Mulvad:~# systemctl start wg-quick@mullvad
root@Ruter-Mulvad:~# systemctl enable wg-quick@mullvad
root@Ruter-Mulvad:~# systemctl status wg-quick@mullvad
* wg-quick@mullvad.service - WireGuard via wg-quick(8) for mullvad
   Loaded: loaded (/lib/systemd/system/wg-quick@.service; enabled; vendor preset: enabled)
   Active: active (exited) since Mon 2024-06-10 12:57:07 UTC; 8s ago
     Docs: man:wg-quick(8)
           man:wg(8)
           https://www.wireguard.com/
           https://www.wireguard.com/quickstart/
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8
   Main PID: 412 (code=exited, status=0/SUCCESS)
     Tasks: 0 (limit= 47959)
    Memory: 0B
       CPU: 0
   CGroup: /system.slice/system-wg\x2dquick.slice/wg-quick@mullvad.service

Jun 10 12:57:07 Ruter-Mulvad wg-quick[412]: [#] ip link set mtu 1420 up dev mullvad
Jun 10 12:57:07 Ruter-Mulvad wg-quick[455]: [#] resolvconf -a tun.mullvad -m 0 -x
Jun 10 12:57:07 Ruter-Mulvad wg-quick[468]: /etc/resolvconf/update.d/libc: Warning: /etc/resolv.conf is not a symbolic link to /run/resolvconf/res
Jun 10 12:57:07 Ruter-Mulvad wg-quick[412]: [#] wg set mullvad fwmark 51820
Jun 10 12:57:07 Ruter-Mulvad wg-quick[412]: [#] ip -4 route add 0.0.0.0/0 dev mullvad table 51820
Jun 10 12:57:07 Ruter-Mulvad wg-quick[412]: [#] ip -4 rule add not fwmark 51820 table 51820
Jun 10 12:57:07 Ruter-Mulvad wg-quick[412]: [#] ip -4 rule add table main suppress_prefixlength 0
Jun 10 12:57:07 Ruter-Mulvad wg-quick[412]: [#] sysctl -q net.ipv4.conf.all.src_valid_mark=1
Jun 10 12:57:07 Ruter-Mulvad wg-quick[412]: [#] nft -f /dev/fd/63
Jun 10 12:57:07 Ruter-Mulvad systemd[1]: Finished WireGuard via wg-quick(8) for mullvad.
lines 1-23/23 (END)
```

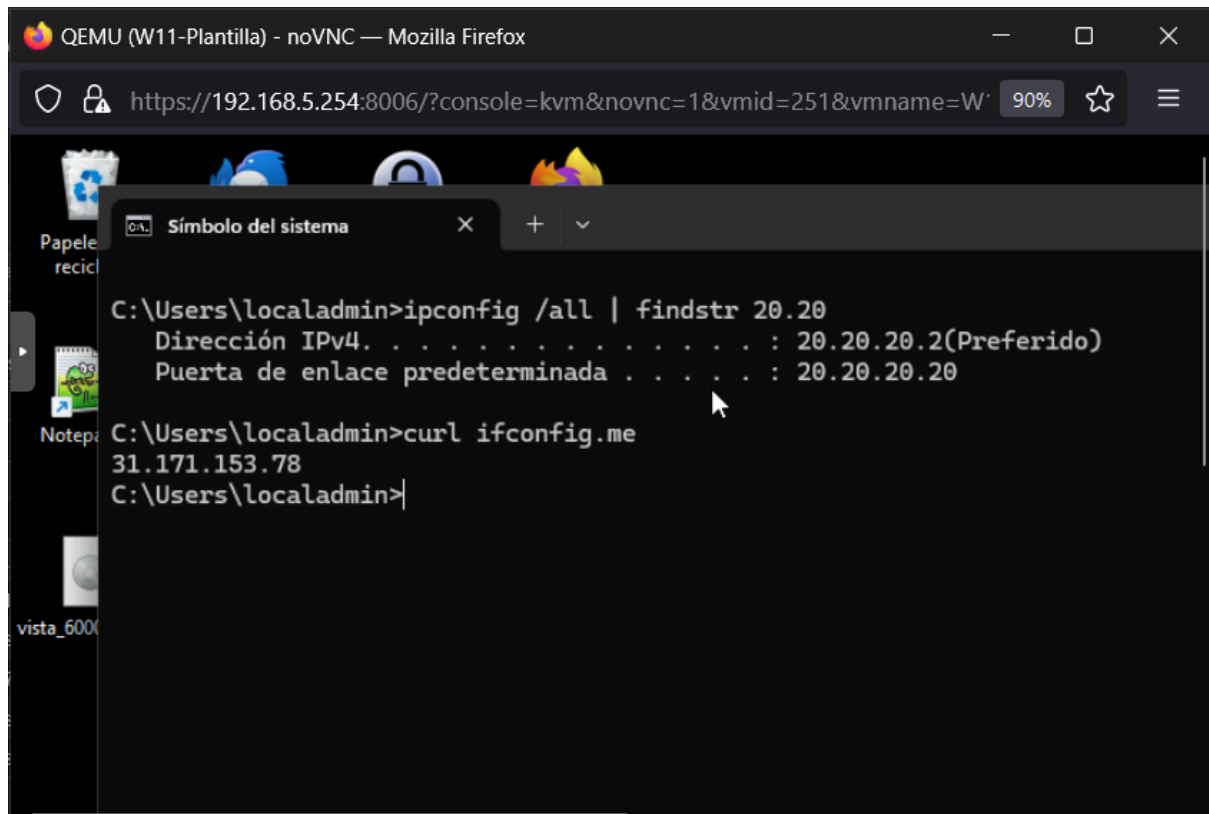
por curiosidad miro el archivo de systemd que genera wireguard

```
root@Ruter-Mulvad:~# cat /lib/systemd/system/wg-quick@.service
[Unit]
Description=WireGuard via wg-quick(8) for %i
After=network-online.target nss-lookup.target
Wants=network-online.target nss-lookup.target
PartOf=wg-quick.target
Documentation=man:wg-quick(8)
Documentation=man:wg(8)
Documentation=https://www.wireguard.com/
Documentation=https://www.wireguard.com/quickstart/
Documentation=https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8
Documentation=https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/wg-quick up %i
ExecStop=/usr/bin/wg-quick down %i
ExecReload=/bin/bash -c 'exec /usr/bin/wg synconf %i <(exec /usr/bin/wg-quick strip %i)'
Environment=WG_ENDPOINT_RESOLUTION_RETRIES=infinity

[Install]
WantedBy=multi-user.target
root@Ruter-Mulvad:~#
```

Ahora iniciamos windows y establecemos la nueva puerta de enlace , la 20.20.20.2/24



Ya salimos por el rúter con mulvad!!

Conexión remota

Vamos a conectarnos a nuestras máquinas windows de forma remota por rdp , cómo funcionaría en un entorno real.

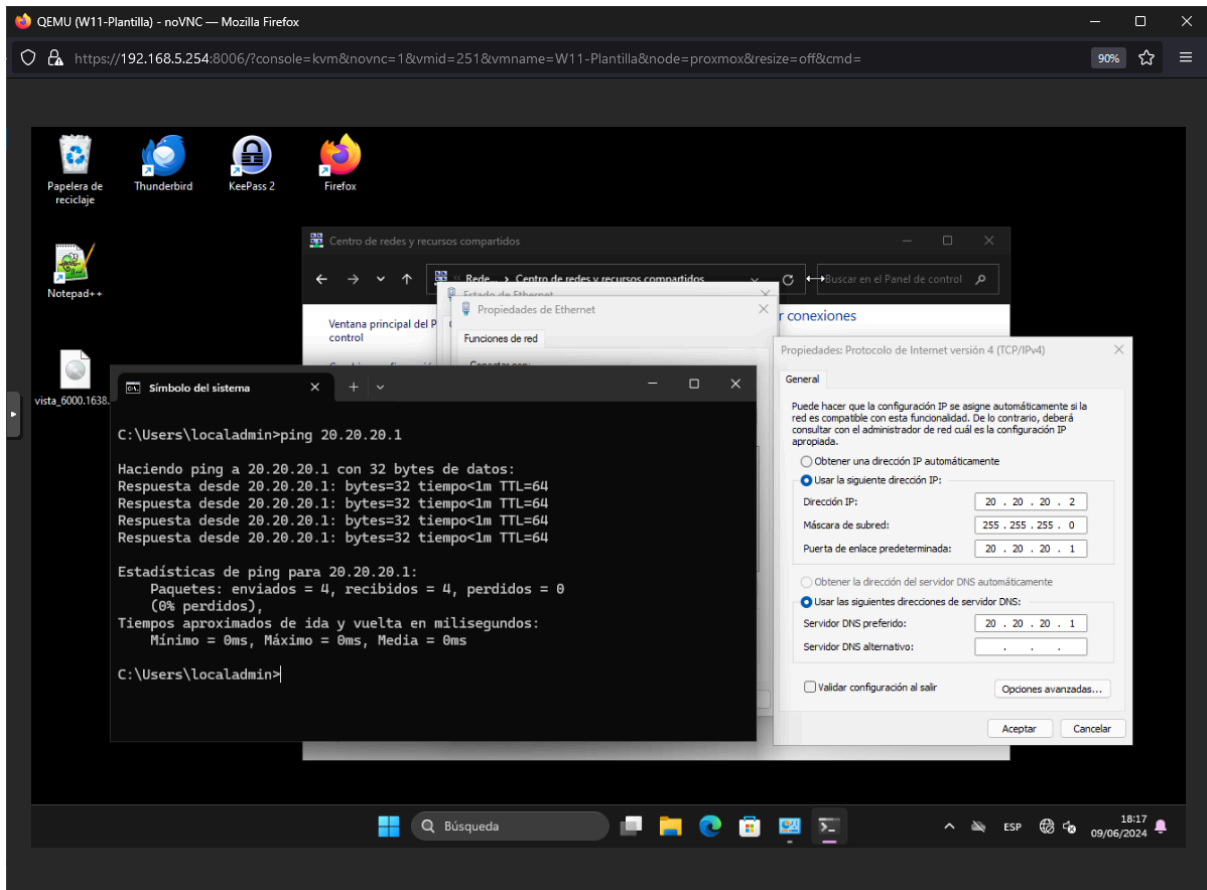
Para ello voy a usar una máquina virtual cnectada a una nueva interfaz de red de prueba

Añado la siguiente nat en el servidor proxmox

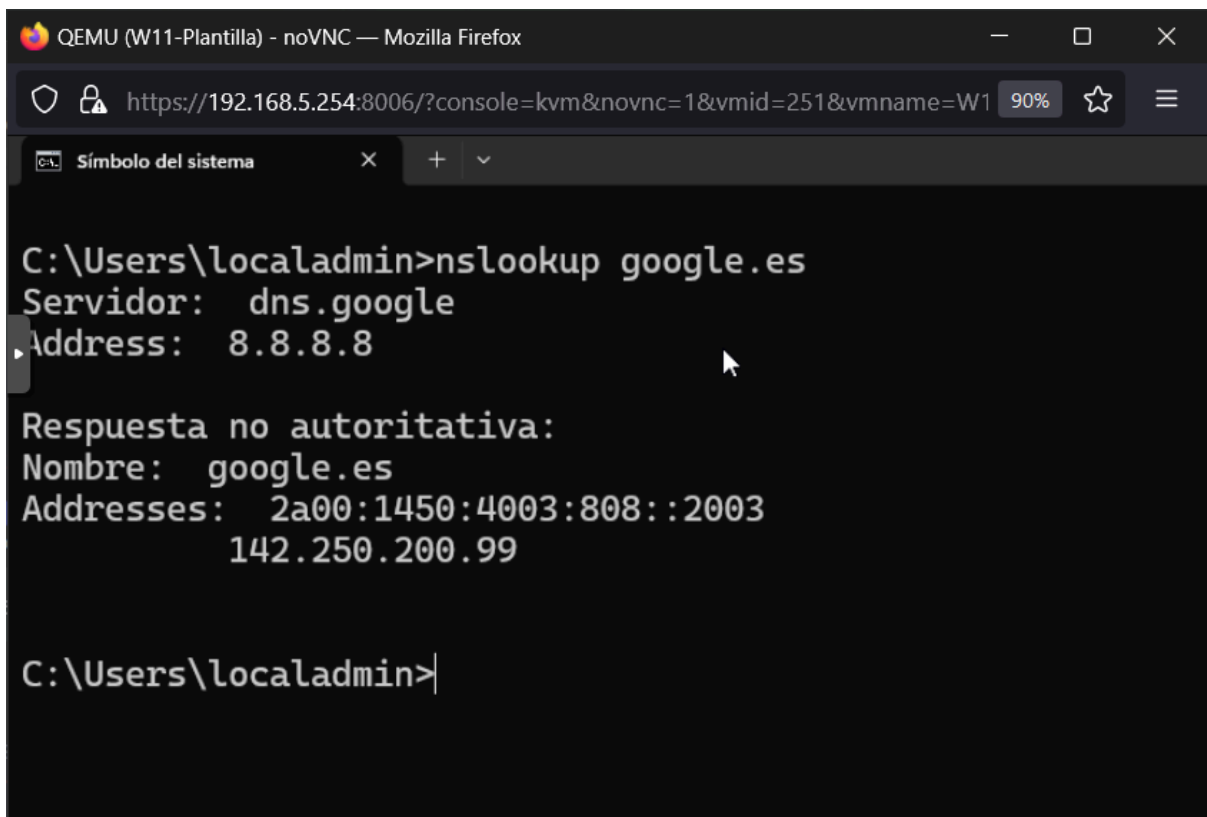
Configuramos el /etc/network/interfaces

```
#Red Interna de prueba 20.20.20.0/24
iface vmb4 inet manual
    address 20.20.20.1/24
    bridge-ports none
    bridge-stp off
    bridge-fd 0
    up /usr/sbin/brctl setageing vmb4 0
    up /usr/sbin/brctl setfd vmb4 0
```

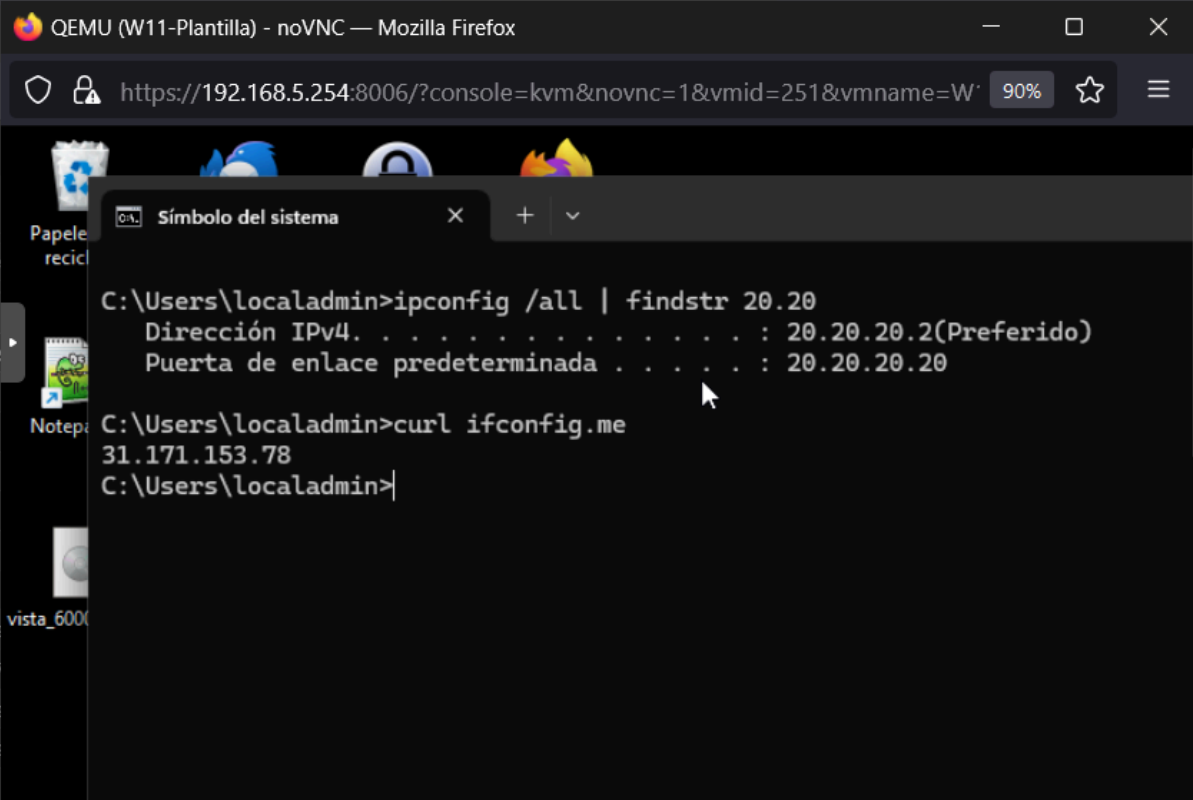
Y configuro la red para la máquina de windows que vamos a usar en esta simulación



comporbamos también la salida a internet



En este punto la máquina windows del entorno de prueba está conectada al router con la VPN de Mullvad que está redirigiendo el tráfico a servidores externos a nuestra máquina por lo que podemos probar las conexiones remotas como si realmente no estuviésemos en la red local



The screenshot shows a Windows command prompt window titled "Símbolo del sistema" (System Symbol) with a dark background. The window is open over a Firefox browser window titled "QEMU (W11-Plantilla) - noVNC — Mozilla Firefox". The browser's address bar shows the URL "https://192.168.5.254:8006/?console=kvm&novnc=1&vmid=251&vmname=W". The command prompt shows the following commands and output:

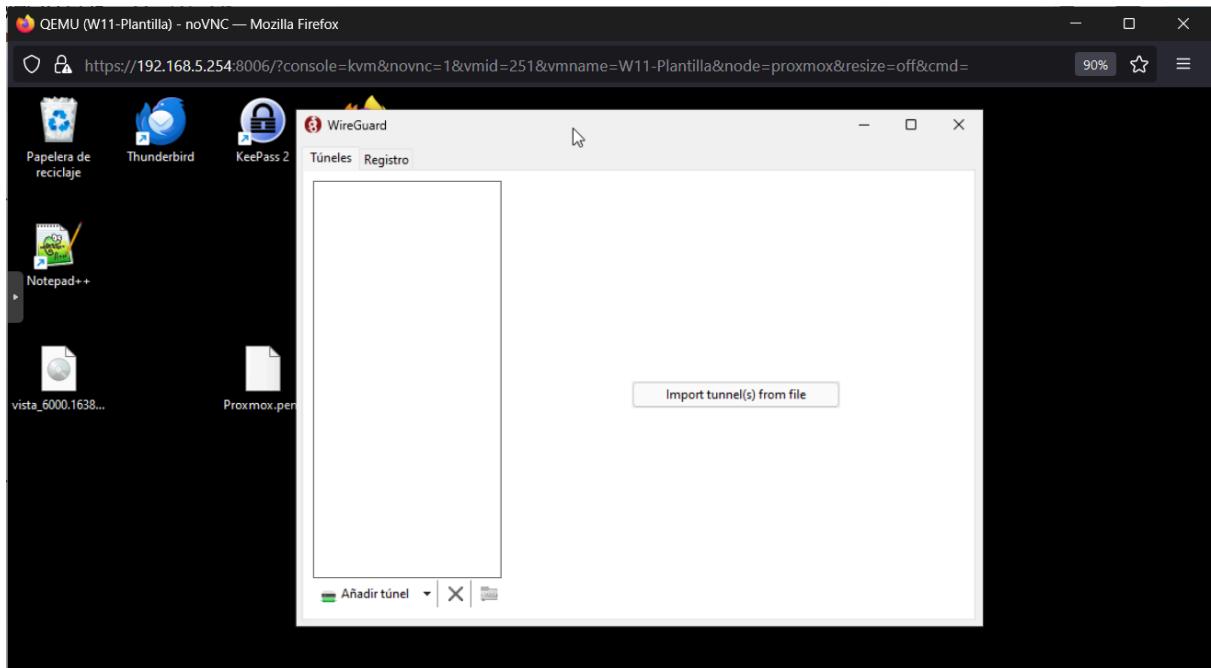
```
C:\Users\localadmin>ipconfig /all | findstr 20.20
Dirección IPv4. . . . . : 20.20.20.2(Preferido)
Puerta de enlace predeterminada . . . . . : 20.20.20.20

C:\Users\localadmin>curl ifconfig.me
31.171.153.78
C:\Users\localadmin>
```

Vamos a descargar el cliente de wireguard para conectarnos a nuestra “red de trabajo” si sale bien deberíamos tener conexión con la red 192.168.20.0/24

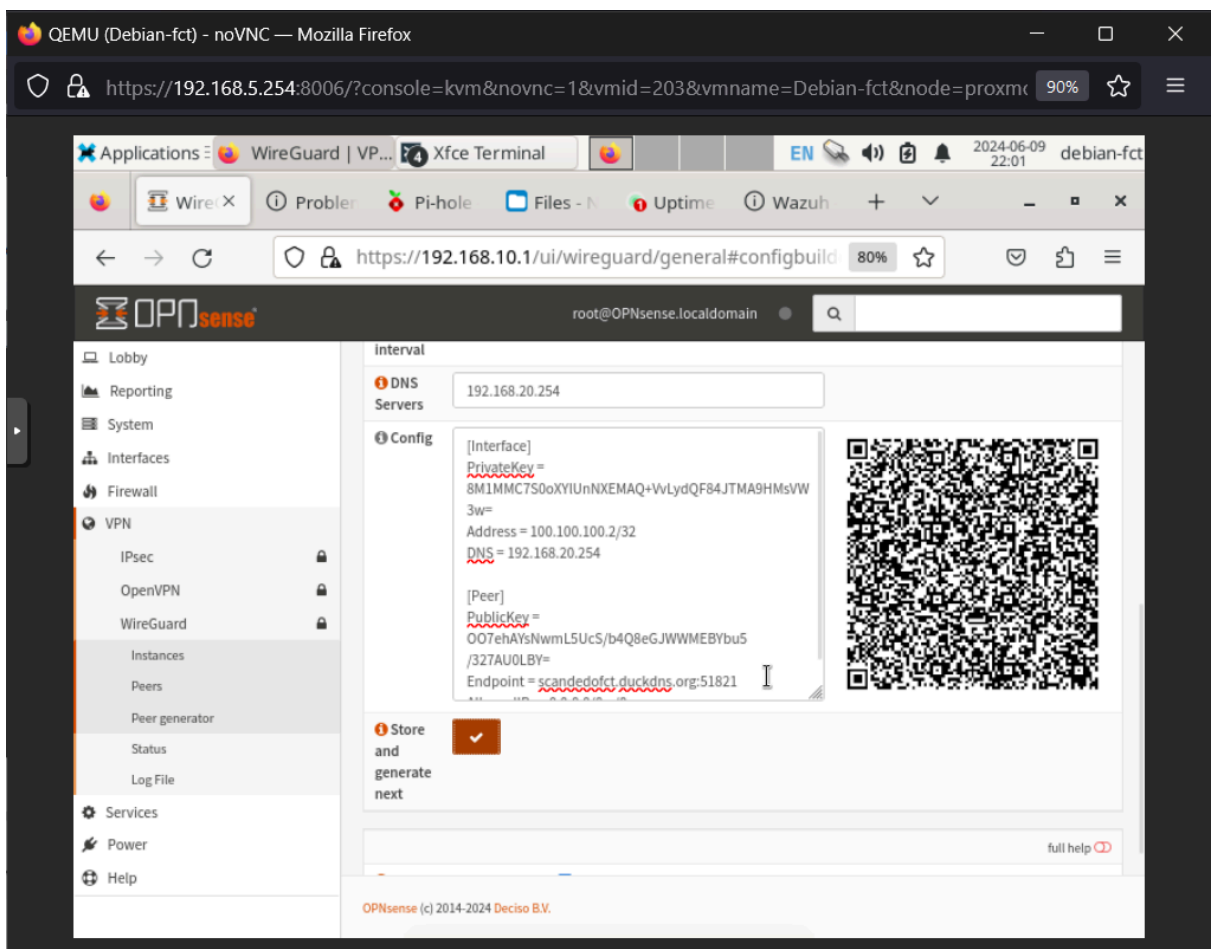
Podemos descargar el cliente de wireguard para windows desde su [página oficial](#)

Nos descargará un ejecutable que instalamos



La interfaz es realmente sencilla, solo debemos importar el peer que previamente configuramos en el router OpnSense

Vamos a crear un nuevo peer para este cliente01



[Interface]

PrivateKey = QFXfTaOU51ra1z224X0baE1HTu9jLcwQrNb+RZVPoFE=

Address = 100.100.100.50/32

DNS = 192.168.20.254

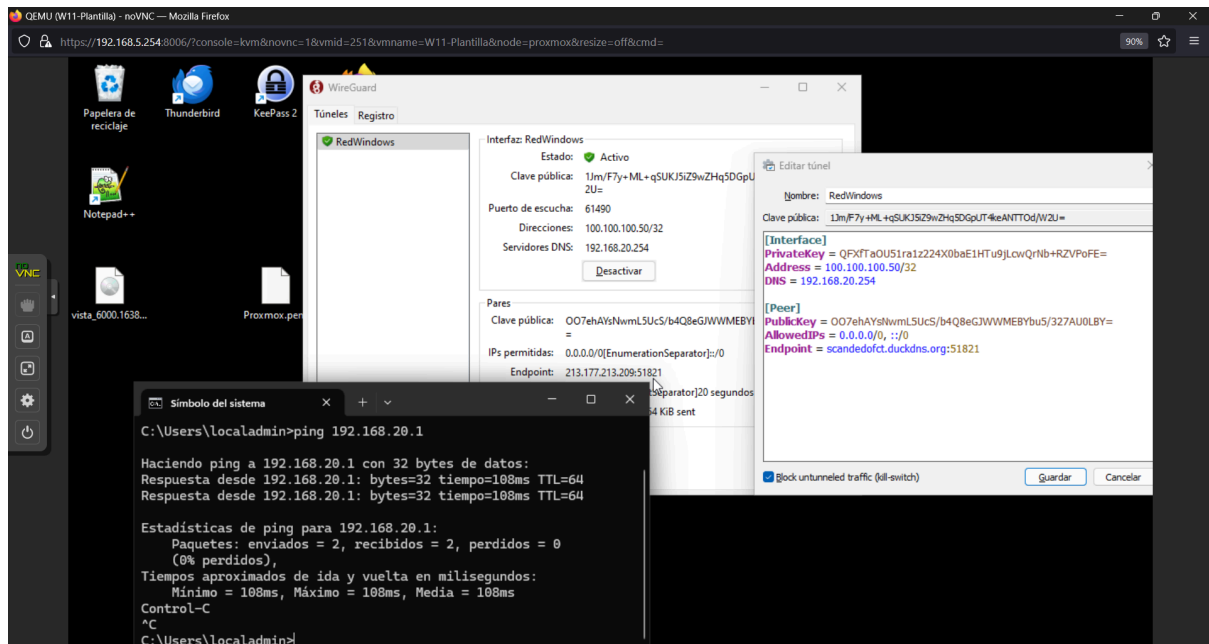
[Peer]

PublicKey = OO7ehAYsNwmL5UcS/b4Q8eGJWWMEBYbu5/327AU0LB=

Endpoint = scandedofct.duckdns.org:51821

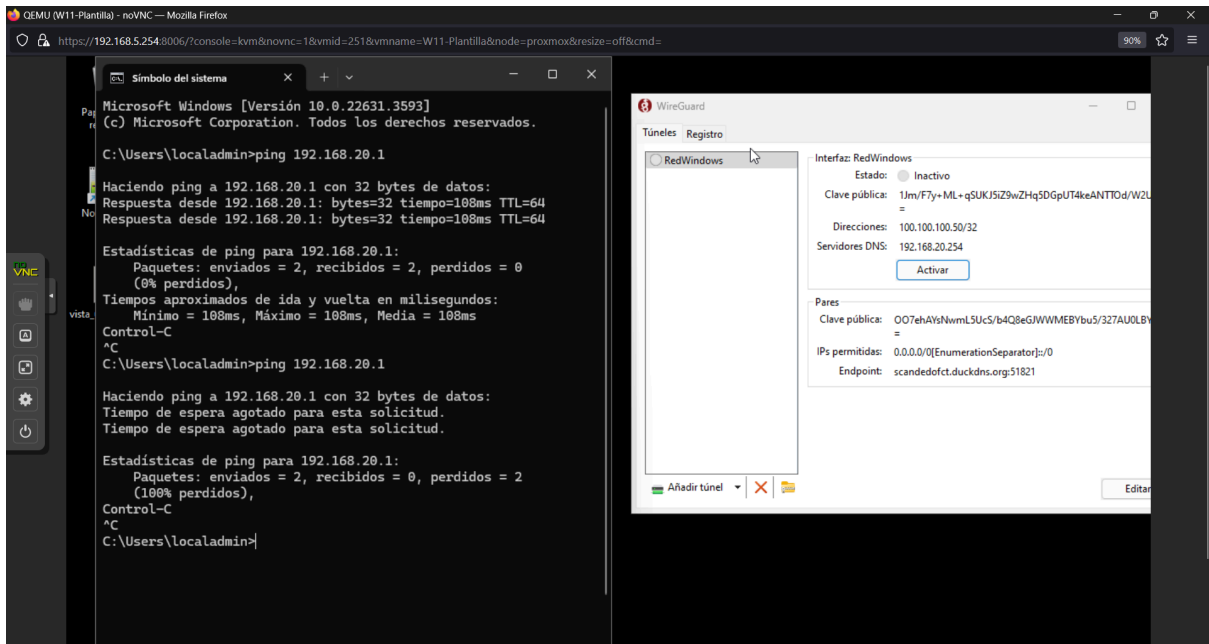
AllowedIPs = 0.0.0.0/0,::/0

Este archivo de configuracion lo importamos en el cliente de wireguard de windows.

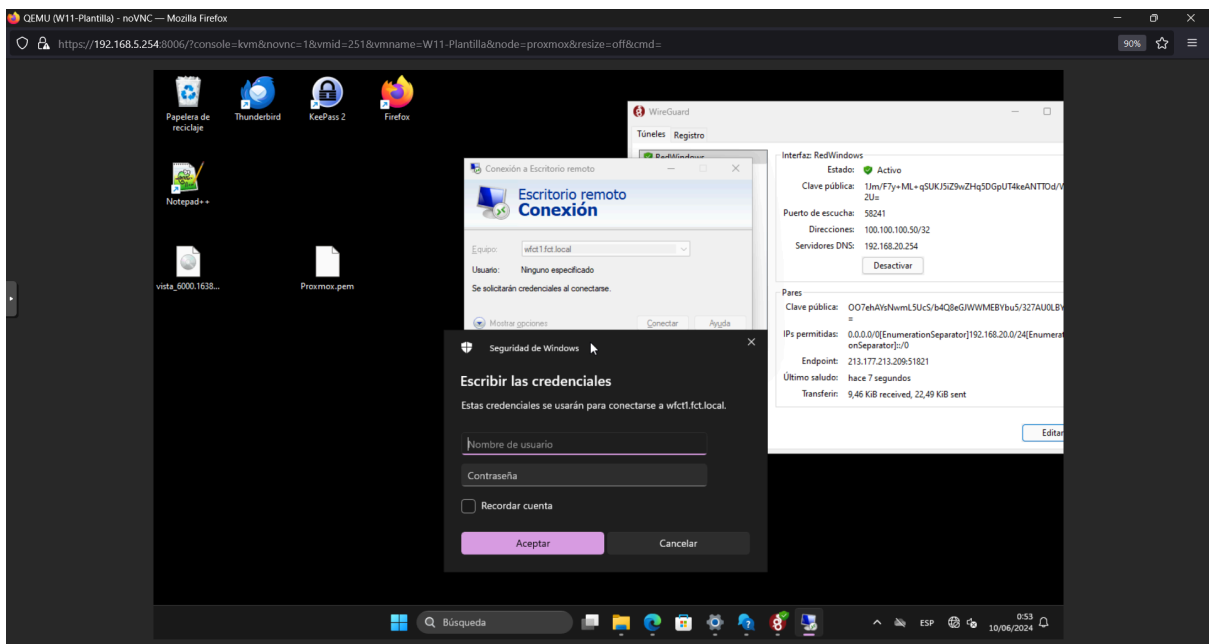


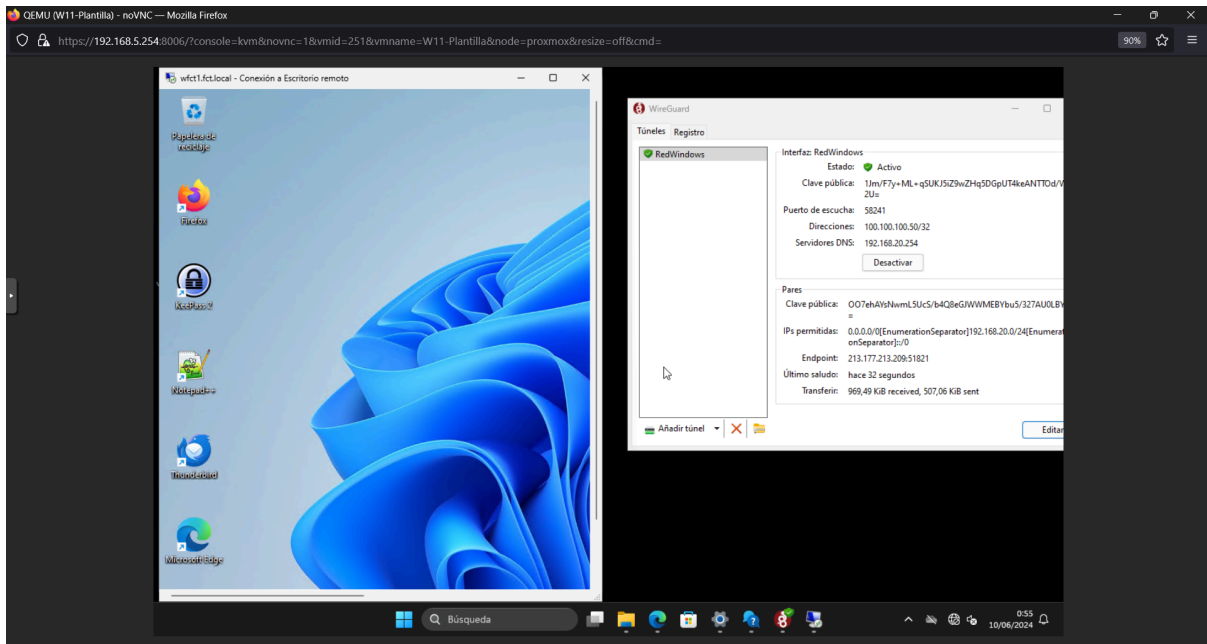
Activamos la conexion y vemos que podemos lanzar un ping a la 192.168.20.1

Si desactivamos la conexion nos fallan los pings



vamos a probar a conectarnos a un equipo de la red de windows por RDP





Vamos a instalar [microsoft remote desktop](#) para gestionar nuestras conexiones rdp y credenciales de forma cómoda

Podemos instalarlo desde la app de la tienda de microsoft o por winget.

Vamos a hacer uso de winget por usar una herramienta diferente, winget es el gestor de paquetes de windows al igual que podría ser apt o pacman para sistemas basados en UNIX

Nombre	Id	Versión	Coincidencia	Origen
Remote Desktop	Microsoft.RemoteDesktopClient	1.2.5405.0	Tag: remotedesktop	winget
EasyConnect	lstratman.easyconnect	3.1.0.105	Tag: remotedesktop	winget
NETworkManager	BornToBeRoot.NETworkManager	2024.1.7.0	Tag: remotedesktop	winget
vSpatial Remote Desktop	vSpatialInc.vSpatialRemoteDesktop	2023.1.8472.30549		winget
Chrome Remote Desktop Host	Google.ChromeRemoteDesktopHost	125.0.6422.31		winget
Remote Desktop Plus	Donkz.RemoteDesktopPlus	7.0.1		winget
Remote Desktop Manager Agent	Devolutions.RemoteDesktopManagerAgent	2024.1.31.0		winget
Remote Desktop Manager	Devolutions.RemoteDesktopManager	2024.1.31.0		winget

Winget install Microsoft.RemoteDesktopClient

```

proximox2.scandedofet.xyz
Windows PowerShell
PS C:\Users\localadmin\Desktop> winget install Microsoft.RemoteDesktopClient
Encontrado Remote Desktop [Microsoft.RemoteDesktopClient] Versión 1.2.5405.0
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Descargando https://query.prod.cms.rt.microsoft.com/cms/api/am/binary/RW1kB76
29.5 MB
El hash del instalador se verificó correctamente
Iniciando instalación de paquete...
Instalado correctamente
PS C:\Users\localadmin\Desktop>

```

[Wazuh](#)

Wazuh es un sistema de detección de intrusos basado en host de código abierto y libre. Realiza análisis de registro, comprobación de integridad, supervisión del registro de Windows, detección de rootkits, alertas basadas en el tiempo y respuesta activa

Escojo Wazuh para montar en mi infraestructura EDR porque es un programa de código abierto y gratuito. Funciona a modo de Fortinet y se instala el cliente en todos los Endpoints de la red (servidores, pc's, laptops...)

Componentes principales de Wazuh:

W.Indexer	Es un motor de análisis y búsqueda de texto completo altamente escalable. Este componente central indexa y almacena alertas generadas por el servidor Wazuh.
W.server	analiza los datos recibidos de los agentes y los procesa utilizando inteligencia sobre amenazas. Un único servidor puede analizar datos de miles de agentes y escalar cuando se configura como un clúster. También se utiliza para gestionar los agentes, configurándose de forma remota cuando sea necesario.
W.dashboard	El panel de Wazuh es la interfaz de usuario web para la visualización, el análisis y la gestión de datos. Incluye paneles de control para cumplimiento normativo, vulnerabilidades, integridad de archivos, evaluación de configuración, eventos de infraestructura en la nube, entre otros.
W.Agent	Los agentes de Wazuh se instalan en los endpoints como pcs portátiles, de escritorio, servidores, instancias en la nube o máquinas virtuales. Proporcionan capacidades de prevención, detección y respuesta a amenazas.

Instalar W.Indexer

[W.Indexer Instalación y requisitos](#)

Vamos a instalar el motor de búsqueda de texto de Wazuh

	RAM (minimo)	CPU cores (minimo)	RAM (recoendado)
Wazuh Indexer	2 GB	2	8 GB

Requisitos de espacio en disco

La cantidad de datos depende de las alertas generadas por segundo (APS). Esta tabla detalla el espacio estimado en disco necesario por agente para almacenar 90 días de alertas en un servidor indexador de Wazuh, dependiendo del tipo de endpoints monitorizados.

Endpoints monitorizados	APS	Almacenamiento en el indexador de Wazuh (GB/90 días)
Servers	0.25	3.7
Estaciones de trabajo	0.1	1.5
Dispositivos de red	0.5	7.4

Por ejemplo, para un entorno con 80 estaciones de trabajo, 10 servidores y 10 dispositivos de red, el almacenamiento necesario en el servidor indexador de Wazuh para 90 días de alertas es de 230 GB.

Vamos a realizar la Instalación del Indexer. Las instrucciones están detalladas en la siguiente página de Wazuh : [Instalación del Indexer](#)

Config.yml

```
GNU nano 5.4 cor
nodes:
# Wazuh indexer nodes
indexer:
- name: node-1
  ip: "10.0.2.10"
#- name: node-2
# ip: "<indexer-node-ip>"
#- name: node-3
# ip: "<indexer-node-ip>"

# Wazuh server nodes
# If there is more than one Wazuh server
# node, each one must have a node_type
server:
- name: wazuh-1
  ip: "10.0.2.11"
# node_type: master
#- name: wazuh-2
# ip: "<wazuh-manager-ip>"
# node_type: worker
#- name: wazuh-3
# ip: "<wazuh-manager-ip>"
# node_type: worker

# Wazuh dashboard nodes
dashboard:
- name: dashboard
  ip: "10.0.2.12"
```

Si conseguimos instalarlo correctamente podemos hacerle un curl al puerto 9200 para ver si el Indexer está en funcionamiento

```
root@wazuh-Indexer:~# tar -axf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
-O | grep -P "'admin\'" -A 1
  indexer_username: 'admin'
  indexer_password: 'HDC*2JCG74fZ*fFSc??KH8x04eUp07Bt'
root@wazuh-Indexer:~# curl -k -u admin:HDC*2JCG74fZ*fFSc??KH8x04eUp07Bt https://10.0.2.10:9200
0
{
  "name" : "node-1",
  "cluster_name" : "wazuh-indexer-cluster",
  "cluster_uuid" : "luHTiSbAQaWHXwhKRk5Pxx",
  "version" : {
    "number" : "7.10.2",
    "build_type" : "rpm",
    "build_hash" : "db90a415ff2fd428b4f7b3f800a51dc229287cb4",
    "build_date" : "2023-06-03T06:24:25.112415503Z",
    "build_snapshot" : false,
    "lucene_version" : "9.6.0",
    "minimum_wire_compatibility_version" : "7.10.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "The OpenSearch Project: https://opensearch.org/"
}
root@wazuh-Indexer:~#
```

Instalar W.Server

[W.Server Instalación y requisitos](#)

Requisitos

Verifica los sistemas operativos compatibles y los requisitos de hardware recomendados para la instalación del servidor Wazuh. Asegúrate de que tu entorno del sistema cumpla con todos los requisitos y de que tengas privilegios de usuario root.

Sistemas operativos recomendados

El servidor Wazuh se puede instalar en un sistema operativo Linux de 64 bits. Wazuh admite las siguientes versiones del sistema operativo:

- Amazon Linux 2
- CentOS 7, 8
- Red Hat Enterprise Linux 7, 8, 9
- Ubuntu 16.04, 18.04, 20.04, 22.04

Requisitos de hardware

El servidor Wazuh se puede instalar como un nodo único o como un clúster multinodo.

Recomendaciones de hardware

Componente	Mínimo	Recomendado
	RAM (GB)	CPU (núcleos)
Wazuh Server	2	2

Requisitos de espacio en disco

La cantidad de datos depende de las alertas generadas por segundo (APS). Esta tabla detalla el espacio en disco estimado necesario por agente para almacenar 90 días de alertas en un servidor Wazuh, según el tipo de endpoints monitoreados.

Endpoints monitoreados	APS	Almacenamiento en el servidor Wazuh (GB/90 días)
Servidores	0.25	0.1
Estaciones de trabajo	0.1	0.04

Dispositivos de red	0.5	0.2
----------------------------	-----	-----

Por ejemplo, para un entorno con 80 estaciones de trabajo, 10 servidores y 10 dispositivos de red, el almacenamiento necesario en el servidor Wazuh para 90 días de alertas es de 6 GB.

—

Instalar W.Dashboard

Este componente central es una interfaz web flexible e intuitiva para extraer, analizar y visualizar datos de seguridad. Proporciona paneles de control listos para usar, lo que le permite navegar sin problemas a través de la interfaz de usuario.

Sistemas operativos recomendados

Wazuh puede instalarse en un sistema operativo Linux de 64 bits. Wazuh admite las siguientes versiones del sistema operativo:

- Amazon Linux 2
- CentOS 7, 8
- Red Hat Enterprise Linux 7, 8, 9
- Ubuntu 16.04, 18.04, 20.04, 22.04

Requisitos de hardware

El dashboard de Wazuh puede instalarse en un nodo dedicado o junto con el indexador de Wazuh.

Recomendaciones de hardware

Componente	Mínimo	
	RAM (GB)	CPU (núcleos)
Dashboard Wazuh	4	2

Compatibilidad del navegador

El dashboard de Wazuh admite los siguientes navegadores web:

- Chrome 95 o posterior
- Firefox 93 o posterior
- Safari 13.7 o posterior

Otros navegadores basados en Chromium también pueden funcionar. Internet Explorer 11 no es compatible.

```

root@wazuh-Indexer:~# bash wazuh-install.sh --wazuh-dashboard dashboard
21/05/2024 11:43:07 INFO: Starting Wazuh installation assistant. Wazuh version: 4.7.4
21/05/2024 11:43:07 INFO: Verbose logging redirected to /var/log/wazuh-install.log
21/05/2024 11:43:12 INFO: Wazuh web interface port will be 443.
21/05/2024 11:43:15 INFO: Wazuh repository added.
21/05/2024 11:43:15 INFO: --- Wazuh dashboard ----
21/05/2024 11:43:15 INFO: Starting Wazuh dashboard installation.
21/05/2024 11:44:05 INFO: Wazuh dashboard installation finished.
21/05/2024 11:44:05 INFO: Wazuh dashboard post-install configuration finished.
21/05/2024 11:44:05 INFO: Starting service wazuh-dashboard.
21/05/2024 11:44:05 INFO: wazuh-dashboard service started.

21/05/2024 11:45:10 INFO: Initializing Wazuh dashboard web application.

```

Una vez instalado se nos mostrarán las credenciales por terminal , accedemos desde el navegador web para ver si está correctamente instalado

The image shows two screenshots of the Wazuh dashboard. The top screenshot is the login page at `https://10.0.2.12/app/login?`, featuring the Wazuh logo and a login form with fields for Username and Password, and a Log in button. The bottom screenshot is the main overview page at `https://10.0.2.12/app/wazuh#/overview/?g={filters:{},refreshInterval(pause:{value:0,time:(from:now-24h,to:now)})&a={columns:{},source:{},filters:{},index:wazuh-alerts-*,interval:auto,query:{language:}}`. It displays a summary of agents (Total, Active, Disconnected, Pending, Never connected) and a grid of modules including Security Information Management, Auditing and Policy Monitoring, Threat Detection and Response, and Regulatory Compliance.

Wazuh Dashboard Overview

Agents Summary:

- Total agents: 0
- Active agents: 0
- Disconnected agents: 0
- Pending agents: 0
- Never connected agents: 0

Modules:

- SECURITY INFORMATION MANAGEMENT**
 - Security events: Browse through your security alerts, identifying issues and threats in your environment.
 - Integrity monitoring: Alerts related to file changes, including permissions, content, ownership and attributes.
- AUDITING AND POLICY MONITORING**
 - Policy monitoring: Verify that your systems are configured according to your security policies baseline.
 - System auditing: Audit users behavior, monitoring command execution and alerting on access to critical files.
 - Security configuration assessment: Scan your assets as part of a configuration assessment audit.
- THREAT DETECTION AND RESPONSE**
 - Vulnerabilities: Discover what applications in your environment are affected by well-known vulnerabilities.
 - MITRE ATT&CK: Security events from the knowledge base of adversary tactics and techniques based on real-world observations.
- REGULATORY COMPLIANCE**
 - PCI DSS: Global security standard for entities that process, store or transmit payment cardholder data.
 - NIST 800-53: National Institute of Standards and Technology Special Publication 800-53 (NIST 800-53) sets guidelines for federal information systems.
 - TSC: Trust Services Criteria for Security, Availability, Processing.
 - GDPR: General Data Protection Regulation (GDPR) sets guidelines for

Una vez terminada la instalación y accedido al panel de Administración podemos empezar a instalar el programa en los Endpoints. Vamos a empezar con el windows server de la red 192.168.20.24

Desde la página del manager podemos gestionar un nuevo agente de forma sencilla. cubrimos los pasos para agregar un nuevo agente

The screenshot shows the 'Deploy new agent' form in the Wazuh management interface. The form is titled 'Deploy new agent' and has a 'Refresh' button. It is divided into two main sections: 'Select the package to download and install on your system:' and 'Server address:'. The first section has three tabs: 'LINUX', 'WINDOWS', and 'macOS'. Under the 'WINDOWS' tab, there are two radio buttons: 'MSI 32/64 bits' (selected) and 'Apple silicon'. The second section, 'Server address:', has a text input field with the value '192.168.20.254'.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

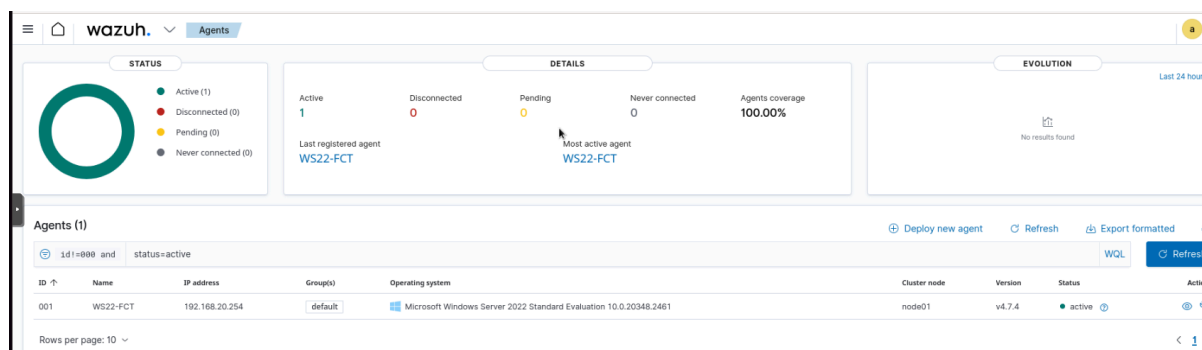
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.4-1.msi -OutFile $(env.tmp)\wazuh-agent; msixexec.exe /i $(env.tmp)\wazuh-agent /q WAZUH_MANAGER='192.168.20.254' WAZUH_AGENT_NAME='WS22-FCT2' WAZUH_REGISTRATION_SERVER='192.168.20.254'
PS C:\Users\Administrator> NET START WazuhSvc

The Wazuh service was started successfully.

C:\Users\Administrator>
```

Ahora en Wazuh-Agents podemos observar cómo se nos añade windows Server



Gracias a Wazuh que opera a nivel del NT-SYSTEM en el sistema operativo podemos monitorizar todo tipo de eventos a tiempo real y visualizar la implicación a nivel de seguridad que tiene y el impacto para la empresa.

Por hacer una prueba intento loguearme al sistema operativo cómo administrador del AD y me registra el inicio de sesión , incluso el intento fallido de inicio de sesión

Security Alerts					
Time ↓	Technique(s)	Tactic(s)	Description	Level	Rule ID
May 23, 2024 @ 09:51:17.729	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Login failure - Unknown user or bad password.	5	60122
Table JSON Rule					
@timestamp	2024-05-23T07:51:17.729Z				
_id	1Rupl889gU4eJ322vj				
agent.id	001				
agent.ip	192.168.20.254				
agent.name	WS22-FCT				
data.win.eventdata.authenticationPackageName	Negotiate				
data.win.eventdata.failureReason	%%2313				
data.win.eventdata.ipAddress	127.0.0.1				
data.win.eventdata.ipPort	0				
data.win.eventdata.keyLength	0				
data.win.eventdata.logonProcessName	User32				
data.win.eventdata.logonType	2				
data.win.eventdata.processid	0x5d8				

Se registra todo lo que pasa en el sistema operativo en los logs del Indexer, y además tenemos la posibilidad de generar todo tipo de gráficos para el monitoreo en la infraestructura.

Snort

Snort es un sistema de detección de intrusiones de red (NIDS) gratuito y de código abierto, desarrollado por Sourcefire.

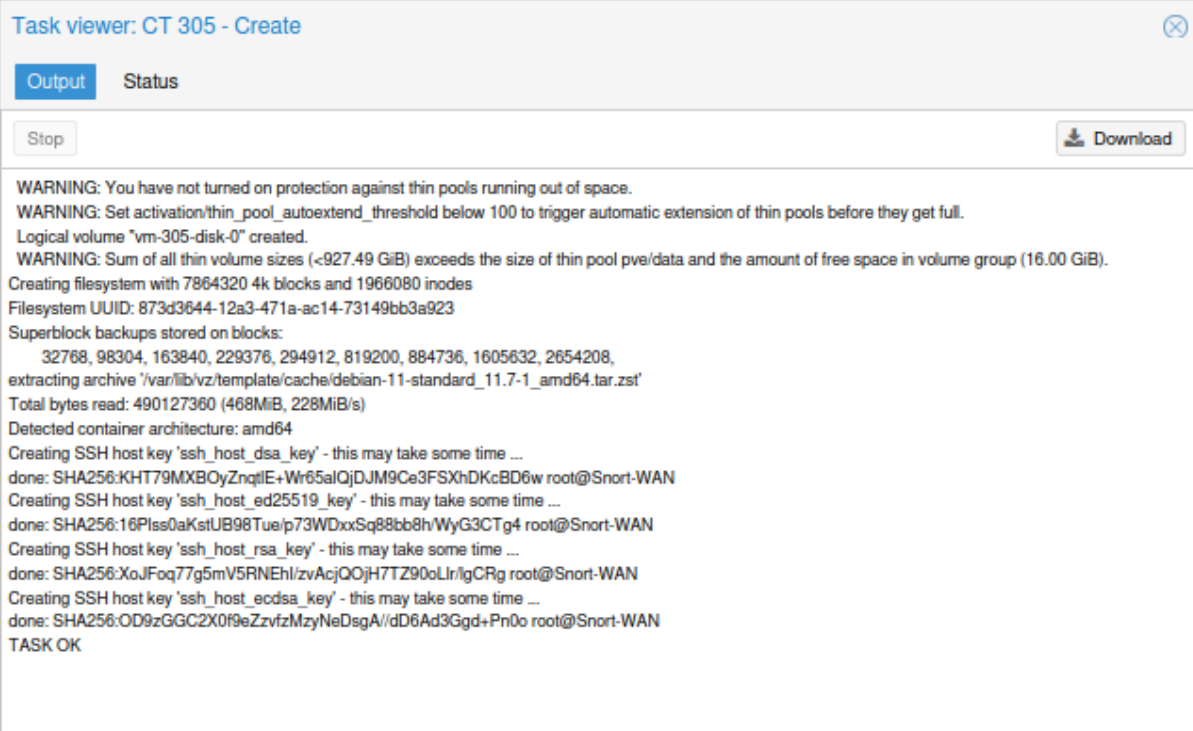
Funciona analizando el tráfico de red en tiempo real para detectar diversas formas de ataques y amenazas. Podemos configurar snort para registrar tráfico, generar alertas en tiempo real y realizar acciones específicas basadas en las reglas definidas por el usuario.

Propuesta de Implementación de Snort

Vamos a crear una máquina dentro de la red que esté monitoreando el tráfico de red constantemente y que registre si alguien hace un ping al servidor proxmox dentro de la red **10.0.2.0/24** , también se me ocurrió implementar mediante script que me envíe un mensaje al teléfono móvil cada vez que ocurra

1. Preparar una Máquina para Snort

Vamos a crear un CT en Proxmox que monitoree el tráfico que llega al servidor, por poner un ejemplo vamos a programar una alerta snort que salte cuando enviemos tráfico **icmp** al servidor windows



```
Task viewer: CT 305 - Create

Output  Status

Stop Download

WARNING: You have not turned on protection against thin pools running out of space.
WARNING: Set activation/thin_pool_autoextend_threshold below 100 to trigger automatic extension of thin pools before they get full.
Logical volume "vm-305-disk-0" created.
WARNING: Sum of all thin volume sizes (<927.49 GiB) exceeds the size of thin pool pve/data and the amount of free space in volume group (16.00 GiB).
Creating filesystem with 7864320 4k blocks and 1966080 inodes
Filesystem UUID: 873d3644-12a3-471a-ac14-73149bb3a923
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
extracting archive 'var/lib/vz/template/cache/debian-11-standard_11.7-1_amd64.tar.zst'
Total bytes read: 490127360 (468MiB, 228MiB/s)
Detected container architecture: amd64
Creating SSH host key 'ssh_host_dsa_key' - this may take some time ...
done: SHA256:KHT79MXBOyZnqtIE+Wr65alQjDJM9Ce3FSXhDKcBD6w root@Snort-WAN
Creating SSH host key 'ssh_host_ecdsa_key' - this may take some time ...
done: SHA256:16Plss0aKstUB98Tue/p73WDxxSq88bb8h/WyG3CTg4 root@Snort-WAN
Creating SSH host key 'ssh_host_rsa_key' - this may take some time ...
done: SHA256:XoJFq77g5mV5RNEhl/zvAcjQOjH7TZ90oLlrIgCRg root@Snort-WAN
Creating SSH host key 'ssh_host_ecdsa_key' - this may take some time ...
done: SHA256:OD9zGGC2X0f9eZzvfzMzyNeDsgA//dD6Ad3Ggd+Pn0o root@Snort-WAN
TASK OK
```

2. Instalar Snort

Para instalar snort he creado un script de bash con detección de errores, para que si falla en algún punto salga del script y notifique el usuario, haciendo uso de las dobles pipes (||) y de error_exit que imprime un mensaje si falla.

```
#!/bin/bash

# Verificar si el script se ejecuta con permisos de sudo
if [ "$EUID" -ne 0 ]; then
    echo "Este script requiere permisos de sudo. Por favor, introduce la contraseña:"
```

```

sudo -v || { echo "Error: no se pudo obtener permisos de sudo"; exit 1; }
fi

# Función para manejar errores
error_exit() {
    echo "Error en el paso: $1"
    exit 1
}

echo "Actualizando el sistema..."
sudo apt update && sudo apt upgrade -y || error_exit "Actualización del sistema"

echo "Instalando dependencias..."
sudo apt install -y build-essential libpcap-dev libpcrc3-dev libdumbnet-dev bison flex
zlib1g-dev wget libluajit-5.1-dev libssl-dev cmake libdaq-dev || error_exit "Instalación de
dependencias"

echo "Instalando dependencias y Snort..."
sudo apt install -y snort || error_exit "Instalación de Snort"

echo "Configurando Snort "
# En esta parte del script me decanto por crear un usuario y un grupo dedicado para snort que
va a ser el #que pueda gestionar los recursos de snort por motivos de seguridad, sort podría
ejecutarse como usuario root pero en vez de ello dejamos a un usuario de bajos privilegios
que lo haga para que en caso de que se detecte alguna vulnerabilidad con snort el atacante
no llegue a tener un usuario con privilegios de root

sudo ln -s /usr/local/bin/snort /usr/sbin/snort || error_exit "Creación de enlace simbólico para
Snort"
sudo groupadd snort || error_exit "Creación del grupo snort"
sudo useradd snort -r -s /sbin/nologin -c SNORT_IDS -g snort || error_exit "Creación del
usuario snort"
sudo mkdir /etc/snort /var/log/snort /usr/local/lib/snort_dynamicrules || error_exit "Creación de
directorios para Snort"

sudo chown -R snort:snort /etc/snort /var/log/snort /usr/local/lib/snort_dynamicrules || error_exit
"Cambio de propiedad de directorios para Snort"

```

Le damos permisos de ejecución e iniciamos el script

```
root@Snort-WAN:~# ./instalar-snort
Actualizando el sistema...
Hit:1 http://security.debian.org bullseye-security InRelease
Hit:2 http://deb.debian.org/debian bullseye InRelease
Hit:3 http://deb.debian.org/debian bullseye-updates InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
69 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  adduser base-files bind9-dnsutils bind9-host bind9-libs bsdxtrutils bsduutils cpio dbus distro-info-data dpkg
  fdisk file krb5-locales less libblkid1 libbsd0 libc-bin libc-l10n libc6 libcurl3-gnutls libdbus-1-3 libfdisk1
  libgnutls30 libgssapi-krb5-2 libjson-c5 libk5crypto3 libkrb5-3 libkrb5support0 libmagic-mgc libmagic1 libmount1
  libncurses6 libncursesw6 libnftables1 libnftables1 libnftables1 libnftables1 libnftables1 libnftables1 libperl5.32 libsmartcols1
  libssl1.1 libsystemd0 libtinfo6 libudev1 libuuid1 libuv1 locales logrotate mount ncurses-base ncurses-bin
  ncurses-term nftables openssh-client openssh-server openssh-sftp-server openssl perl perl-base perl-modules-5.32
  postfix ssh systemd systemd-sysv systemd-timesyncd tar tzdata udev util-linux
69 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 45.5 MB of archives.
After this operation, 147 kB of additional disk space will be used.
Get:1 http://security.debian.org bullseye-security/main amd64 bsduutils amd64 1:2.36.1-8+deb11u2 [149 kB]
```

Verificamos que esta instalado escribiendo snort en la terminal

```
root@Snort-WAN:~# snort
Running in packet dump mode

==== Initializing Snort ====
Initializing Output Plugins!
pcap DAQ configured to passive.
Acquiring network traffic from "eth0".
Decoding Ethernet

==== Initialization Complete ====

,,_
o" )~ Version 2.9.20 GRE (Build 82)
''' By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
    Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
    Copyright (C) 1998-2013 Sourcefire, Inc., et al.
    Using libpcap version 1.10.0 (with TPACKE_V3)
    Using PCRE version: 8.39 2016-06-14
    Using ZLIB version: 1.2.11

Commencing packet processing (pid=2073)
WARNING: No preprocessors configured for policy 0.
05/28-19:36:58.815377 fe80::be24:11ff:fe72:e019 -> ff02::2
IPv6-ICMP TTL:255 TOS:0x0 ID:0 IpLen:40 DgmLen:56
+++++
```

3. Configurar Snort

- + Creación del archivo de reglas de snort

Para poner a funcionar el snort necesitamos especificar unas reglas . La ubicación de la ruta del archivo de instrucciones la vamos a especificar en /etc/snort/snort.conf así que podemos guardar las reglas en un archivo en el que el usuario snort tenga acceso . en mi caso me decanto por /etc/snort/rules/local.rules

Nuestra regla quiere crear una alerta si detecta un ping a nuestro servidor proxmox 10.0.2.2

alert icmp any any -> 10.0.2.2 any (msg:"Ping a Servidor Proxmox Detectado"; sid:1000001; rev:1;)

Desglosemos cada parte de la regla:

Acción de la Regla

- **alert**: Esta es la acción que Snort tomará cuando la regla coincida. En este caso, "alert" significa que Snort generará una alerta.

Protocolo

- **icmp**: Este es el protocolo de red que esta regla monitorea. En este caso, es el protocolo ICMP (Internet Control Message Protocol), que se usa comúnmente para mensajes de control de red, como el comando "ping".

Dirección de Origen

- **any**: Esta palabra clave significa cualquier dirección IP de origen. La regla no está restringida a una dirección IP específica de origen.

Puerto de Origen

- **any**: Para ICMP, los puertos no son relevantes, pero esta sintaxis se mantiene por consistencia. Se usa "any" para indicar cualquier puerto de origen.

Dirección de Destino

- **192.168.20.254**: Esta es la dirección IP de destino específica a la que la regla se aplica. En este caso, es la IP del servidor que estás monitoreando.

Puerto de Destino

- **any**: Similar al puerto de origen, "any" se usa aquí para indicar cualquier puerto de destino, aunque no es relevante para ICMP.

Opciones de la Regla

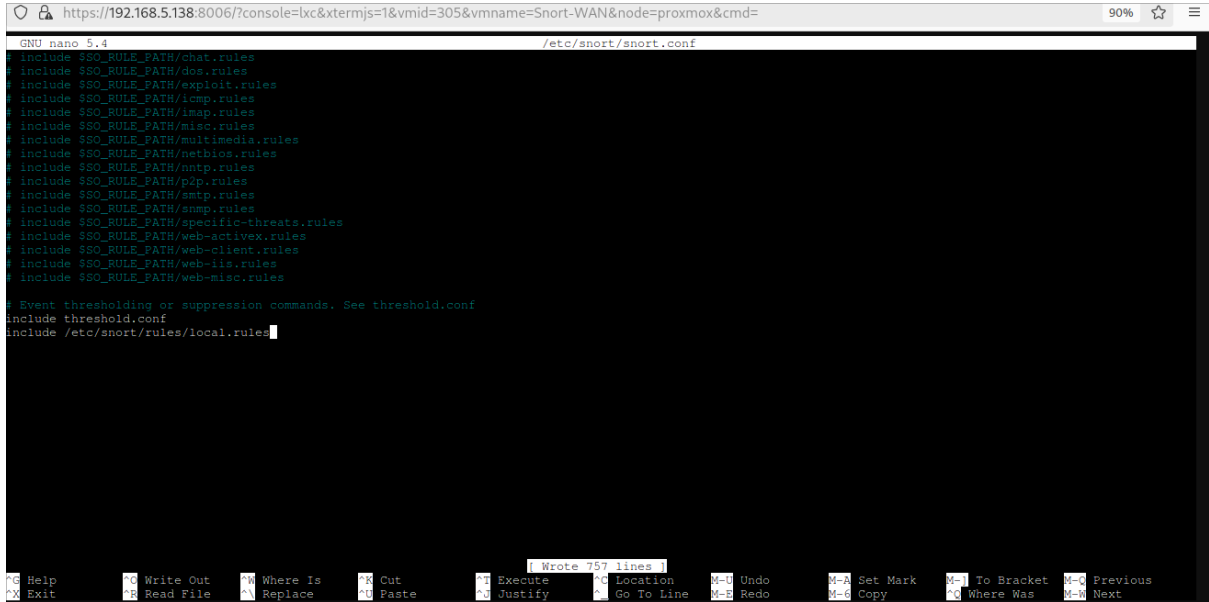
- **(msg:"Ping a servidor Proxmox detectado"; sid:1000001; rev:1;)**: Estas son las opciones de la regla, encerradas entre paréntesis.
 - **msg:"Ping a servidor Proxmox detectado"**: Este es el mensaje que se generará cuando la regla coincida
 -

- **sid:1000001**: El ID de la regla (Signature ID). Cada regla debe tener un SID único. Los SIDs en el rango de 1000000 y superiores se reservan para reglas personalizadas locales.
- **rev:1**: La revisión de la regla. Es un número que se incrementa cada vez que se modifica la regla para identificar versiones.

Una vez sabemos la sintaxis de snort podemos crear todo tipo de triggers para el monitoreo del tráfico de red de nuestro EDR

Añadimos la siguiente línea a nuestro archivo de configuración de snort : `/etc/snort/snort.conf`

```
include /etc/snort/rules/local.rules
```



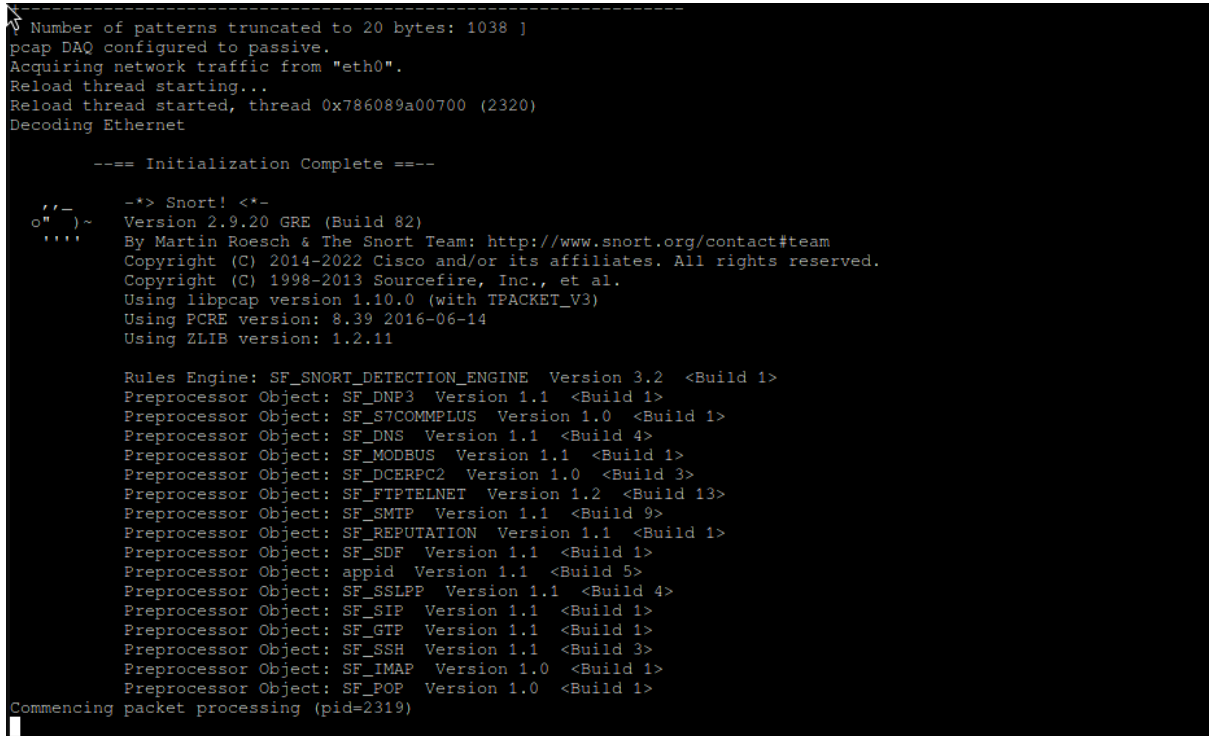
```
GNU nano 5.4 /etc/snort/snort.conf
# include $SO_RULE_PATH/char.rules
# include $SO_RULE_PATH/dos.rules
# include $SO_RULE_PATH/exploit.rules
# include $SO_RULE_PATH/icmp.rules
# include $SO_RULE_PATH/imap.rules
# include $SO_RULE_PATH/misc.rules
# include $SO_RULE_PATH/multimedia.rules
# include $SO_RULE_PATH/netbios.rules
# include $SO_RULE_PATH/nntp.rules
# include $SO_RULE_PATH/p2p.rules
# include $SO_RULE_PATH/smtp.rules
# include $SO_RULE_PATH/snmp.rules
# include $SO_RULE_PATH/specific-threats.rules
# include $SO_RULE_PATH/web-activex.rules
# include $SO_RULE_PATH/web-client.rules
# include $SO_RULE_PATH/web-iis.rules
# include $SO_RULE_PATH/web-misc.rules

# Event thresholding or suppression commands. See threshold.conf
include threshold.conf
include /etc/snort/rules/local.rules
```

Para iniciar snort lo podemos hacer con el siguiente comando:

```
snort -D -c /etc/snort/snort.conf -i eth0
```

Le indicamos el archivo de configuracion y la interfaz de escucha de red,



```
-----
Number of patterns truncated to 20 bytes: 1038 ]
pcap DAQ configured to passive.
Acquiring network traffic from "eth0".
Reload thread starting...
Reload thread started, thread 0x786089a00700 (2320)
Decoding Ethernet

==== Initialization Complete ====

--> Snort! <*-
o" )~
  ~~~~
    Version 2.9.20 GRE (Build 82)
    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
    Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
    Copyright (C) 1998-2013 Sourcefire, Inc., et al.
    Using libpcap version 1.10.0 (with TPACKET_V3)
    Using PCRE version: 8.39 2016-06-14
    Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_S7COMMPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_MODEBUS Version 1.1 <Build 1>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Commencing packet processing (pid=2319)
```

Vamos a probar el trigger. Para realizar la prueba hago un ping desde dos maquinas diferentes al servidor proxmox. Una maquina desde la red **192.168.10.0/24** y otra desde la misma red la **10.0.2.0/24**

```

root@Deb-Ansible:~# ping 10.0.2.1
PING 10.0.2.1 (10.0.2.1): 56(84) bytes of data.
64 bytes from 10.0.2.1: icmp_seq=1 ttl=64 time=0.224 ms
64 bytes from 10.0.2.1: icmp_seq=2 ttl=64 time=0.260 ms
64 bytes from 10.0.2.1: icmp_seq=3 ttl=64 time=0.216 ms
^C
--- 10.0.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 203ms
rtt min/avg/max/mdev = 0.216/0.233/0.260/0.019 ms
root@Deb-Ansible:~# ping 10.0.2.2
PING 10.0.2.2 (10.0.2.2): 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.329 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=63 time=0.390 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=63 time=0.303 ms
^C
--- 10.0.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 204ms
rtt min/avg/max/mdev = 0.303/0.340/0.390/0.036 ms
root@Deb-Ansible:~#

root@Wazuh-Server:~# ip a | grep inet
inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
    inet 10.0.2.11/24 brd 10.0.2.255 scope global eth0
    inet6 fe80::b24:11ff:fe5b:58e/64 scope link
root@Wazuh-Server:~# ping 10.0.2.2
PING 10.0.2.2 (10.0.2.2): 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=64 time=0.131 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=64 time=0.053 ms
^C
--- 10.0.2.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 106ms
rtt min/avg/max/mdev = 0.053/0.092/0.131/0.039 ms
root@Wazuh-Server:~#

[Number of patterns truncated to 20 bytes: 1038 ]
pcap DAQ configured to passive.
Acquiring network traffic from "eth0".
Reload thread starting...
Reload thread started, thread 0x723419c00700 (517)
Decoding Ethernet

---== Initialization Complete ===---

--> Snort! <--
Version 2.9.20 GRE (Build 92)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.0 (with TPACKET_V3)
Using FUSE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_S7COMMPPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_MQTBUS Version 1.1 <Build 1>
Preprocessor Object: SF_DCEBPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_FIPELNET Version 1.2 <Build 13>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>
Preprocessor Object: SF_SSLFP Version 1.1 <Build 4>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_POP Version 1.0 <Build 1>

Commencing packet processing (pid=516)
05/28-22:04:32.540983  [**] [1:1000001:1] Ping a Servidor Proxmox Detectado [**] [Priority
: 0] (ICMP) 10.0.2.2 -> 10.0.2.11
05/28-22:04:33.403301  [**] [1:1000001:1] Ping a Servidor Proxmox Detectado [**] [Priority
: 0] (ICMP) 10.0.2.2 -> 10.0.2.11
05/28-22:04:33.588936  [**] [1:1000001:1] Ping a Servidor Proxmox Detectado [**] [Priority
: 0] (ICMP) 10.0.2.2 -> 10.0.2.1
05/28-22:04:32.411489  [**] [1:1000001:1] Ping a Servidor Proxmox Detectado [**] [Priority
: 0] (ICMP) 10.0.2.2 -> 10.0.2.1
05/28-22:04:33.635451  [**] [1:1000001:1] Ping a Servidor Proxmox Detectado [**] [Priority
: 0] (ICMP) 10.0.2.2 -> 10.0.2.1

```

A la izquierda tenemos ambas máquinas y a la derecha el log de snort.

Podemos observar que snort está monitoreando el tráfico de la red 10.0.2.0/24 y nos triggerea los paquetes icmp que van dirigidos al servidor proxmox **10.0.2.2** tal como le indicamos

Anexo

He tenido que cambiar la configuracion de red del puente de proxmox vmbr2 que maneja la red 10.0.2.0/24 ya que tras investigar mucho, descubrí que proxmox crea una especie de vlan por cada máquina virtual que encapsula el tráfico de red para que se pueda usar la red con normalidad pero que los dispositivos conectados a ella no puedan sniffear paquetes que viajen por la red.

se anaden las siguientes lineas en la configuracion /etc/network/interfaces de proxmox:

```

up /usr/sbin/brctl setageing vmbr2 0
up /usr/sbin/brctl setfd vmbr2 0

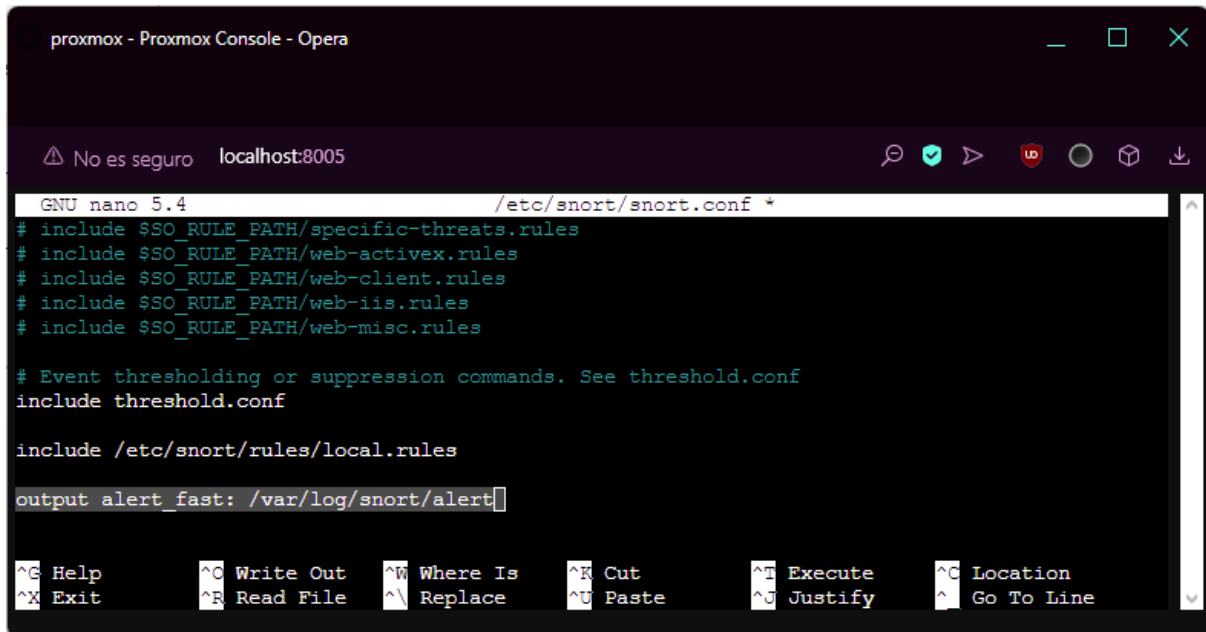
```

Anexo

Notificaciones al Teléfono e interfaz Web (NTFY)

Vamos a configurar la máquina snort para que nos envíe una notificación al correo electrónico cada vez que salte una alerta

para ello vamos a añadir la siguiente línea al archivo de configuración



The screenshot shows a terminal window titled "proxmox - Proxmox Console - Opera". The address bar indicates "localhost:8005". The terminal is running GNU nano 5.4 editing the file "/etc/snort/snort.conf". The configuration includes several rule paths and a threshold configuration. The line "output alert_fast: /var/log/snort/alert" is being edited, with the cursor at the end of the line. A help menu is visible at the bottom of the terminal.

```
proxmox - Proxmox Console - Opera
localhost:8005
GNU nano 5.4 /etc/snort/snort.conf *
# include $SO_RULE_PATH/specific-threats.rules
# include $SO_RULE_PATH/web-activex.rules
# include $SO_RULE_PATH/web-client.rules
# include $SO_RULE_PATH/web-iis.rules
# include $SO_RULE_PATH/web-misc.rules

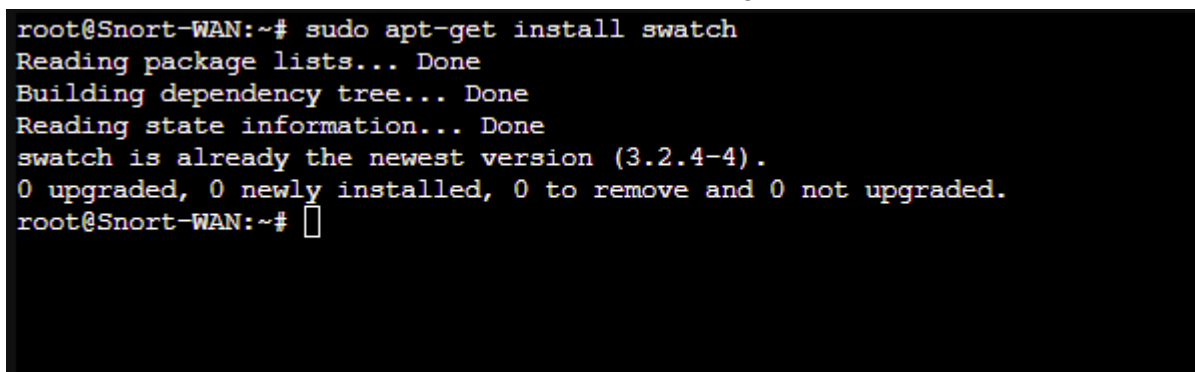
# Event thresholding or suppression commands. See threshold.conf
include threshold.conf

include /etc/snort/rules/local.rules

output alert_fast: /var/log/snort/alert
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Ahora las alertas de snort se guardarán en el archivo **/var/log/snort/alert**

vamos a instalar swatch para monitorizar el archivo de logs:



The screenshot shows a terminal window with the command "sudo apt-get install swatch" being executed. The output shows that swatch is already the newest version (3.2.4-4) and no packages were upgraded, newly installed, or removed.

```
root@Snort-WAN:~# sudo apt-get install swatch
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
swatch is already the newest version (3.2.4-4).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@Snort-WAN:~#
```

El archivo de logs (**/var/snort/alert**) guarda la alerta con el siguiente mensaje:

```
05/29-08:25:16.528047 [**] [1:1000001:1] Ping a Servidor Proxmox Detectado [**]
[Priority: 0] {ICMP} 10.0.2.2 -> 10.0.2.1
```

Con esta información vamos a crear un script que envíe un POST a un servidor de NTFY con la información del log, pero vamos a abrir el servidor a través de un túnel de cloudflare para poder enrutar el tráfico de forma segura a mi servidor local.

Voy a montar mi servidor http de ntfy en mi raspberry-pi4 .

```
root@snort-WAN:~# sudo docker run -d cloudflare/cloudflared:latest tunnel --no-autoupdate run --token eyJhYjIyMTM4MzE5MTdtYTQzMzY3ZmYzYXU0eWEyMGNhbmNkLWVkdjBjZWJ3MGtEOTZCOONw1  
ILRtTlMgTnNjc3MySG9ic0Fvbi1wey61K16tmiZV0xwWpNPf1XRZdaTaAwNW1ZMOxUbGXFVi10TdFNPF1GV7Joa1UxtUR0ay99  
Unable to find image 'cloudflare/cloudflared:latest' locally  
latest: Pulling from cloudflare/cloudflared  
7161936ae90: Pull complete  
2ae710d8afe: Pull complete  
d462aa345367: Pull complete  
0f8b424aa0b9: Pull complete  
d55767654e5: Pull complete  
e9022d091192a: Pull complete  
d858bc252ad: Pull complete  
1069fc2daed1: Pull complete  
b40161cd83fc: Pull complete  
53f8d93a3a65: Pull complete  
307c1adaab60: Pull complete  
2772bd5fb3df: Pull complete  
8852bb06792: Downloading [=====]           ]    1.22MB/1.893MB  
4ba746eb059: Download complete  
6e7c833d53d: Downloading [====]              ]    974.1kB/18.98MB
```

Cloudflare

Support

Zero Trust overview

Analytics

Gateway

Access

Networks New

Tunnels

Routes

DEX

My Team

Logs

Settings

Networks / Tunnels

Tunnels

Tunnels establish a secure connection between Cloudflare's edge and your infrastructure.

[Learn more](#)

Your tunnels Showing 1 - 1

Manage the configurations of your existing tunnels.

[+ Create a tunnel](#)

Search by tunnel name

Tunnel name ↑	Connector type	Connector ID	Tunnel ID	Routes	Status	Uptime
NTFY	cloudflared	a83fd79f-edfc-4d8f-b1cc-1a2e2e3d7b02-27	dfc-1a2e2e3d7b02-9b0b-6772a22c7595f	--	HEALTHY	1 hours ⋮

1-1 Items per page: 10< 1 of 1 page >

Para activar el dominio en cloudflare necesitaremos agregar los NS en nuestro registro DNS igual que haríamos con el bind9

scandedofct.xyz

Active

Starred

Free plan

Page Rules are deprecated. You should consider alternative [Rules](#) options due to their enhanced configurability. Refer to [the migration guide](#) for details.

Overview

scandedofct.xyz

Active

Starred

Free plan

DNS Setup: Full ⓘ

Import and Export ▾

⚙️ Dashboard Display Settings

Search DNS Records

▾ Add filter

Search

+ Add record

Type	Name	Content	Proxy status	TTL	Actions
CNAME	ntfy	d5ceb70a-996d-...	Proxied	Auto	Edit ▶

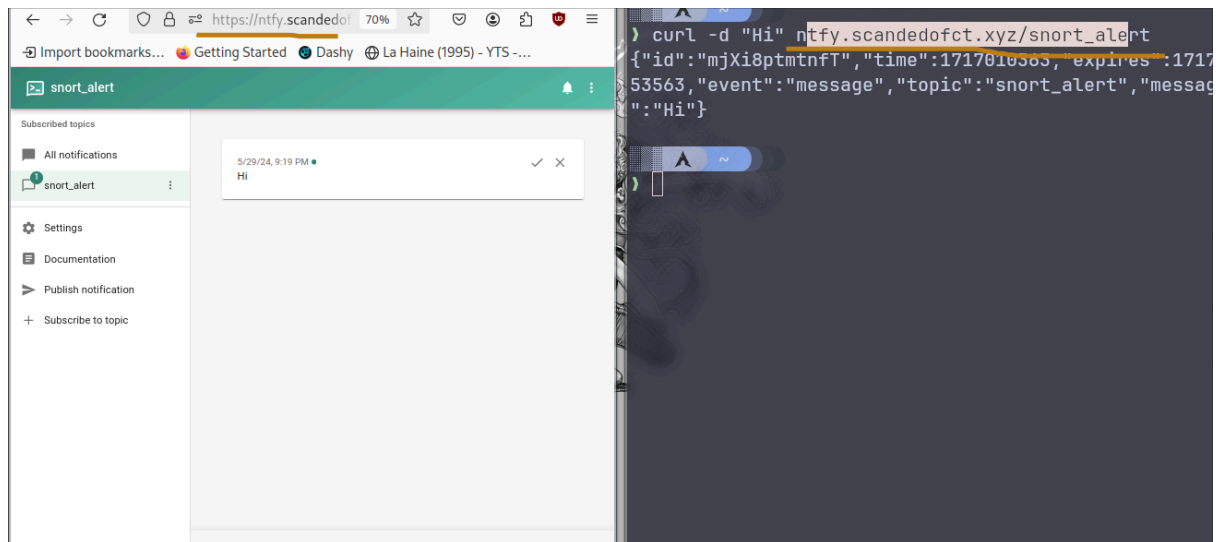
Cloudflare Nameservers

Every DNS zone on Cloudflare is assigned a set of Cloudflare-branded nameservers.

Type	Value
NS	ariadne.ns.cloudflare.com
NS	pete.ns.cloudflare.com

Podemos ver que nuestros Nameservers estén bien configurados con páginas de escaneo de dominios como whois.com, tambien añadí un subdominio que usaremos para mandar mensajes a ntfy

Conectado a la red de mi movil puedo acceder al tunel de cloudfare apuntando a mi dominio.



¿Cómo funciona el tunel de cloudflare?

Para entender cómo funciona un tunel de cloudflare vamos a seguir el tráfico que seguiría un paquete de petición GET a un sitio web que está alojado en mi red local pero la petición se la realizamos ala DNS

1. Cliente realiza la petición:
 - Tu navegador o aplicación cliente quiere acceder a un recurso en **ntfy.scandedo.xyz**.
2. Resolución de DNS:
 - El cliente consulta al servidor DNS para obtener la dirección IP asociada con **ntfy.scandedofct.xyz**.
 - En este caso, Cloudflare es el servicio de DNS que responde con la IP de uno de sus servidores.
3. Conexión al servidor de Cloudflare:
 - El cliente se conecta al servidor de Cloudflare usando la IP proporcionada y el puerto 80 (HTTP).
4. Establecimiento del túnel:
 - Cloudflare tiene un túnel configurado que redirige el tráfico desde **ntfy.scandedofct.xyz** hacia tu servidor web ubicado en la IP interna **192.168.5.10:80**.
5. Transmisión de la petición a través del túnel:
 - La petición GET que el cliente envió a **ntfy.scandedo.xyz** ahora es redirigida por Cloudflare a través del túnel seguro hacia tu red interna.
 - Cloudflare sabe a qué servidor interno redirigir la petición basándose en la configuración que hiciste (indicando que las peticiones a **ntfy.scandedo.xyz** deben ser enviadas a **192.168.5.10:80**).

6. Recepción en el servidor web:

- Tu servidor web en **192.168.5.10:80** recibe la petición GET que fue originada por el cliente.
- El servidor procesa la petición, busca el recurso solicitado y prepara una respuesta (por ejemplo, una página web, un archivo, etc.).

7. Respuesta al cliente:

- La respuesta del servidor web es enviada de vuelta a través del túnel de Cloudflare.
- Cloudflare recibe la respuesta desde **192.168.5.10:80** y la envía de vuelta al cliente que hizo la solicitud inicial.

8. Cliente recibe la respuesta:

- El cliente recibe la respuesta del recurso solicitado y lo muestra o procesa según corresponda (por ejemplo, mostrando una página web en el navegador).

La petición GET viaja desde el cliente a través de los servidores de Cloudflare, que tienen un túnel configurado para redirigir el tráfico hacia tu servidor web interno en la IP **192.168.5.10:80**, y la respuesta sigue el camino inverso de vuelta al cliente.

¿Qué es NTFY?

Ntfy es un servicio de notificaciones HTTP pub-sub

Nos permite enviar notificaciones a tu teléfono o escritorio a través de scripts desde cualquier PC o servidor, el software es 100% código abierto, podemos observar el código en su [github](#)

Gracias a NTFY podemos abrir un servidor web en nuestra red local que permita **PETICIONES POST O PUT** para poder recibir notificaciones

Para ello vamos a crear un contenedor de docker que hostee nuestro servidor de **ntfy** que escuche en nuestro puerto 8888

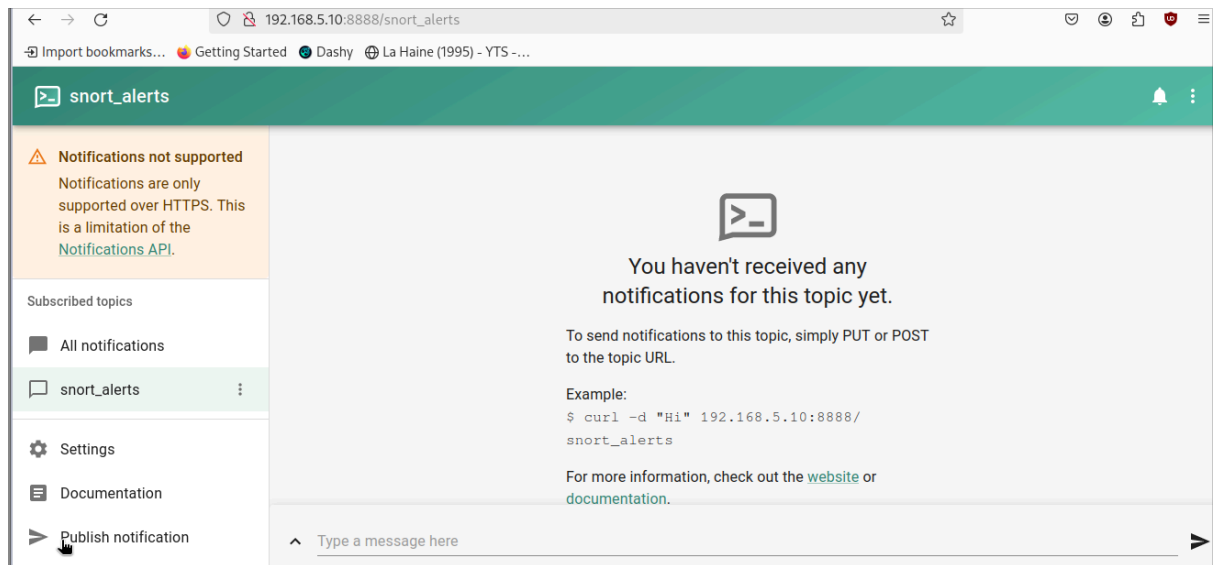
```
services:
  ntfy:
    image: binwiederhier/ntfy
    container_name: ntfy
    command:
      - serve
    environment:
      - TZ=Europe/Madrid
    volumes:
```

- /var/cache/ntfy:/var/cache/ntfy
- /etc/ntfy:/etc/ntfy

ports:

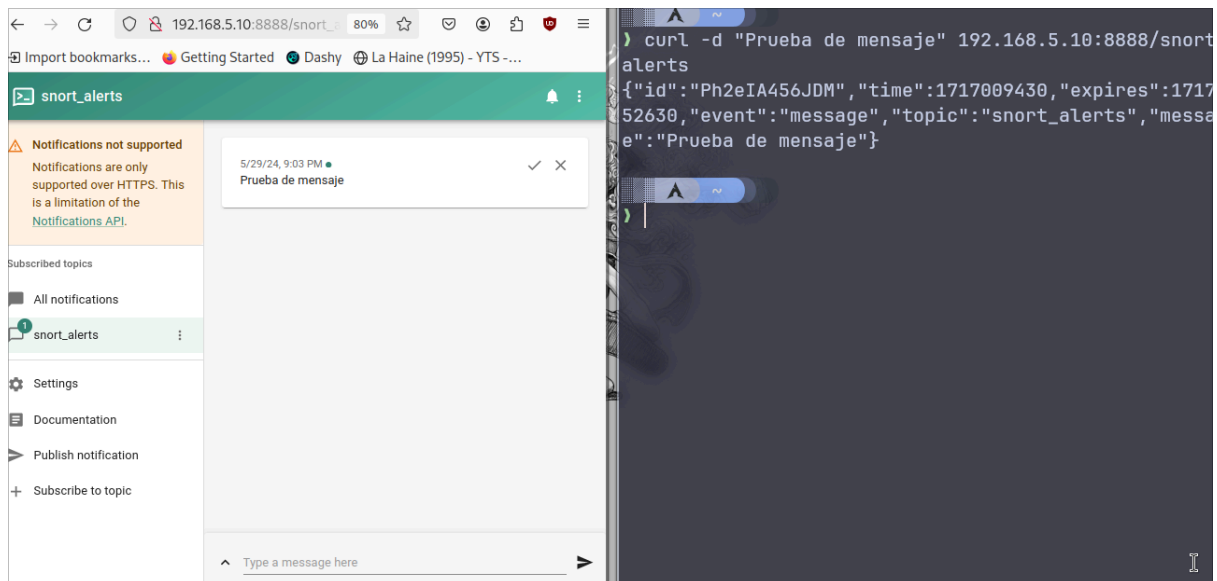
- 8888:80

restart: unless-stopped

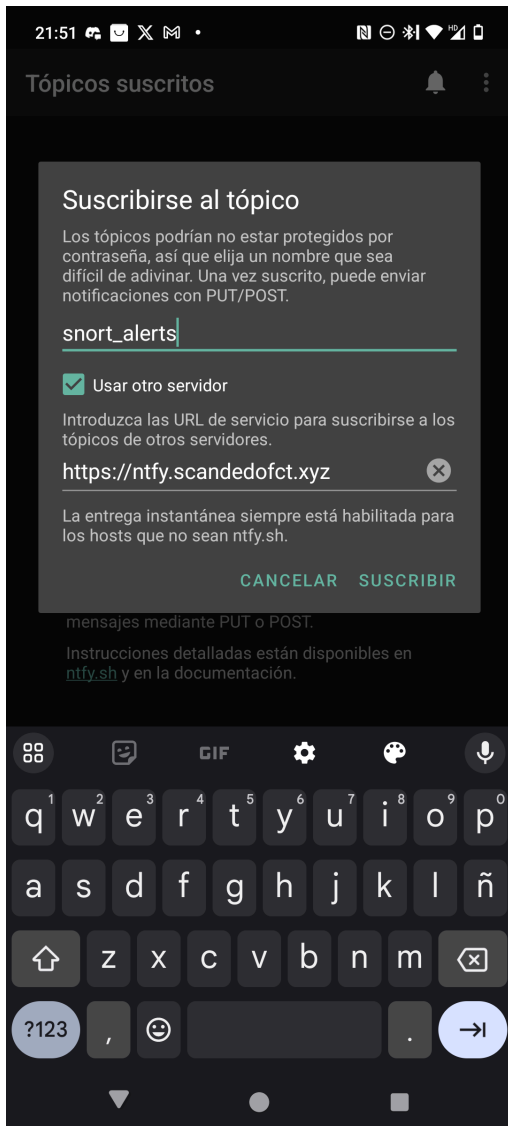


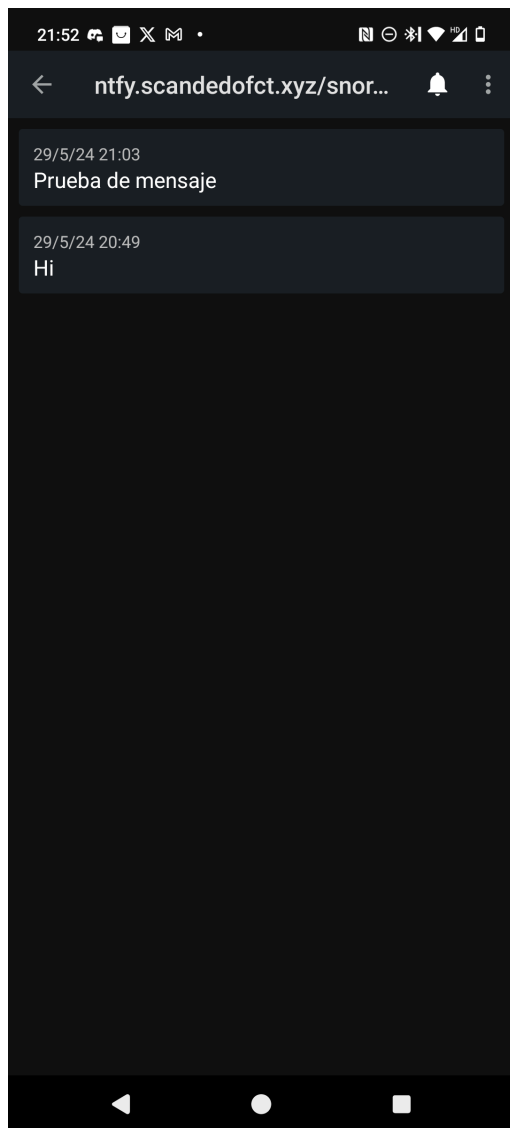
Una vez iniciado el docker podremos ver la página web de ntfy

Con un curl -d para hacer un post podemos mandar mensajes a la página web



Una vez creado nuestro tunel de cloudflare podremos enviar mensajes apuntando a nuestro tunel de cloudflare, incluso desde nuestras maquinas virtuales fuera de la red , ya que enviaremos la peticion a los servidores de cloudflare de forma segura





Voy a crear el script que usará swatch para mandar notificaciones al servidor de **ntfy** ,
Necesitamos darle permisos de ejecución (chmod +x) al usuario con el que funcionará el
servicio de swatch .

```
#!/bin/bash

# DomainName que apunta a servidor ntfy desde el tunel de cloudfare
NTFY_URL="https://ntfy.scandedofct.xyz/snort_alerts"

# Leer la línea de entrada de Swatch
line="$1"

# Verificar si la línea contiene la alerta, si $?=0 entonces es true
```

```
echo "$line" | grep "Ping a Servidor Proxmox Detectado" > /dev/null
if [ $? = 0 ]; then
    # Extraer la IP de origen (se asume que la IP está en la segunda columna)
    IP_ORIGEN=$(echo "$line" | awk '{print $(NF-0)}')
    # Enviar notificación mediante POST
    curl -d "Ping al servidor proxmox detectado desde la IP: $IP_ORIGEN" "$NTFY_URL"
fi
```

Una vez tenemos el script lo dejo en una ruta del sistema a donde swatch pueda tener acceso . para este caso vamos a usar la cuenta de root y guardo el script en **/root/script/ntfy-alert.sh**

Ahora necesitamos un archivo de configuración de swatch donde vamos a darle instrucciones para que revise la línea “Ping a Servidor Proxmox Detectado” dentro del archivo de logs de snort situado en **/var/log/snort/alert**

Nuestro archivo .swatchdogrc se ve de esta forma:

```
watchfor /Ping a Servidor Proxmox Detectado/
exec /root/script/ntfy-alert.sh
```

Podemos iniciar swatch de forma manual con el siguiente comando:

```
swatchdog --config-file=/root/.swatchrc --tail-file=/var/log/snort/alert
```

con --config-file le indicamos la ruta del archivo de configuraciones y con --tail file le indicamos que archivo debe monitorizar. En nuestro caso el archivo de logs de snort

Cómo quiero que swatch esté monitorizando constantemente el archivo de logs de Snort opto por crear un servicio de Linux en **/etc/systemd/system** para poder iniciarlo con systemctl y habilitarlo para que se inicie junto con el inicio del sistema igual que el servicio de snort.

Vamos a crear un servicio para automatizar la ejecución de swatch

Automatización con Proceso del sistema

```
GNU nano 5.4 /etc/systemd/system/swatch.service *
[Unit]
Description=Servicio swatch para las alertas de snort

[Service]
ExecStart=/usr/bin/swatchdog --config-file=/root/.swatchdogrc --tail-file=/var/log/>
Restart=always
User=root

[Install]
WantedBy=multi-user.target
█
```

Ahora reiniciamos los daemons e iniciamos el servicio

systemctl daemon-reload

systemctl start swatch

systemctl enable swatch

podemos hacer un enable si queremos que se inicie junto el sistema operativo

Vamos a comprobar que el servicio de swatch está funcionando y que ejecuta el script que envía el aviso al servidor web

```
root@Snort-WAN:~# systemctl status swatch
* swatch.service - Servicio swatch para las alertas de snort
   Loaded: loaded (/etc/systemd/system/swatch.service; enabled; vendor preset: en
   Active: active (running) since Wed 2024-05-29 22:59:39 UTC; 8min ago
   Main PID: 2240 (swatchdog)
     Tasks: 3 (limit: 28663)
    Memory: 23.9M
       CPU: 546ms
    CGroup: /system.slice/swatch.service
            └─2240 /usr/bin/perl /usr/bin/swatchdog --config-file=/root/.swatchdog
            └─2241 /usr/bin/perl /root/.swatchdog_script.2240
            └─2243 /usr/bin/tail -n 0 -F /var/log/snort/alert

May 29 23:06:26 Snort-WAN swatchdog[2401]: [158B blob data]
May 29 23:06:26 Snort-WAN swatchdog[2401]: ("id":"y564GI9vesaI","time":1717023986,"
May 29 23:06:57 Snort-WAN swatchdog[2411]: % Total % Received % Xferd Average
May 29 23:06:57 Snort-WAN swatchdog[2411]: [158B blob data]
May 29 23:06:57 Snort-WAN swatchdog[2411]: ("id":"2CukYvY9YRGx","time":1717024017,"
May 29 23:06:58 Snort-WAN swatchdog[2420]: % Total % Received % Xferd Average
May 29 23:06:58 Snort-WAN swatchdog[2420]: [158B blob data]
May 29 23:06:58 Snort-WAN swatchdog[2420]: ("id":"zdeskZnqACcw","time":1717024018,"

root@Snort-WAN:~# ping 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.362 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=63 time=0.418 ms

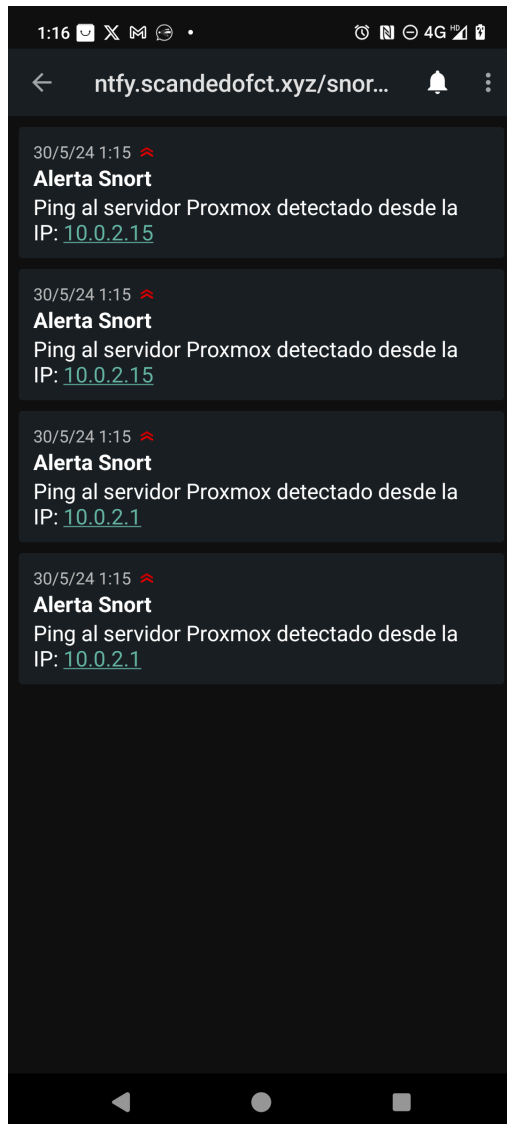
--- 10.0.2.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1030ms
rtt min/avg/max/mdev = 0.362/0.390/0.418/0.028 ms
root@Arch-Docker-FCI ~]#
```

El servicio está funcional y ejecutando el script cuando detecta que se añade la línea especificada en el .swatchdogrc en el archivo del log, el script se encarga de recoger con awk la ip del atacante y mandarla por curl al servidor web de **ntfy**

En el script podríamos apuntar a la 192.168.5.10:8888 pero prefiero apuntar al túnel de cloudflare ya que quiero que la red wan quede aislada de enviar tráfico a mi LAN

Entonces el paquete que envío por curl viaja de tal forma que llega al nameserver de cloudflare y cloudflare se encarga de enrutar el tráfico a mi máquina local gracias al Docker que desplegamos en la máquina. De esta forma creamos un túnel seguro de cloudflare y no hace falta abrir puertos a internet en mi router. Todo el tráfico viaja a través de cloudflare de forma segura.

Revisamos que las notificaciones llegasen al servidor de **ntfy**



Podemos aplicar esta documentación para automatizar todo tipo de notificaciones que nos ayuden al monitoreo del sistema, por ejemplo, me interesa recibir cada 30 min una notificación al móvil que me avise de la temperatura de mi servidor Proxmox. Para ello hago uso de un script y del comando sensors que me indica la temperatura de los dispositivos de mi placa:

```

root@proxmox:~# sensors
k10temp-pci-00c3
Adapter: PCI adapter
Tctl:      +35.6°C
Tccd1:     +32.2°C
Tccd2:     +34.5°C

nvme-pci-0400
Adapter: PCI adapter
Composite:  +27.9°C (low = -0.1°C, high = +82.8°C)
              (crit = +89.8°C)

nvme-pci-0100
Adapter: PCI adapter
Composite:  +39.9°C (low = -40.1°C, high = +83.8°C)
              (crit = +87.8°C)
Sensor 1:   +59.9°C (low = -273.1°C, high = +65261.8°C)
Sensor 2:   +34.9°C (low = -273.1°C, high = +65261.8°C)

root@proxmox:~# █

```

El script luce de la siguiente manera

```

#!/bin/bash

# Obtener la temperatura de la CPU , grepea sólo el num ej 30.45
CPU_TEMP=$(sensors | grep 'Tctl' | awk '{print $2}' | tr -d '+°C')

# Convertir la temperatura a un número entero para poder usar con -gt
CPU_TEMP_INT=$(printf "%.0f" $CPU_TEMP)

# Enviar la temperatura al servidor ntfy
curl -d "Temperatura actual de la CPU: $CPU_TEMP°C" ntfy.scandedofct.xyz/CPU_TEMP

# Si la temperatura es mayor a 60°C, enviar alerta
if [ $CPU_TEMP_INT -gt 60 ]; then
    curl -d "ALERTA: Temperatura ALTA de la CPU: $CPU_TEMP°C"
    ntfy.scandedofct.xyz/CPU_TEMP
fi

```

** Sintaxis

printf "%.0f" convierte el número que grepeamos con awk tr y grep en un entero.

**

Automatización con Crontab

Ahora podemos crear una regla de cron para que ejecute este script cada 30 min:

```
crontab -e
```

Para abrir el archivo de configuración de crontab donde le especificaremos cada cuanto queremos ejecutar una tarea y las instrucciones de que debe de hacer

```
GNU nano 7.2 /tmp/crontab.2$GNU1/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
*/5 * * * * ~/duckdns/duck.sh >/dev/null 2>&1
*/30 * * * * /root/scripts/NTFY/CPUtemp.sh
```

En mi caso le indico que quiero ejecutar el script cada 30 min .

Si ejecuto el script ahora vemos que nos llega una notificación al servidor de **ntfy**



Voy a crear otro documento donde detallo todos los procesos que monitorizo a través de ntfy y sus respectivos scripts

[DOC CON SCRIPTS DE MONITOREO](#)

```
sudo systemctl enable proxmox-login-monitor.service
sudo systemctl start proxmox-login-monitor.service
```

Gestion de logs centralizada

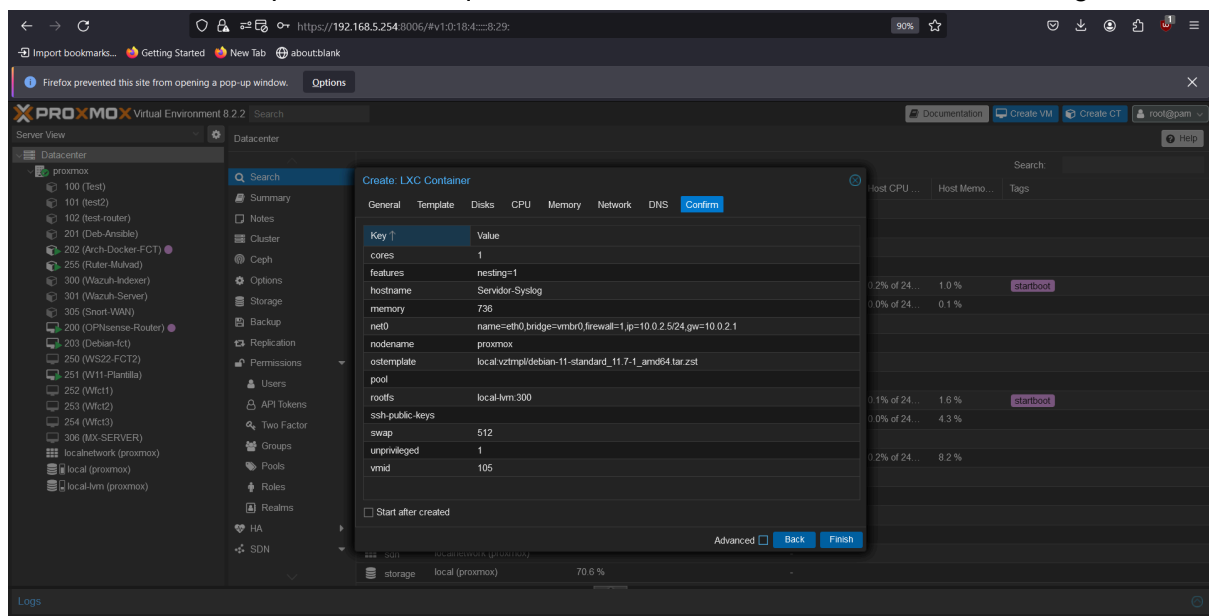
Como implementación final del sistema de detección y respuesta ante amenazas vamos a implementar el uso de diferentes herramientas para centralizar los logs de todos los sistemas que componen nuestra red

Vamos a crear un servidor de syslog centralizado para que nuestras máquinas críticas dentro de la infraestructura envíen los logs a este servidor y en caso de ataque podamos detectar el momento y la causa.

Configurar un servidor centralizado de syslog implica varios pasos, incluyendo la configuración del servidor syslog central y la configuración de los clientes para enviar sus logs a este servidor.

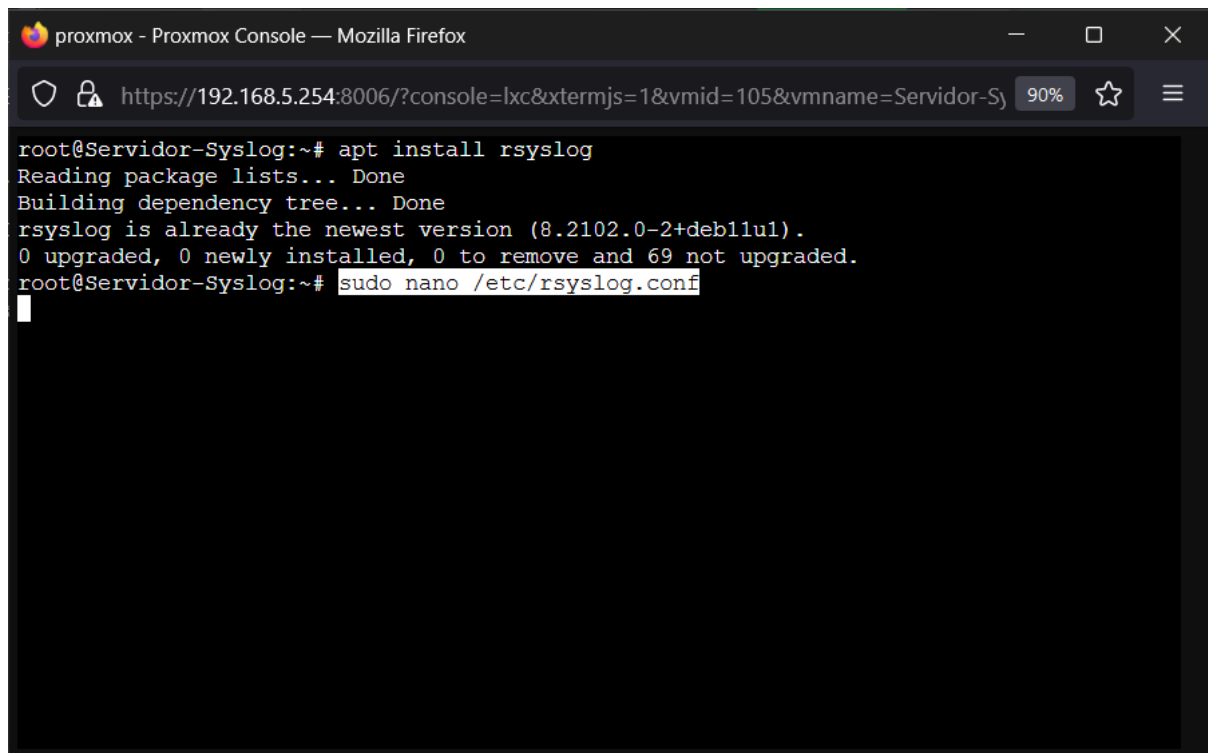
Gracias a ansible vamos a crear un playbook de forma que pueda realizar la instalación en todos los dispositivos linux de mi red de forma remota centralizada y eficiente.

Creo una nueva máquina 10.0.2.5 que va a ser nuestro servidor centralizado de logs



Los dos servidores syslog más comunes son rsyslog y syslog-ng. Para el proyecto vamos a usar rsyslog.

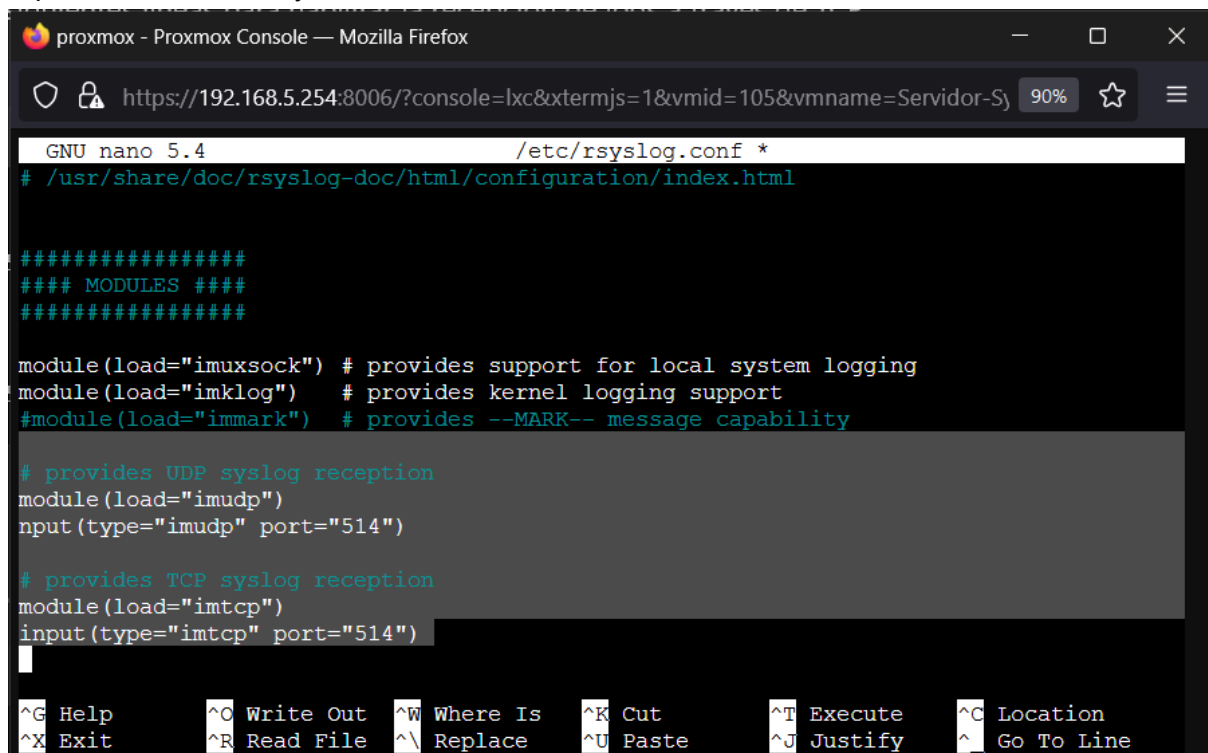
Instalamos rsyslog y vamos a la configuración para establecerlo como servidor centralizado



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lxc&xtermjs=1&vmid=105&vmname=Servidor-Sy 90% ☆ ≡

root@Servidor-Syslog:~# apt install rsyslog
Reading package lists... Done
Building dependency tree... Done
rsyslog is already the newest version (8.2102.0-2+deb11u1).
0 upgraded, 0 newly installed, 0 to remove and 69 not upgraded.
root@Servidor-Syslog:~# sudo nano /etc/rsyslog.conf
```

Descomentamos las siguientes lineas en el archivo de configuracion para recibir trafico por el puerto 514 en UDP y TCP



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lxc&xtermjs=1&vmid=105&vmname=Servidor-Sy 90% ☆ ≡

GNU nano 5.4 /etc/rsyslog.conf *
# /usr/share/doc/rsyslog-doc/html/configuration/index.html

#####
### MODULES ###
#####

module(load="imuxsock") # provides support for local system logging
module(load="imklog") # provides kernel logging support
#module(load="immark") # provides --MARK-- message capability

# provides UDP syslog reception
module(load="imudp")
input(type="imudp" port="514")

# provides TCP syslog reception
module(load="imtcp")
input(type="imtcp" port="514")

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Tenemos que habilitar los puertos tambien en el firewall (ufw) aunque yo uso proxmox como firewall asi que abro los puertos alli

Una vez configurado vamos a reiniciar rsyslog

```
systemctl restart rsyslog
```

Configuración de los clientes Syslog con ansible

Para configurar los clientes de syslog tenemos que seguir los mismos pasos, instalar rsyslog en el sistema y editar el archivo de configuración añadiendo la ip del servidor de logs al final de la configuración

```
** @192.168.1.100:514 # UDP
** @@192.168.1.100:514 # TCP
```

Así luce nuestro playbook >

```
---
- name: Configurar rsyslog como cliente para enviar logs a un
servidor central
  hosts: all
  become: yes

  vars:
    servidor_syslog: "10.0.2.5"
    archivo_conf_rsyslog: "/etc/rsyslog.conf"
    servicio_rsyslog: "rsyslog"

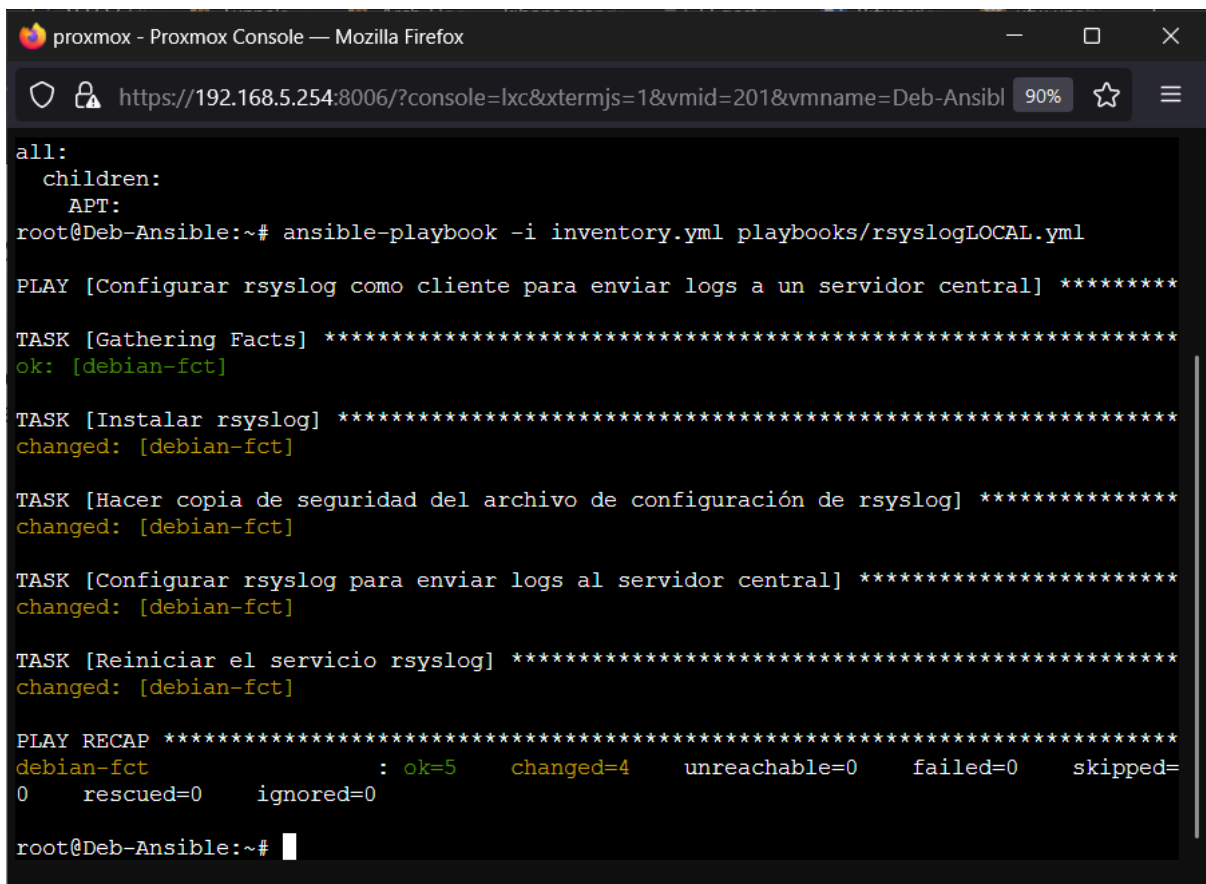
  tasks:
    - name: Instalar rsyslog
      apt:
        name: rsyslog
        state: present
        when: ansible_os_family == "Debian"

    - name: Hacer copia de seguridad del archivo de configuración de
rsyslog
      copy:
        src: "{{ archivo_conf_rsyslog }}"
        dest: "{{ archivo_conf_rsyslog }}.bak"
        backup: yes
```

```
- name: Configurar rsyslog para enviar logs al servidor central
  lineinfile:
    path: "{{ archivo_conf_rsyslog }}"
    insertafter: EOF
    line: "*. * @{{ servidor_syslog }}:514" # UDP

- name: Reiniciar el servicio rsyslog
  service:
    name: "{{ servicio_rsyslog }}"
    state: restarted
```

Vamos a ejecutar el playbook en el host Debian para demostrar el funcionamiento, para ello ajusto el parámetro host en el playbook de “all” a “debian-fct” que es como figura en el inventario :



```
proxmox - Proxmox Console — Mozilla Firefox
https://192.168.5.254:8006/?console=lx&xtermjs=1&vmid=201&vmname=Deb-Ansibl 90% ☆ ≡

all:
  children:
    APT:
root@Deb-Ansible:~# ansible-playbook -i inventory.yml playbooks/rsyslogLOCAL.yml

PLAY [Configurar rsyslog como cliente para enviar logs a un servidor central] *****

TASK [Gathering Facts] *****
ok: [debian-fct]

TASK [Instalar rsyslog] *****
changed: [debian-fct]

TASK [Hacer copia de seguridad del archivo de configuración de rsyslog] *****
changed: [debian-fct]

TASK [Configurar rsyslog para enviar logs al servidor central] *****
changed: [debian-fct]

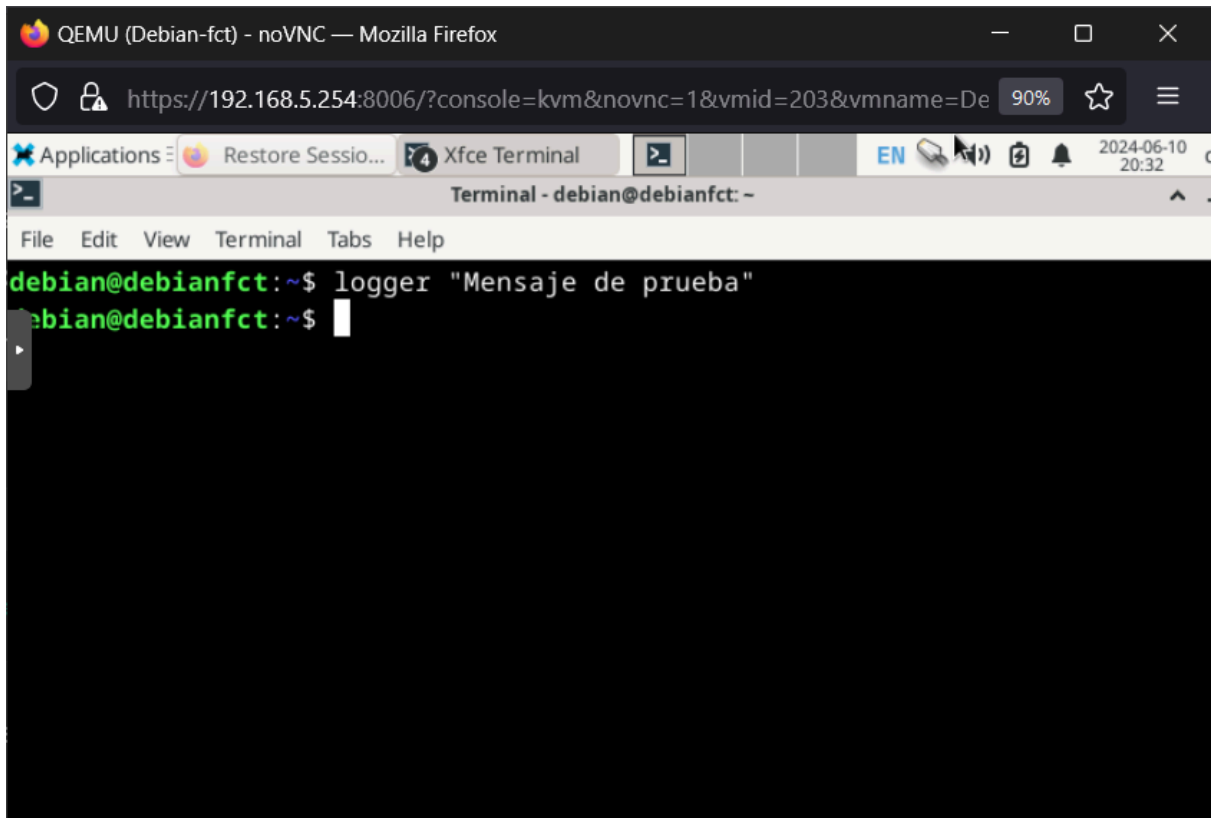
TASK [Reiniciar el servicio rsyslog] *****
changed: [debian-fct]

PLAY RECAP *****
debian-fct : ok=5    changed=4    unreachable=0    failed=0    skipped=
0    rescued=0    ignored=0

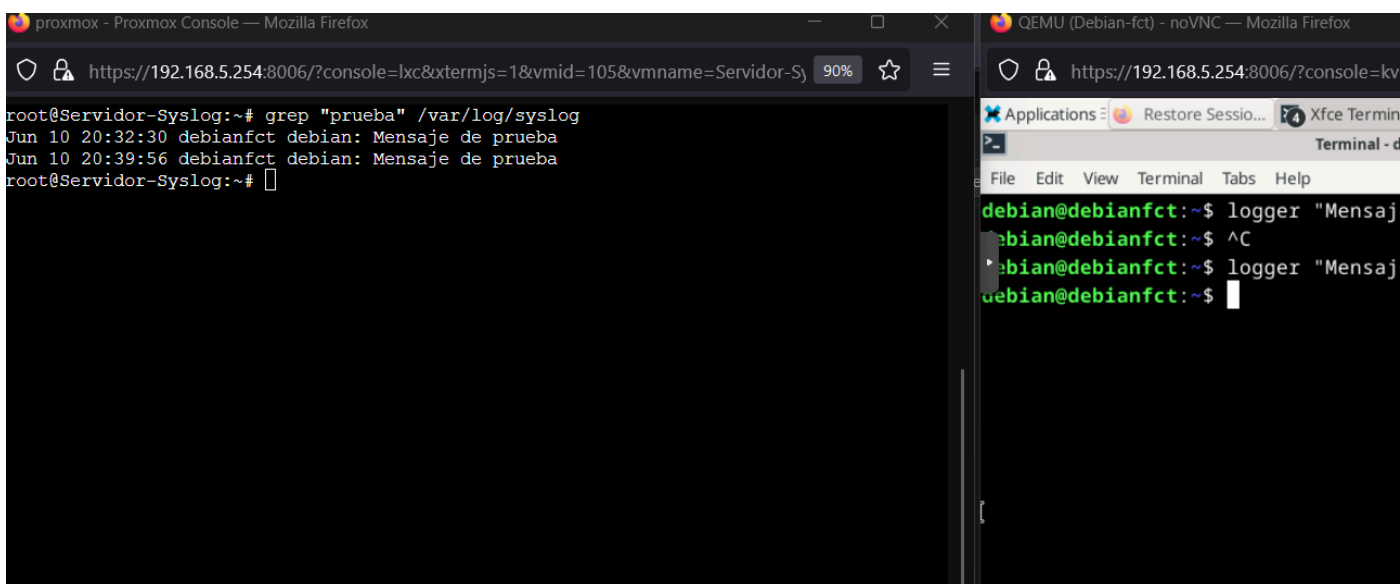
root@Deb-Ansible:~#
```

Ejecutamos el playbook y todas las tareas se ejecutan en el host de forma exitosa.

Para probar el funcionamiento podemos dirigirnos a nuestra maquina debian y guardar algo en el log:



Ahora en nuestro servidor centralizado de syslog deberiamos poder filtrar por ese mensaje



Ahora los logs de nuestras máquinas más críticas se guardan todas en un mismo log .